University of Applied Sciences and Arts Dortmund

Softwaretechnik 2/B

Prolog

Fachhochschule Dortmund
University of Applied Sciences and Arts

© SoSe 2026 Diepenbrock/Hirsch/Sachweh/Zeppenfeld

Fachhochschule Dortmund
University of Applied Sciences and Arts

Studiengänge

we focus on students

	Modulnummer	Studiengang	Semester
WiSe SWT 1/A SoSe SWT 2/B	44121	BA Informatik VR: Praktische Informatik/ Technische Informatik/ Data Science	4.
	44122	BA Informatik Dual VR: Softwaretechnik + WPF-Katalog	4. (erste Hälfte)
SoSe SWT 2/B	44121	BA Medizinische Informatik	4.
	44121	BA Medizinische Informatik Dual	4.
	44121	BA Wirtschaftsinformatik	4.

Fachhochschule Dortmund
University of Applied Sciences and Arts

Gruppenbildung im Praktikum

we focus on students

	Modulnummer	Studiengang	Semester
WiSe SWT 1/A SoSe SWT 2/B	44121	BA Informatik VR: Praktische Informatik/ Technische Informatik/ Data Science	4.
	44122	BA Informatik Dual VR: Softwaretechnik + WPF-Katalog	4. (erste Hälfte)
SoSe SWT 2/B	44121	BA Medizinische Informatik	4.
	44121	BA Medizinische Informatik Dual	4.
	44121	BA Wirtschaftsinformatik	4.

Gruppenbildung nur innerhalb eines Clusters möglich!!!

Fachhochschule Dortmund
University of Applied Sciences and Arts

Das Dozententeam

we focus on students

Sabine Sachweh

Andreas Diepenbrock

Martin Hirsch

Philip Wizenty

Klaus Zeppenfeld

Fachhochschule Dortmund
University of Applied Sciences and Arts

Die studentischen Hilfskräfte (SHKs)

we focus on students

Micha Keiten

Yahya Hasan Bek

Ahmad Bader Eddin Lababidi

Gabriel Nunes Trautmann

Angelika Kopia

Rodolphe Loic Souassi Tatang

Niloufar Oladzadabasabadi

Yasaman Oladzadabasabadi

Philipp Mikos

Fachhochschule Dortmund
University of Applied Sciences and Arts

Fragen zur Lehrveranstaltungsplanung

we focus on students

Bitte richten Sie Fragen zur Lehrveranstaltungsplanung an
Prof. Dr. Klaus Zeppenfeld

- Büro: **EFS 42**, Raum C.3.35

- E-Mail: klaus.zeppenfeld@fh-dortmund.de
- WWW: https://www.fh-dortmund.de/personen/Klaus-Zeppenfeld/index.php

Fachhochschule Dortmund
University of Applied Sciences and Arts

Termine

we focus on students

Vorlesung:	**Softwaretechnik 2/B**	
Studiengänge:	PI, TI und Informatik Dual, MI, MI Dual und WI	
Vorlesung/ Besprechung:	Montag:	14:15 – 17:35 (bis zum 8.6.26 einschließlich)
Praktikum:	siehe Plan	

Veranstaltungsnummer: 44121 und 44122 !!!

Fachhochschule Dortmund
University of Applied Sciences and Arts

Stundenplan

we focus on students

gültig ab 27.04.2026

Stundenplan SoSe 2026

		Montag	Dienstag		Mittwoch		Donnerstag	Freitag
1.	8:30 - 9:15		P		P	Inf- Dual		
2.	9:20 - 10:05							
3.	10:15 - 11:00		P		P		P	P
4.	11:05 - 11:50							
5.	12:00 - 12:45		P	Inf- Dual	P	Inf- Dual	P	P
6.	12:50 - 13:35							
	Pause				Pause		Pause	
7.	14:15 - 15:00	Sabine SWT2 V A.E.01	P		P		P	P
8.	15:05 - 15:50	Sabine SWT2 V A.E.01						
9.	16:00 - 16:45	SWT1 V A.E.01	P		P		P	P
10.	16:50 - 17:35	SWT2 V A.E.01						

Vorlesung am Montag **nur in der ersten Semesterhälfte** bis zum 08.06.2026!!!

Fachhochschule Dortmund
University of Applied Sciences and Arts

Projektarbeit

we focus on students

- Die **Gruppenanmeldung** erfolgt durch den oder die Gruppensprecher:in **per Mail** an **klaus.zeppenfeld@fh-dortmund.de** plus Angabe möglicher Praktikumstermine (Mail-Template wird noch bekannt gegeben)

- Der Praktikumstermin und Gruppentutor wird dem Gruppensprecher per Mail zugesandt

- ACHTUNG: Die Projektarbeit ist die Prüfungsleistung der Veranstaltung!
  Prüfungstermine: 27.-31.07.25

!!! Veranstaltungsnummer: 44121 + 44122

Fachhochschule Dortmund
University of Applied Sciences and Arts

Veranstaltungsplanung für das SoSe 2026

we focus on students

Einstieg

Tools und Bibliotheken

UML-Modellierung

EU-Compliance

Woche	Datum VL	#	Thema
KW 17	20.04.26	1	Prolog + Einführung
		2	Historie und Verteilte Architekturen
KW 18	27.04.26	3	Git und Projektmanagement
		4	Maven
KW 19	04.05.26	5	Architekturmodellierung
		6	Architektur-Konzept Spring-Boot
KW 20	11.05.26	7	UML
		8	EU-Compliance I
KW 21	18.05.26	Blockwoche	
KW 22	25.05.26	Pfingsten	
KW 23	01.06.26	9	Entwurfsmuster
		10	EU-Compliance II
KW 24	08.06.26	11	Message Broker
		12	Docker
KW 25	15.06.26	keine Vorlesung	
KW 26	22.06.26	keine Vorlesung	
KW 27	29.06.26	keine Vorlesung	
KW 28	06.07.26	keine Vorlesung	
KW 29	13.07.26	keine Vorlesung	
KW 30	20.07.26	13	Vorlesungsabschluss/Beratung/Abschlusspräsentation
KW 31	**27.-31.07.2026**	***Mündliche Prüfung (InnoLab) - DUAL am 29.07.2026***	

Fachhochschule Dortmund
University of Applied Sciences and Arts

Praktikum für das SoSe 2026 – PI und TI

we focus on students

Woche	Datum VL	#	Kohorte	Thema	Praktikum Software Engineering (PI/TI)
KW 17	20.04.26	1	I/TI/WI/MI/Du	Prolog + Einführung	Teambildung
		2	I/TI/WI/MI/Du	Historie und Verteilte Architekturen	
KW 18	27.04.26	3	I/TI/WI/MI/Du	Git und Projektmanagement	Epics und User Stories + Entwicklungsinfrastruktur aufsetzen und testen + Kanban-Board und Burn-down Chart
		4	I/TI/WI/MI/Du	Maven	
KW 19	04.05.26	5	I/TI/WI/MI/Du	Architekturmodellierung	Epics und User Stories + Entwicklungsinfrastruktur aufsetzen und testen + Kanban-Board und Burn-down Chart
		6	I/TI/WI/MI/Du	Architektur-Konzept Spring-Boot	
KW 20	11.05.26	7	I/TI/WI/MI/Du	UML	Epics und User Stories + Entwicklungsinfrastruktur aufsetzen und testen + Kanban-Board und Burn-down Chart
		8	I/TI/WI/MI/Du	EU-Compliance I	
KW 21	18.05.26	Blockwoche			frei
KW 22	25.05.26	Pfingsten			frei
KW 23	01.06.26	9	I/TI/WI/MI/Du	Entwurfsmuster	Architekturkonzept + API-Definition + Beginn Implementierung
		10	I/TI/WI/MI/Du	EU-Compliance II	
KW 24	08.06.26	11	I/TI/WI/MI/Du	Message Broker	Architekturkonzept + API-Definition + Beginn Implementierung
		12	I/TI/WI/MI/Du	Docker	
KW 25	15.06.26	keine Vorlesung			Architekturkonzept + API-Definition + Beginn Implementierung
KW 26	22.06.26	keine Vorlesung			Implementierung und Test
KW 27	29.06.26	keine Vorlesung			Implementierung und Test
KW 28	06.07.26	keine Vorlesung			Implementierung und Test
KW 29	13.07.26	keine Vorlesung			Implementierung und Test
KW 30	20.07.26	13	PI/TI/WI/MI	Vorlesungsabschluss/Beratung/Abschlusspräsentation	Vorlesungsabschluss/Beratung/Abschlusspräsentation
KW 31	**27.-31.07.202**	***Mündliche Prüfung (InnoLab) - DUAL am 29.07.2026***			

Fachhochschule Dortmund
University of Applied Sciences and Arts

Praktikum für das SoSe 2026 – Inf Dual

we focus on students

Woche	Datum VL	#	Kohorte	Thema	Praktikum Programmierung (Dual)
KW 17	20.04.26	1	I/TI/WI/MI/Du	Prolog + Einführung	Teambildung
		2	I/TI/WI/MI/Du	Historie und Verteilte Architekturen	
KW 18	27.04.26	3	I/TI/WI/MI/Du	Git und Projektmanagement	Epics und User Stories + Entwicklungsinfrastruktur aufsetzen und testen + Kanban-Board und Burn-down Chart
		4	I/TI/WI/MI/Du	Maven	Epics und User Stories + Entwicklungsinfrastruktur aufsetzen und testen + Kanban-Board und Burn-down Chart
KW 19	04.05.26	5	I/TI/WI/MI/Du	Architekturmodellierung	Epics und User Stories + Entwicklungsinfrastruktur aufsetzen und testen + Kanban-Board und Burn-down Chart
		6	I/TI/WI/MI/Du	Architektur-Konzept Spring-Boot	Architekturkonzept + API-Definition + Beginn Implementierung
KW 20	11.05.26	7	I/TI/WI/MI/Du	UML	Architekturkonzept + API-Definition + Beginn Implementierung
		8	I/TI/WI/MI/Du	EU-Compliance I	Architekturkonzept + API-Definition + Beginn Implementierung
KW 21	18.05.26	Blockwoche			
KW 22	25.05.26	Pfingsten			
KW 23	01.06.26	9	I/TI/WI/MI/Du	Entwurfsmuster	Implementierung und Test
		10	I/TI/WI/MI/Du	EU-Compliance II	Implementierung und Test
KW 24	08.06.26	11	I/TI/WI/MI/Du	Message Broker	Implementierung und Test
		12	I/TI/WI/MI/Du	Docker	Vorlesungsabschluss/Beratung/Abschlusspräsentation
KW 25	15.06.26	keine Vorlesung			
KW 26	22.06.26	keine Vorlesung			
KW 27	29.06.26	keine Vorlesung			
KW 28	06.07.26	keine Vorlesung			
KW 29	13.07.26	keine Vorlesung			
KW 30	20.07.26	13	PI/TI/WI/MI	Vorlesungsabschluss/Beratung/Abschlusspräsentation	
KW 31	27.-31.07.20	***Mündliche Prüfung (innoLab) - DUAL am 29.07.2026***			

Fachhochschule Dortmund
University of Applied Sciences and Arts

Praktikum für das SoSe 2026 – MI, MI Dual und WI

we focus on students

Woche	Datum VL	#	Kohorte	Thema	Praktikum Software Engineering (WI/MI)
KW 17	20.04.26	1	/TI/WI/MI/Du	Prolog + Einführung	Teambildung
		2	/TI/WI/MI/Du	Historie und Verteilte Architekturen	
KW 18	27.04.26	3	/TI/WI/MI/Du	Git und Projektmanagement	Fallbeispiel designen
		4	/TI/WI/MI/Du	Maven	
KW 19	04.05.26	5	/TI/WI/MI/Du	Architekturmodellierung	Gewünschter KI-Einsatz
		6	/TI/WI/MI/Du	Architektur-Konzept Spring-Boot	
KW 20	11.05.26	7	/TI/WI/MI/Du	UML	Regulationslandschaft Überblick erarbeiten
		8	/TI/WI/MI/Du	EU-Compliance I	
KW 21	18.05.26	Blockwoche			
KW 22	25.05.26	Pfingsten			
KW 23	01.06.26	9	/TI/WI/MI/Du	Entwurfsmuster	Techn. KI - Ansätze
		10	/TI/WI/MI/Du	EU-Compliance II	
KW 24	08.06.26	11	/TI/WI/MI/Du	Message Broker	Techn. KI - Ansätze
		12	/TI/WI/MI/Du	Docker	
KW 25	15.06.26	keine Vorlesung			KI in skizziertem Unternehmen rechtskonform einsetzen? Gesetze + Lösungsmuster (Infrastruktur + Software + Modelle + Architekturmuster)
KW 26	22.06.26	keine Vorlesung			KI in skizziertem Unternehmen rechtskonform einsetzen? Gesetze + Lösungsmuster (Infrastruktur + Software + Modelle + Architekturmuster)
KW 27	29.06.26	keine Vorlesung			KI in skizziertem Unternehmen rechtskonform einsetzen? Gesetze + Lösungsmuster (Infrastruktur + Lösungen + Muster)
KW 28	06.07.26	keine Vorlesung			Handlungsempfehlung
KW 29	13.07.26	keine Vorlesung			Handlungsempfehlung
KW 30	20.07.26	13	PI/TI/WI/MI	Vorlesungsabschluss/Beratung/Abschlusspräsentation	Vorlesungsabschluss/Beratung/Abschlusspräsentation
KW 31	**27.-31.07.202**	***Mündliche Prüfung (InnoLab) - DUAL am 29.07.2026***			

Unterlagen

Die Unterlagen zur Vorlesung und Übung werden im Ilias-System in Form von PDF- und ZIP-Dateien zur Verfügung gestellt:

- Vorlesungsfolien,
- Aufgabenblätter das Projekt

Ab Anfang Mai wird der Zugang Passwort geschützt sein.

Fachhochschule Dortmund
University of Applied Sciences and Arts

Teams und Prüfung

we focus on students

Im Projekt arbeiten Sie in **Teams von 6-7 Personen.**

In der Prüfung am Ende des Semesters wird sowohl die **Team- wie auch Einzelleistung geprüft**, so dass die Teammitglieder sehr verschiedene Noten erhalten können.

Das Semester gliedert sich in **drei Phasen**:

- **Architekturmodellierung** (Arbeitsweise + Gruppenergebnis) **20%** der Note
- **Projektmanagement** (Arbeitsweise + Einzelleistung) **30%** der Note
- **Implementierung und Test** (Arbeitsweise + Einzelleistung) **50%** der Note

Fachhochschule Dortmund
University of Applied Sciences and Arts

Digitale Vorbereitung

we focus on students

Digitale **Unterlagen**:

- Vermittlung von Konzepten, Methoden und Werkzeugen der Softwaretechnik
- Im aktuellen Semester primär Softwarearchitektur, Verteilte Systeme und EU-Compliance
- Zweite von vier Softwaretechnik-Veranstaltungen (A, B, C, D)
- Die **Vorlesungen** finden (montags ab 14:15 Uhr) statt.
- Die Vorlesungen finden nur in der ersten Hälfte des Semesters statt, dafür aber doppelt so lange.

Unterlagen im Ilias

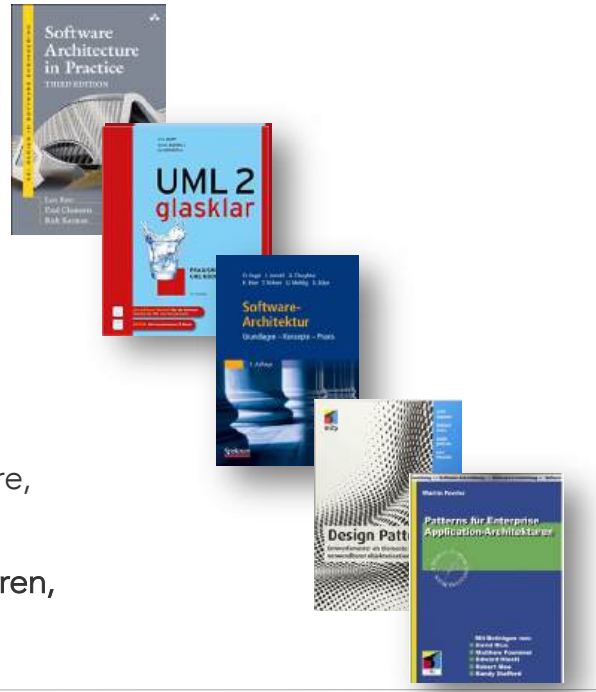
Voraussetzungen/Literatur

Kenntnisse aus den Veranstaltungen

„Einführung in die Informatik 1 und 2“ sowie „Programmierkurs 1“ und „SWT 1“

Literaturempfehlung:

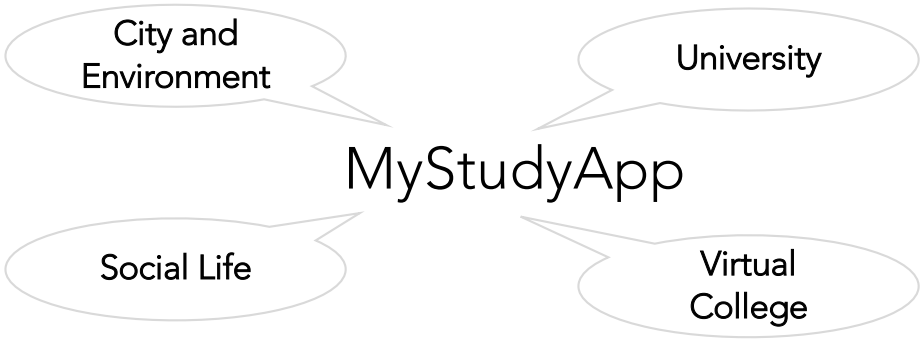
- Bass et al: **Software Architecture in Practice**, 3. Auflage, Addison Wesley, 2012.
- Rupp et al: **UML 2 glasklar**, 4. Auflage, Hanser, 2012.
- Vogel et al: **Software-Architektur:** Grundlagen – Konzepte – Praxis, 2. Auflage, Spektrum, 2009.
- Gamma et al.: **Entwurfsmuster:** Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software, mitp, 2014.
- M. Fowler: **Patterns für Enterprise Application-Architekturen,** 1. Auflage (Taschenbuch), mitp, 2003.



Projekte

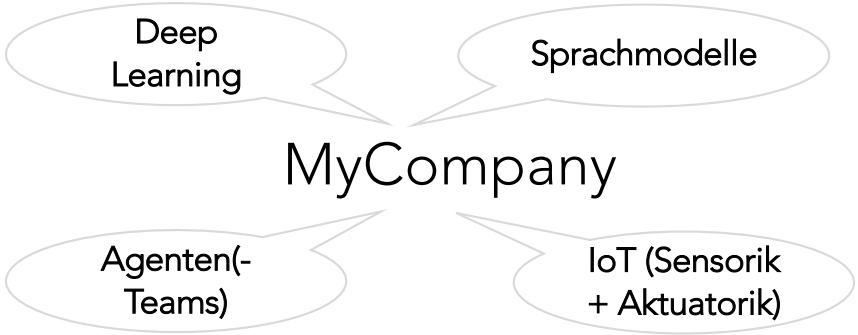
Praktikumsaufgaben

PI/TI und Inf Dual



Fokus: Architektur u. Entwicklungspipeline

WI/MI und MI Dual



Fokus: Neue Technologien und EU-Compliance

Fachhochschule Dortmund
University of Applied Sciences and Arts

Praktikumsaufgaben: MyStudyApp

we focus on students

1. City and Environment

2. University

3. Social Life

4. Virtual College

Praktikum: Anforderungen

- Jedes Projekt hat mindestens **zwei serverseitige Komponenten**, die je nach Usecase auch ein externes System simulieren können.
- Jedes Projekt realisiert **synchrone und asynchrone Kommunikation** zwischen Komponenten, abhängig davon an welcher Stelle im Usecase welche Kommunikation sinnvoll ist.
- Jedes Projekt setzt für die asynchrone Kommunikation über Komponentengrenzen einen **Message-Broker bzw. einen Message-Server** ein.
- Jedes Projekt soll **Daten persistieren. DBMS-Auswahl** begründen.
- Jedes Projekt muss die **Kommunikationsschnittstellen ausführlich dokumentieren**.

Fachhochschule Dortmund
University of Applied Sciences and Arts

Arbeitsergebnisse für die Abschlusspräsentation

we focus on students

motivieren
vorstellen
reflektieren

- Wrapper:
  - 1. Folie: Gruppenthema -> Teaser mit 2-3 Frontend-Auszügen
  - 2. Folie: Team mit (akt. Bildern, Namen u. Matrikelnummern)
- Beschreibung des Technologiestacks
  (Begründung bei individuellen Komponenten wie z.B. Datenbank)
- Verteilungsdiagramm inkl. Zoom-In in die erstellten Artefakte
  (Klassendiagramme + Code dazu)
  - Fokus: Businesslogik mit Entwurfsmustern
  - Fokus: Kommunikation
  - Fokus: Persistenz
- Projektdokumentation
  - Team: Kompetenz- und Rollenverteilung, Aufgabenverteilung
  - Software: Projektumsetzungsplanung über drei Sprints
  - Abweichungen/Verschiebungen von der Planung und warum
- Reflexion

Jedes Teammitglied stellt seinen Verantwortungs-bereich vor.!!!

Fokus: Architektur u. Entwicklungspipeline

Fachhochschule Dortmund
University of Applied Sciences and Arts

Praktikum PI/TI/Inf Dual

we focus on students

Für Dualis Zeitplan auf die Hälfte der Zeit reduziert!!!

KW	Datum	Praktikum	Aufgabe
18	27.04.26	Praktikum 1	Inhaltliche Konzeption: Epics +User Stories Orga: Kanban-Board + Burn-down Chart
19	04.05.26	Praktikum 2	Entwicklungsinfrastruktur aufsetzen und testen
20	11.05.26	Praktikum 3	Architektur modellieren + API-Definition + Muster Arbeitsverteilung für Implementierung planen
ab 24	08.06.26	Praktikum 4	Entwicklung in drei Sprints + Projektdokumentation

15 Wochen im Semester * 4 SWS = 60 SWS = 60*0,75 h = 45 h
5 ECTS entsprechen 150 h => 150 h – 45 h (für V + P) = 105 h
105 h/15 Wochen = **7 h/Woche** selbstständiges Arbeiten

Neben Vorlesung und Praktikum müssen Sie ca. 7 Stunden pro Woche für die Veranstaltung einplanen.

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Praktikumsinhalt: KI und EU-Compliance
Fallbeispiele aus unterschiedl. Branchen

- Krankenhaus (MI)
- Energieunternehmen (WI)
  z.B. Übertragungsnetzbetreiber
- Finanzunternehmen (WI)

Fachhochschule Dortmund
University of Applied Sciences and Arts

Praktikum: Anforderungen

we focus on students

- Jedes Projekt betrachtet **ein Unternehmen**, das **vier „Neue Technologien"** in passenden Anwendungsszenarien einsetzen möchte.
- Folgende Gesetze sollten Sie im Blick haben:
  GDPR (DSGVO), AI-Act, NIS2, Data Act (→ nächste Folie)
- Jedes Projekt überlegt sich ein Beschreibungsformat für die Beschreibung
  - der gesetzlichen Herausforderungen,
  - der technischen Umsetzungsvarianten
  - der Bewertung der einzelnen Varianten bezogen auf die gesetzlichen Anforderungen.
- Jedes Projekt soll eine begründete Handlungsempfehlung für das betrachtete Unternehmen aussprechen.
- Jedes Projekt muss jeden Schritt ordentlich dokumentieren und Quellen angeben.

Fachhochschule Dortmund
University of Applied Sciences and Arts

Europäische Gesetze

we focus on students

Gesetz	Beschreibung
GDPR	Regelt die Verarbeitung personenbezogener Daten. Primär adressiert sind damit faktisch alle Branchen, die personenbezogene Daten verarbeiten, besonders SaaS/Cloud, E-Commerce, HR, Gesundheitswesen, Finanzsektor, AdTech/MarTech und öffentliche Verwaltung.
AI-Act	Regelt das Inverkehrbringen, den Einsatz und die Überwachung von KI-Systemen risikobasiert. Primär adressiert sind Anbieter und Betreiber von KI, besonders in Biometrie, kritischer Infrastruktur, Bildung, Beschäftigung/Recruiting, Zugang zu wesentlichen Diensten, Strafverfolgung, Migration und Justiz; zusätzlich auch Anbieter von General-Purpose-AI-Modellen.
CRA	Stellt horizontale Cybersicherheitsanforderungen für Produkte mit digitalen Elementen auf. Primär adressiert sind Hersteller, Importeure und Händler von Hard- und Software, besonders IoT, Consumer Electronics, Embedded Systems, industrielle Steuerungstechnik und Enterprise-Software.
NIS2	Regelt Cybersicherheitspflichten für wesentliche und wichtige Einrichtungen. Primär adressiert sind Energie, Verkehr, Bankwesen, Finanzmarktinfrastrukturen, Gesundheit, Trinkwasser, Abwasser, digitale Infrastruktur, IKT-Management-/Managed-Services, öffentliche Verwaltung und Weltraum; außerdem u. a. Post/Kurier, Abfall, Chemie, Lebensmittel, Produktion und Forschung.
DORA	Regelt die digitale operationelle Resilienz im Finanzsektor. Primär adressiert sind Banken, Versicherungen, Investmentfirmen und andere regulierte Finanzunternehmen sowie deren ICT-Drittdienstleister.
Data Act	Regelt fairen Zugang zu und Nutzung von Daten, insbesondere aus vernetzten Produkten und verbundenen Diensten, sowie Themen wie Cloud-Switching. Primär adressiert sind IoT-/Gerätehersteller, Industrie 4.0, Automotive/Connected Vehicles, Smart Home, Maschinenbau und Cloud-/Data-Service-Provider.
Data Governance Act	Schafft Regeln für die sichere Weiterverwendung bestimmter öffentlicher Daten, für Datenvermittlungsdienste und Data Altruism. Primär adressiert sind öffentliche Stellen, Datenintermediäre und datenintensive Bereiche wie Gesundheit, Verkehr, Energie, Umwelt und Finanzwesen.
Digital Services Act	Regelt Verantwortlichkeiten für digitale Vermittlungsdienste. Primär adressiert sind Intermediäre, Hosting-Anbieter, Online-Plattformen, Marktplätze und Suchmaschinen, also vor allem Plattformwirtschaft, Social/Community-Plattformen, Content-Hosting und E-Commerce-Marktplätze.
European Accessibility Act	Regelt Barrierefreiheitsanforderungen für bestimmte Produkte und Dienstleistungen. Primär adressiert sind Hersteller und Anbieter in Consumer Electronics, Telekommunikation, Bankdienstleistungen, Personenverkehr und E-Books.

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Deutsche Gesetze

Gesetz	Beschreibung
BSIG	Zentrales deutsches IT-Sicherheitsgesetz mit dem neuen NIS-2-Regime. Primär adressiert sind besonders wichtige und wichtige Einrichtungen, also vor allem KRITIS-nahe und größere regulierte Unternehmen in Sektoren wie Energie, Verkehr, Gesundheit, Wasser, digitale Infrastruktur, öffentliche Verwaltung, Produktion und Forschung.
BSI-Kritisverordnung	Definiert, welche Anlagen als kritische Infrastruktur gelten. Primär adressiert sind KRITIS-Betreiber, insbesondere in klassischen KRITIS-Sektoren wie Energie, Wasser, Gesundheit, IT/TK, Transport/Verkehr und weiteren kritischen Versorgungsbereichen.
KRITIS Dachgesetz	Regelt die physische Resilienz kritischer Anlagen. Primär adressiert sind Betreiber kritischer Anlagen, also vor allem KRITIS-Sektoren wie Energie, Gesundheit, IT/TK, Transport/Verkehr, Wasser, Abfall sowie staatlich besonders relevante Versorgungsbereiche.
Digitale Dienste Gesetz	Deutsches Begleitgesetz für digitale Dienste, u. a. mit nationalen Regelungen und Zuständigkeiten rund um den DSA. Primär adressiert sind Diensteanbieter digitaler Dienste, besonders Plattformen, Hosting-Anbieter, Online-Marktplätze und sonstige Online-Dienste.
TDDDG	Regelt Datenschutz und Privatsphäre in Telekommunikation und digitalen Diensten, besonders den Zugriff auf Endgeräte und damit praktisch das Cookie-/Tracking-Thema. Primär adressiert sind Telekommunikationsanbieter, aber auch Webseiten-, App- und Digital-Service-Anbieter, die Tracking, Cookies oder ähnliche Technologien einsetzen.
BFSG	Deutsches Gesetz und Verordnung zur Barrierefreiheit bestimmter Produkte und Dienstleistungen. Primär adressiert sind Hersteller, Händler, Importeure und Dienstleister in Computer/Smartphones/Selbstbedienungsterminals, Telekommunikation, Bankdienstleistungen, Personenverkehr und E-Commerce.

Fachhochschule Dortmund
University of Applied Sciences and Arts

Arbeitsergebnisse für die Abschlusspräsentation

we focus on students

motivieren
vorstellen
reflektieren

- Wrapper:
  - 1. Folie: Gruppenthema -> Einordnung des Unternehmens
  - 2. Folie: Team mit (akt. Bildern, Namen u. Matrikelnummern)
- Beschreibung der Anwendungsszenarien für die „Neuen Technologien“
- Überblick über relevante Gesetze und gesetzliche Anforderungen (Datenschutz, Sicherheit, Souveränität, KI, …)
- Herausforderungen bei Umsetzung der Szenarien
- Mögliche Realisierungsalternativen für jede Technologie
- Bewertung der Alternativen bzgl. der gesetzlichen Anforderungen
- Handlungsempfehlung
- Reflexion

Jedes Teammitglied stellt seinen Verantwortungs-bereich vor.!!!

Fokus: Neue Technologien und EU-Compliance

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Praktikum WI/MI/MI Dual

KW	Datum	Praktikum	Aufgabe
18	27.04.26	Praktikum 1	Inhaltliche Konzeption: Unternehmen designen Orga: Kanban-Board + Burn-down Chart
19	04.05.26	Praktikum 2	Anwendungsszenarien für „Neue Technologien“ def.
20	11.05.26	Praktikum 3	Überblick über relevante Gesetze und Inhalte
23	01.06.26	Praktikum 4	Umsetzungsalternativen der Alternativen → Architekturdiagramm-Level + ggf. Schnittstellen
25	26.06.26	Praktikum 5	Bewertung der Alternativen bzgl. der gesetzlichen Anforderungen
28	06.07.26	Praktikum 6	Handlungsempfehlung

15 Wochen im Semester * 4 SWS = 60 SWS = 60*0,75 h = 45 h
5 ECTS entsprechen 150 h => 150 h – 45 h (für V + P) = 105 h
105 h/15 Wochen = **7 h/Woche** selbstständiges Arbeiten

Neben Vorlesung und Praktikum müssen Sie ca. 7 Stunden pro Woche für die Veranstaltung einplanen.

www.fh-dortmund.de

facebook.com/fhdortmund instagram.com/fhdortmund twitter.com/fh_dortmund youtube.com/FachhochschuleDO

University of Applied Sciences and Arts Dortmund

Softwaretechnik 2/B

Prolog

Fachhochschule Dortmund
University of Applied Sciences and Arts

© SoSe 2026 Diepenbrock/Hirsch/Sachweh/Zeppenfeld

Struktur

1. Einführung

1.1 Einführung und Begriffe

1.2 Verteilungsmodelle und Middleware-Technologien (im Überblick)

~~1.3 Kriteriengeleitete Evaluation~~

Wo stehen wir jetzt?

OOA-Modell beschreibt ***WAS*** erstellt werden soll, aber nicht ***WIE***.

Was muss geklärt werden?

OOA-Modell

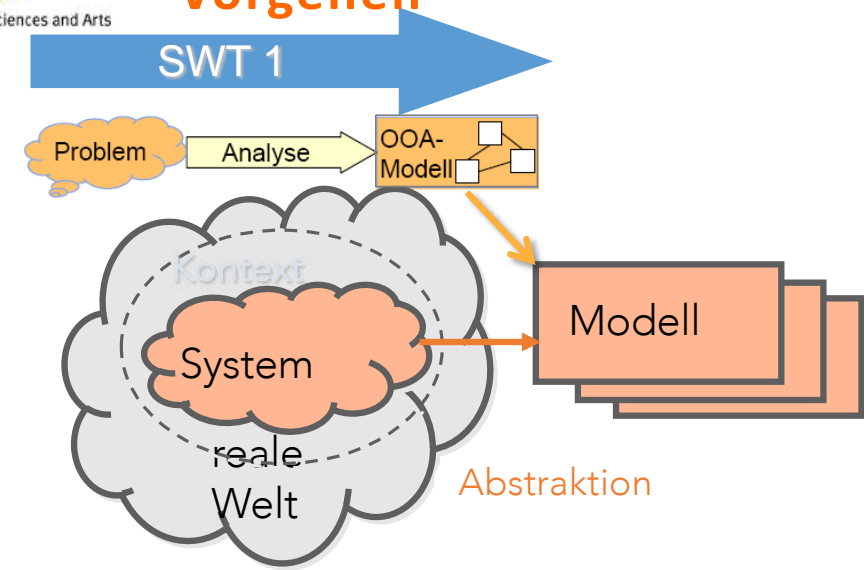
Implementierung/
Realisierung

Vorgehen

Beispiel: Hausbau

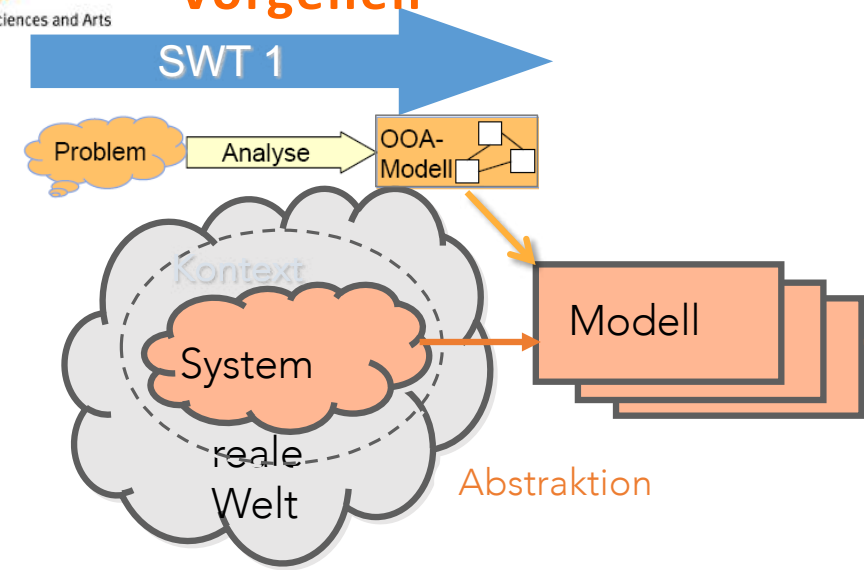
we
focus
on
students

Vorgehen



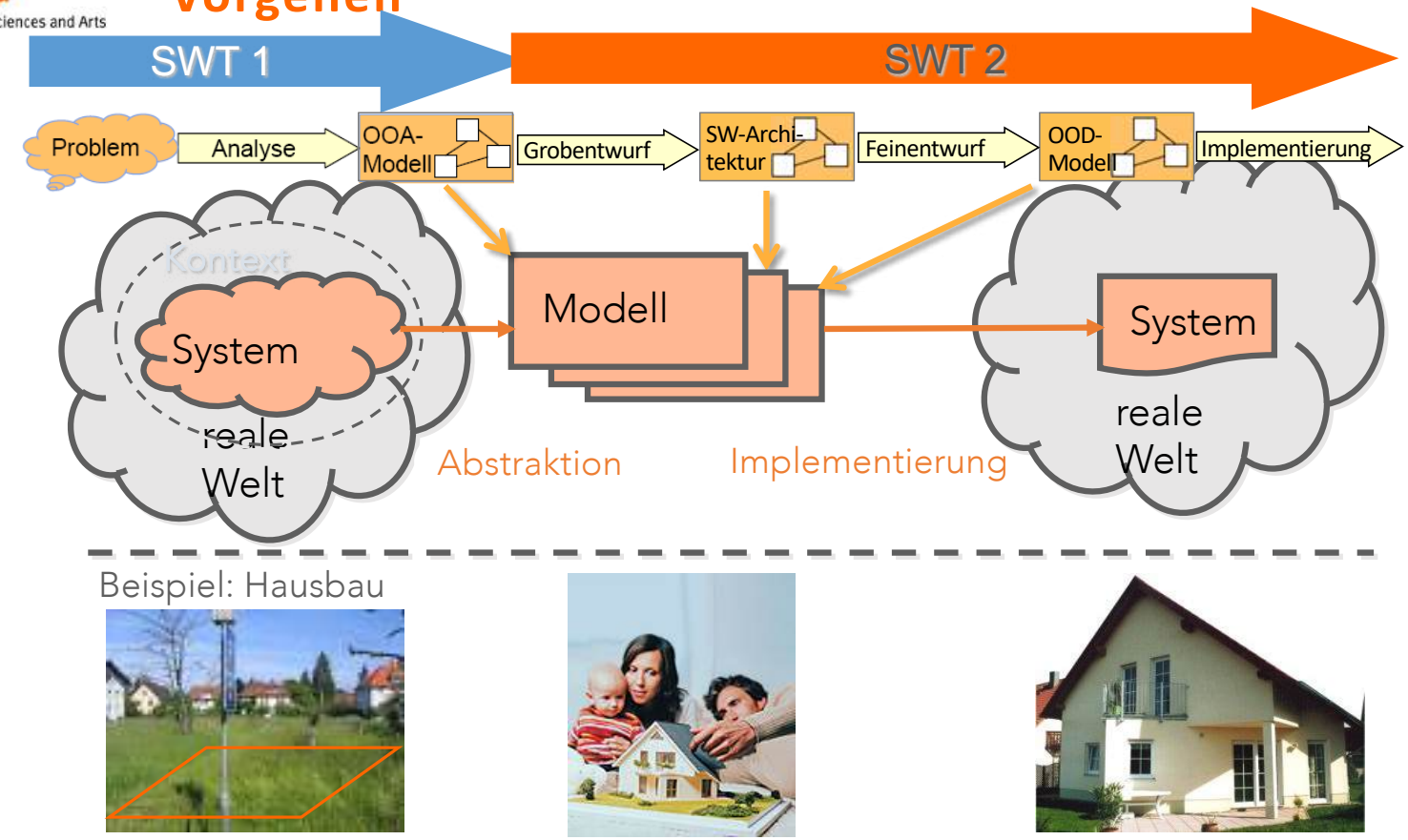
Beispiel: Hausbau

Vorgehen



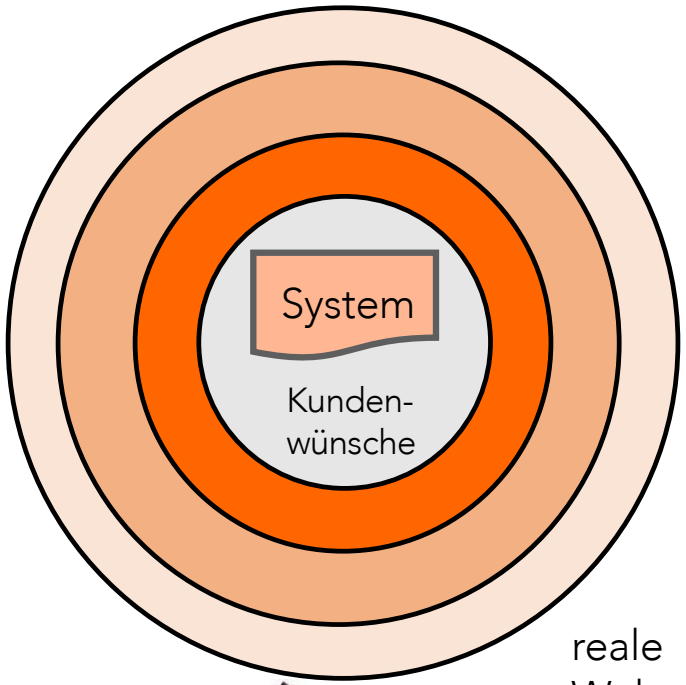
Beispiel: Hausbau

Vorgehen



Beispiel: Hausbau

Vorgehen



reale Welt

Technologie
Deutsches Recht
EU Recht

Implizite Anforderungen

Begriffe

Architektur

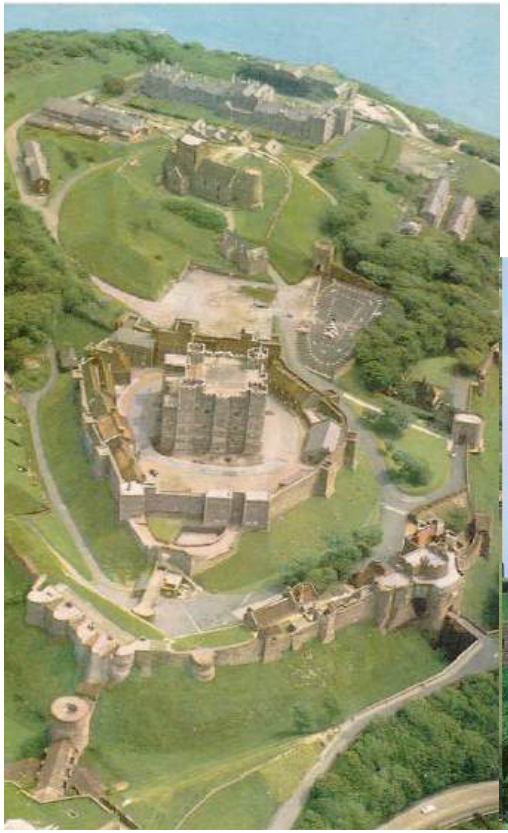
- Architektur ist die Kombination von utilitas, firmitas und venustas.
  frei nach Vitruvius (Römischer Architekt 90-20 v. Chr.)
  - Funktionalität (Zweckmäßigkeit)
  - Stabilität (Beständigkeit)
  - Ästhetische Gestaltung (Eleganz)

Titusbogen
(am Eingang zum Forum Romanum)

Pont du Gard
(Aquädukt)

Amphitheatrum Flavium
(heute bekannt als Kolosseum)

Zielgerichtete Architekturen



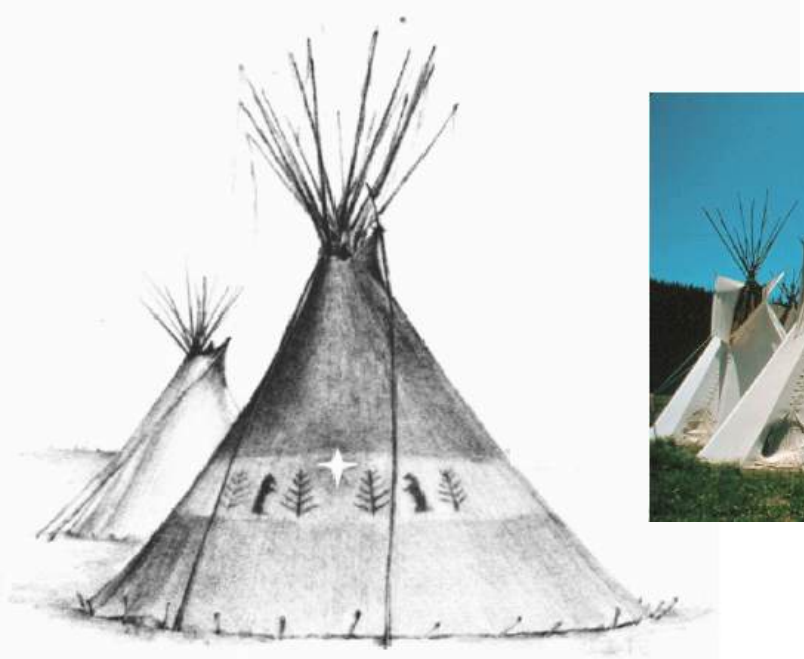
Zielgerichtete Architekturen

Burj Khalifa, 560 Meter

Burj al Arab, 7 Sterne Hotel

Zielgerichtete Architekturen

Zielgerichtete Architekturen



Zielgerichtete Architekturen

Und hier ???

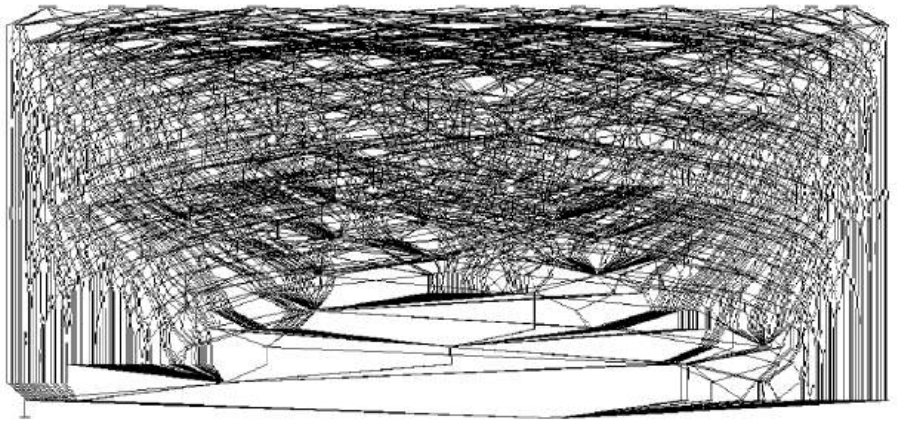
Begriffe

Softwarearchitektur (1)

- Softwarearchitektur ist die Kombination von utilitas, firmitas und venustas.
  frei nach Vitruvius (Römischer Architekt 90-20 v. Chr.)
  - **Funktionalität**: das Softwaresystem erfüllt seine Anforderungen
  - **Stabilität**: das Softwaresystem ist langlebig und stabil gegenüber Veränderungen
  - **Ästhetische Gestaltung**:
    die Strukturen des Softwaresystems weisen eine Klarheit auf, die das Verständnis und die Kommunikation im Team erleichtert

[vgl. Rausch 2006]

Softwarearchitektur
oder
Chaos ?



[Quelle: Erich Gamma, "100 OO Frameworks, Pitfalls and Lessons Learned", 1997]

Softwarearchitektur (2)

- Der Name „Softwarearchitektur" ist als Analogie zu sehen, als eine programmatische Aussage, eine Absichtserklärung:

  „da, wo die Architektur heute steht, da wollen wir auch hin“ [Vgl. Rausch 2006]

Softwarearchitektur (3)

- „Softwarearchitektur:
  Das ist die Königsdisziplin des Software Engineering."

  [Ernst Denert, 2004]

Begriffe

Softwarearchitektur (4)

- „Die Softwarearchitektur eines Systems ist die Menge an Strukturen, die erforderlich sind, um über das System nachzudenken. [Konkret die Strukturen], die **Softwareelemente**, **Beziehungen** zwischen ihnen und **Eigenschaften** von beiden umfassen.“ [Vgl. Bass et al. 20012]

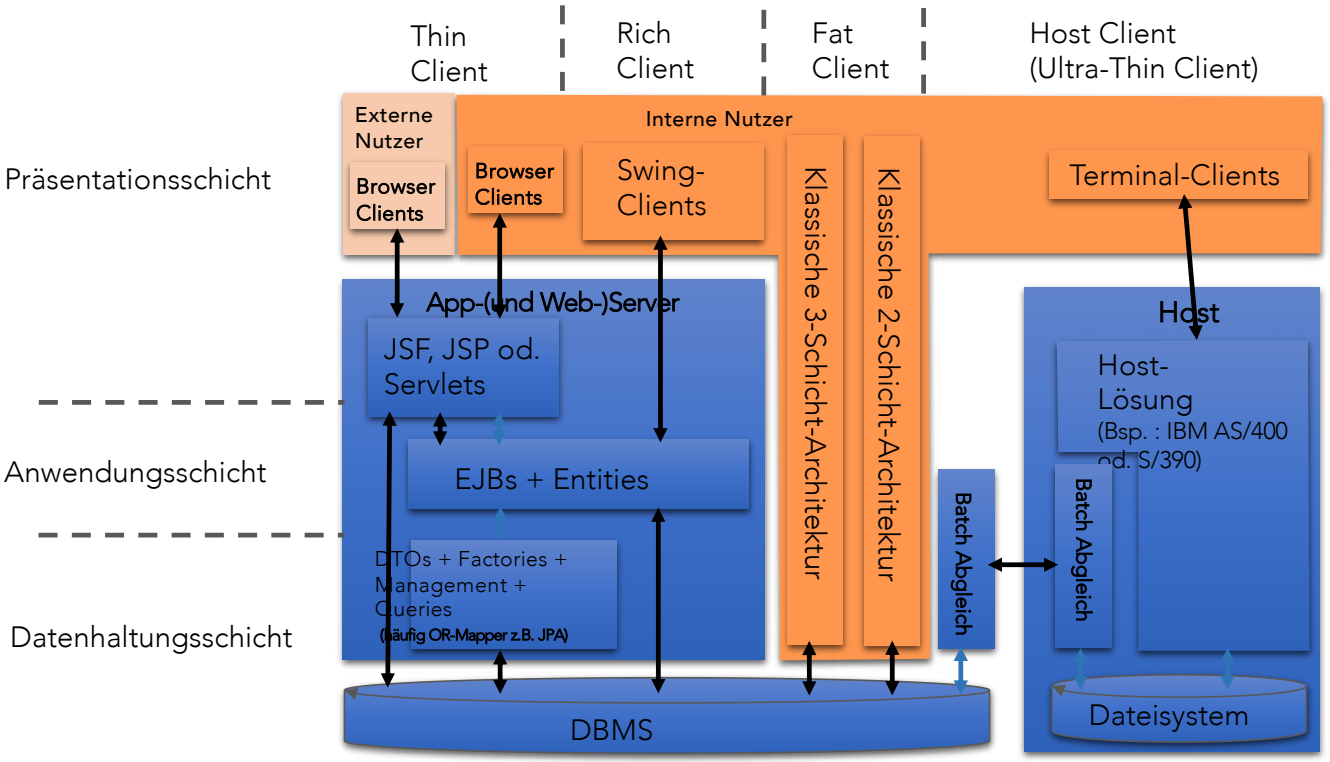
Weitere Definitionen

- Es existieren über 90 Definitionen des Begriffs „Softwarearchitektur"

[vgl. http://www.sei.cmu.edu]

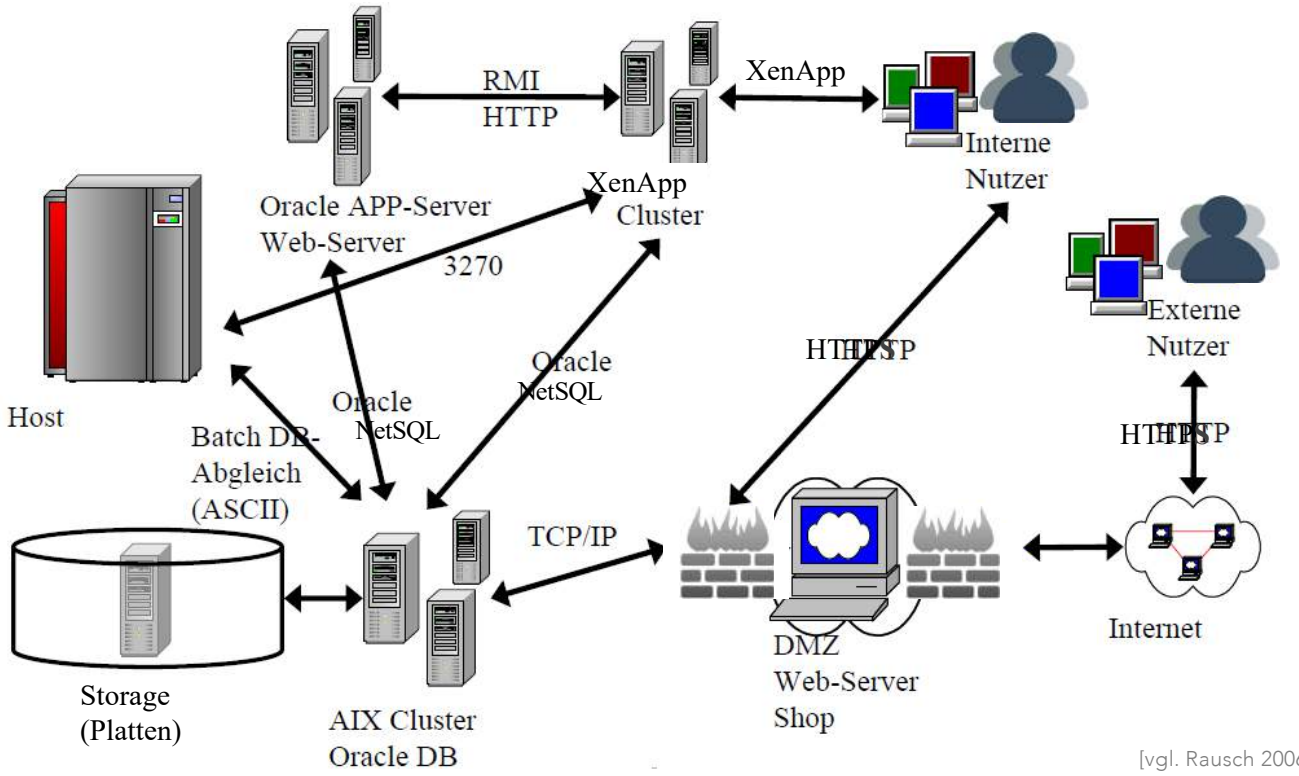
Beispiel: Softwarearchitekturen

we focus on students



[angelehnt an Rausch 2006]

Beispiel: Systemarchitektur



[vgl. Rausch 2006]

Struktur

1. Einführung

1.1 Einführung und Begriffe

1.2 Verteilungsmodelle und Middleware-Technologien (im Überblick)

~~1.3 Kriteriengeleitete Evaluation~~

Technologien

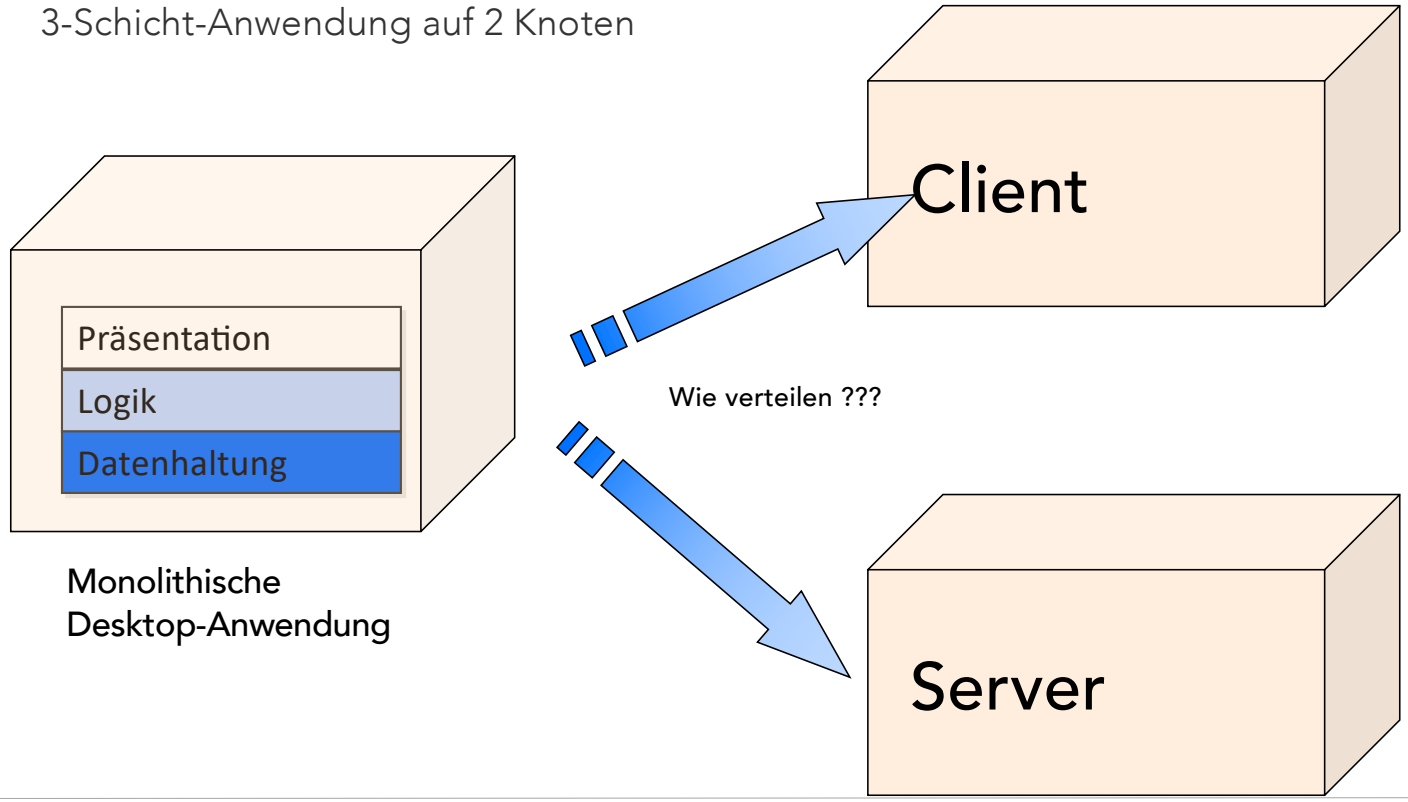
Entscheidung: Monolithisches oder verteiltes System

- **monolithische Desktop-Anwendung** = Anwendung, die
  - lokal auf dem Arbeitsplatzrechner (engl.: Desktop) installiert ist.
  - Beispiele:
    - die meisten Office-Programme,
    - e-Mail-Programme,
    - CAD-Systeme,
    - Computerspiele,
    - Integrierte Entwicklungsumgebungen
    - Web-Browser …
- **Verteilte-Anwendungen** = komplexe Anwendung, die
  - in einem verteilten System abläuft (→ nächste Folie).
  - Beispiele:
    - Reisebuchungssyteme (z.B. ab-in-den-urlaub),
    - SAP,
    - ebay, amazon, facebook, …

Technologie-Entscheidung: Verteilungsmodell

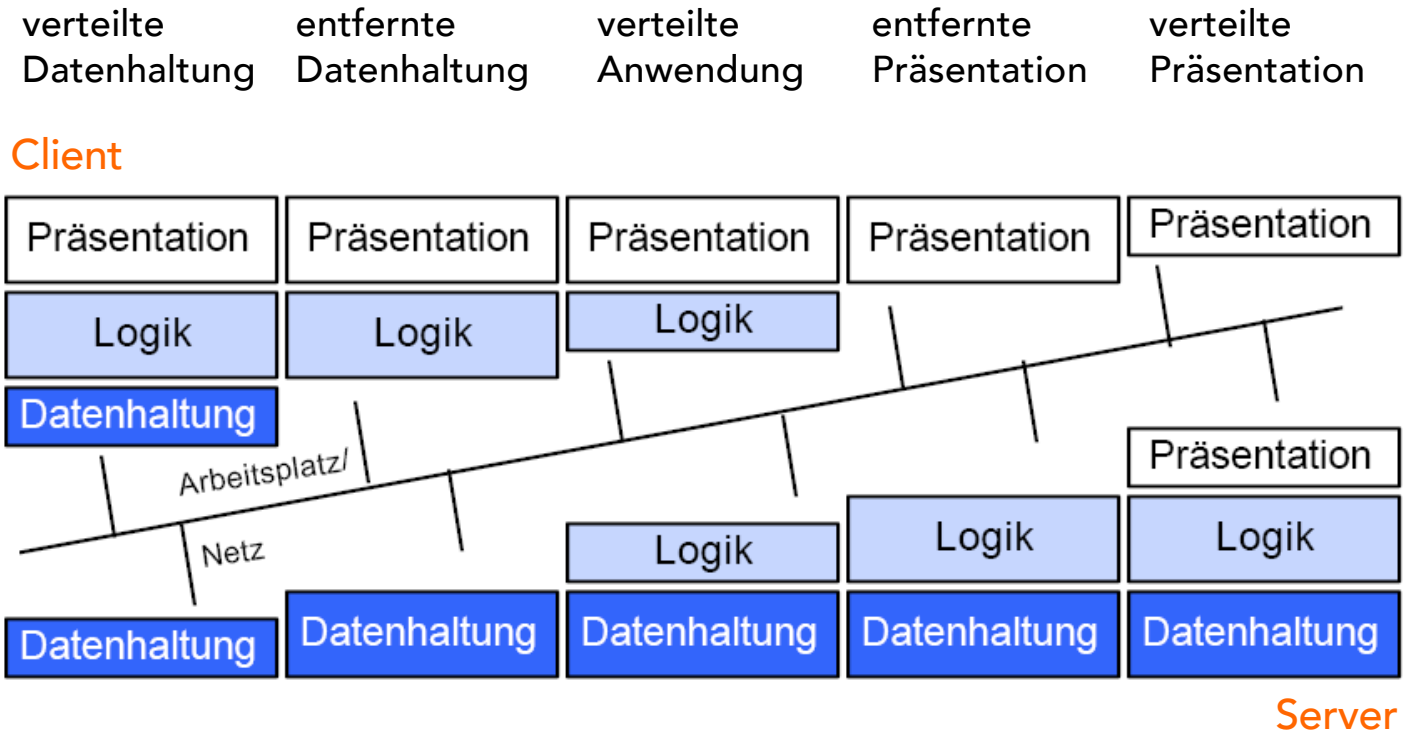
we focus on students

3-Schicht-Anwendung auf 2 Knoten



Monolithische Desktop-Anwendung

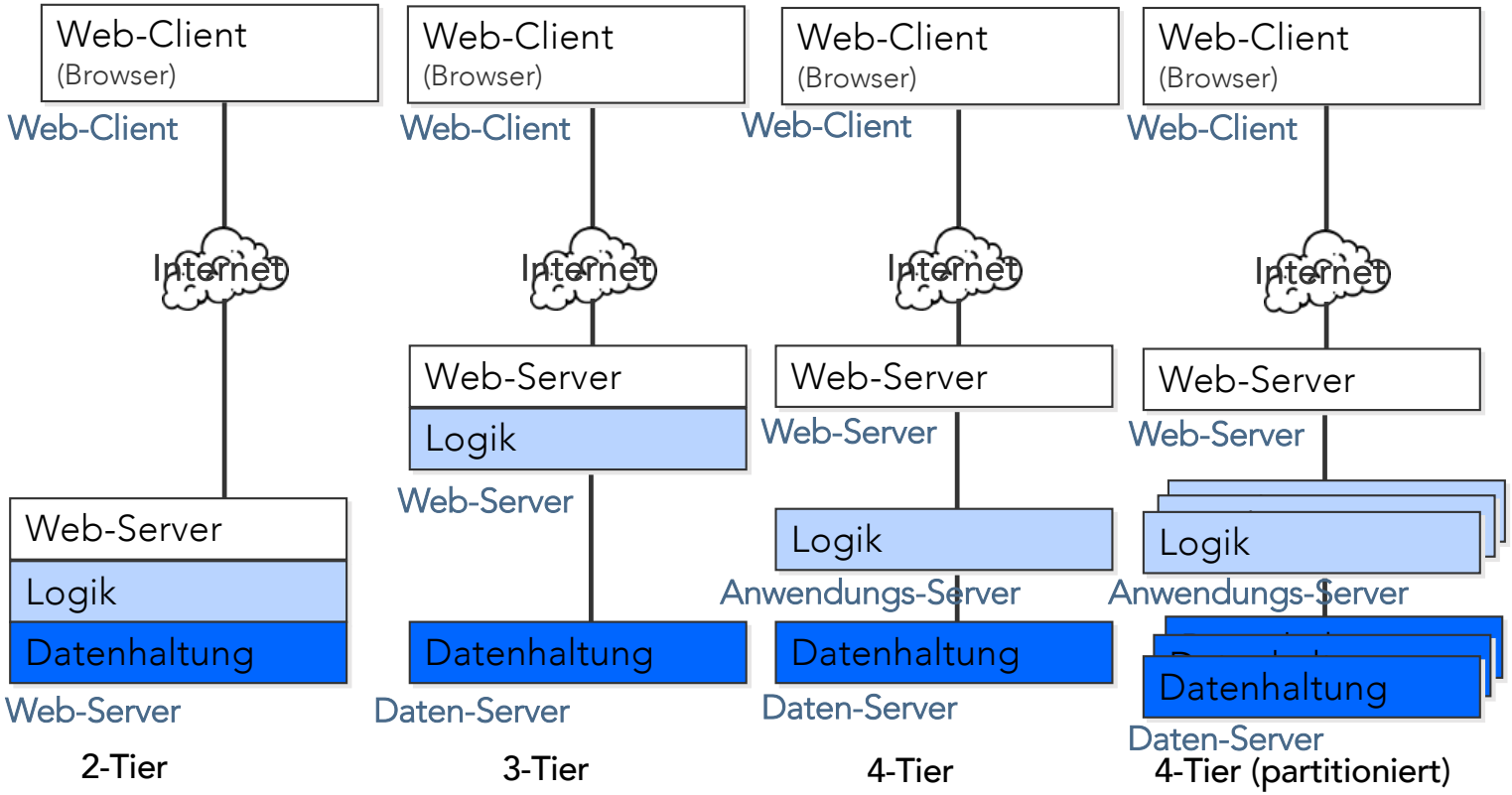
Verteilungsmodelle in einer 2-Tier-Architektur



Verteilungsmodelle in einer n-Tier-Architektur

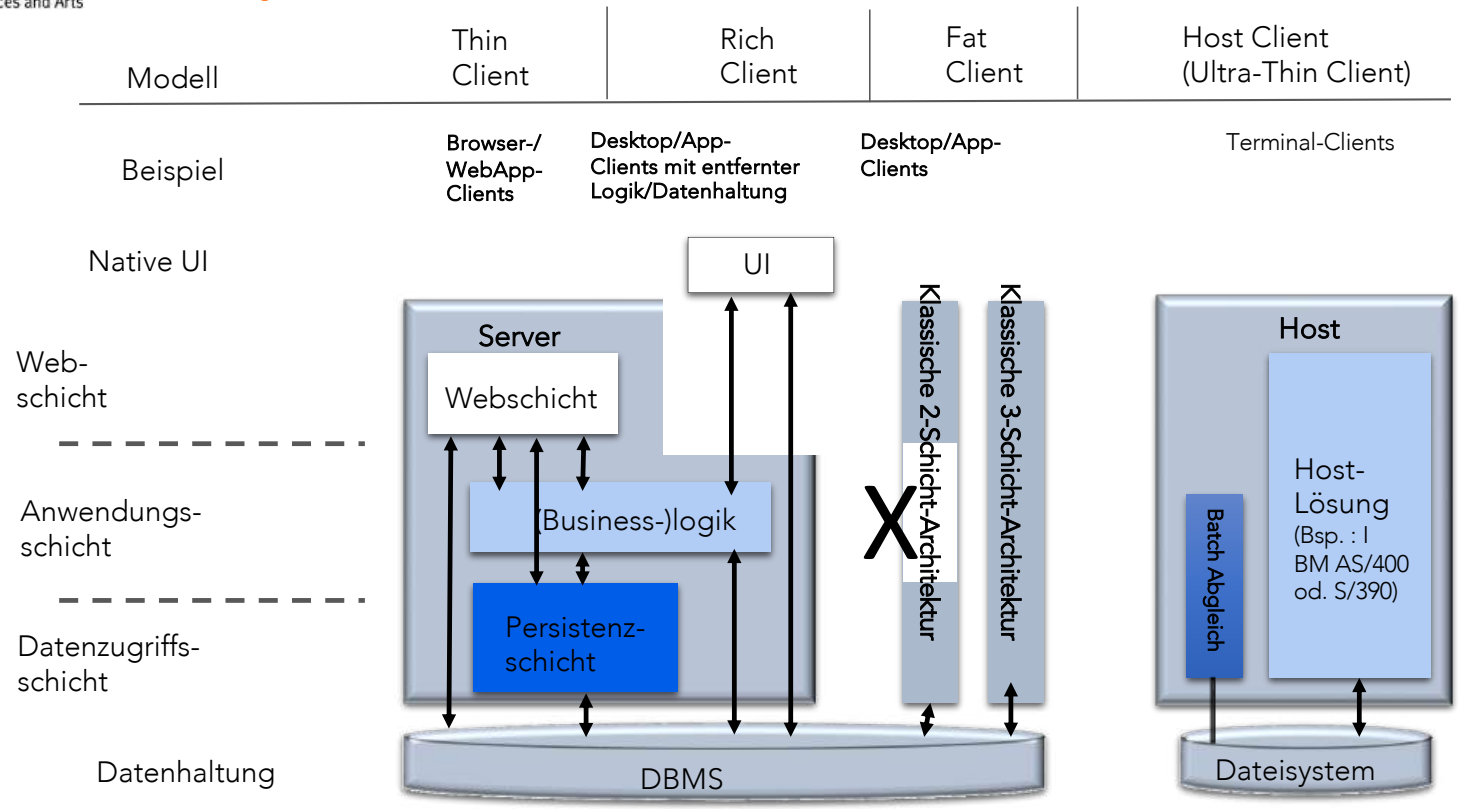
(einige Beispiele)

we focus on students



Beispiele für Softwarearchitekturen

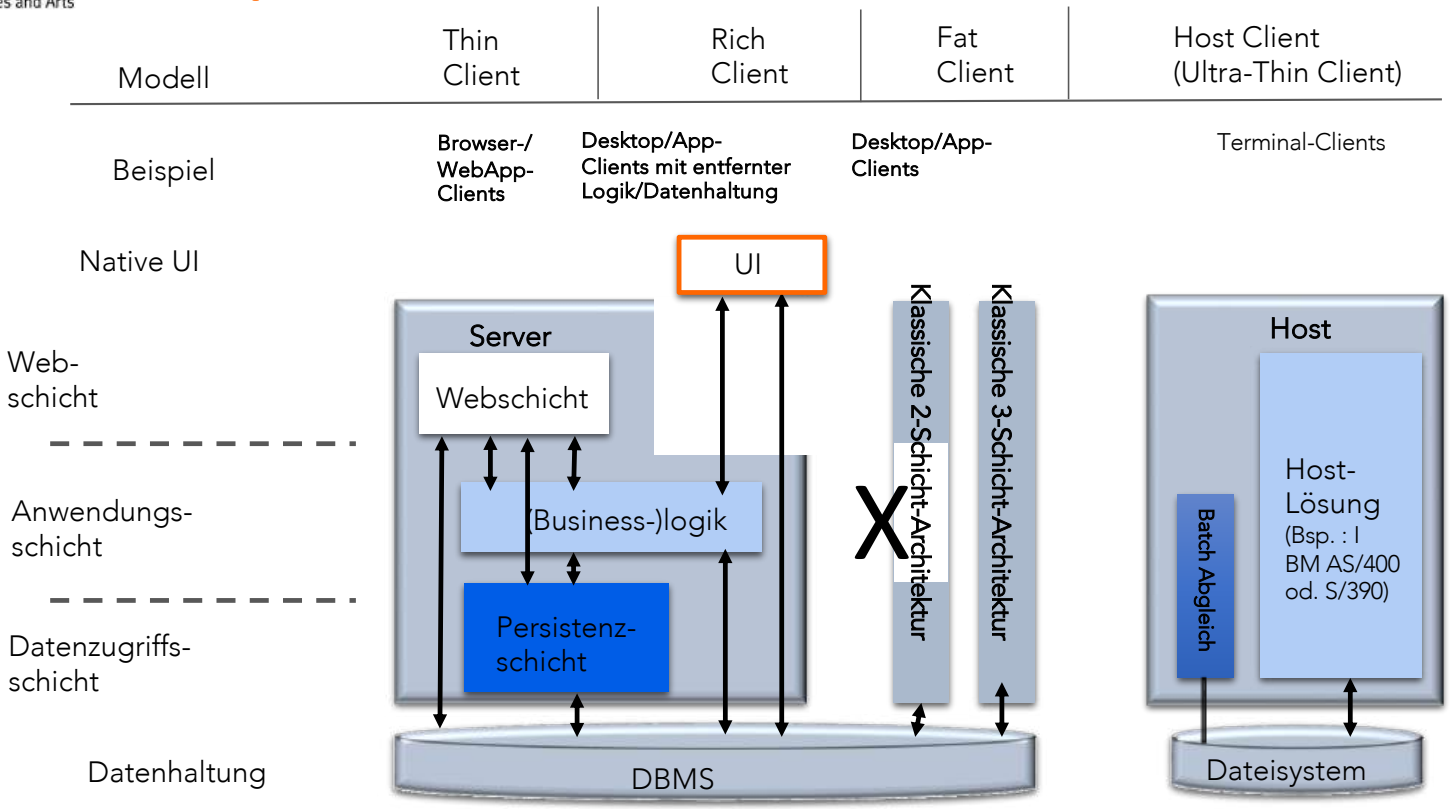
we focus on students



[angelehnt an Rausch 2006]

Beispiele für Softwarearchitekturen

we focus on students



[angelehnt an Rausch 2006]

Beliebte Desktop-Frameworks

Desktop-Anwendungs-Frameworks:
Diese Frameworks werden verwendet, um Desktop-Anwendungen mit grafischen Benutzeroberflächen (GUIs) zu erstellen.

Beispiele (Java):

- JavaFX
- Swing
- SWT (Standard Widget Toolkit)
- Apache Pivot

Beispiele (.NET):

- Windows Forms (WInForms)
- Windows Presentation Foundation (WPF)
- Universal Windows Platform (UWP)
- Xamarin
- WinUI
- MAUI

Beliebte App-Frameworks

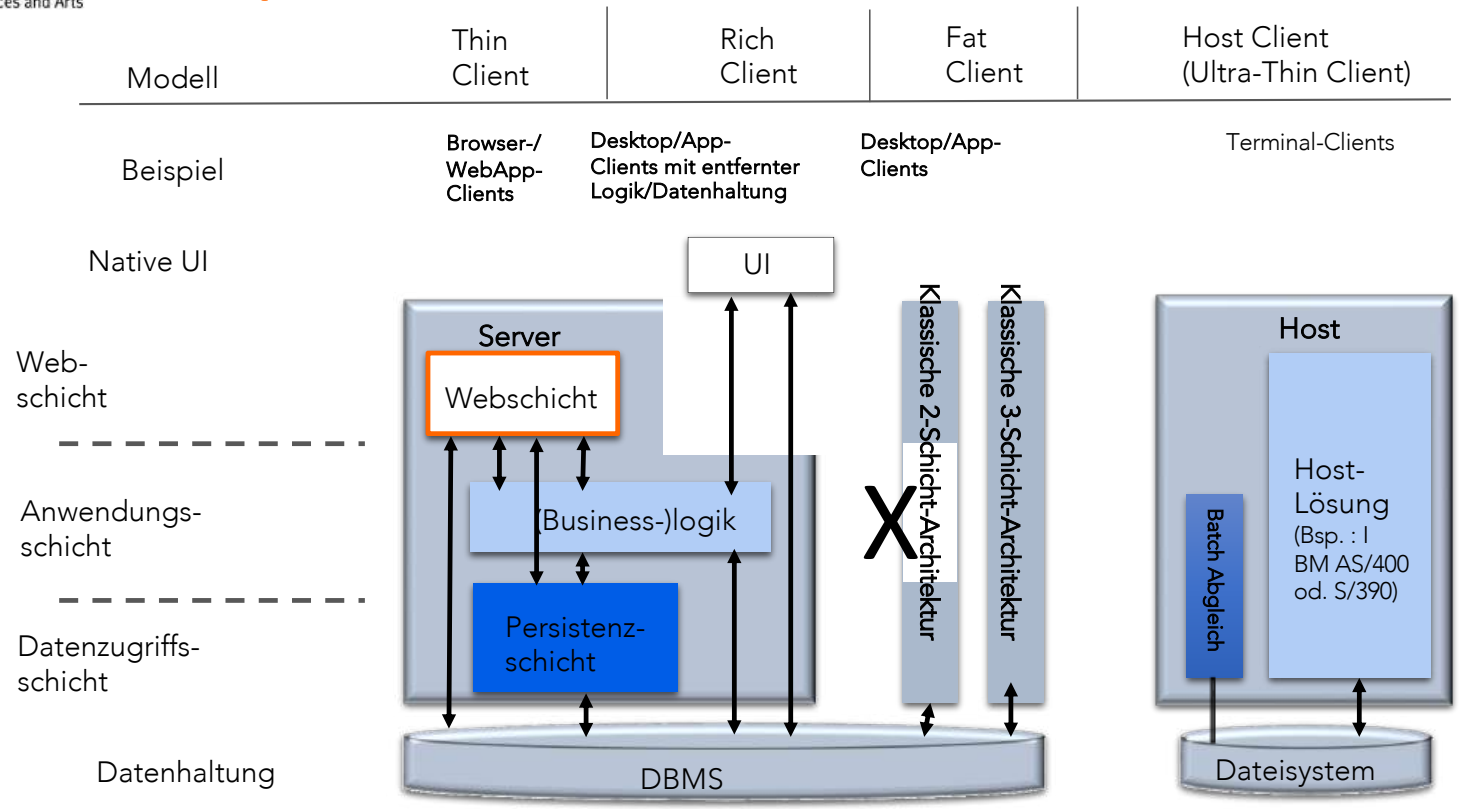
App-Frameworks (Crossplattform):
Diese Frameworks werden verwendet, um die Entwicklung mobiler Apps zu unterstützen.

Beispiele:

- **ReactNative** (2013, Facebook, JS: Microsoft Office, Skype, and Xbox Game Pass, Facebook, Desktop Messenger)
- **Flutter** (2017 von Google, Dart: Google Ads/Earth, Alibaba, Bosch)
- **Kotlin Multiplatform** (KMP) (Kotlin: McDonald's, Netflix, Forbes, 9GAG, Cash App, Philips)
- **Ionic** (2013, kann Elemente von Angular, React, and Vue integrieren, JS: T-Mobile, BBC Children's & Education)
- **Cordova/PhoneGap** (kann auf Kameras u. Sensorik zugreifen, nicht so performant)
- **.NET MAUI** (2022, Microsoft, Desktop + App (Crossplattform), C#+XAML: NBC Sports Next, Escola Agil, Irth Solution) für: Windows, macOS + iOS u. Android + Tizen (Linux)

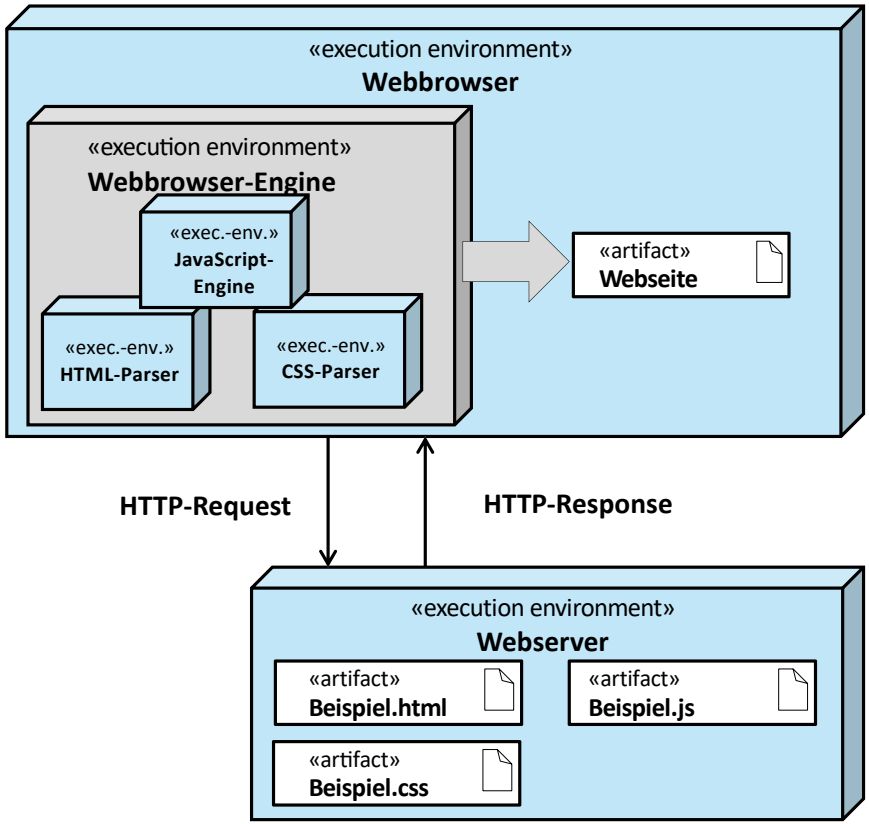
Beispiele für Softwarearchitekturen

we focus on students



[angelehnt an Rausch 2006]

Anzeige einer Webseite im Browser



Progressive Web App (PWA)

Eine PWA ist eine App, die auf Webtechnologien (JavaScript, Css, HTML) basiert. Sie ist im Web auffind- und verlinkbar, simulieren jedoch ähnliche Funktionen wie „native Apps“:

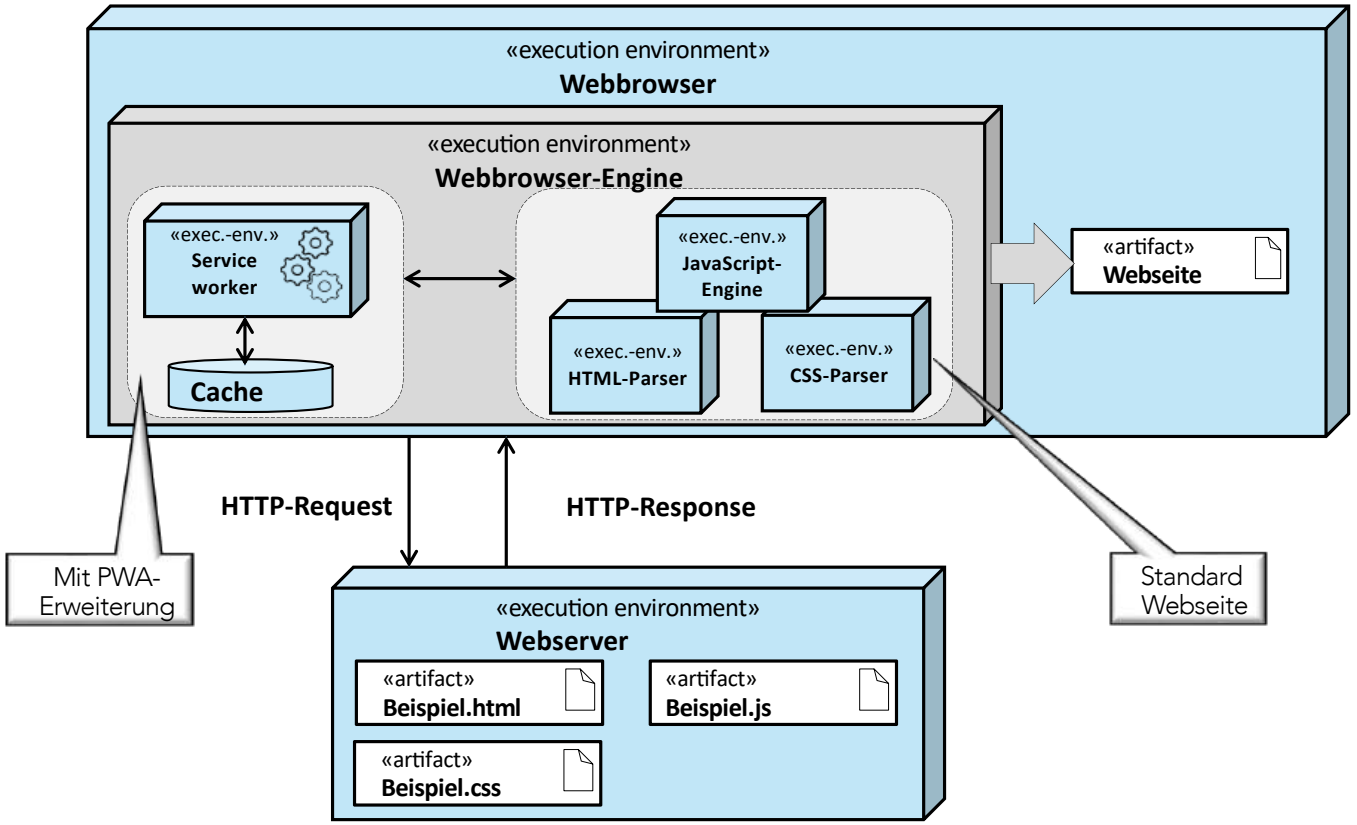
- sie kann **über Browser installiert** werden und funktioniert auch offline,
- sie **erzeugt eine Instanz des Webbrowsers**, um als eigenständige Anwendung zu laufen,
- **bei jedem Aufruf** wird die App **automatisch aktualisiert**,
- sie kann **Push-Benachrichtigungen** senden,
- sie **kann auf die Hardware der Endgeräte zugreifen**, wie Kamera, Mikrofon, GPS oder das Dateisystem,
- sie **arbeitet mittels Service Worker** im Hintergrund.

⇒ Eine PWA kann **über eine Suchanfrage gefunden und direkt über den Browser aufgerufen werden** oder mit einem Klick-Installation direkt über den Startbildschirm, wenn sie mit einem Icon dort abgelegt sind.

[angelehnt an Rausch 2006]

Funktionsweise einer PWA

we focus on students

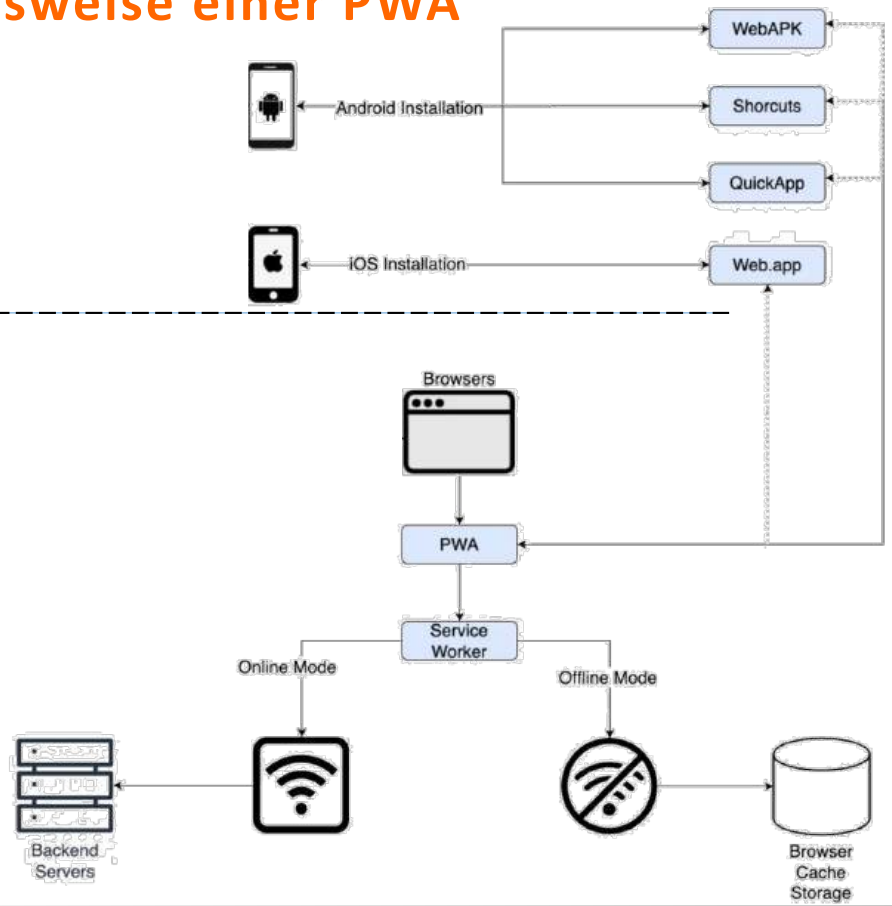


[angelehnt an Rausch 2006]

Funktionsweise einer PWA

Installation

Arbeitsmodi



Vergleich klassische Webseite und Progressive Web App (PWA)

Klassische (responsive) Webseite

- Verfügbarkeit: Web
- geringe Entwicklungskosten

- responsive
- nicht-installierbar
- verlinkbar
- nicht-offline nutzbar

PWA

- Verfügbarkeit: Multi-Plattform
- geringe Entwicklungskosten (one for all)
- responsive
- ohne App-Store
- installierbar
- verlinkbar
- offline nutzbar
- Service Worker arbeitet im Hintergrund

Beliebte Web-Frameworks

Web-Frontend-Frameworks:
Zur Vereinfachung der Entwicklung von Webanwendungen zu vereinfachen durch Unterstützung der Verarbeitung von HTTP-Anfragen, Sitzungsverwaltung, Formularvalidierung, Datenbindung und anderen Funktionen.

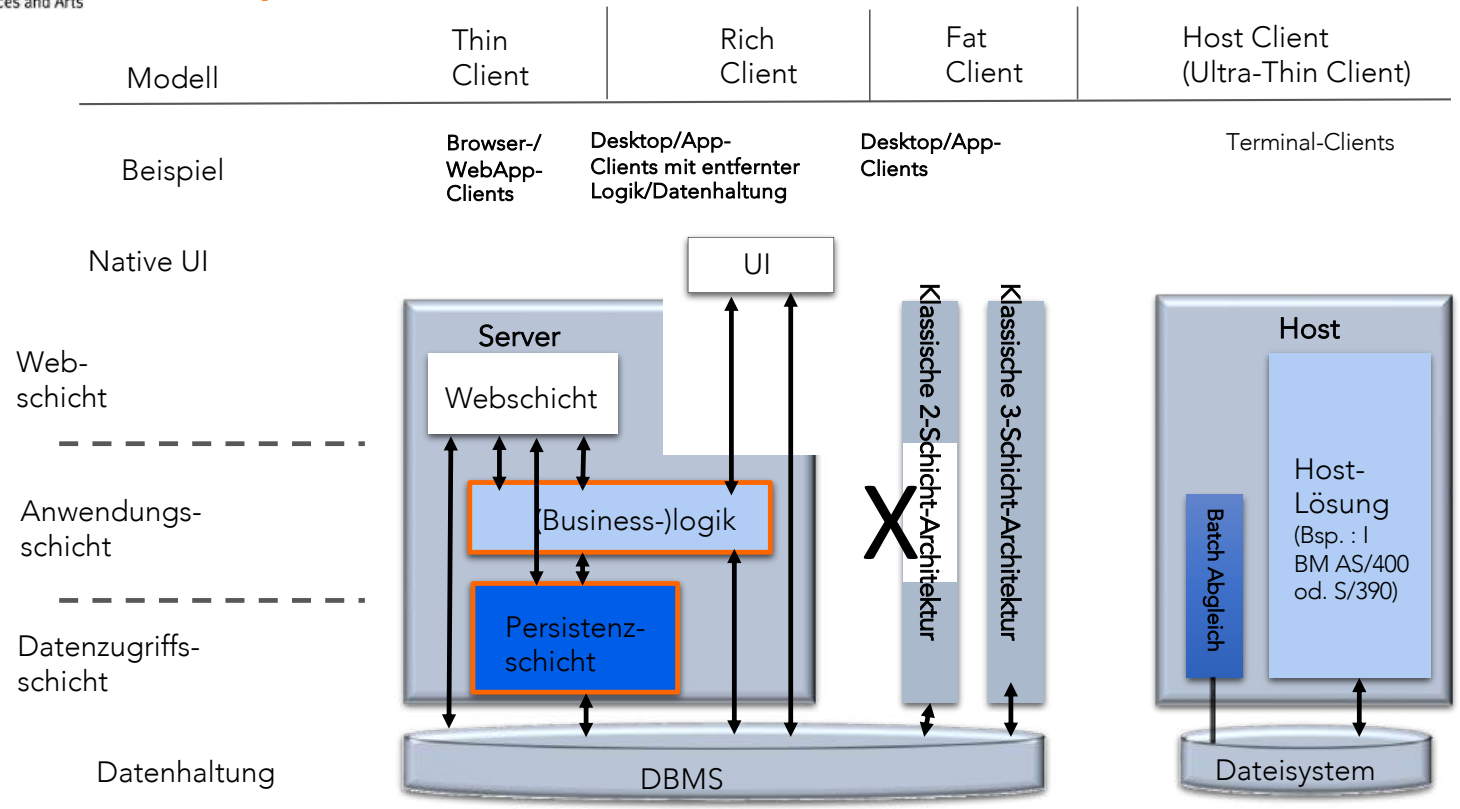
Beispiele früher:
Spring MVC, JavaServer Faces (JSF), Struts, Play Framework

Beispiele heute:

- ReactDOM (2013 von Facebook, JS: Facebook, Netflix, Airbnb)
- Angular (2016 von Google, JS→ TypeScript: Google, PayPal, IBM)
- Vue.js (2014 von Angular-Entwickler, JS: Xiaomi, GitLab, Nintendo)
- Vaadin (2002) hybrid: serverseitiges Java + WebComponents)
- Apache Pivot (sowohl Web wie auch Desktop)
- Sencha Touch (2010 WebApp, gutes Look and Feel, nicht so performant)

Beispiele für Softwarearchitekturen

we focus on students



[angelehnt an Rausch 2006]

Beliebte Businesslogik- und Persitenz-Frameworks

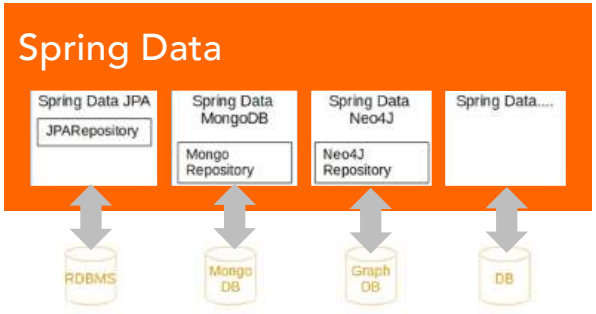
we focus on students

Businesslogik- und Persistenz-Frameworks:
Diese Frameworks bieten Funktionen zur Erstellung robuster, großer, verteilter und datenintensiver Anwendungen, sogenannter „Enterprise Anwendungen“: weltweit, große Nutzeranzahl, 24/7, international.

Beispiele:

- **Jakarta EE** (früher Java EE)
  Framework für große komponentenbasierte Java-EE-Anwendungen
- .Net – Ökosystem
- Spring Framework mit Spring Data (Persistenz-Abstraktionsschicht)

- JPA-kompatible Implementierungen
  - **Hibernate** (OR-Mapping, Java)
  - **EclipseLink** (OR-Mapping, Java)
  - **NHibernate** (OR-Mapping, .Net)



Spezielle Frameworks

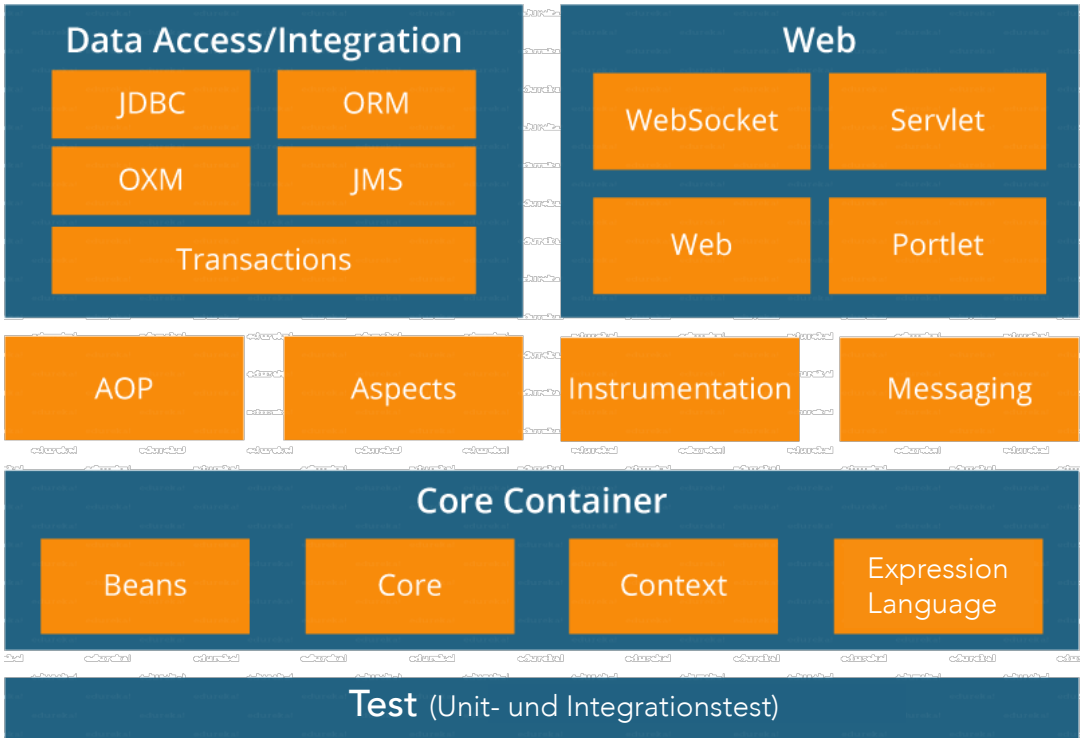
Node.js (2009)

- ist eine JavaScript-Laufzeitumgebung, die es ermöglicht dass JavaScript-Code außerhalb eines Web-Browsers ausgeführt werden kann.
- es wird in der JavaScript-Laufzeitumgebung V8 ausgeführt und
- es basiert auf einer ressourcensparenden Architektur, die eine besonders große Anzahl gleichzeitig bestehender Netzwerkverbindungen unterstützt.

Microservice-Frameworks
wurden entwickelt, um Microservices-Architekturen zu entwickeln und zu verwalten.
Sie enthalten Tools für die Service-Erkennung, Lastausgleich, Fehlertoleranz und mehr.

- Beispiele:
  - Spring Boot
  - Dropwizard
  - Micronaut
  - Quarkus

Beispiel: Spring Framework

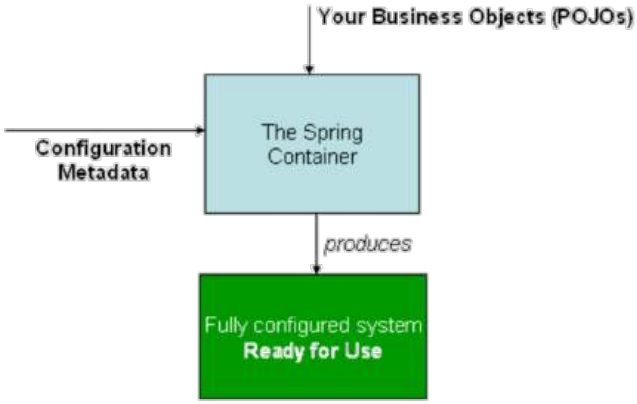


Beans und Core

- Die Module **Beans** und **Core** stellen die grundlegenden Bestandteile des Frameworks dar, einschließlich der Inversion of Control (IoC) - und Dependency-Injection (DI) -Funktionen.

- **Beans** bezeichnen zentrale Objekte der Anwendung, die vom IoC-Container verwaltet werden. Die Bean wird vom Container instanziiert, assembliert und verwaltet (→ Lebenszyklus).

  Dabei macht die BeanFactory programmierte Singletons überflüssig und ermöglicht die Entkopplung der Beschreibung von Abhängigkeiten von der eigentlichen Programmlogik.



Context

- Das **Context-Modul** baut auf den Core- und Beans-Modulen auf.
- Es ermöglicht den **Zugriff** auf Objekte über eine Art Register.
- Das Context-Modul bietet ergänzend zum Beans-Modul weitere Unterstützung z.B. für **Internationalisierung, Ereignisweiterleitung, Laden von Ressourcen, entfernte Aufrufe, die Erstellung von Kontext**en und weiteren Themen.

Expression-Languague

- Das Expression-Languague-Modul wird zur **Arbeit auf einem Objektgraphen** während der Laufzeit genutzt. Attribute können gelesen, geschrieben und zugewiesen werden und Methoden auf diesen Objekten ausgeführt werden.

Weitere Module

AOP | Aspects | Instrumentation | Messaging

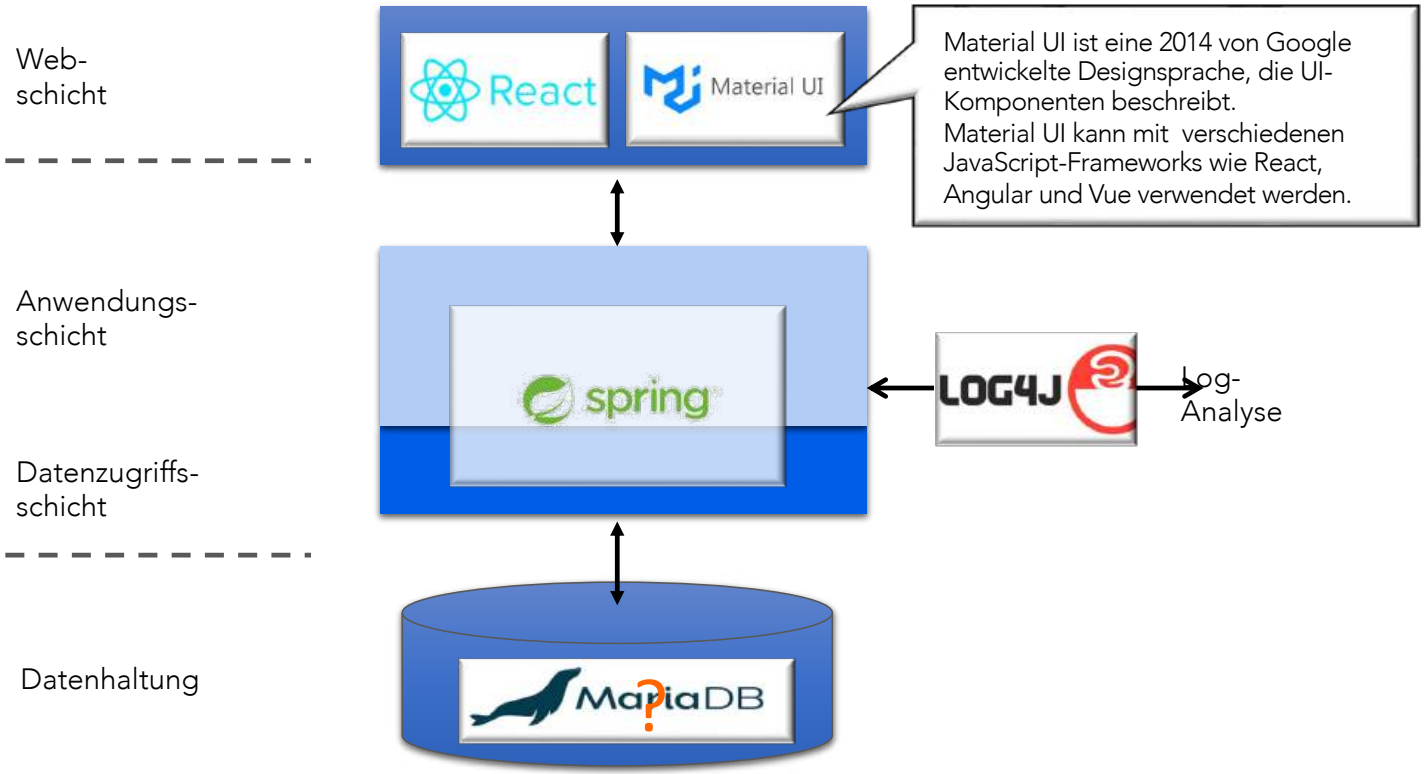
- **Aspektorientierten Programmierung (AOP) i**st ein Programmierparadigma für um generische Funktionalitäten über mehrere Klassen hinweg zu implementieren, s.g. **Cross-Cutting-Concerns.** So lässt sich beispielsweise die Anpassung des Lebenszyklusses der Beans realisieren, ein spezifischer Transaktionsschutz oder Logging.

- Das Modul **Aspects** integration AspectJ. AspectJ ist eine aspektorientierte Erweiterung von Java.

- **Instrumentierung** bezeichnet das Hinzufügen von Bytecodes zu Methoden, um Daten für die Nutzung durch Tools zu sammeln. Dieses Modul unterstützt die Instrumentatierung von Klassen und die Implementierung von Classloadern.

- Das **Messaging-Modul** enthält Konzepte wie Message, MessageChannel, MessageHandler und Weiterenzur Umsetzung Messaging-basierter Anwendungen. Das Modul enthält außerdem eine Reihe von Annotationen zum Zuordnen von Nachrichten zu Methoden.

Technologiestack einer (Progressive) Web App



Struktur

1. Einführung

1.1 Einführung und Begriffe

1.2 Verteilungsmodelle und Middleware-Technologien (im Überblick)

~~1.3 Kriteriengeleitete Evaluation~~

www.fh-dortmund.de

facebook.com/fhdortmund instagram.com/fhdortmund twitter.com/fh_dortmund youtube.com/FachhochschuleDO

Fachhochschule
Dortmund
University of Applied Sciences and Arts

we
focus
on
students

Softwaretechnik 2

Architekturstile im Zeitverlauf I

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Architekturstile

nach Gernot Starke 2017

Stil	Intention	Beispiele
Datenflusssysteme	Besteht aus Folge (Sequenz) von Operationen auf Daten	Batch-Sequentiell Pipes und Filter Prozesssteuerung
Datenzentrische Systeme	gemeinsam verwendeter, zentraler Datenbestand	Repository Blackboard
Hierarchische Systeme	Bestehen aus Bausteinen in unterschiedlichen Ebenen einer Hierarchie	Haupt- und Unterprogramm, Master-Slave, Schichten Ports und Adapter virtuelle Maschine
Verteilte Systeme	Bestehen aus Speicher- und Verarbeitungsbausteinen, die über Kommunikationsnetze interagieren	Client-Server, CQRS, Broker-Architekturen, serviceorientierte Architekturen Peer-to-Peer
Ereignisbasierte Systeme	Bestehen aus voneinander unabhängigen Bausteinen, die über Ereignisse (Events) kommunizieren, sich implizit aufrufen.	Ungepufferte Kommunikation: Broadcast, Publisher-Subscriber Gepufferte Kommunikation: Message-Queue, Message-Service
Interaktionsorientierte Systeme	Systeme mit (grafischer) Bedienoberfläche	Model-View-Controller u.a.
Heterogene Systeme	Verwenden mehrere Architekturstile parallel	beispielsweise REST

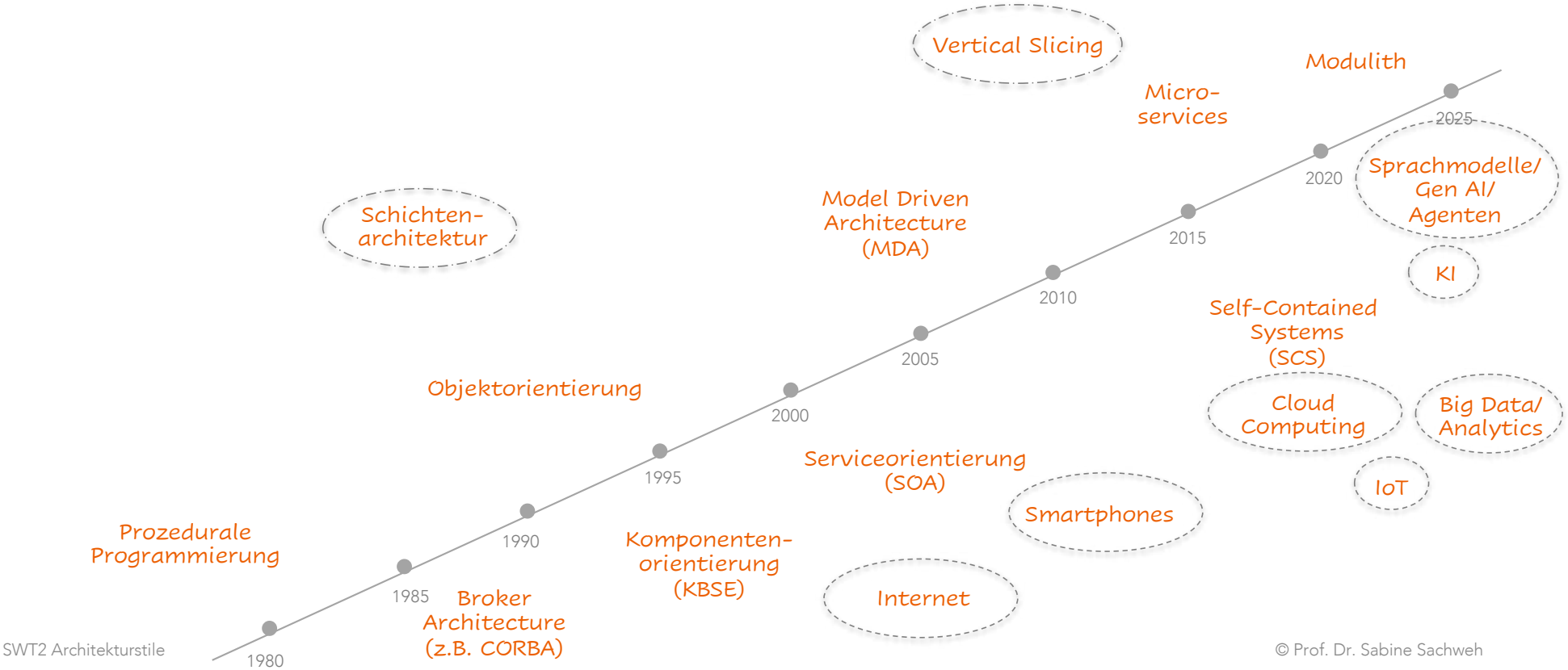
© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Architekturstile im Zeitverlauf I

Broker

KBSE

SOA

Fachhochschule
Dortmund
University of Applied Sciences and Arts

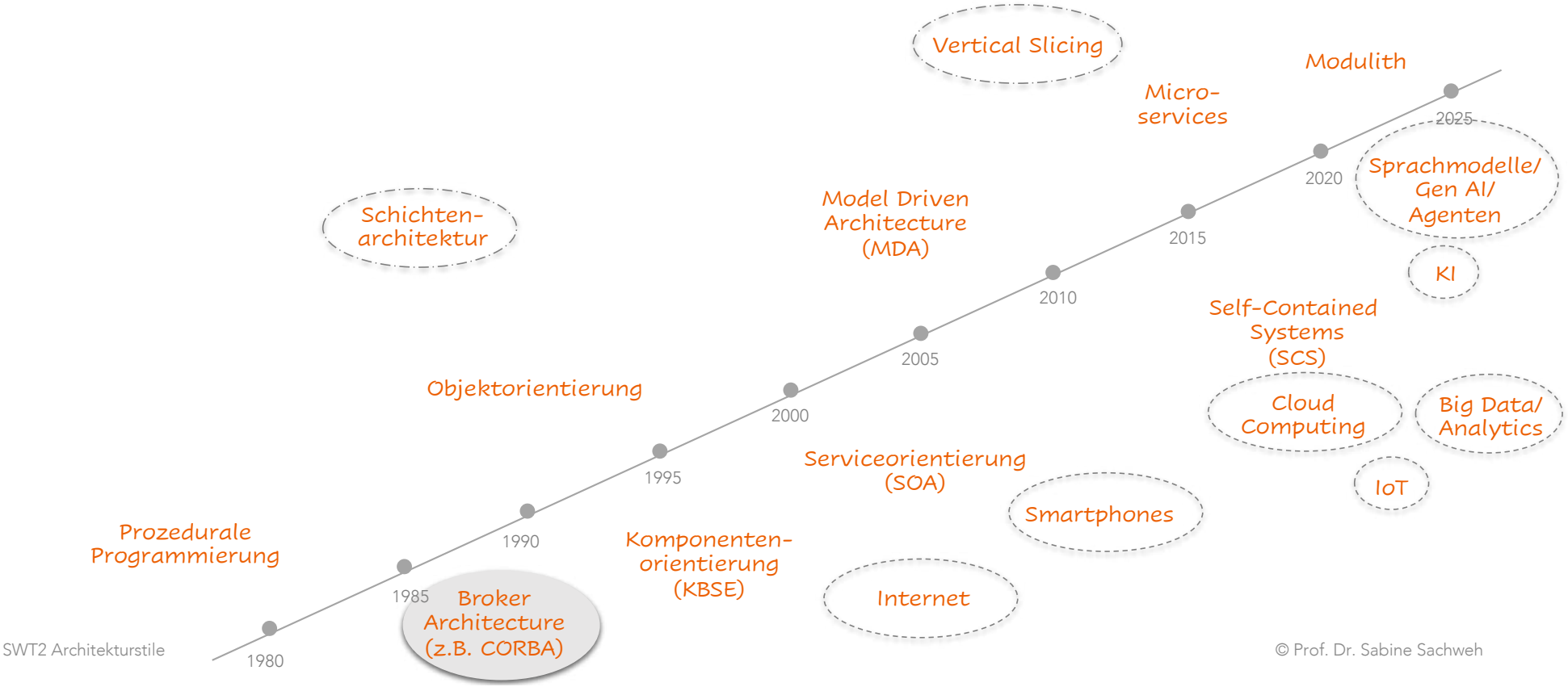
Broker

am Beispiel CORBA

Entwicklungs- und Architekturtrends im Zeitverlauf

we focus on students

Unvollständige Auswahl großer Softwaretechnik-Trends



© Prof. Dr. Sabine Sachweh

Einordnung

- Ende der 80ger Jahre wurden IT-Systeme komplexer und vermehrt eingesetzt
- zu nehmender Bedarf des Datenaustausches zwischen den Systemen
- Anwendungslandschaften „wuchsen langsam zusammen“
- leistungsfähigere Hardware und Betriebssysteme lieferten eine geeignete Basis für Client/Server-Systeme
- zusätzlich stieg der Druck auch ältere Systeme anzubinden und zu warten

⇒ benötigt wurde eine offene und flexible Kommunikationsinfrastruktur

© Prof. Dr. Sabine Sachweh

Object Management Group (OMG)

- **Object Management Group (OMG)**
  Internationaler Zusammenschluss von akademischen Instituten, Hardwarehe Softwareentwicklern, Netzwerkbetreibern und kommerziellen Nutzern
- Gründung im Jahre **1989**, schnell wachsend, mittlerweile mehr als 800 Mitglieder
- **Vision**
  Interoperabilität maximieren
- **Fokus**
  Integration von Systemen und Anwendungen über heterogene Plattformen hinweg
- **Ziel**
  Sammeln und Aufarbeiten von technologischem Know-how für die Entwicklung verteilter,
  auf Basis verschiedener Plattformen entstandener und laufender Software
- Bekannte Ergebnisse: **CORBA** (1991) und **UML** (1997)

© Prof. Dr. Sabine Sachweh

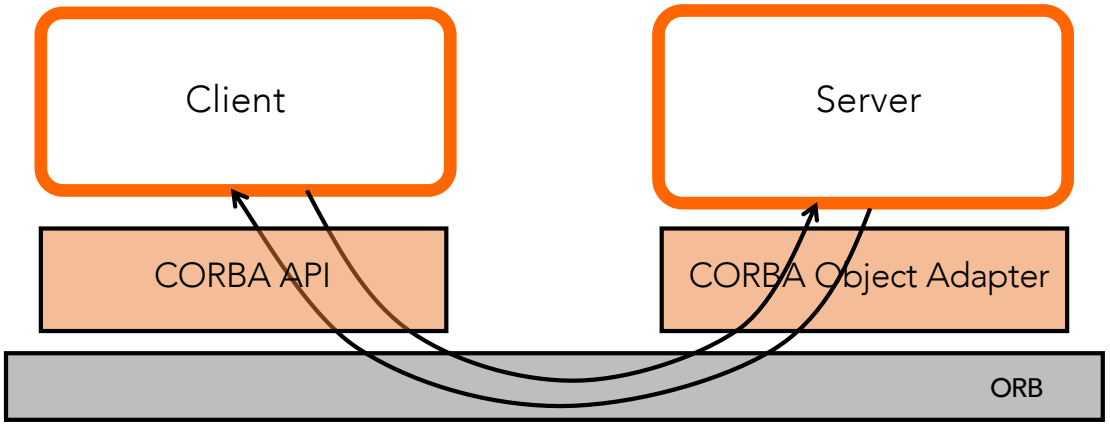
CORBA-Eigenschaften

CORBA zeichnet sich vor allem durch folgende Eigenschaften aus:

- Grundlage für eine mehrschichtige Client/Server-Architektur
- Sprach- und Systemunabhängigkeit
- Interoperabilität zwischen heterogenen Systemen
- offener Standard
- stabile Basis für verteilte Objekte
- Verfügbarkeit von CORBA auf allen bekannten Betriebssystemen
- CORBA macht Netzwerkkommunikation transparent für Entwickler

© Prof. Dr. Sabine Sachweh

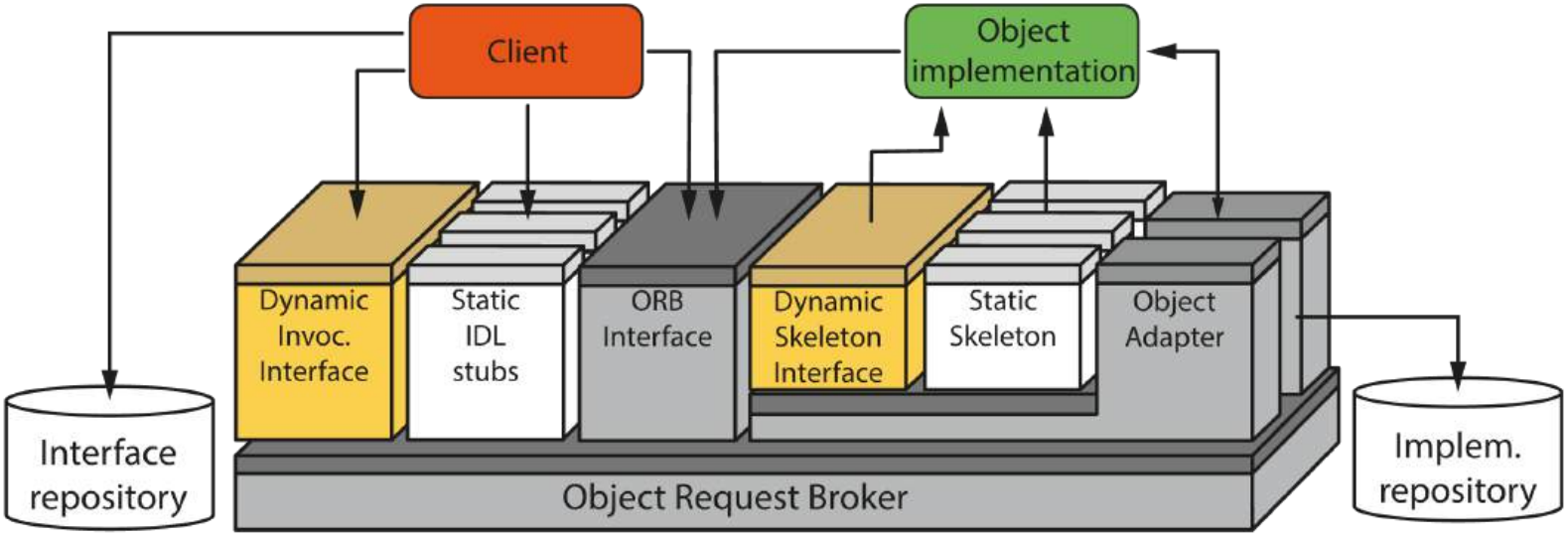
CORBA-Grundkonzept (Broker)



- Literatur: OMG Einführung in CORBA: http://www.omg.org/gettingstarted/corbafaq.htm
  Java-Beispiel: https://docs.oracle.com/javase/7/docs/technotes/guides/idl/jidlExample.html

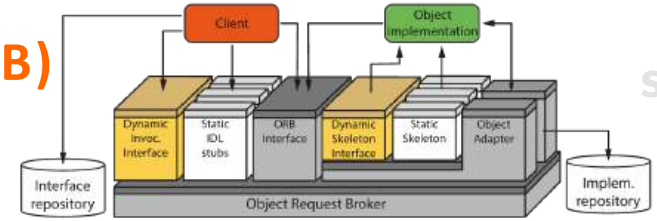
© Prof. Dr. Sabine Sachweh

CORBA-Architektur



© Prof. Dr. Sabine Sachweh

Object Request Broker (ORB)

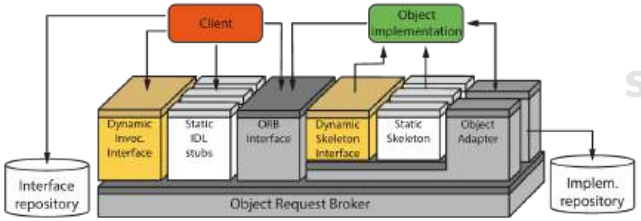


we focus on students

- Kommunikationsbus zwischen CORBA-Objekten, der einen Mechanismus anbietet, um **Anfragen des Clients** an die entsprechende Implementierung **weiterzuleiten**, **unabhängig** davon, an welchem **Ort** diese im Netzwerk liegen und unter welchem **Betriebssystem** sie dort laufen.
- Eine **Anfrage** des Clients, bestehend aus dem **Methodenaufruf und den Parametern**, wird serialisiert (**marshaling**) und über das Netzwerk an den Server geschickt.
- Die Informationen werden auf der Server-Seite wieder umgewandelt (**unmarshaling**) und die **gewünschte Operation** ausgeführt.
- **Rückgabewerte** werden auf die gleiche Weise wieder über den ORB an den Client gesendet.
- Der ORB hat also die Aufgabe, die entsprechende **Objektimplementierung zu finden**, sie **zu starten**, falls erforderlich, die **Anfrage an das Objekt** zu leiten und entsprechende **Rückgabewerte** wieder **an den Client zurückzugeben**.

© Prof. Dr. Sabine Sachweh

Stubs und Skeletons



we focus on students

- ein **Stub** stellt die Verbindung zwischen Client und ORB dar, d.h.
  - er stellt dem Client eine Methodenaufrufschnittstelle zur Verfügung und
  - leitet Aufrufe von Methoden serialisiert an den ORB weiter.
- ein **Skeleton** stellt die Verbindung zwischen ORB und Server dar, d.h.
  - es empfängt die Nachricht (s.o.) auf der Server-Seite vom ORB
  - baut einen „richtigen" Methodenaufruf zusammen
  - leitet den Aufruf an das Server-Objekt weiter und
  - wartet auf den Rückgabewert, den es entsprechend auf die Reise schickt.
- keine generische Aufgabe, d.h., Stubs und Skeletons sind nicht Teil eines fertig implementierten ORBs sondern werden aus einer Schnittstellen-Beschreibung generiert.
- Schnittstellen-Beschreibung wird in einer **Interface Definition Languague (IDL)** beschrieben

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Interface Definition Languague (IDL)

we focus on students

- Programmiersprachen unabhängig, orientiert sich an der Syntax von C++
- es existieren unterschiedliche Bindungen an Programmiersprachen wie z.B. C, C++, Smalltalk, Ada, COBOL, Java
- eine IDL-Beschreibung enthält
  - den Namen der Schnittstelle
  - den Namen der Methode
  - die erlaubten Typen aller Ein- und Ausgabeparameter
- eine IDL-Beschreibung ist Basis der Stub- und Skeleton- Generierung

```
module <identifier>
{
  <type declarations>;
  <constant declarations>;
  <exception declarations>;
       interface <identifier> [:<inheritance>]
       {
           <type declarations>;
           <constant declarations>;
           <attribute declarations>;
           <exception declarations>;
           [<op_type>]<identifier>(<parameter_list>)
           [raises exception] [context];
                        .
                        .
       }
        interface <identifier> [:<inheritance>]
                .
}
```

Definition eines **Namensraums**

Definition einer CORBA **Klasse**

Definition einer **Methode**

Definition einer CORBA **Klasse**

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Interface Definition Languague (IDL)

we focus on students

- Basisdatentypen
  - **void**
  - **boolean** – true und false
  - **char** – 8Bit
  - **wchar** – implementationsabhängig, normalerweise 16 Bit
  - **float** – IEEE einfache Genauigkeit
  - **double** – IEEE doppelte Genauigkeit
  - **long double** – IEEE erweiterte doppelte Genauigkeit
  - (unsigned) **long** – 32 Bit Integer
  - (unsigned) **long long** – 64 Bit Integer
  - (unsigned) **short** – 16 Bit
  - **octet** – C und C++: char und unsigned char, Java: byte
  - **string** (feste und variable Zeichenlänge) – C: Zeichenarray, C++: Cstring, Java: String
- Komplexe Datentypen
  - **enum** (Erzeugung von Typen mit vordefinierte Wertemenge):

    ```
    enum Tage { Montag, Dienstag, ... };
    ```
  - **struct** (Strukturtyp, unterschiedliche Elementwerten, mit call-by-value übergeben):

    ```
    struct Datum { short Jahr, short Monat, short Millisekunde };
    ```


© Prof. Dr. Sabine Sachweh

Interface Definition Languague (IDL)

- Beispiel:

```
module MyAnimals {

        // Class definition of Dog
        interface Dog: Pet, Animal {

                attribute integer age;
                exception notInterested (string explanation);

                void bark(in short how_long)
                        raises (NotInterested);
                void sit(in string where)
                        raises (NotInterested);
                void growl(in string at_whom)
                        raises (NotInterested);
        };

        // class Definition of Cat
        interface Cat: Animal {

                void eat();
                void hereKitty();
        };
}; // end of MyAnimals
```

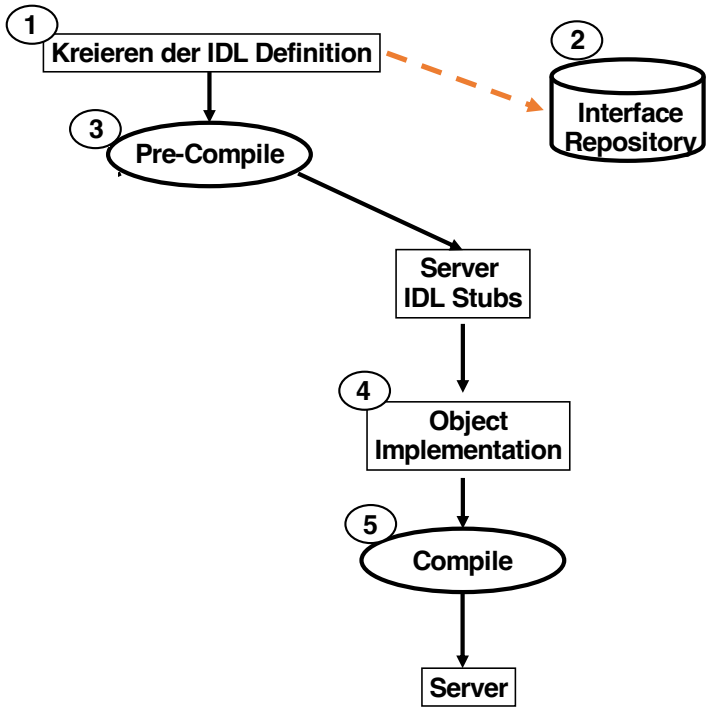
Definition eines Namensraums

Definition einer Klasse

Definition einer Klasse

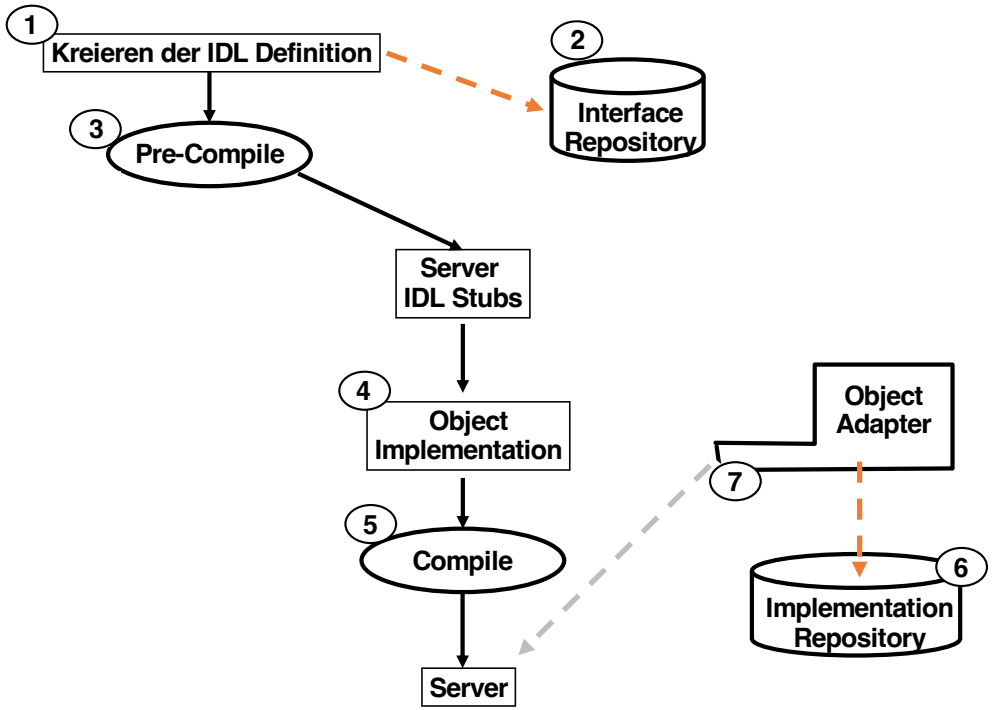
© Prof. Dr. Sabine Sachweh

Verwendung einer IDL-Beschreibung



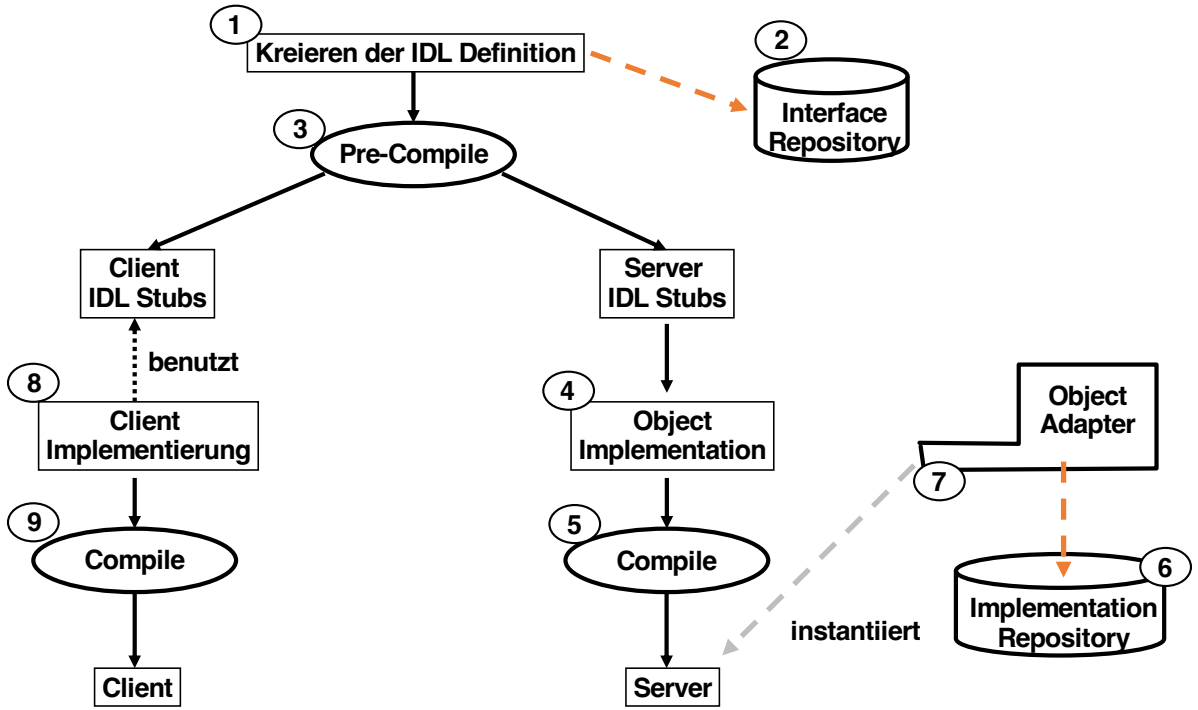
© Prof. Dr. Sabine Sachweh

Verwendung einer IDL-Beschreibung



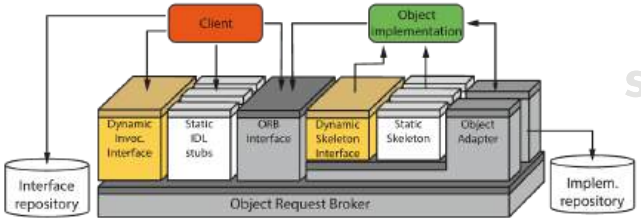
© Prof. Dr. Sabine Sachweh

Verwendung einer IDL-Beschreibung



© Prof. Dr. Sabine Sachweh

Dynamic Interfaces

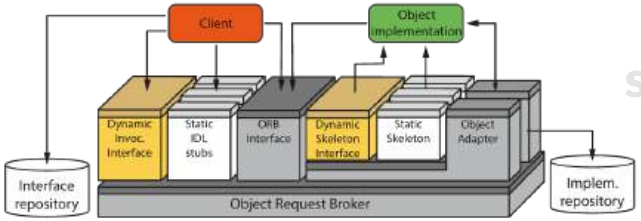


we focus on students

- Die Aufgabe der eben besprochenen **Stubs und Skeletons** ist beschränkt auf die in einer Applikation zur IDL-Übersetzungszeit bekannten **(statischen) Schnittstellen**.
- CORBA erlaubt aber auch die **dynamische Definition und Verwendung entfernter Objekte**
  - auf der **Client-Seite** gibt es dazu das **Dynamic Invocation Interface** und das **Interface Repository** und
  - auf der **Server-Seite** gibt es das **Dynamic Skeleton Interface** und das **Implementation Repository**
- in diesen Fällen kann ein Client das Objekt und dessen Schnittstelle, also die Namen und Parameter der Methoden, über das **Interface Repository** abfragen, falls das Server-Objekt zuvor registriert wurde
- der Aufruf einer konkreten Methoden erfolgt dann über das **Dynamic Invocation Interface**
- das **Dynamic Skeleton Interface** unterstützt den Aufruf von Methoden auf Objekten, die **keine IDL-Beschreibung** besitzen; diese müssen sich zur Laufzeit beim Dynamic Skeleton Interface registrieren, das die Informationen im **Implementation Repository** ablegt

© Prof. Dr. Sabine Sachweh

Portable Object Adapter



we focus on students

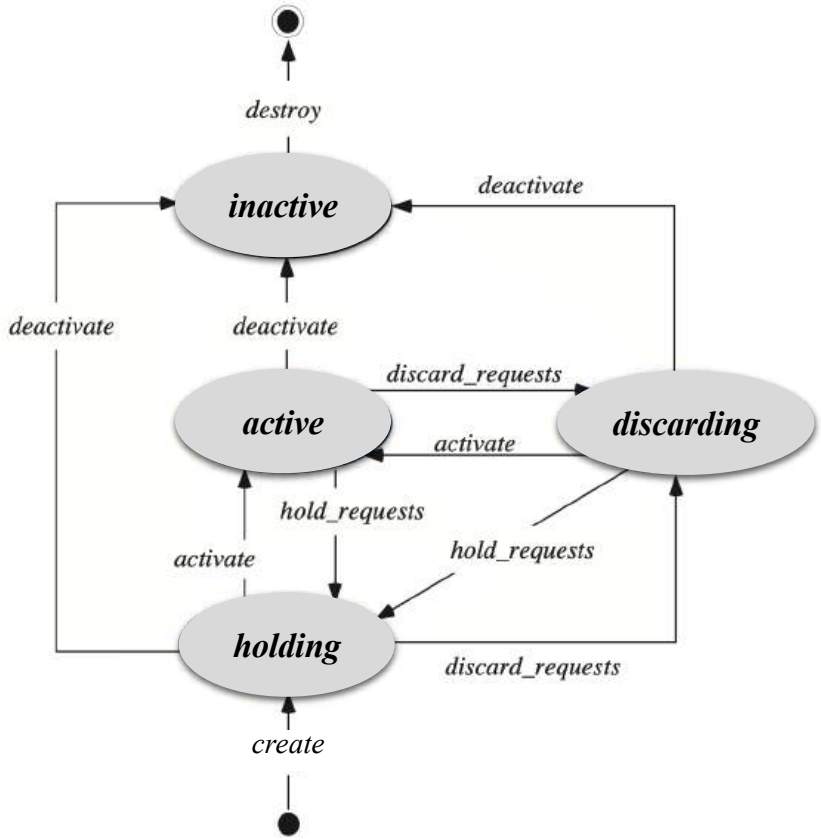
- Der Portable Object Adapter (POA) ist der **Teil des ORB, der serverseitige Ressourcen für Skalierbarkeit verwaltet**.
- **Objektimplementierungen werden deaktiviert**, wenn sie nichts zu tun haben und werden **erneut aktiviert**, sobald sie wieder benötigt werden, so dass die Hardwareressourcen besser ausgenutzt werden können.
- Der POA ist die **primäre Schnittstelle, um Dienste des ORBs zu nutzen,** zu denen u.a. folgende Dienste zählen:
  - Generieren und Interpretieren von Objektreferenzen,
  - Methodenaufruf,
  - Sicherheit von Aufrufen,
  - Aktivieren und Deaktivieren von Objekten und Implementierungen,
  - Zuordnen von Objektreferenzen zu Implementierungen und
  - Registrieren von Implementierungen
- Obwohl es sich architektonisch um ein separates Teil des ORBs handelt und ORBs potentiell mehr als einen Objektadaptertyp haben können, ist der POA keine Software, die Sie separat vom ORB verfügbar ist, **da die Schnittstellen, die den ORB und den POA verbinden, proprietär sind.**
- Der POA ist der zweite Objektadapter, den OMG entwickelt hat, da der Vorgänger, der **Basic Object Adapter (BOA)** nicht alle Unternehmens- und Internetanforderungen erfüllen konnte.

© Prof. Dr. Sabine Sachweh

Portable Object Adapter

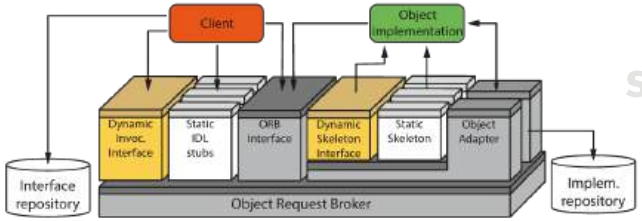
we focus on students

- Der Object Adapter verwaltet quasi den Lebenzyklus der Objektimplementationen.
- Er definiert vier Zustände, in die ein Objekt während seiner Lebensdauer eintreten kann:
  - holding: überbrückt nach dem Start die Verzögerung bis Objekt im Zustand active und arbeitsfähig ist. Zwischenzeitliche Anfragen werden in einer Message Queue zwischen gespeichert.
  - active: Normalzustand, Objekt beantwortet Anfragen.
  - discarding, in diesem Zustand werden Anfragen nicht bearbeitet und eine Exception wird zurückgeschickt. Der Zustand wird beispielsweise bei Überlastung eingesetzt.
  - inactive, in diesem Zustand können Objektreferenzen zwar genutzt werden; Methodenaufrufe werden aber vom Object Adapter zurückgehalten und blockiert, bis das Objekt aktiv ist. Dieser Zustand wird üblicherweise für Verwaltungsaufgaben eingesetzt wie z.B. um eine Objektimplementierung auszutauschen oder vor einem Server-Shutdown.



© Prof. Dr. Sabine Sachweh

ORB Interface



we focus on students

- das ORB Interface stellt einige nützliche Hilfsfunktionen für lokale Dienste bereit, wie z.B.:
  - Initialisierung des Object Request Broker
  - Initialisierung einer Client/Server-Anwendung
  - Programmierschnittstelle zum Interface Repository
    - Konvertierung einer Objektreferenz in eine Zeichenfolge und umgekehrt
    - Erstellen von Argumentlisten, die für dynamische Methodenaufrufe benötigt werden
  - ...

- OMG Einführung in CORBA: http://www.omg.org/gettingstarted/corbafaq.htm
- Java-Beispiel: https://docs.oracle.com/javase/7/docs/technotes/guides/idl/jidlExample.html

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Common Object Services

Wichtige (Unterstützungs-)Services

we focus on students

- **Naming Service** der es Serverobjekten ermöglicht, mittels eines festgelegten Namens angesprochen zu werden.
- **Trading Service** ermöglicht die Objektsuche zur Laufzeit auf Basis von Objekteigenschaften
- **Event Service** ermöglicht lose gekoppelte, ereignisbasierte n:m Kommunikation.
- **Life Cycle Service** stellt Operationen zum Kopieren, Verschieben und Löschen von Objekten zur Verfügung.
- **Relationship Service** ermöglicht die Modellierung von Beziehungen zwischen Objekten.
- **Weitere Dienste:** Externalization Service, Persistent Object Service, Concurrency Control Service, Transaction Service, Property Service, Licensing Service, Object Collection Service, Query Service, Time Service, Security Service, Notification Service als Erweiterung zum Event Service.

© Prof. Dr. Sabine Sachweh

Common Object Services (Überblick)

Service	Description
Collection	Facilities for grouping objects into lists, queue, sets, stacks, etc.
Query	Facilities for querying collections of objects in a declarative manner; various collections possible
Concurrency	Facilities to allow concurrent access to shared objects
Transaction	Flat and nested transactions on method calls over multiple objects
Event	Facilities for asynchronous communication through events
Notification	Advanced facilities for event-based asynchronous communication (e.g. filtering)
Externalization	Facilities for marshaling and unmarshaling of objects (like Java's serialization)
Life cycle	Facilities for creation, deletion, copying, and moving of objects
Licensing	Facilities for attaching a license to an object
Naming	Facilities for systemwide name of objects
Property	Facilities for associating (attribute, value) pairs with objects
Trading	Facilities to publish and find the services an object has to offer based on constraints
Persistence	Facilities for persistently storing objects; persistence transparency is provided
Relationship	Facilities for expressing relationships between objects (compare database schemes)
Security	Mechanisms for secure channels, authorization, and auditing
Time	Provides the current time within specified error margins

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Komponentenbasierte Systeme (KBSE)

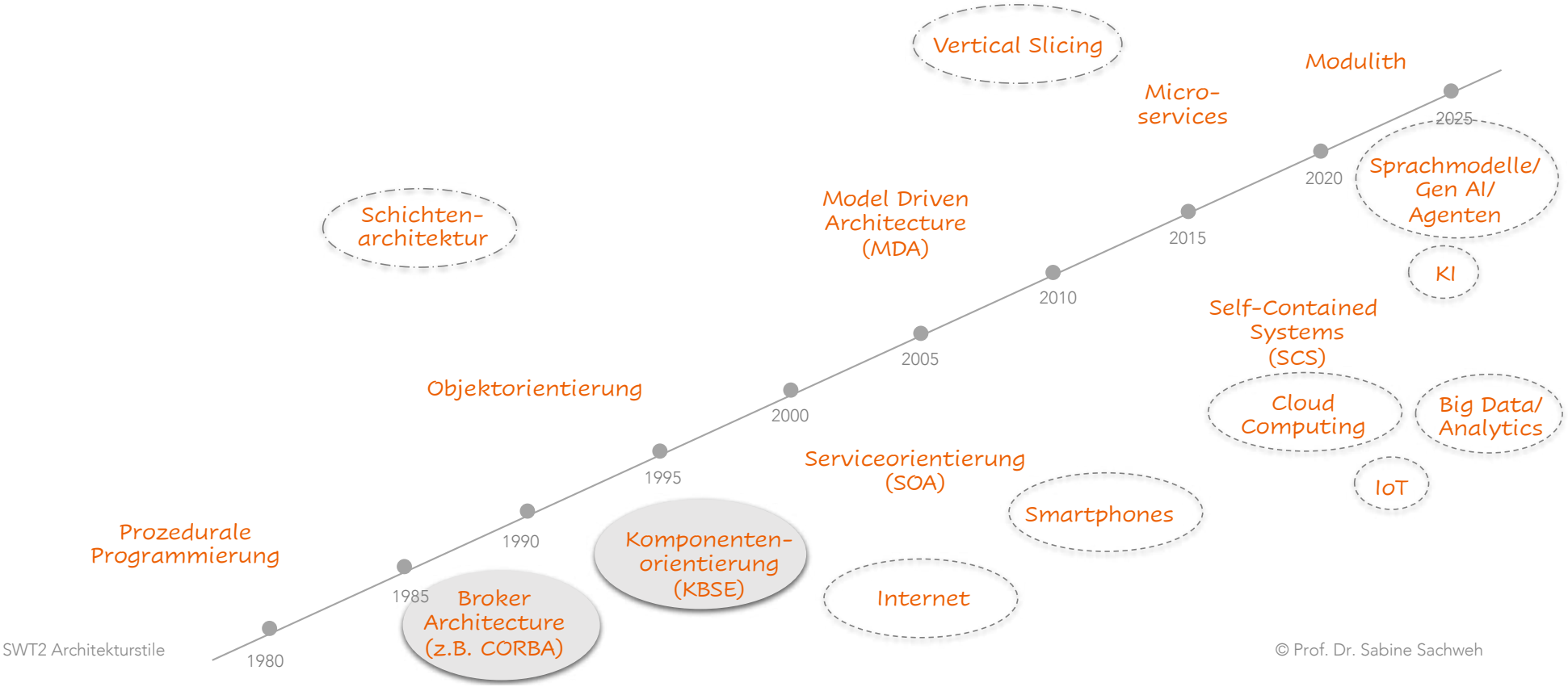
am Beispiel EJB

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Entwicklungs- und Architekturtrends im Zeitverlauf

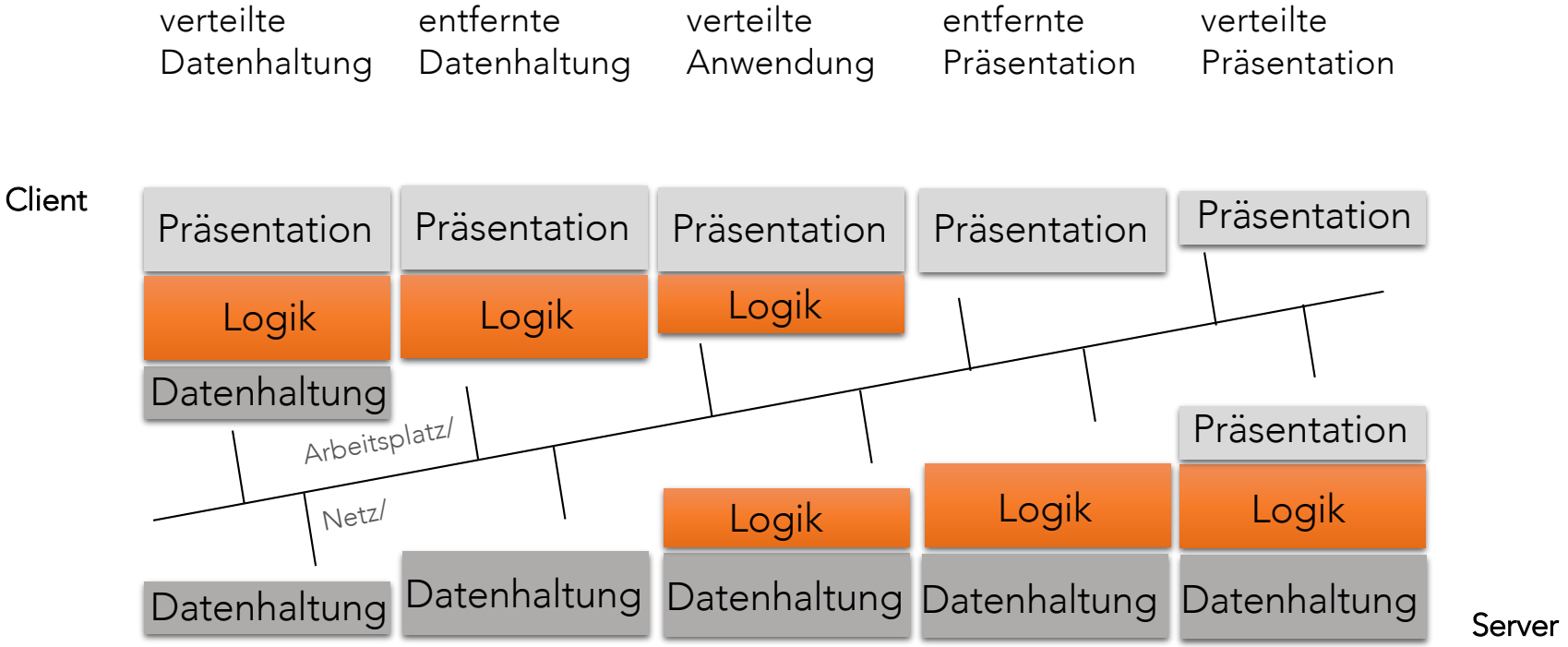
Unvollständige Auswahl großer Softwaretechnik-Trends



© Prof. Dr. Sabine Sachweh

Schichtenmodell (2-Tier bzw. Client/Server)

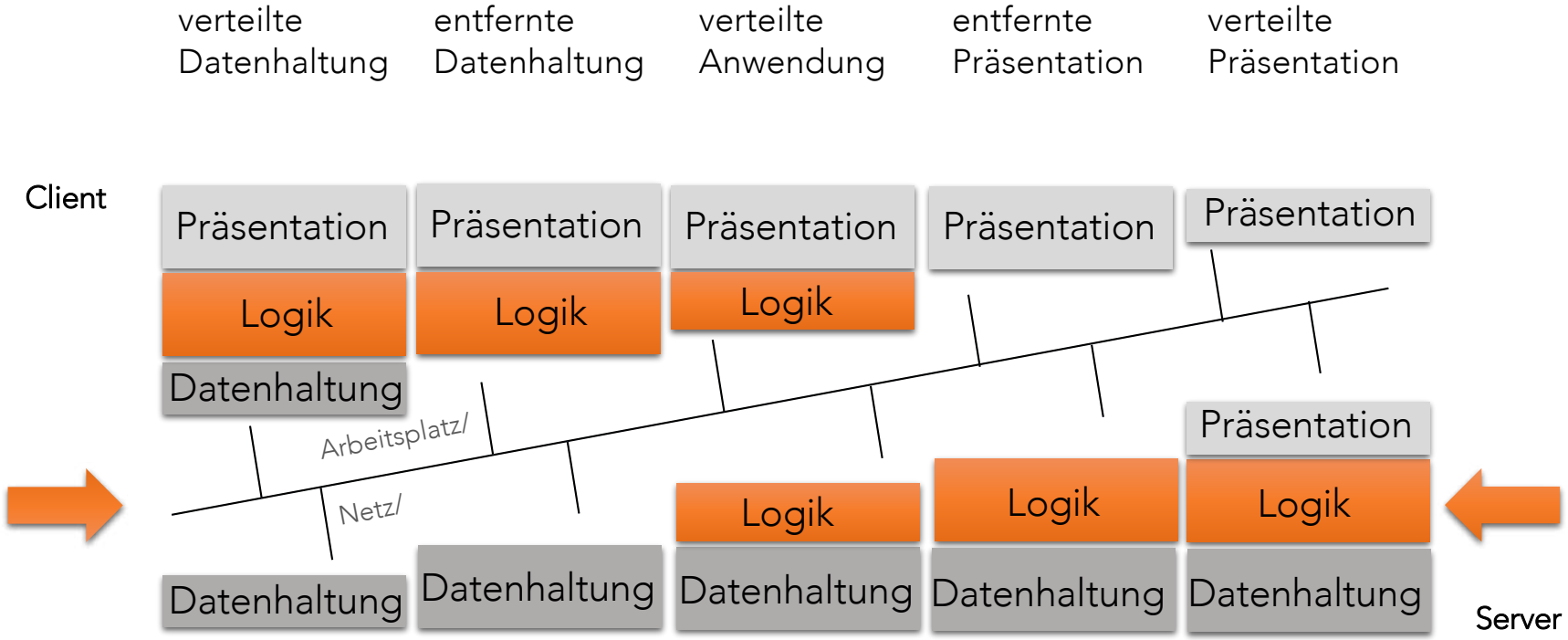
we focus on students



© Prof. Dr. Sabine Sachweh

Schichtenmodell (2-Tier bzw. Client/Server)

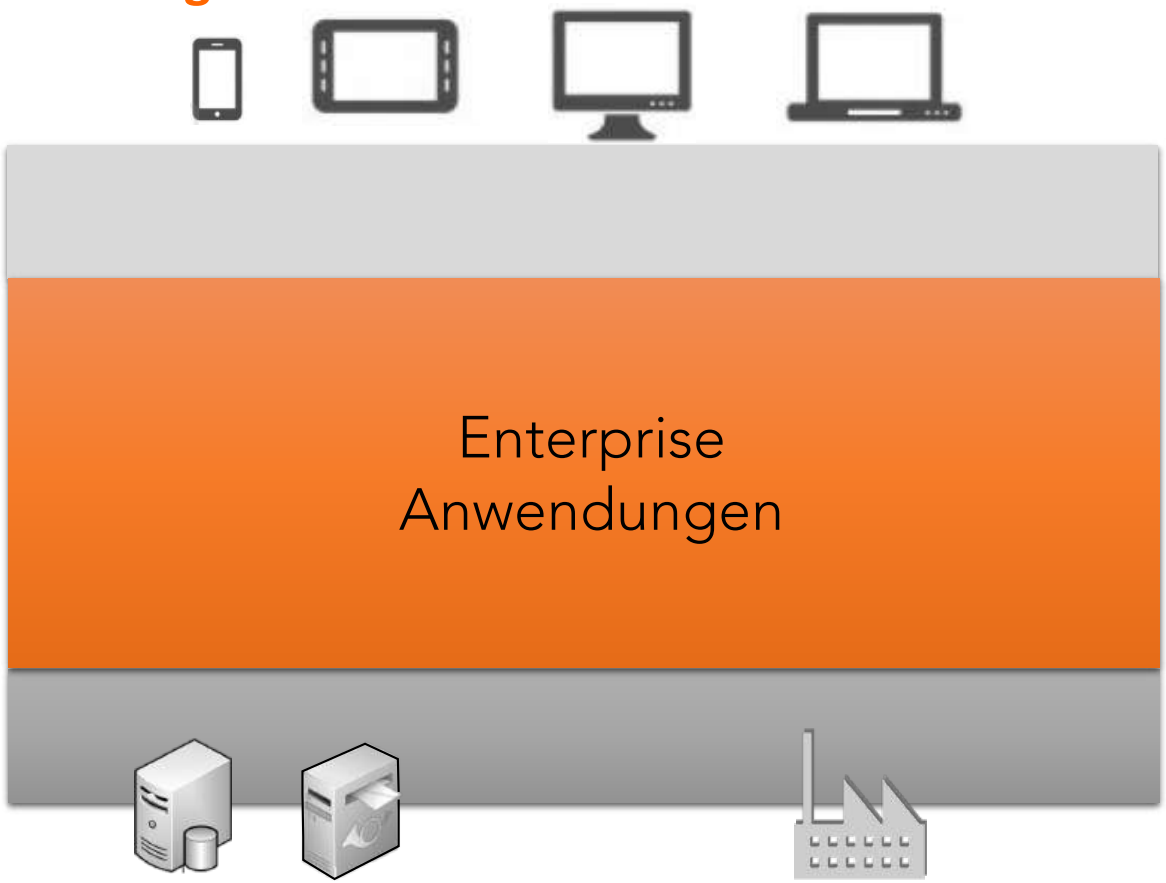
we focus on students



© Prof. Dr. Sabine Sachweh

Trennung der Verantwortlichkeiten

we focus on students

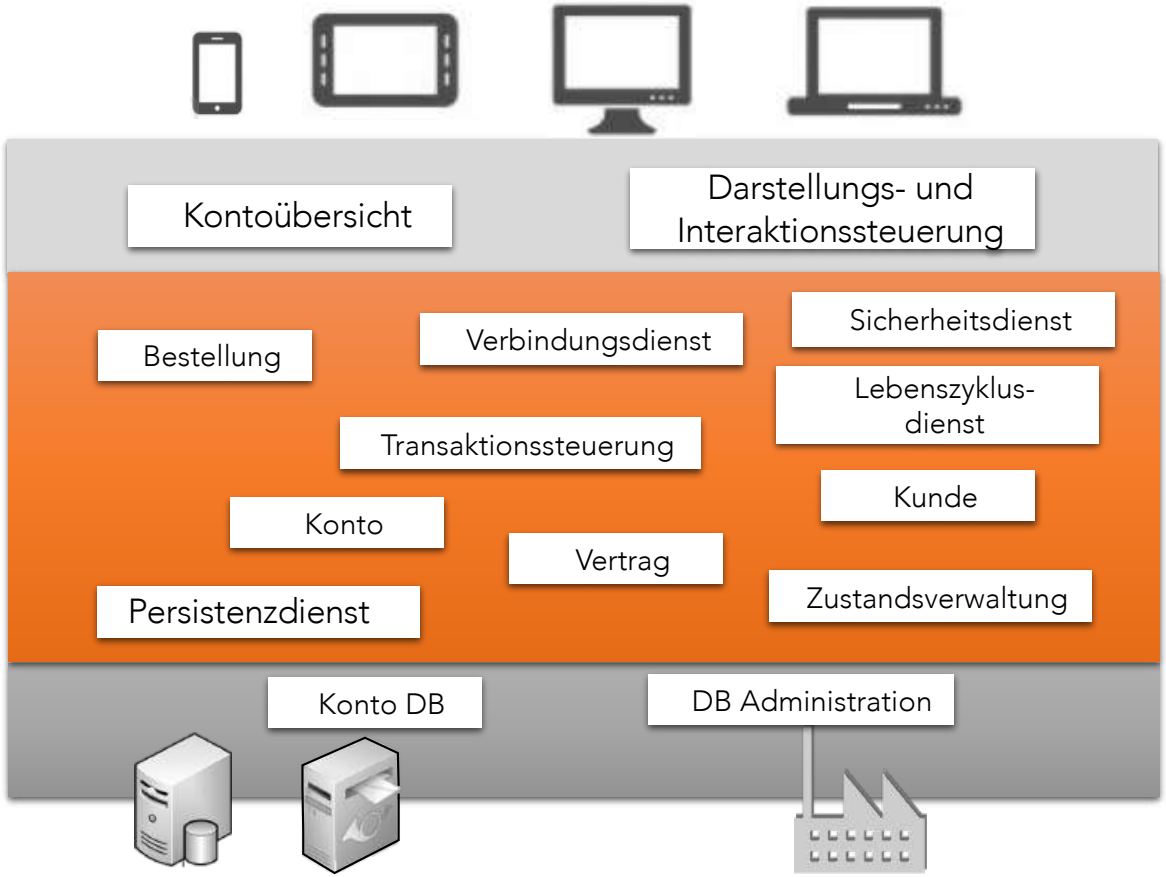


viele verschiedene Clients

viele verschiedene Datenquellen

© Prof. Dr. Sabine Sachweh

Trennung der Verantwortlichkeiten



viele verschiedene Clients

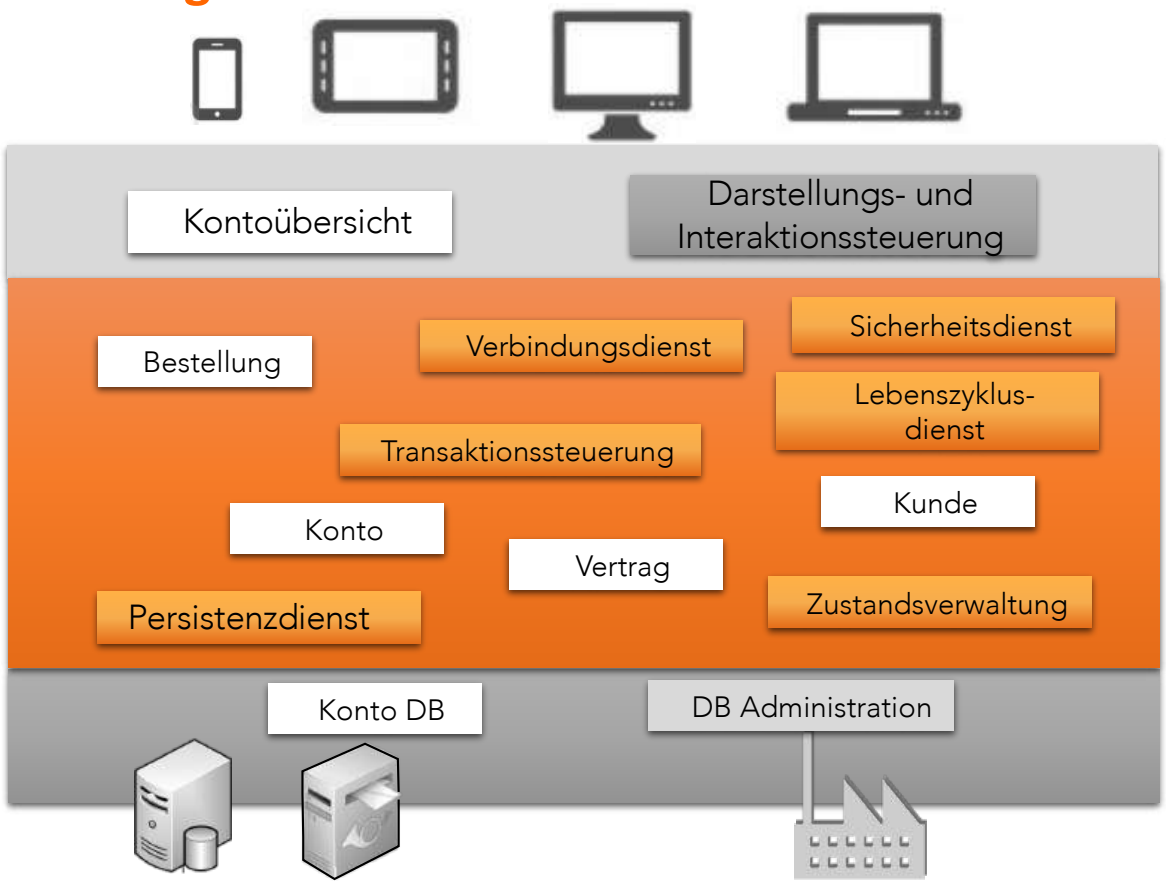
viele verschiedene Datenquellen

© Prof. Dr. Sabine Sachweh

Trennung der Verantwortlichkeiten

we focus on students

viele verschiedene Clients



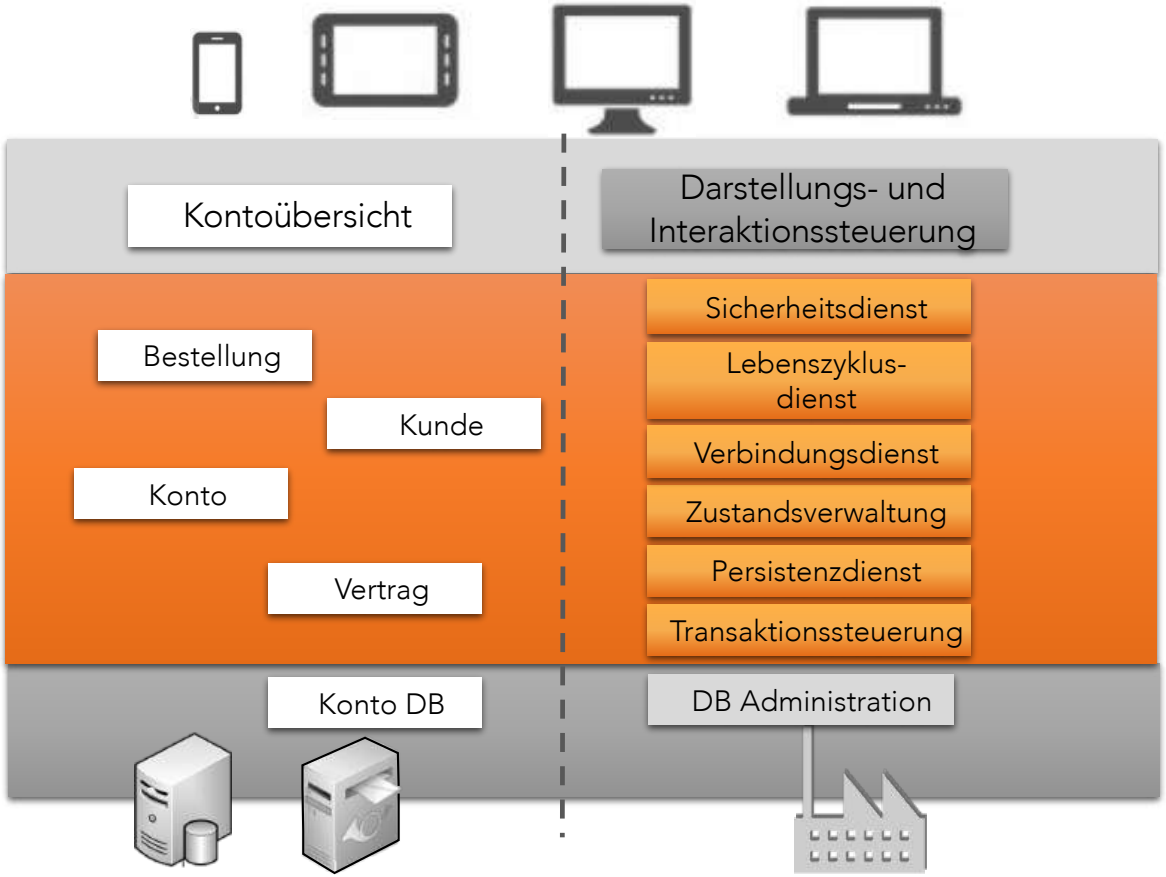
viele verschiedene Datenquellen

© Prof. Dr. Sabine Sachweh

Trennung der Verantwortlichkeiten

we focus on students

viele verschiedene Clients



fachlich

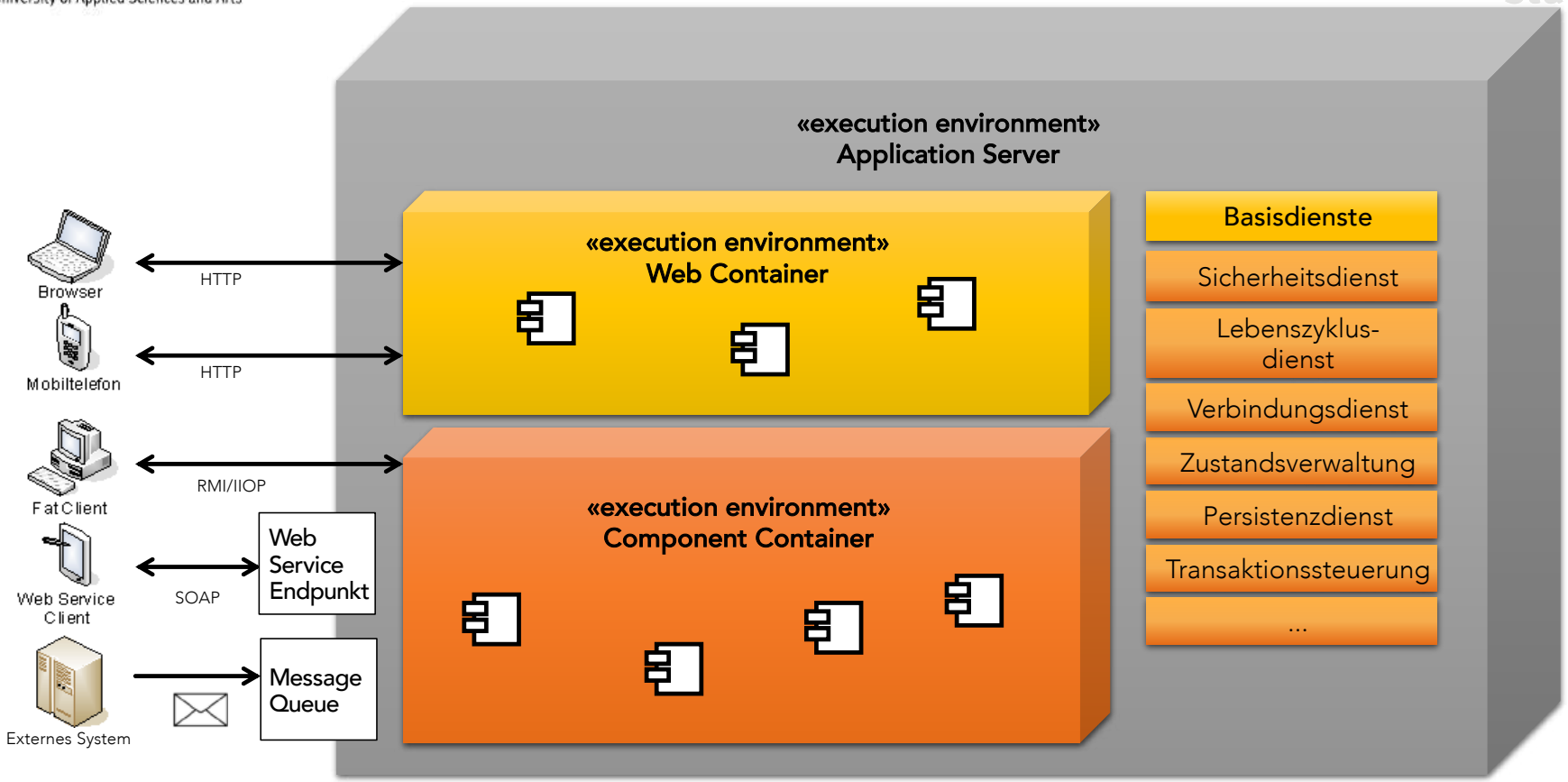
technisch

viele verschiedene Datenquellen

© Prof. Dr. Sabine Sachweh

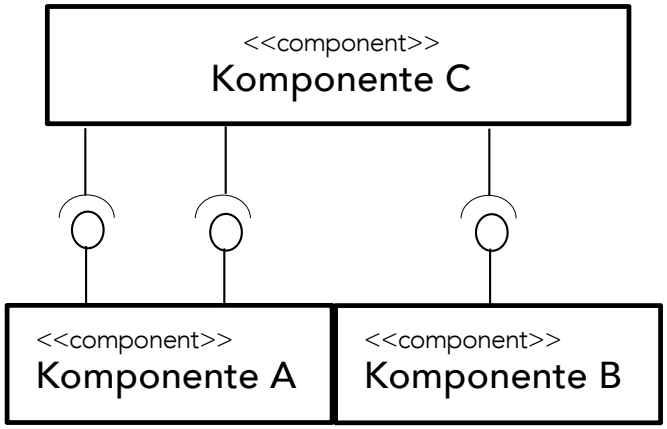
Anwendungsserver (engl. Application Server)

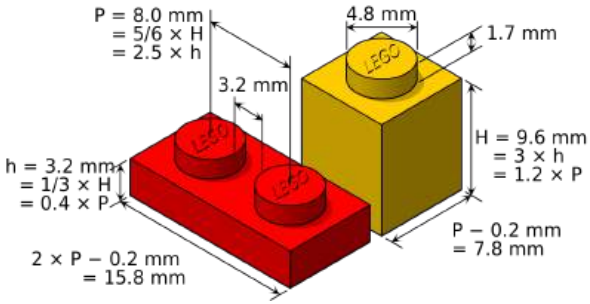
we focus on students



© Prof. Dr. Sabine Sachweh

Komponente – Lego Metapher



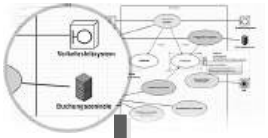


Eine Komponente ist ...

- geschlossener Teil eines Systems
- ohne starke Abhängigkeit
  - von anderen Komponenten
    - geringe Kopplung, hohe Kohäsion
  - von der Umgebung
    - systemunabhängiger Softwarebaustein
    - Interoperabilität:
      eine Komponente kann über Adressbereiche, Netzwerke, Sprachen, Betriebssysteme und verschiedene Tools hinweg aufgerufen werden
- mit expliziten Abhängigkeiten
- Komponenten werden oft als Archive verpackt

© Prof. Dr. Sabine Sachweh

Eine Komponente ...



… hat eine Spezifikation

… hat eine Implementierung

… kann ausgeliefert werden

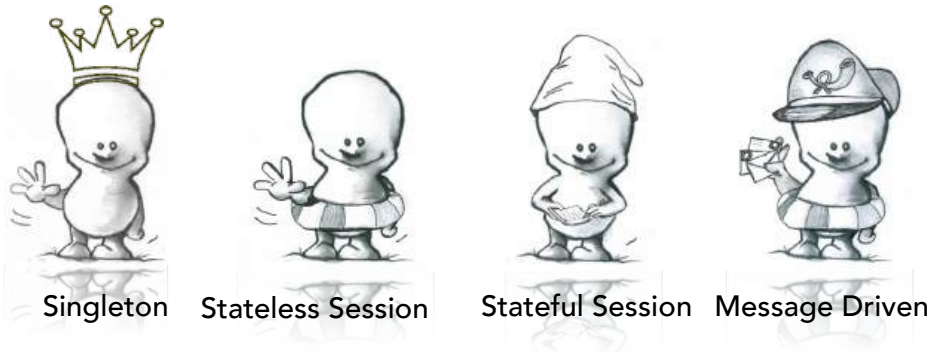
… entspricht einem Standard
(Corba/CCM, .Net/COM+, Java EE/EJB)

… kann verpackt werden

© Prof. Dr. Sabine Sachweh

Komponenten-Frameworks und -standards

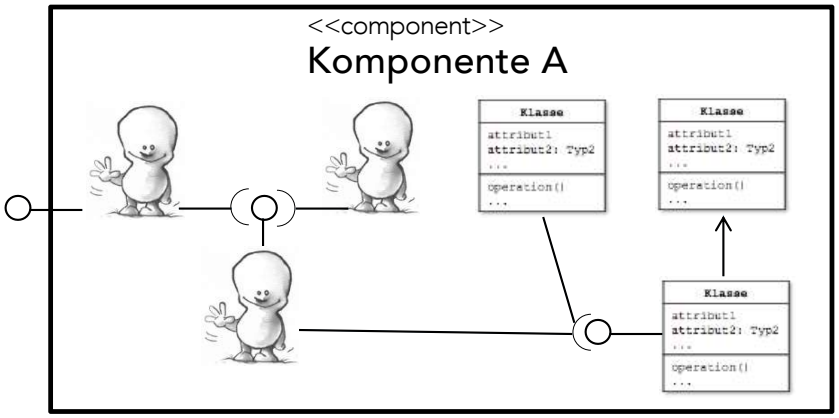
- es gibt Komponentenmodelle als Frameworks und Standards in verschiedenen Programmiersprachen und auf verschiedenen Plattformen
- Beispiele für bekannte Vertreter:
  - JavaEE/EJB
  - Microsoft .Net/COM+
  - Corba/CCM
- im weiteren Folgenden wird exemplarisch kurz auf den JavaEE/EJB-Ansatz eingegangen

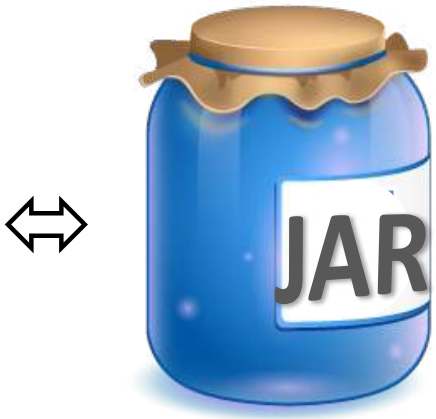


Java Enterprise Beans (EJBs)

© Prof. Dr. Sabine Sachweh

Komponente (EJB-Modul)





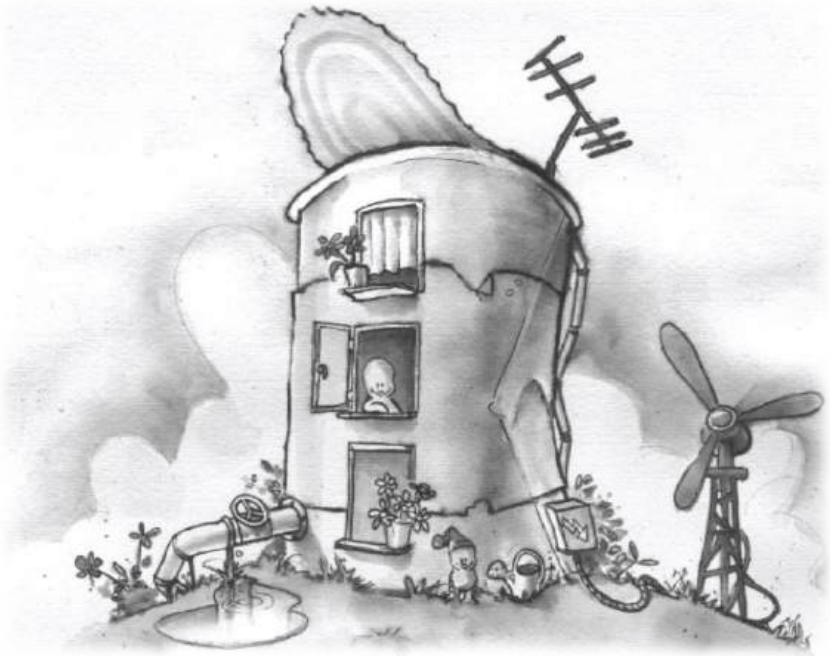
Interfaces (lokal, remote), **Beans** und (Hilfs-)**Klassen**

© Prof. Dr. Sabine Sachweh

Container

- stellt Dienste bereit
  (Implementierungen übergreifender Anforderungen)

- umgibt Komponenten bzw. die enthaltenen Beans, gibt Clients die Illusion von Komponenten, die eng mit ihrer Umgebung verknüpft sind

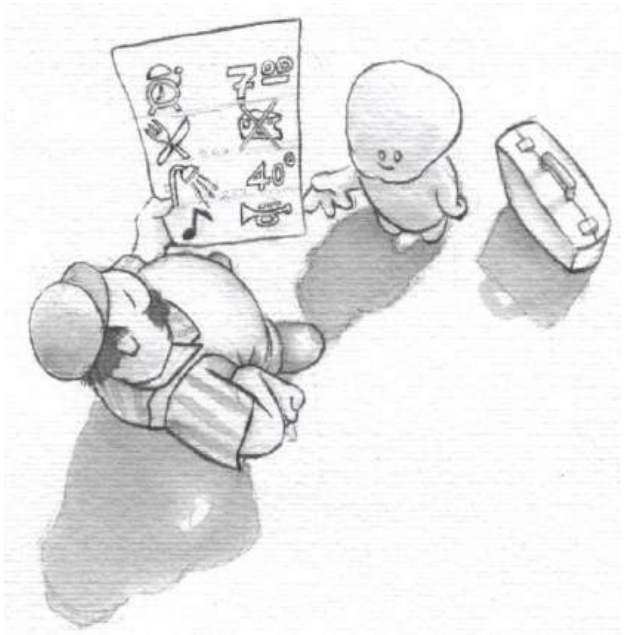


Wo die kleinen Beans wohnen …

© Prof. Dr. Sabine Sachweh

Deployment Descriptoren

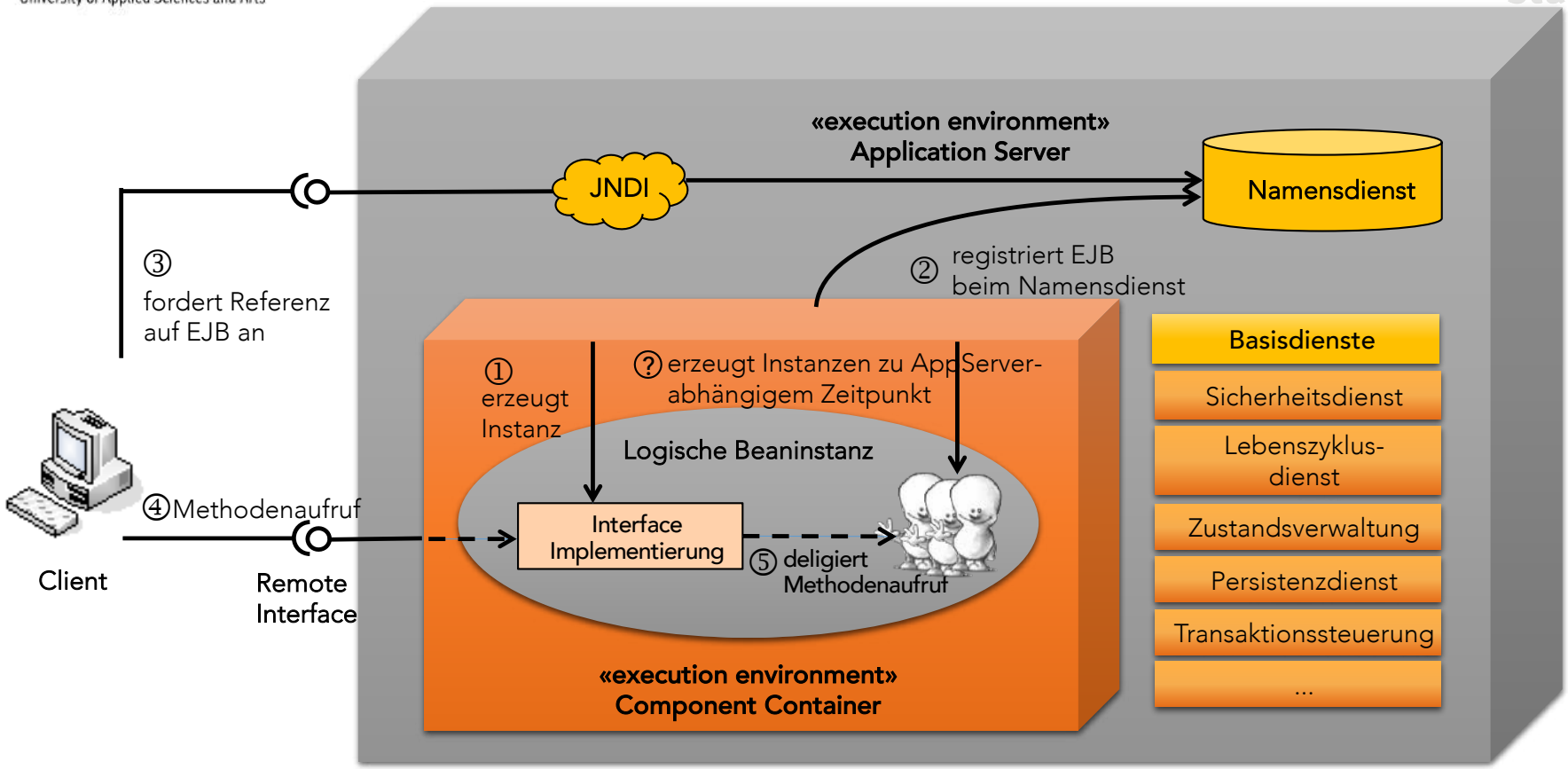
- Der Container ist verantwortlich für die Unterstützung nicht-funktionaler Anforderungen oder zur Verknüpfung von Beans.
  - Er braucht dazu Informationen darüber:
    - was unter Transaktionsschutz laufen soll
    - Sicherheitseinstellungen
    - gesendete Events
    - anpassbare Eigenschaften
- Eine Bean muss auf Nachfrage folgende Informationen über sich liefern, wie z.B: Schnittstellen, Eigenschaften, Events, etc.



© Prof. Dr. Sabine Sachweh

Beispiel: Aufruf über Remote Interface

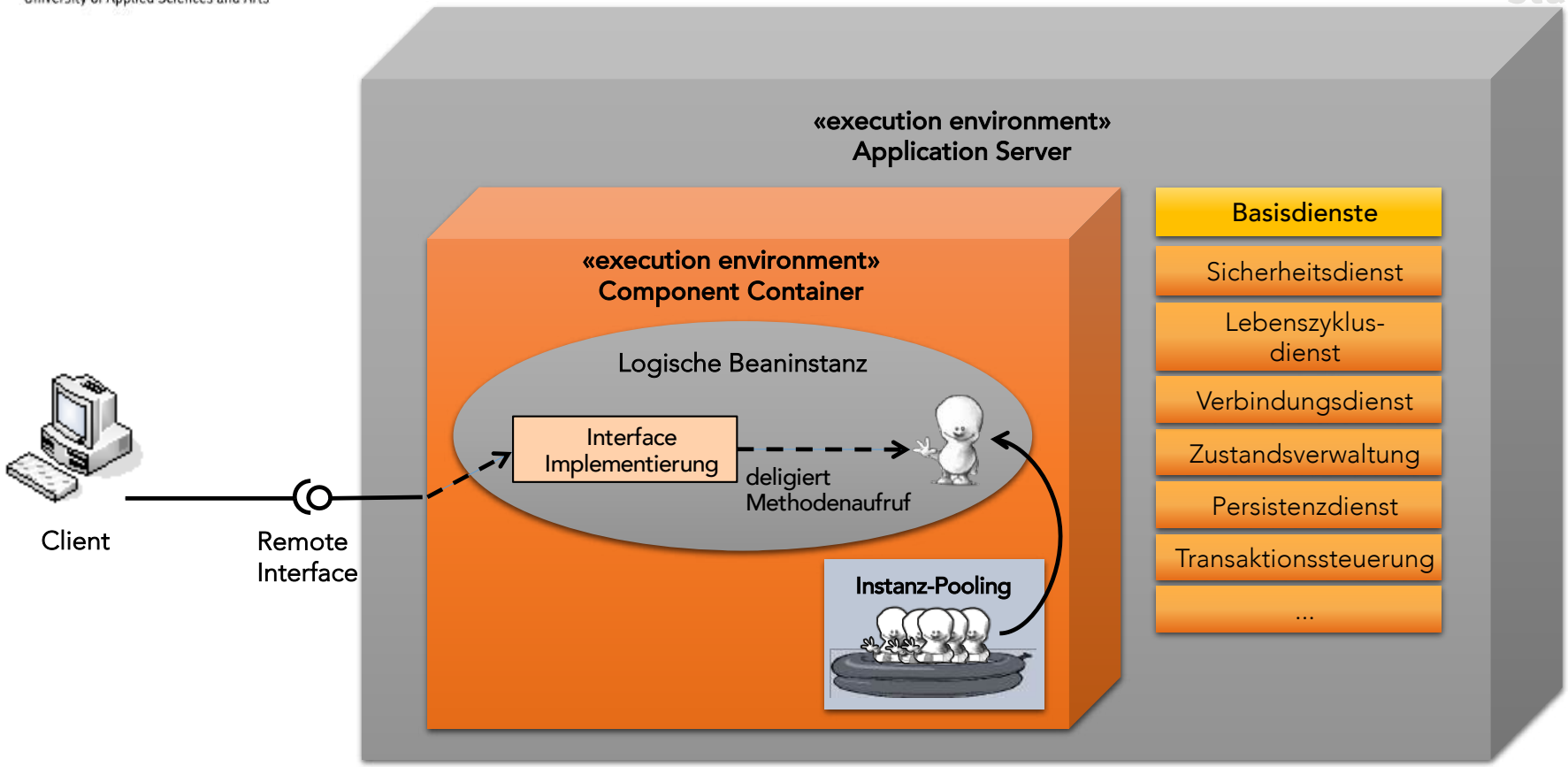
we focus on students



© Prof. Dr. Sabine Sachweh

Beispiel: Stateless Session Beans - Pooling

we focus on students



© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

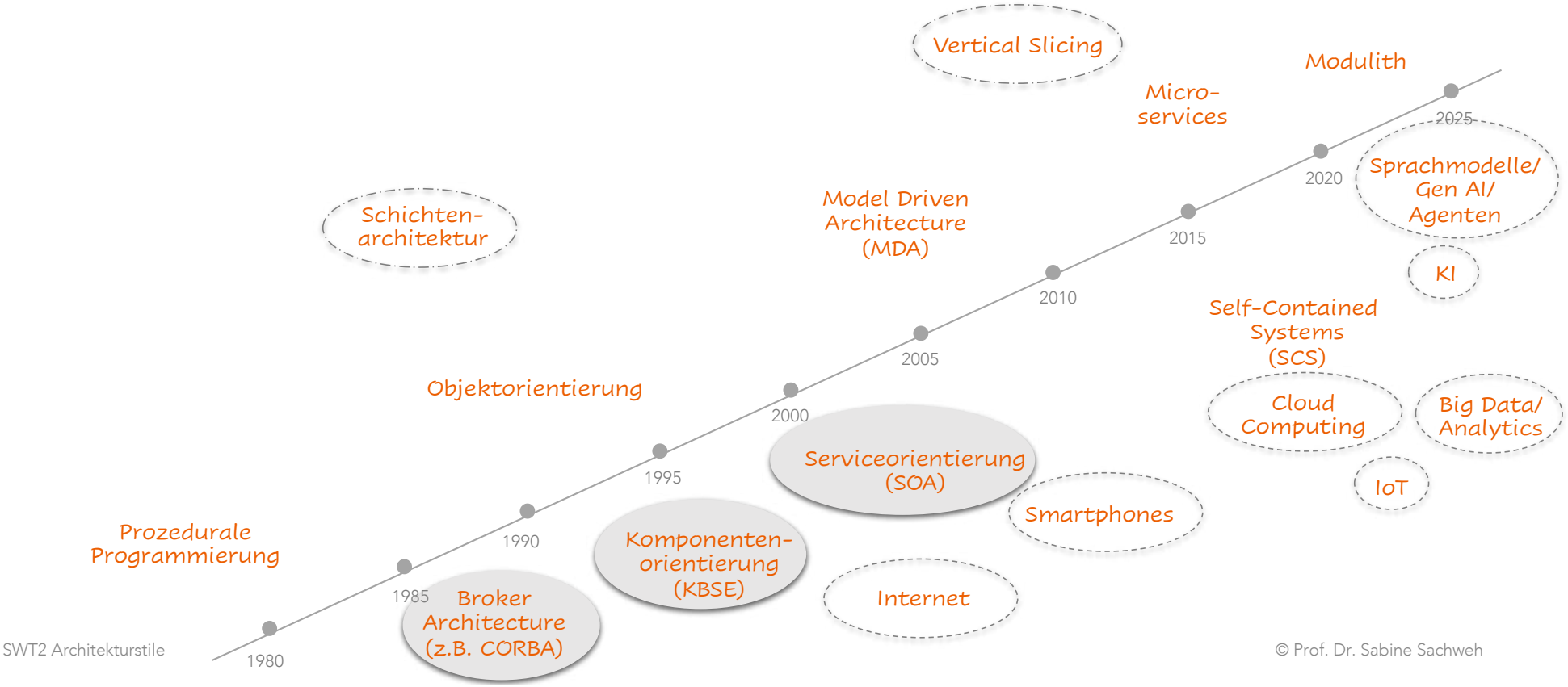
Service Orientierte Architekturen (SOA)

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends

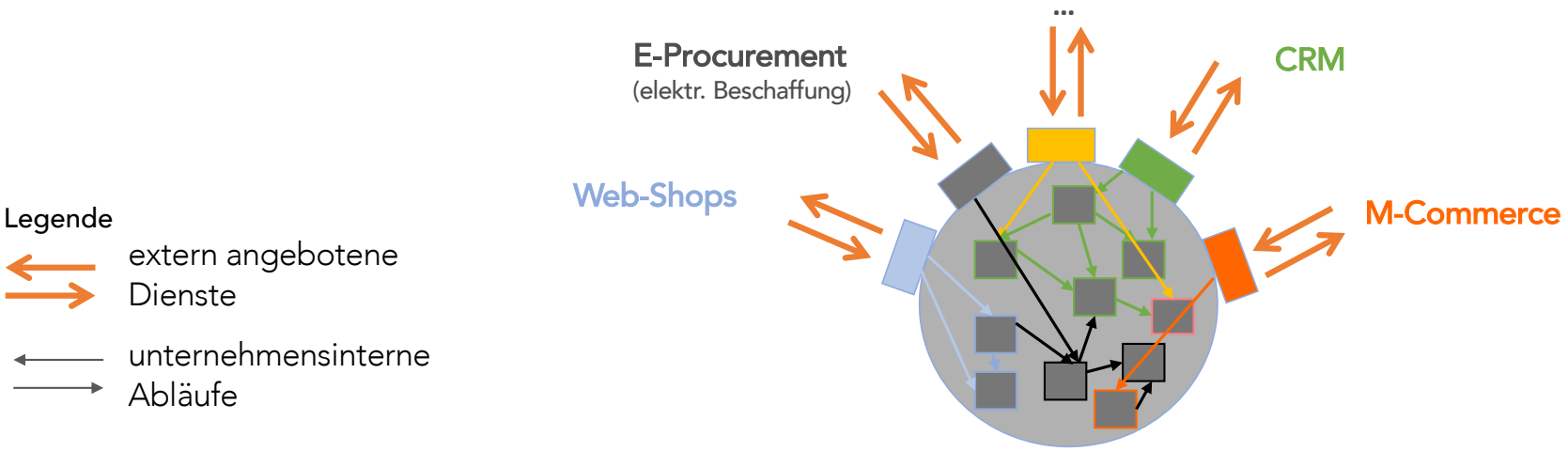


© Prof. Dr. Sabine Sachweh

Motivation

Unternehmensinterne Abläufe und externe Angebote

- Software Systeme müssen den ständig neuen Herausforderungen bzw. ständig wechselnden „Trends“ gewachsen sein



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Serviceorientierte Architektur (SOA)

we focus on students

Begriffsdefinitionen (viele verschiedene, teils gegensätzliche Definitionen

- Der Begriff „serviceorientierte Architektur" wurde **1996 von** dem Marktforschungs-unternehmen **Gartner** erstmalig genutzt in der Research Note SPA-401-068.
- **SOA ist ein Paradigma für die Organisation und Nutzung verteilter Funktionalitäten**, die dezentral verwaltet sein können. → Standardisiert [1)]
- SOA ist also ein Software Design Ansatz mit dem Grundgedanken, dass **Geschäftsprozesse im Vordergrund** stehen. [2)]
- SOA soll die **flexible Anpassung von Geschäftsprozessen** an die sich ständig wandelnden Umweltbedingungen der Unternehmens ermöglichen. [2)]
- Dies erfolgt durch eine **Architektur grobkörniger Services** die miteinander **kommunizieren** und **flexibel zusammengestellt** werden können. [2)]

1) [http://docs.oasis-open.org/soa-rm/soa-ra/v1.0/cs01/soa-ra-v1.0-cs01.pdf]
2) [http://www.till-rausch.de/assets/baxml/soa_akt.pdf]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Dimensionen einer SOA

we focus on students

1. Ein unternehmensweiter IT-Ansatz

- Fokus auf Business/IT-Alignment
- Modularisierung von Anwendungen
- Von der Anwendungs- zur Service-Landschaft
- Governance-Fokus

2. Eine Softwarearchitektur

- Services mit formal beschriebenen Schnittstellen
- Wiederverwendung durch Aufruf
- Trennung von Schnittstelle und Implementierung
- Realisierbar mit beliebigen Technologien

3. Das Konzept hinter SOAP/WSDL/WS-*

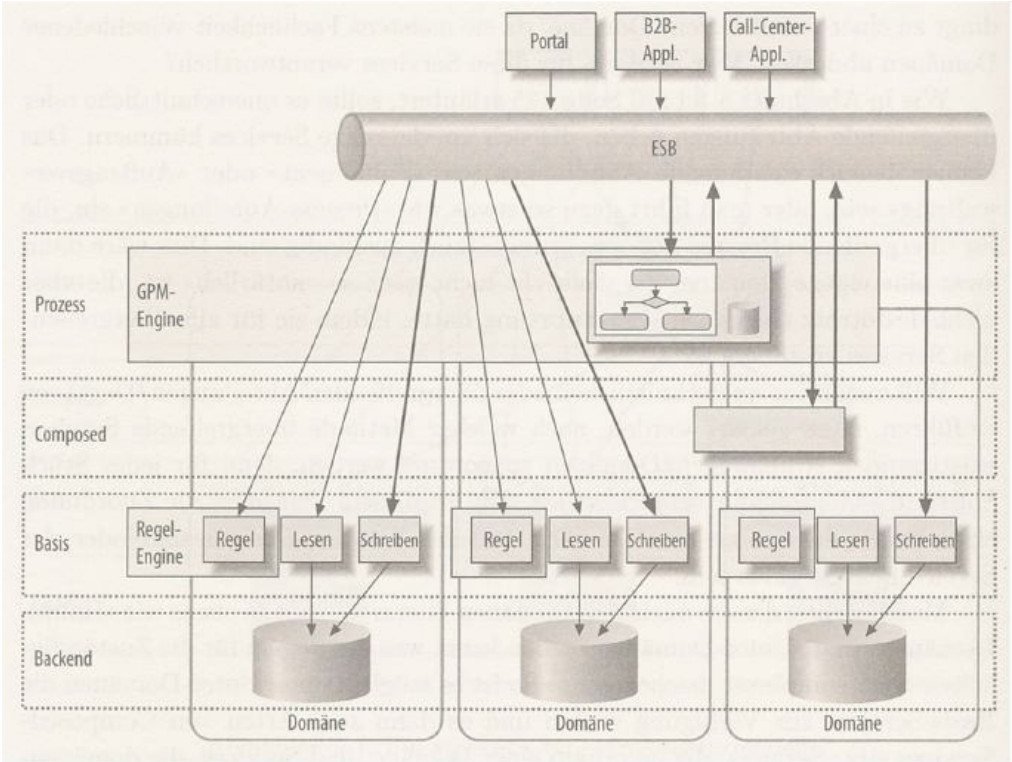
- Selbstbeschreibende Nachrichten (XML, XSD)
- Envelope/Header/Body-Konzept
- Dynamic lookup via Registry
- Policy-getriebene Konfiguration
- Anforderungsgetrieben kombinierbare Standards und Spezifikationen

© Prof. Dr. Sabine Sachweh

Eine Softwarearchitektur

Zerlegen und Verdrahten von Anwendungen

- Geschäftsobjekte werden von den **Backend-Systemen** der Domänen verwaltet
- **Basis Services** werden von der anbietenden Domäne verwaltet
- (Geschäfts-)Logik auf Ebene der Basis-Services wird mit einer **Rule-Engine** implementiert
- **Composed Services** sind Domänen-übergreifend definiert
- (Geschäfts-)**Prozess-Services** werden von einem GPM-Tool bzw. einer **Process Engine** verwaltet und ausgeführt



[Nicolai Josuttis (2008) :SOA in der Praxis: System-Design für verteilte Geschäftsprozess]

© Prof. Dr. Sabine Sachweh

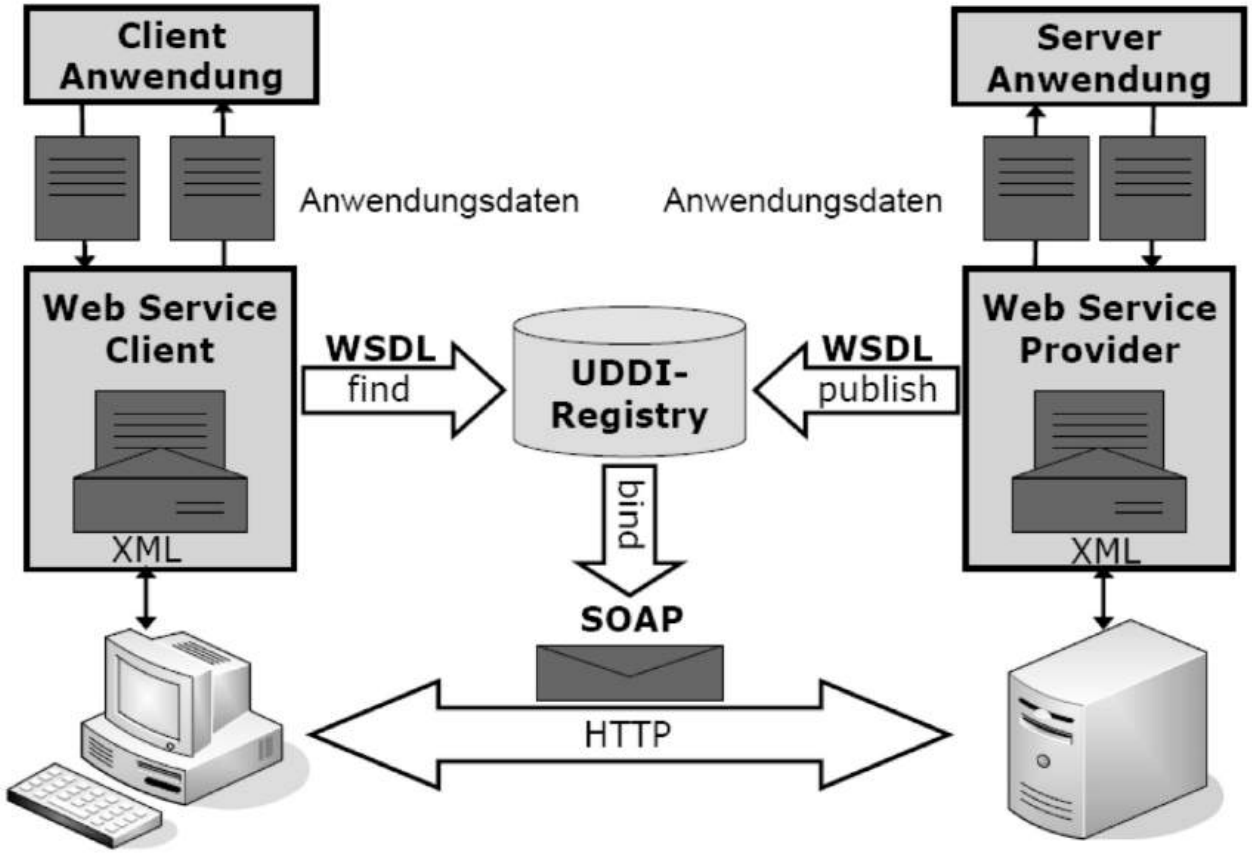
Das Konzept hinter SOAP/WSDL/WS-*

- Mit **WSDL (Web Service Description Language)** wird die Schnittstelle eines Web-Services XML-basiert spezifiziert,
- via **SOAP (früher: Simple Object Access Protocol)** werden Prozedurfernaufrufe übermittelt und
- mit **UDDI (Universal Discovery, Description and Integration)**, einem zentralen Verzeichnisdienst für angebotene Web Services, können andere Web-Services aufgefunden werden.

© Prof. Dr. Sabine Sachweh

we focus on students

Zusammenspiel der Standards



© Prof. Dr. Sabine Sachweh

SOA und Technologie-Bindung

- **Komplizierte SOA-Modelle** und der **unübersichtliche Wald von Standards**, die in Zusammenhang mit Web Services entwickelt werden, können die **zentralen Vorteile** von SOA verdecken.
- Die **wahren Stärken des SOA-Modells** liegen jedoch
  - in der **einfachen Erweiterung der üblichen Schichtung** von Anwendungen und Architekturen **um eine Service- und Orchestrierungsschicht** und
  - in der Möglichkeit, mit Hilfe von SOA, **Ordnung in eine bestehende heterogene Systemlandschaft zu bringen**.
- Das **SOA-Modell sollte technologieneutral formuliert werden** und gleichzeitig den Einsatz einzelner Produkte oder der gesamten SOA-Produktpalette eines oder mehrerer Hersteller gestatten.

© Prof. Dr. Sabine Sachweh

SOA und Technologie-Alternativen

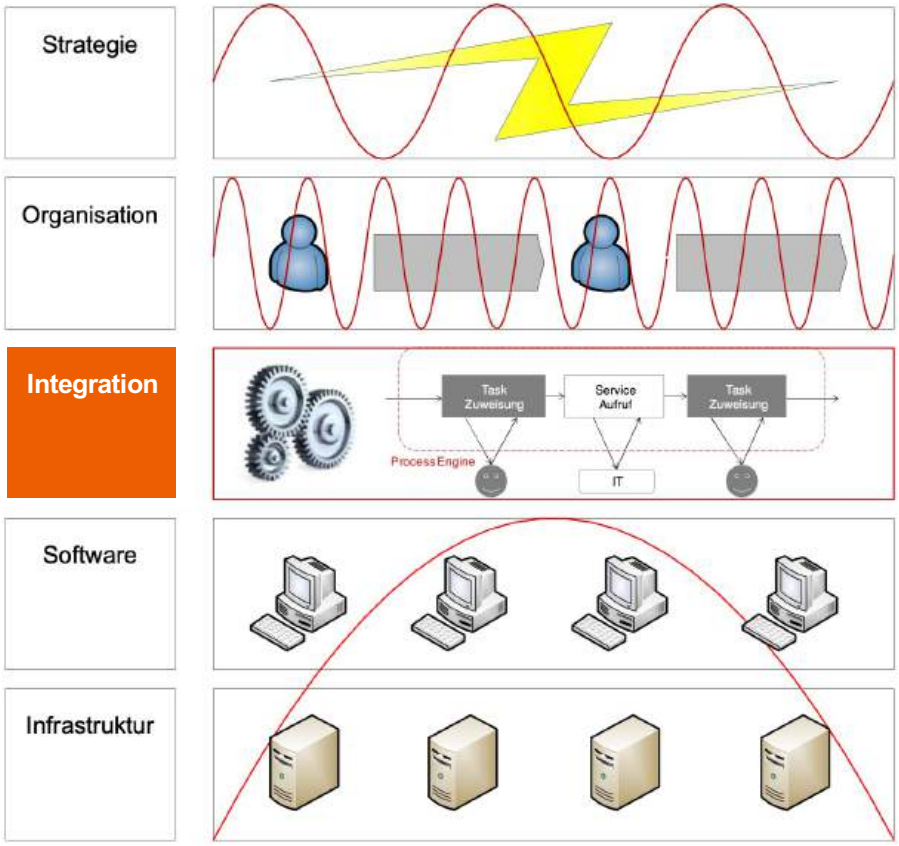
- Die **Ideen der SOA** können auch übernommen werden **ohne WSDL, SOAP und UDDI** zu verwenden.
- Was müssen auch **keine Infrastruktur-Plattformen und ESBs** eingesetzt werden
- Statt **SOAP-Webservices** können auch REST-Webservices verwendet werden, um Anwendungen zu integrieren.
- Somit lassen sich SOAs auch **weniger Technologie-fokussiert** umsetzen.

© Prof. Dr. Sabine Sachweh

Service ≠ Service ≠ Service

we focus on students

- Strategie-Ebene: 1-2 Jahre
- Prozess-Ebene: 3-6 Monate
- Software-Ebene: 6-10 Jahre



[In Anlehnung an: Prof. Dr. Robert Winter, Universität St. Gallen]

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Architekturstile im Zeitverlauf I

Broker

KBSE

SOA

Fachhochschule
Dortmund
University of Applied Sciences and Arts

we
focus
on
students

Softwaretechnik 2

Architekturstile im Zeitverlauf II

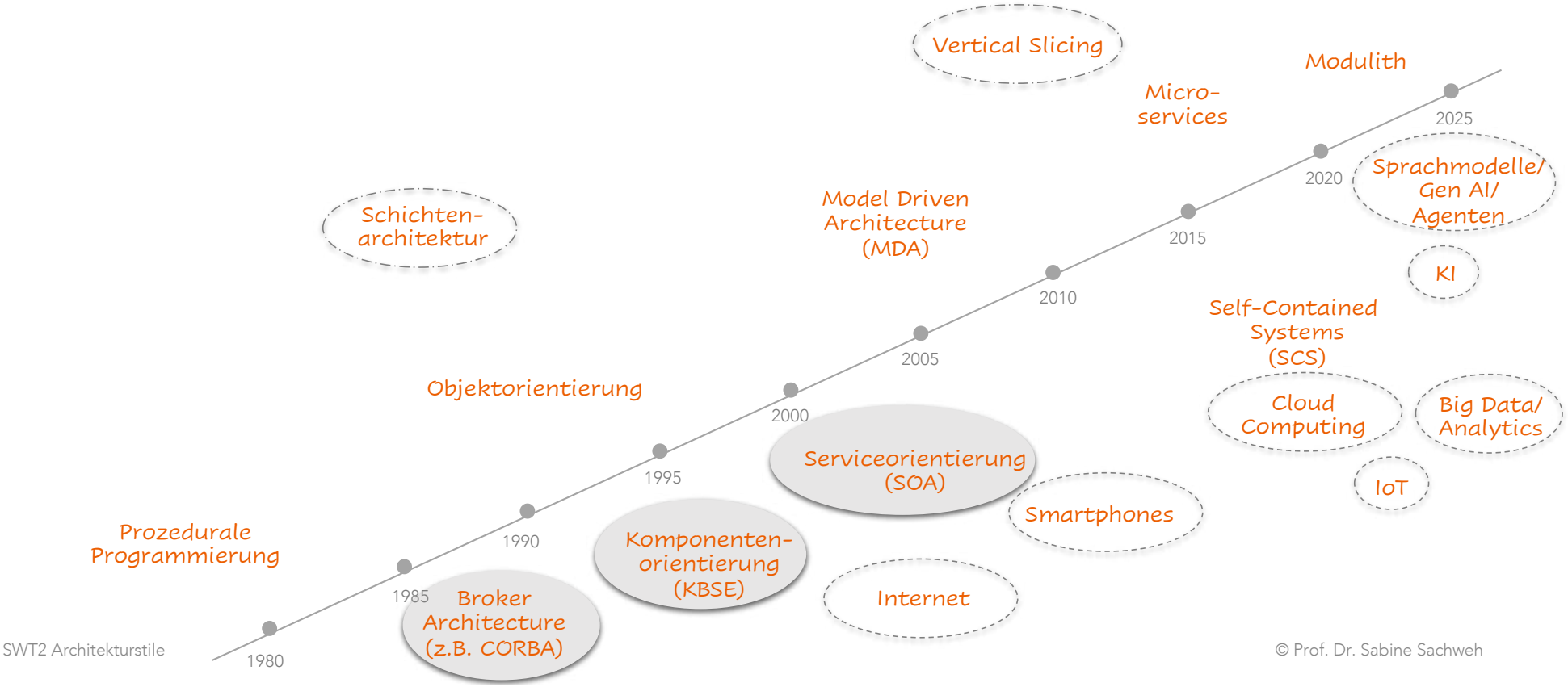
© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Architekturstile im Zeitverlauf II

Microservice Architecture (MSA)

Self-contained Systems (SCS)

Modulith

Fachhochschule
Dortmund
University of Applied Sciences and Arts

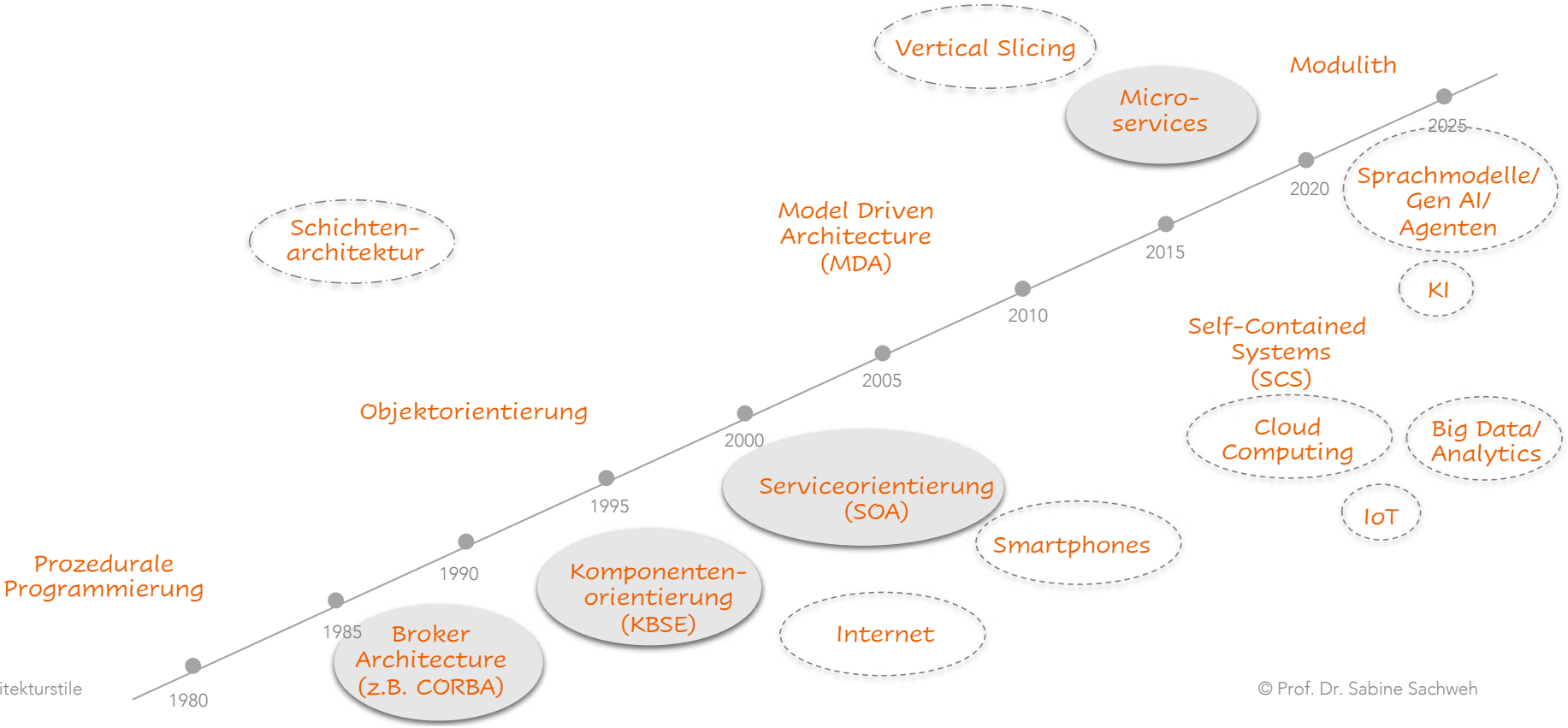
Microservice Architecture (MSA)

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends



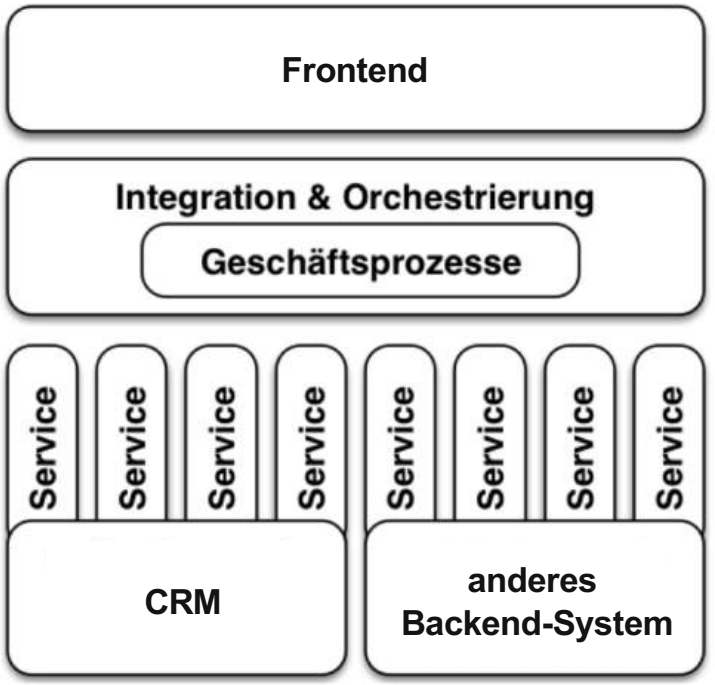
© Prof. Dr. Sabine Sachweh

SOA

Rückblick

we focus on students

- In einer SOA-Landschaft **jede Anwendung in Services** aufgeteilt, auch solche, die schon vor Einführung der SOA existierten.
- Beispielsweise könnte ein **Customer Relationship Management** (CRM) Dienste zum **Anlegen von Kunden**, zum **Abfragen von Informationen** über Kunden oder zum **Anlegen neuer Interaktionen** mit einem Kunden anbieten.
- Das CRM ist ein Backend-System und damit eine **Deployment-Einheit**, da alle Services nur gemeinsam in Produktion gebracht werden können.



[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems", 2016 https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

SOA

Rückblick

- Die **Kommunikation der Services** muss eine **einheitliche Technologie** nutzen, über die alle Services erreichbar sind.
  - asynchron: beispielsweise ESBs (Enterprise Service Bus)
  - synchron: beispielsweise SOAP, das u.a. die Kommunikation über HTTP erlaubt

- Die **Integration und Orchestrierung** wird üblicherweise in **einer eigenen Schicht** umgesetzt. Sie implementiert Geschäftsprozesse mit Hilfe der Services.
  ⇒ Wenn ein neuer Service in den Prozess integriert werden soll, ist dazu nur eine Änderung des Prozesses notwendig. Die Services bleiben unverändert.

- Verschiedenste Frontends können die Orchestrierung und die einzelnen Services nutzen, so dass schnell dedizierte Frontends angeboten werden können.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems“, 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

SOA

Vorteile

we focus on students

Vorteile (u.a.)

- Services können in unterschiedlichsten **Geschäftsprozessen** und/oder unterschiedlichsten **Frontends wiederverwendet** werden.

- **Änderungen** an den **Geschäftsprozessen** sind möglich, ohne dass dazu die Services geändert werden müssen. Es muss lediglich die Orchestrierung angepasst werden.
  → **Flexibilität + Ressourcenschonung**

Herausforderungen (u.a.)

- Allerdings ist eine SOA mit einem **erheblichen Aufwand** und **erheblichen Investitionen** verbunden:
  - Alle IT-Systeme müssen **in Services aufgeteilt werden**, die im Netzwerk ansprechbar sind.
  - Alle **Geschäftsprozesse in der Orchestrierung-Schicht** umgesetzt werden.
  - **Benutzungsschnittstellen müssen neu gestaltet** werden – üblicherweise als Portal.
- Die intendierte Flexibilität kann nur erzielt werden, wenn notwendige **Änderungen** in den Services, der **Orchestrierung oder im Portal isoliert** werden können.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems", 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

SOA

Lessons learned

- Auf der Ebene einer unternehmensweiten IT bietet sich eigentlich immer zumindest eine **Aufteilung in Services** an.
- Dazu ist eine **einheitliche Kommunikationstechnologie** notwendig.
- Es ist aber fraglich, ob eine **vollständige SOA** mit einer **Aufteilung in eine getrennte Orchestrierung und Portal** sinnvoll ist, denn das ist sehr aufwändig und bietet kaum einen Mehrwert.
- Anforderungen **moderner internetbasierter Dienstleistungen** wie beispielsweise Streaming wird dieser Architekturstil vor allem in Bereichen wie **Skalierung**, **Robustheit**, oder **Entwicklungsgeschwindigkeit** nicht gerecht.

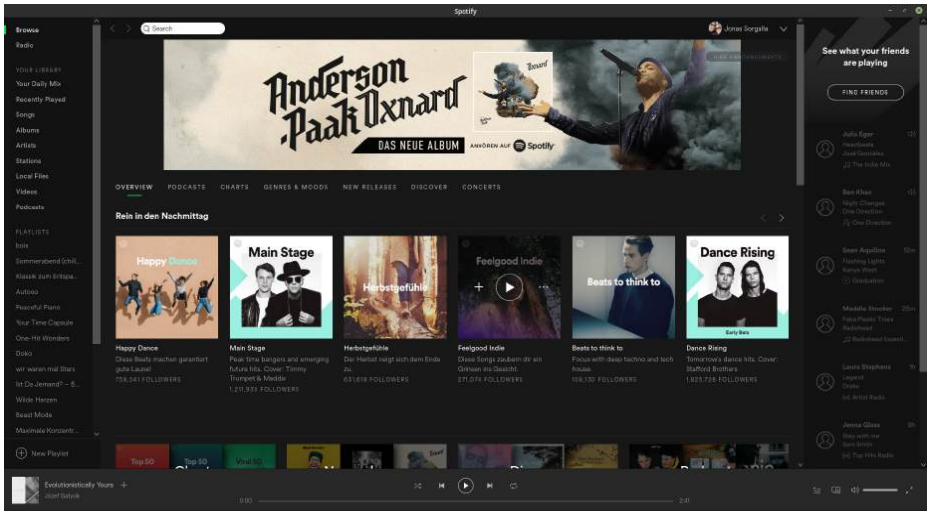
© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Beispiel Spotify

we focus on students

Streaming-Dienstleister heben Skalierung auf ein anderes Niveau



- 75+ Millionen aktive Nutzer jeden Monat
- 58 Länder
- 20.000 Neue Songs täglich
- 2+ Milliarden Playlisten
- „Saisonales Nutzerverhalten“

Traditionelle Architekturstile stoßen hier an ihre Grenzen!

© Prof. Dr. Sabine Sachweh

Microservices

we focus on students

SOA is dead long live services – Services reloaded

Kernidee

Monolithen werden in unabhängige, separate Prozesse zerschnitten!

⇒ **„Microservices"**

(So benannt seit ca. 2011/2012)

"Microservices are **small, autonomous** services that **work together**."

[S. Newman, Building Microservices, M. Loukides und B. MacDonald, Hrsg. O'Reilly Media, 2015.]

© Prof. Dr. Sabine Sachweh

Microservice: Ursprung

- der Begriff "microservice" wurde im **Mai 2011** auf einem Architekten-Workshop in der Nähe von Venedig diskutiert
- um zu beschreiben, was aus Sicht der Teilnehmer einen üblichen Architekturstil beschrieb, den viele von ihnen kürzlich untersucht hatten
- im **Mai 2012** entschied sich die gleiche Gruppe für den Begriff „Microservices“ als angemessene Bezeichnung
- **James Lewis** von ThoughtWorks präsentierte einige Ideen dazu auf der 33rd Degree, einer Java Konferenz, in Krakow in Polen „Microservices – Java, the Unix Way“
  [http://2012.33degree.org/pdf/JamesLewisMicroServices.pdf]
- **Fred George** (ThoughtWorks) und **Adrian Cockcroft** damals Cloud-Architekt bei Netflix (mit seinem Ansatz:"fine grained SOA“) und **Joe Walnes, Dan North, Evan Botcher** und **Graham Tackley** propagierten parallel diesen neuen Architekturstil

© Prof. Dr. Sabine Sachweh

Begriffsdefinition

we focus on students

James Lewis

Martin Folwer

"In short, the microservice architectural style is an approach to developing a single application as a **suite of small services**, each running **in its own process** and communicating with **lightweight** mechanisms, often an HTTP resource API. These services are built around **business capabilities** and **independently deployable** by fully automated deployment machinery."[2)]

- James Lewis & Martin Fowler

[J. Lewis und M. Fowler, "Microservices", https://martinfowler.com/articles/microservices.html, 2014.]

© Prof. Dr. Sabine Sachweh

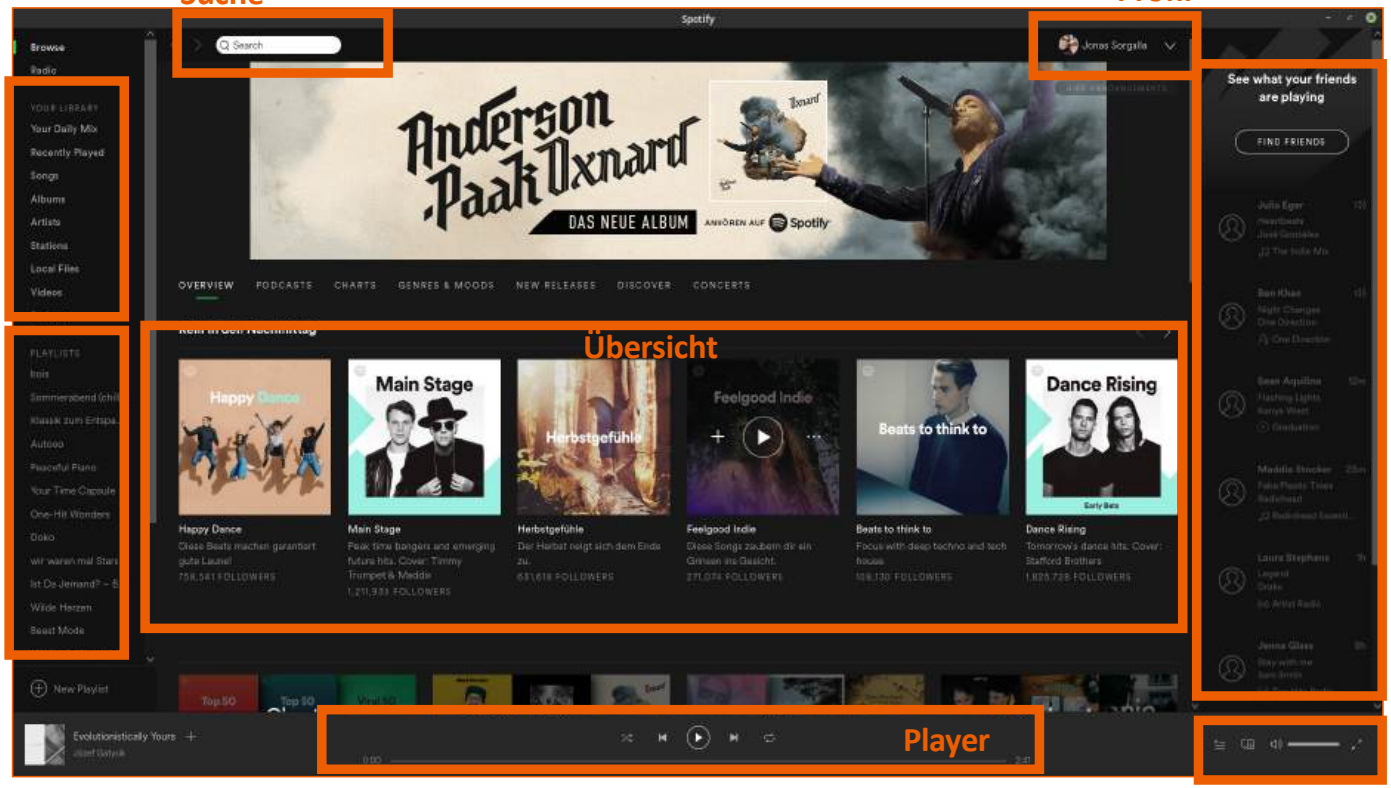
Begriffsdefinition

- dementsprechend sind Microservices definiert ...

  … als Ansatz eine einzelne Anwendung als **Menge von kleinen Services** zu entwickeln, von denen jeder
  - in **seinem eigenen Prozess** läuft und
  - **leichtgewichtig** kommuniziert, häufig über eine HTTP Ressource API
  - diese Services werden ausgerichtet auf **Geschäftsfunktionen** entwickelt
  - sie können **unabhängig depoyed** werden durch ein voll automatisiertes Deployment

Der Gedanke dahinter entspricht weitgehend dem der Unix-Philosophie:
(„**Do One Thing and Do It Well**“)

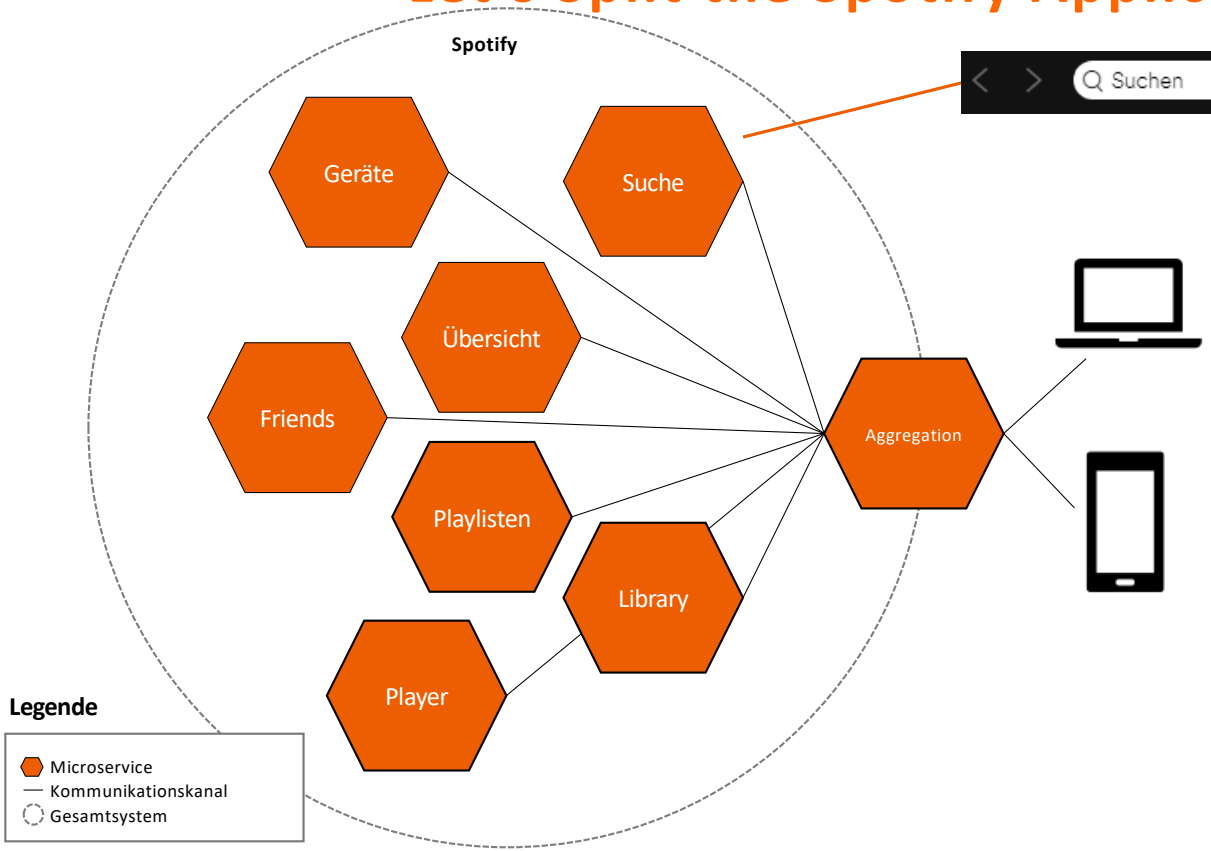
Let‘s Split the Spotify Application



© Prof. Dr. Sabine Sachweh

we focus on students

Let‘s Split the Spotify Application

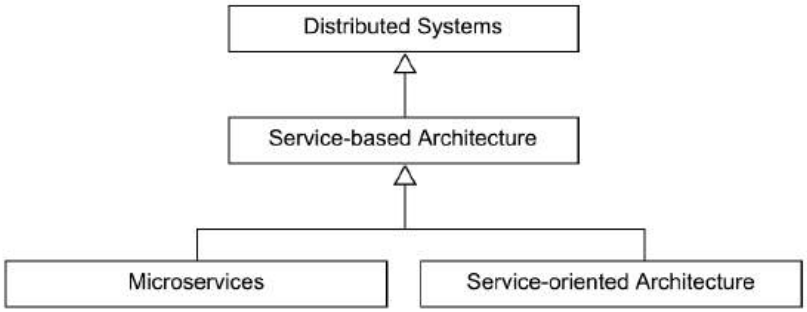


- Jede **Funktion** bzw. **jeder Service** wird als **eigenständige Software** entwickelt → Deployment
- Diese fungieren als **Bausteine** des „großen Ganzen“ bzw. des Systems
- Daten werden über **fest vereinbarte Kommunikationskanäle** zwischen den Bausteinen ausgetauscht (Schnittstellen)
- **Choreographie anstatt Orchestrierung**

© Prof. Dr. Sabine Sachweh

Microservices

Weitere Abgrenzung zur SOA



„Perhaps, Microservice are SOA done right."

- Microservices und SOA haben das **Ziel die Entwicklung von Software zu flexibilisieren**, allerdings auf sehr verschiedene Art.
- Microservices setzen auf einer **anderen Ebene** als SOA an.
  - Microservices dienen zur **Strukturierung einer Anwendung**, während
  - SOA eine Strategie zur **Strukturierung einer gesamten IT eines Unternehmens** ist.

⇒ **SOA und Microservices unterscheiden** sich von der Ausrichtung **fundamental**, auch wenn sie ähnliche Kommunikationsmechanismen nutzen können.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems", 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

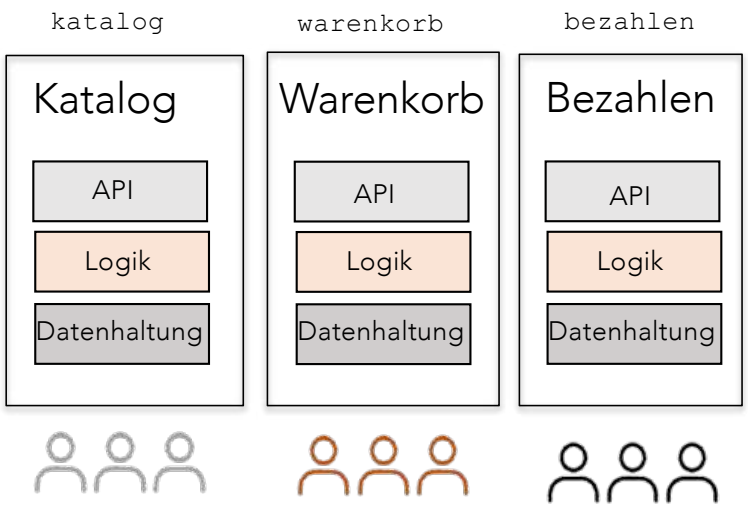
Microservices

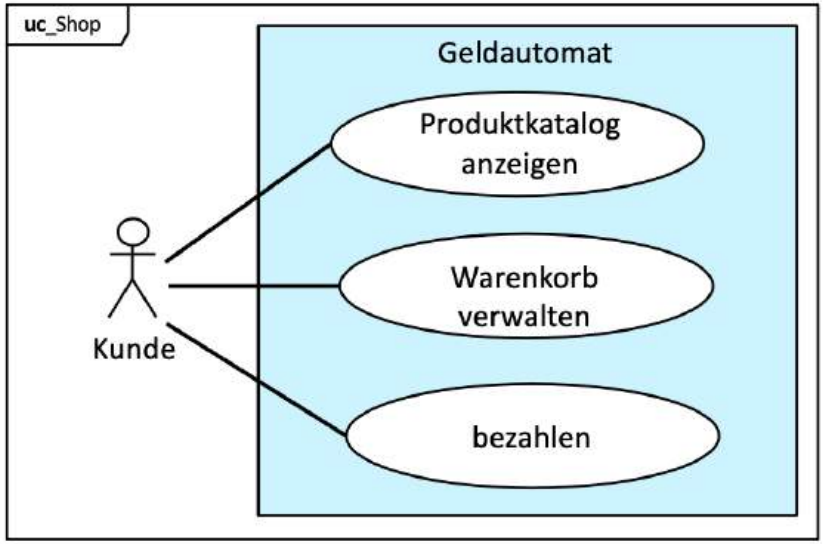
we focus on students

katalog.ui

katalog.service

katalog.data

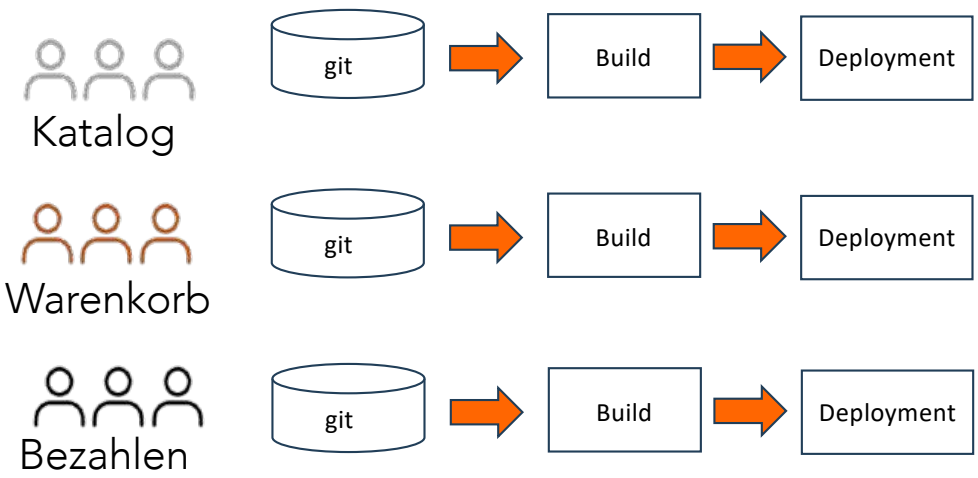


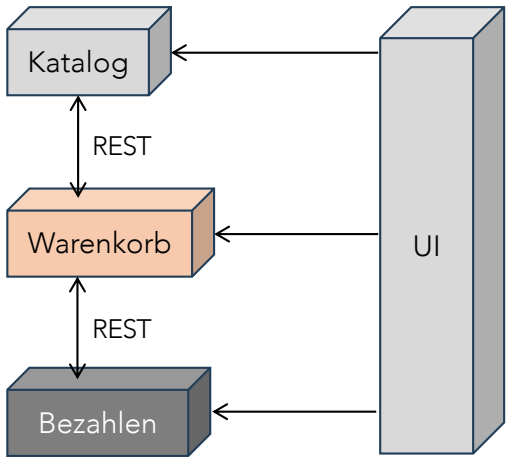


© Prof. Dr. Sabine Sachweh

Microservives

we focus on students



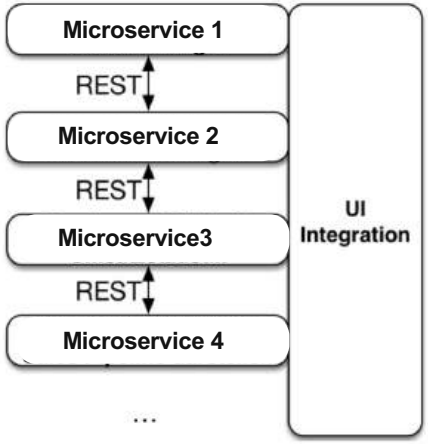


© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Microservices

Weitere Abgrenzung zur SOA



we focus on students

- Microservices können auf ein Projekt beschränkt sein
  ⇒ **keine unternehmensweite Entscheidung**.
- Microservices sind **nur eine Möglichkeit** der Modularisierung. Andere wie beispielsweise Bibliotheken oder andere Mechanismen werden nicht ausgeschlossen.
- Microservices **unabhängig voneinander in Produktion** gebracht werden.
- Jeder Microservice kann in einem **eigenen Prozess**, einer **eigenen virtuelle Maschine** (VM) oder einem **Docker-Container** laufen.
  ⇒ Starke Trennung (Entkopplung) der Microservices.
- Die Microservices müssen **zu einer Anwendung kombiniert** werden, wie beispielsweise
  - über REST-Schnittstellen oder
  - als Teil einer Web-Schnittstelle

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems“, 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Microservices

Ziele

Erhoffte Vorteile durch den Einsatz von Microservices[1)]

- Wiederverwendbarkeit
- Ersetzbarkeit
- Technologische Heterogenität
- Robustheit
- Skalierbarkeit
- Einfache Bereitstellung
- Parallelisierung des Entwicklungsprozesses

1) [S. Newman, Building Microservices, M. Loukides und B. MacDonald, Hrsg. O'Reilly Media, 2015.]

Microservices

Eigenschaften

Microservice-Architekturen legen typischerweise einen Fokus auf **folgende**

Prinzipien[1,2,3]

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

1) [S. Newman, Building Microservices, M. Loukides und B. MacDonald, Hrsg. O'Reilly Media, 2015.]
2) [J. Lewis und M. Fowler, "Microservices", https://martinfowler.com/articles/microservices.html, 2014.]
3) [I. Nadareishvili, R. Mitra, M. Mclarty und M. Amundsen, Microservice Architecture, B. MacDonald und H. Bauer, Hrsg. O'Reilly Media, 2016.]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Single Responsibility

we focus on students

Prinzipien

- [x] Single Responsibility
- [ ] Lose Kopplung, hohe Kohäsion
- [ ] Self-Containment
- [ ] Unabhängigkeit und Autonomie
- [ ] Single Team Ownership

Ein Microservice ist in der Regel für die Realisierung genau einer Geschäftsfunktion verantwortlich.

Entsprechend der Unix-Philosophie:
"Do One Thing and Do It Well"

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Lose Kopplung, hohe Kohäsion

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☒ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

Microservicearchitekturen streben nach einer möglichst losen Kopplung bezüglich:

- **Austauschbarkeit**
  Austausch bzw. Update eines Services sollte sich nicht auf abhängige Services auswirken (bei unveränderten Schnittstellen)
- **Kommunikation**
  Die Kommunikation zwischen Services sollte auf ein Minimum beschränkt werden. Die Kommunikation verläuft durch synchronen oder asynchronen Austausch einfacher Nachrichten.
  - Paradigma: Smart Endpoints, dumb pipes.
  - Typische Protokolle: HTTP (RESTful), AMQP, KAFKA, gRPC, …
  - Typische Nachrichtenformate: JSON, XML, Binary.

Worst Case bei zu enger Kopplung: Verteilter Monolith

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Lose Kopplung, **hohe Kohäsion**

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☒ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

Ein Microservice enthält typischerweise alle zur Umsetzung seiner Geschäftsfunktion relevanten Domänenkonzepte.

- Verringert den Grad der Kopplung.
- Verringert die Wahrscheinlichkeit, dass Änderungen an einem Service Änderungen an anderen Services nach sich ziehen.
- Redundanzen werden in Kauf genommen.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Self-Containment

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☒ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

Neben den relevanten Domänenkonzepten und ihrer Implementierung umfasst jeder Microservice auch alle weiteren zu seinem Betrieb notwendigen Ressourcen, wie bspw.:

- Schnittstellenbeschreibungen
- Datenbanken
- Application- und Web-Container wie Tomcat , Jetty
- Konfigurationen für **OS-Container wie Docker**, Continuous-Integration-Pipelines (z.B. Jenkins-Pipelines) und Build-Management-Tools wie
- Maven oder Gradle
- Ggf. auch Teile der grafischen Oberfläche

Ziel: Verringerung von Abhängigkeiten zwischen den Microservices sowie den zuständigen Teams

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Self-Containment

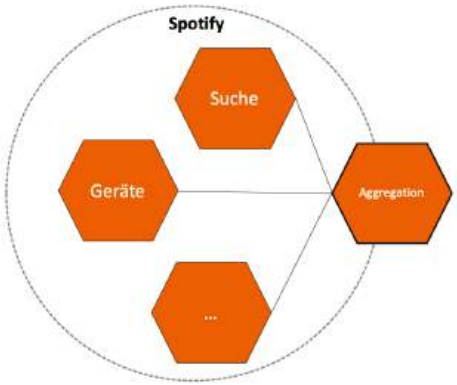
we focus on students

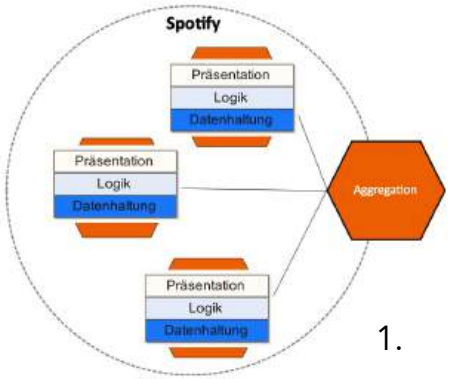
Prinzipien

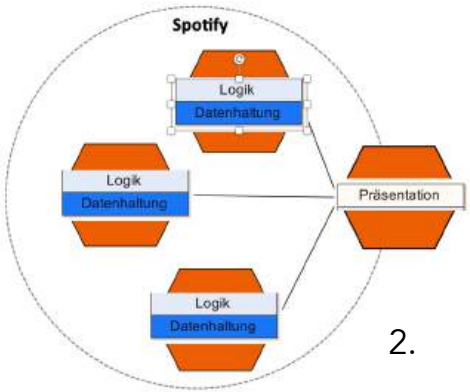
- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☒ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

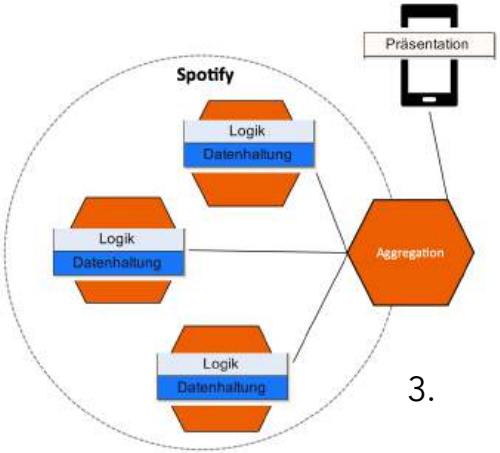
Varianten für die Auslieferung von grafischen Oberflächen

1. Jeder Service hat seine **eigene Präsentationsschicht**
2. Die Präsentation erfolgt **zentral**
3. Die Präsentation erfolgt **komplett clientseitig**









© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Unabhängigkeit und Autonomie

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☒ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

Der Software-Lebenszyklus eines Microservices ist unabhängig von denen anderer Microservices

Dies beinhaltet folgende Phasen:

- Entwicklung
- Test
- Bereitstellung (Deployment)
- Betrieb

"The **golden rule**:
Can you make a change to a service and deploy it by itself without changing anything else?"

Ziel: **Jeder Microservice soll autonom entwickelt, deployed und betrieben werden können.**

[S. Newman, Building Microservices, M. Loukides und B. MacDonald, Hrsg. O'Reilly Media, 2015.]

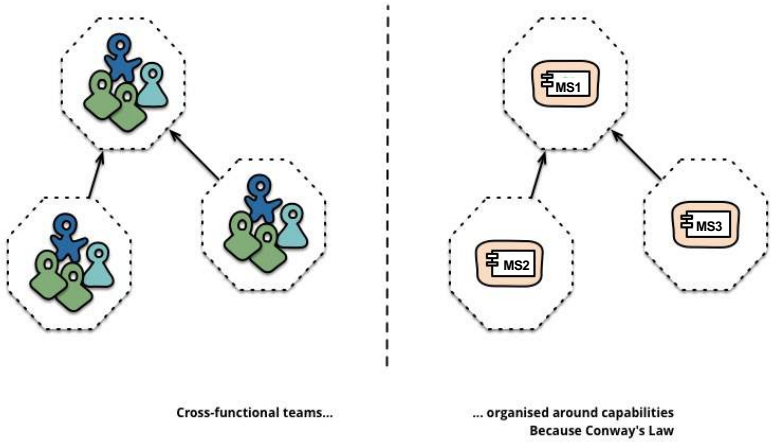
© Prof. Dr. Sabine Sachweh

Single Team Ownership

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☒ Single Team Ownership



[Bild:https://martinfowler.com/articles/microservices.html]

Für jeden Microservice sollte **genau ein Team** verantwortlich sein:[1]

- Das Team ist nicht nur für Entwurf und Entwicklung des Microservices verantwortlich, sondern auch für alle anderen Aspekte (Konfiguration, Deployment, Betrieb)
- Cross-Functional Teams, z.B. DevOps[2]

1) [S. Newman, Building Microservices, M. Loukides und B. MacDonald, Hrsg. O'Reilly Media, 2015.]
2) [I. Nadareishvili, R. Mitra, M. Mclarty und M. Amundsen, Microservice Architecture, B. MacDonald und H. Bauer, Hrsg. O'Reilly Media, 2016.]

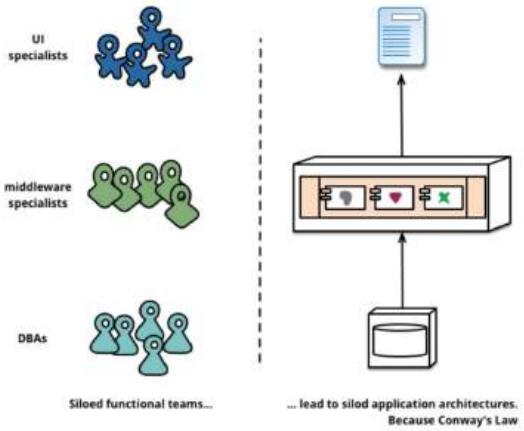
© Prof. Dr. Sabine Sachweh

Single Team Ownership

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☒ Single Team Ownership



- Microservice-Architekturen erfordern in der Regel **mehrere Teams** für die Entwicklung
- Teams können **anhand der technischen Skills organisiert** werden:
  Zum Beispiel können alle Frontend-Entwickler zu einem Team zusammengefasst werden
  ⇒ **Urlaubsvertretung** oder **fachlicher Austausch** sind so sehr einfach.
- Die **Aufteilung der Teams beeinflusst aber die Architektur**.
  ⇒ Gesetz von Melvin Edward Conway

Belegt durch empirische Untersuchungen!

Conway's Law:
Eine Organisation kann nur eine Architektur hervorbringen kann, die ihren Kommunikationsbeziehungen entspricht.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems", 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

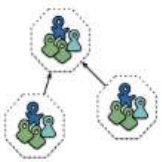
Single Team Ownership

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☒ Single Team Ownership

- Probleme der **Teambildung nach technischen Skills**:
  - Die Arbeit der Teams muss eng koordiniert werden.
  - Eine Verzögerung bei einem Team beeinflusst die anderen Teams.
  - Verantwortlichkeiten können ggf. zwischen den Teams hin- und her verschoben werden



- **Cross-funktionale Teams** sind bei Microservices quasi vorgezeichnet:
  - **Änderung** soll möglichst **nur einen Microservice** betreffen.
  - Team bekommt die **Verantwortung** für **eine bestimmte Fachlichkeit**, die in einem oder mehreren Microservices implementiert ist. (→ 2 Pizza-Teams)
  - Das Team muss daher **technisch breit aufgestellt** sein. Es muss schließlich Backend, Frontend und Datenbank für die Fachlichkeit verantworten.
    → passt zu agilen Teams

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems", 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Single Team Ownership

we focus on students

Prinzipien

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☒ Single Team Ownership

- Vorteile cross-funktionaler Teams:
  - **Unabhängigkeit der Teams** ⇒ wenig Absprachen
  - Jeder Microservice kann mit einem **eigenen Technologiestack** implementiert werden.
  - Jedes Team kann die Technologie nutzen, die **für das jeweilige Problem angemessen** ist.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems“, 2016 https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Microservices

we focus on students

Eigenschaften

Microservice-Architekturen legen typischerweise einen Fokus auf **folgende**

Prinzipien[1,2,3]

- ☐ Single Responsibility
- ☐ Lose Kopplung, hohe Kohäsion
- ☐ Self-Containment
- ☐ Unabhängigkeit und Autonomie
- ☐ Single Team Ownership

1) [S. Newman, Building Microservices, M. Loukides und B. MacDonald, Hrsg. O'Reilly Media, 2015.]
2) [J. Lewis und M. Fowler, "Microservices", https://martinfowler.com/articles/microservices.html, 2014.]
3) [I. Nadareishvili, R. Mitra, M. Mclarty und M. Amundsen, Microservice Architecture, B. MacDonald und H. Bauer, Hrsg. O'Reilly Media, 2016.]

© Prof. Dr. Sabine Sachweh

Integration - Kommunikation

Herausforderung Kommunikation

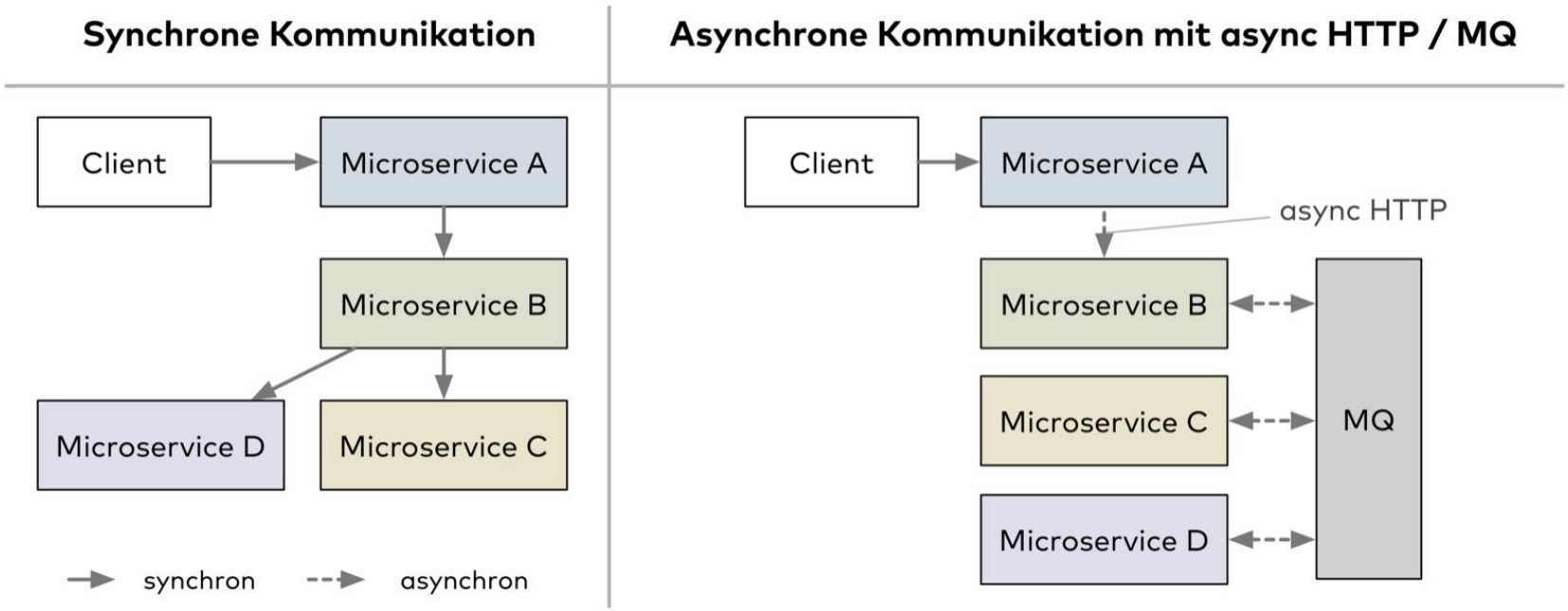
- Die **Probleme** von Microservices sind zum großen Teil mit den **gegenseitigen Netzwerkaufrufen** verbunden.
- Die **einfachste Möglichkeit**, eine monolithische Anwendung zu Microservices zu migrieren, ist die Anwendung aufzuspalten und Methodenaufrufe zwischen Modulen (bzw. Microservices) mit **synchronen Netzwerkaufrufen** zu ersetzen.
- Eine **asynchrone Umsetzung** der Kommunikation bietet allerdings Vorteile wie eine **geringere Latenz** und eine **losere Kopplung** der Microservices.
  ⇒ Die Schwierigkeit ist, sich von der **Frage-Antwort-Interaktion** loszulösen.

[H. Prinz: „Brauchen asynchrone Microservices und SCS ein Service Mesh?", 2020
https://www.innoq.com/de/articles/2020/02/service-mesh-asynchrone-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Integration - Kommunikation

Herausforderung Kommunikation



[H. Prinz: „Brauchen asynchrone Microservices und SCS ein Service Mesh?", 2020
https://www.innoq.com/de/articles/2020/02/service-mesh-asynchrone-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Bestimmung der Servicegröße

we focus on students

- **Risiken** bei **zu kleinen oder zu großen Services**
  - Netzwerkauslastung,
  - Komplexität bei Transaktionen,
  - Infrastrukturaufwand

- **Wie groß ist ein Microservice** bzw. wie groß ist eine „Geschäftsfunktion“?
  - Zeilen an Code → ?
  - Komplexitätsmetriken → ?
  - Anzahl an Testfällen → ?

- Es existiert **kein quantitatives Maß** für die „richtige“ Größe!

© Prof. Dr. Sabine Sachweh

we focus on students

Bestimmung der Servicegröße

Mögliche (unscharfe) Kriterien zur Bestimmung der Größe[1]

- **Geschäftsfunktion**
  Der Microservice sollte alle zur Realisierung seiner Geschäftsfunktion relevanten Domänenkonzepte umfassen.
- **Teamgröße**
  Es sollte nicht mehr als ein Team notwendig sein, um einen Microservice zu entwickeln. Als Teamgröße werden häufig 5 – 7 Personen angegeben („Two-Pizza-Teams").
- **Verstehbarkeit**
  Ein Microservice sollte zur Gänze von einem einzelnen Entwickler verstanden werden können.
- **Ersetzbarkeit**
  Ein Microservice sollte leicht ersetzbar sein („Two-Week-Implementation").

1) [Eberhard Wolff: Das Microservices-Praxisbuch, Hrsg. dpunkt.verlag, 2018.]

© Prof. Dr. Sabine Sachweh

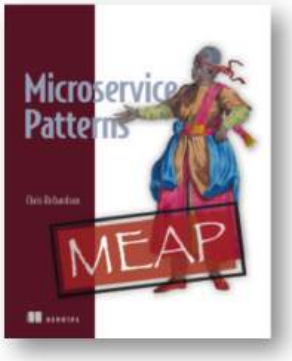
we focus on students

Entwurfsmuster in Microservicearchitekturen

- In der Regel werden bestimmte **Geschäftsfunktions-unabhängige Services** benötigt, sogenannte **infrastrukturelle Services**.

⇒ wiederverwendbare, bewährte Lösungen für die Erstellung von Microservicearchitekturen

(→ Entwurfsmuster) [https://microservices.io/patterns/]



Chris Richardson:
Microservice Patterns.
Hrsg. Manning, 2018.

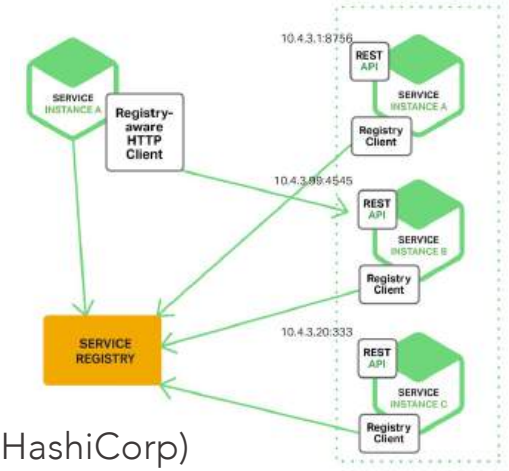
<u>Beispiele:</u>
Service Discovery, API Gateway,
Load Balancer, Cuircuit Breaker,
Authentication,Config Storage …

© Prof. Dr. Sabine Sachweh

we focus on students

Service Discovery

- Woher wissen die einzelnen Services eigentlich, wie sie die anderen Services erreichen können?
- **Service Discovery** ist vergleichbar mit einer SOA-Registry.
  - Services **melden sich an** einer zentralen Stelle unter einem Alias an.
  - Andere **Services können diesen Alias** bei der Service Discovery **anfragen** und erhalten die tatsächliche Adresse.
- **Client-side Service Discovery**
  - Netzwerkadresse einer Serviceinstanz **wird automatisiert bei einer Service Registry** registriert.
  - Sollte die Serviceinstanz terminieren, wird der "Tod" des Service mittels **Heartbeat-Mechanismus** detektiert.
  - Services können mithilfe einer bereitgestellten API die Adresse anderer Serviceinstanzen bei der Registry anfragen.
  - Typische Vertreter: Eureka (Spring Cloud bzw. Netflix OSS) oder Consul (HashiCorp)

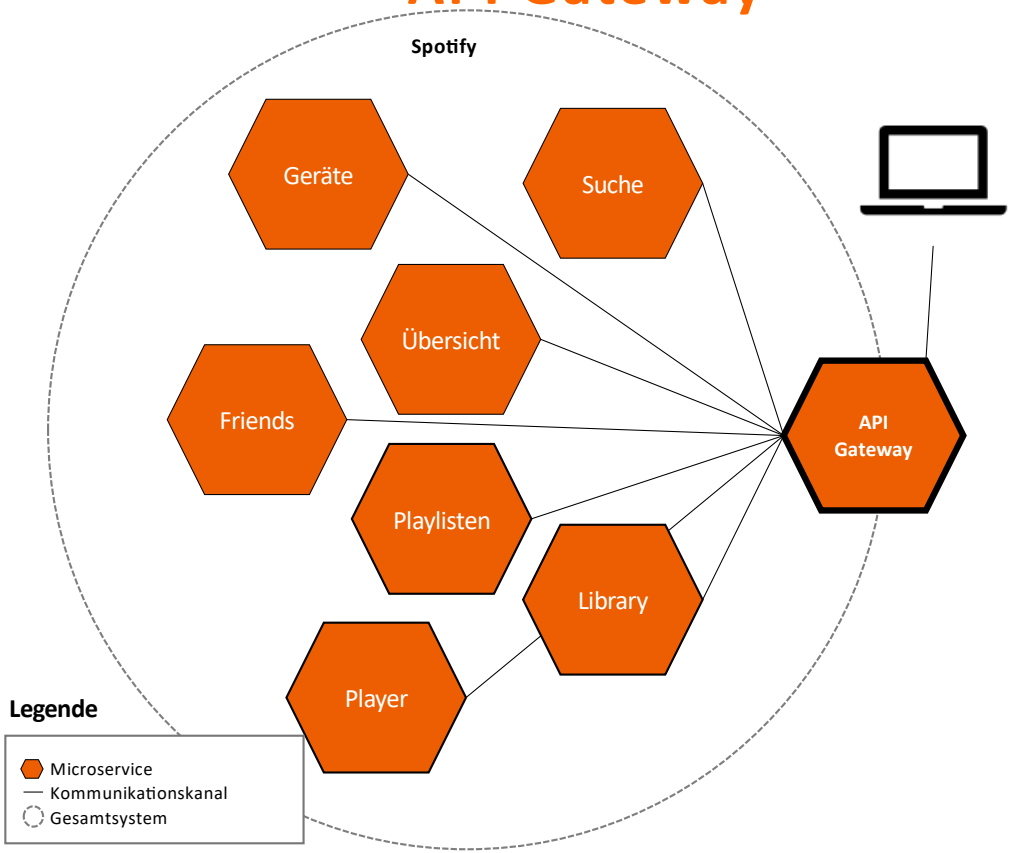


Bildquelle : NGINX,
https://www.nginx.com/blog/service-discovery-in-a-microservices-architecture/

© Prof. Dr. Sabine Sachweh

we focus on students

API Gateway



Das **API Gateway** fungiert für Clienten als zentraler Zugangspunkt zum System

- Für Clienten sind die hintergelagerten Services versteckt, d.h. ein Client merkt u.U. gar nicht, dass es sich um eine Microservicearchitektur handelt.

- API Gateways werden teilweise um weitere Funktionalitäten ergänzt
  - Load Balancing
  - Security
  - …

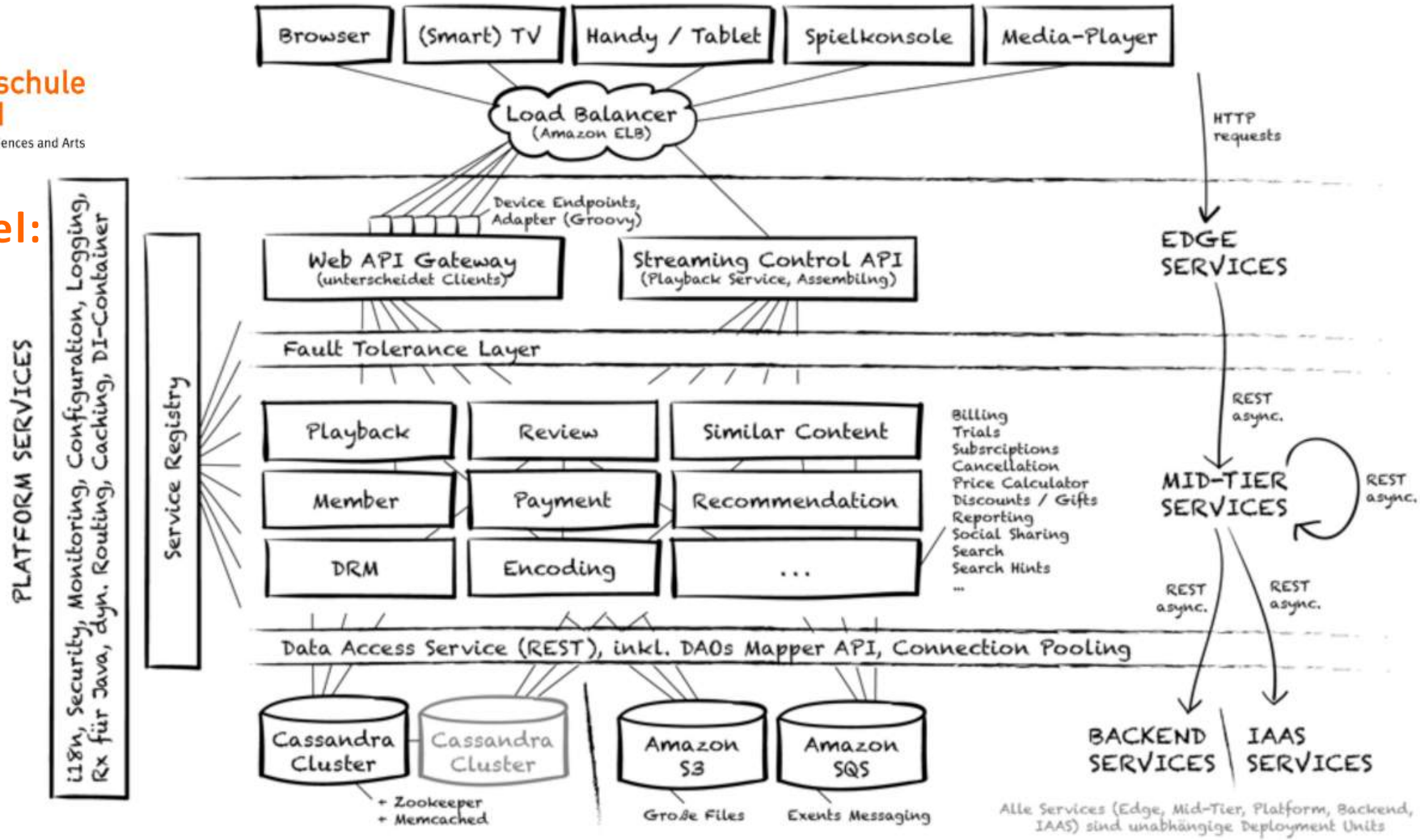
© Prof. Dr. Sabine Sachweh

Anwendungsbeispiele

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Beispiel: Netflix



[Stefan Zörner: Drei zentrale Entwurfsfragen bei vertikalen Anwendungsarchitekturen, Java Forum Nord. Hannover, 2018.]

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

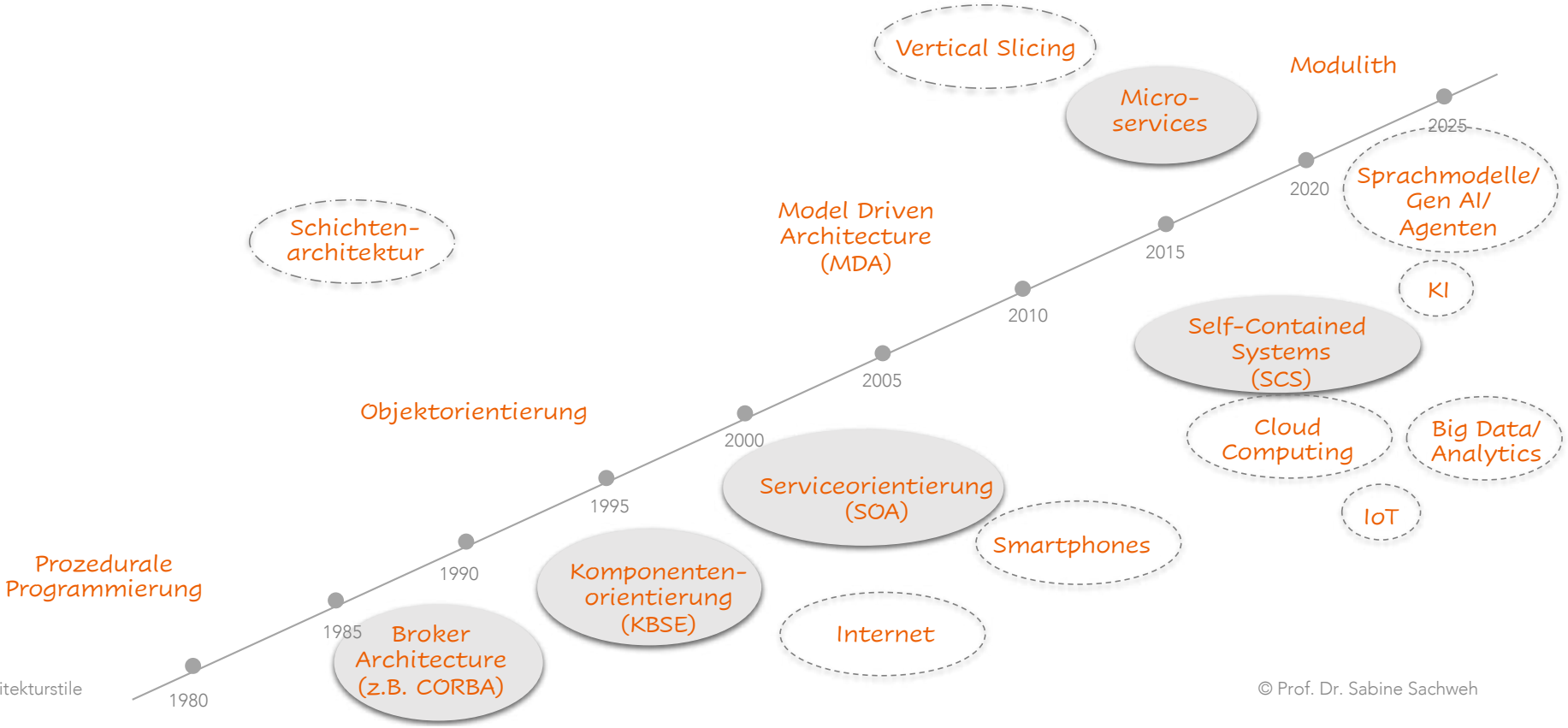
Self Contained Systems (SCS)

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

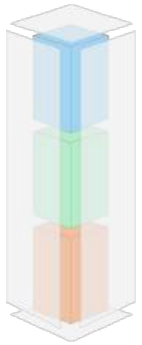
Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends



© Prof. Dr. Sabine Sachweh

Self-contained System (SCS)



- **Microservices** enthalten **viele Freiheitsgrade**, wie beispielsweise die Größe der Services, die Art der Kommunikation und Integration oder das Schneiden der Services
- **Self-contained Systems** (SCSs) sind Microservices mit einer Reihe konkreter **Festlegungen**:
  - Jedes SCS ist eine **eigenständige Web-Anwendung** inkl. Daten, Logik und Code zur Darstellung der Web-Schnittstelle.
  - Jedes SCS soll seine **eigene UI** haben und sich auch **keinen Geschäftscode** mit anderen SCS **teilen**.
  - Ein SCS **kann** auch eine **Service-API** haben, um die Logik für andere SCS oder mobile Clients anzubieten.
  - Jedes SCS wird von **einem Team** verantwortet.
  - Die **Kommunikation** mit Fremdsystemen und anderen SCSs ist nach Möglichkeit **asynchron**.
    → Entkopplung

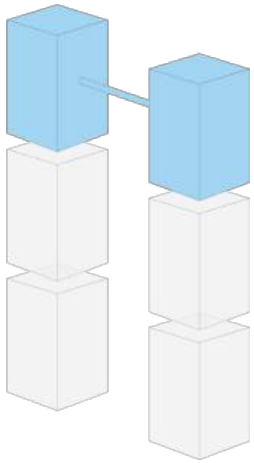
[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems“, 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Self-contained System (SCS)

Integration



we focus on students

- SCSs können zu Anwendungen zusammengesetzt werden.

- Neben der asynchronen Kommunikation wird primär eine Integration auf Ebene der Weboberflächen empfohlen. Beispielsweise über
  - **Weblinks**, die ein Nutzer allerdings explizit anklicken muss; oder über
  - **JavaScript-Code**, der eine Seite nach bestimmten Links scannt und die Links durch den Inhalt der referenzierten Seite ersetzt; oder durch
  - **Features von Web-Servern** wie Server-Side Includes, d.h. der Web Server ersetzt selbst Teile der Web-Seite durch die referenzierten Inhalte.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems“, 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Self-contained System (SCS)

Mikro- und Makro-Architektur

we focus on students

- SCS können größer als Microservices sein (→ komplette Web-Anwendung)
- ein SCS kann intern **aus mehreren Microservices zusammengesetzt sein**
  ⇒ mehrere kleinere Deployment-Einheiten
  ⇒ Komplexität steigt, weil diese Instanzen alle verwaltet bzw. betrieben werden müssen
  ⇒ Aufteilung ist also ein Trade-Off
- Bei der Architektur eines SCS unterscheidet man die Mikro- und Makro-Architektur:
  - **Mikro-Architektur** ⇔ Entscheidungen, die jedes Team für sein SCS selber treffen kann.
  - **Makro-Architektur** ⇔ Entscheidungen, die global für alle Teams und SCSs festgelegt werden.
- Nur **wenige Entscheidungen** müssen zwingend in der **Makro-Architektur** getroffen werden wie beispielsweise das Protokoll für die Kommunikation der SCSs untereinander oder die Technologien für die UI-Integration.

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems", 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Self-contained System (SCS)

Abgrenzung gegenüber Microservices

we focus on students

- Beide Ansätze:
  - Aufteilung eines Systems in kleinere (Service-)Einheiten auf.
  - Ein Team verantwortet Komponente (Microservice oder SCS)
- SCSs sind im Vergleich zu Microservices eher grobgranular
  → komplette Webanwendungen
- SCS fordern noch stärker die lose Kopplung der Komponenten

[Eberhardt Wolff: „Services: SOA, Microservices und Self-contained Systems“, 2016
https://www.innoq.com/de/articles/2016/11/services-soa-microservices-scs/]

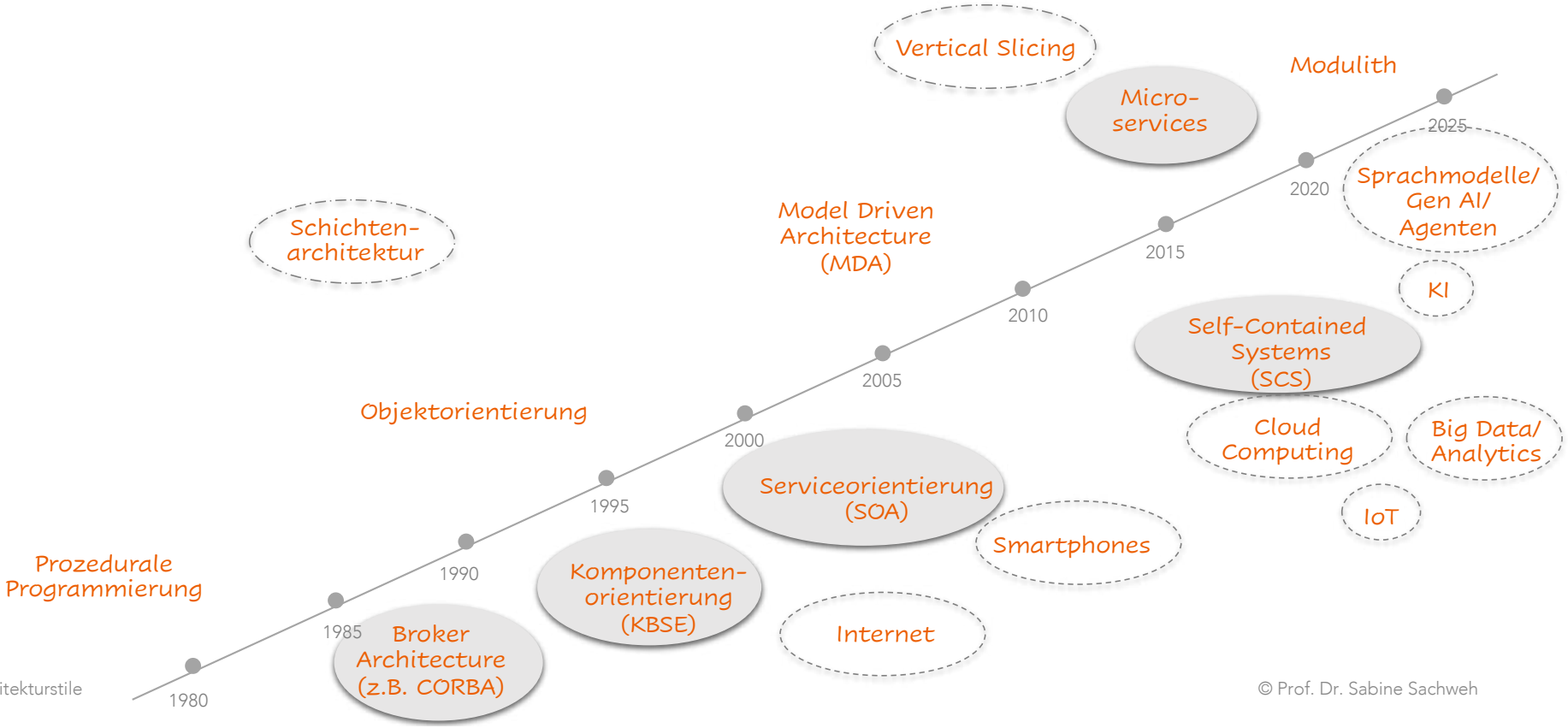
© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Microservices/SCS ⇔ Silver-Bullet?

we focus on students

Eigenschaft	Vorteil	Nachteil
Heterogenität	**Flexibilität** in Technologie-Wahl	Mehr Know-how, **mehr Komplexität**, höhere Betriebskosten
Resilienz (Widerstandsfähigkeit)	Sicherheit gegen Infrastruktur-Ausfall	Muss explizit dafür programmiert werden („Circuit Breaker" Pattern, Auto-Reconnect, etc.)
Skalierbarkeit	Höhere Entwickler-Effizienz, **höhere Runtime-Performance**	**Erhöhte Komplexität**, Service Discovery, schwierigeres Monitoring
Easy Deployment	Jeder einzelne Microservice leichter installierbar/upgradbar	Gesamtanwendung hat **viele Abhängigkeiten**
Fachliche Ausrichtung	Stärkerer **Fokus auf fachliche Einheiten**	Eventuell **mehrere Tech-Stacks** notwendig
Komponierbarkeit	Funktionalitäten **flexibel** zusammenbaubar	**Erhöhte Komplexität** durch Orchestrierung oder Choeographie
Wiederverwendbarkeit	**Funktionalitäten** in mehreren Anwendungen, **nur einmal pflegen**	Alle Anwendungen gleichzeitig betroffen
Austauschbarkeit	Funktionalitäten getrennt austauschbar (Update/Upgrade) → **Flexibilität**	Alle Anwendungen gleichzeitig betroffen

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

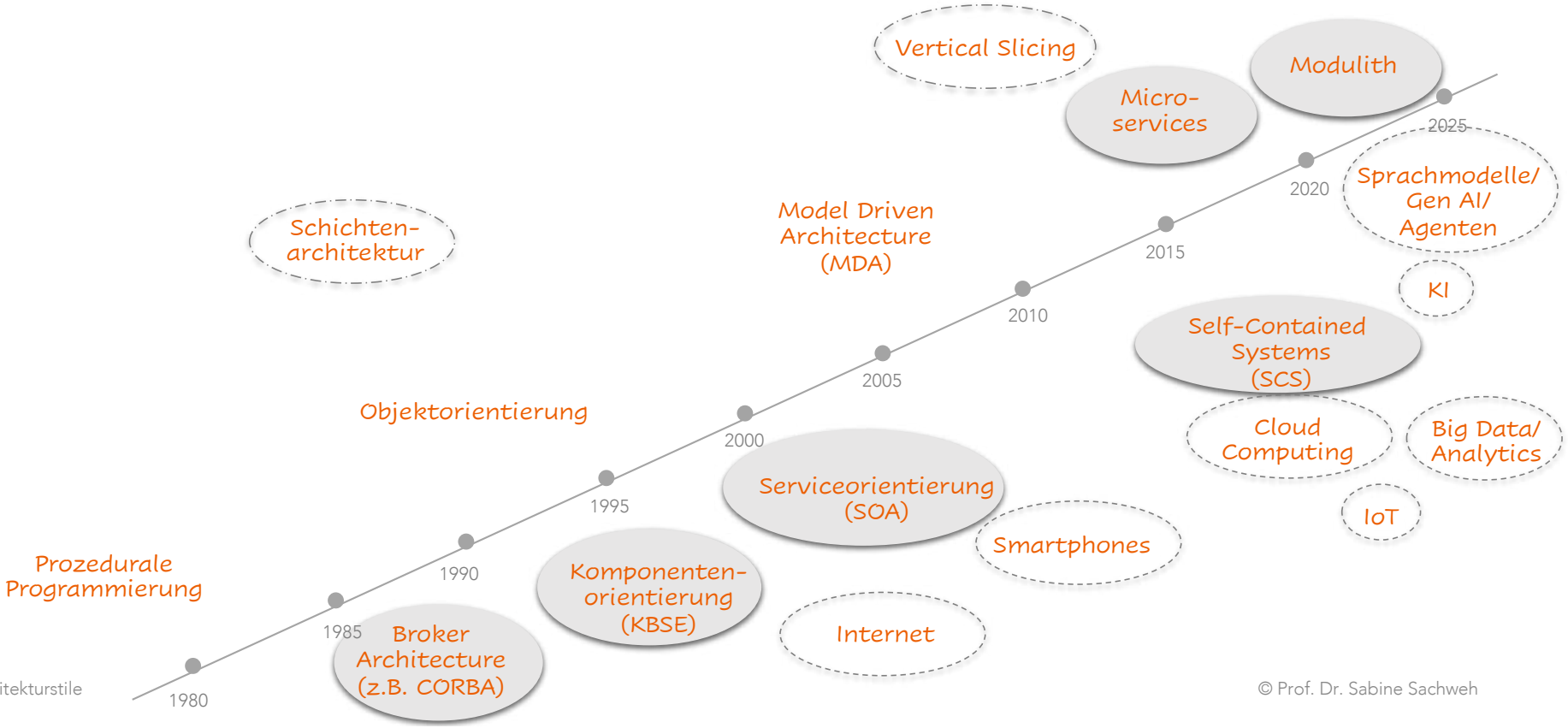
Modulith

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

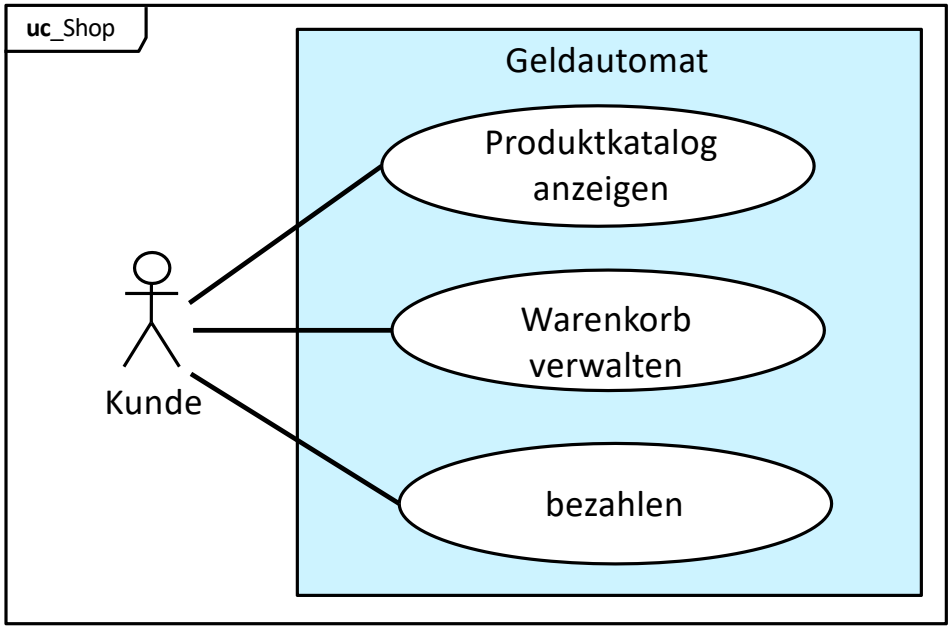
Entwicklungs- und Architekturtrends im Zeitverlauf

Unvollständige Auswahl großer Softwaretechnik-Trends



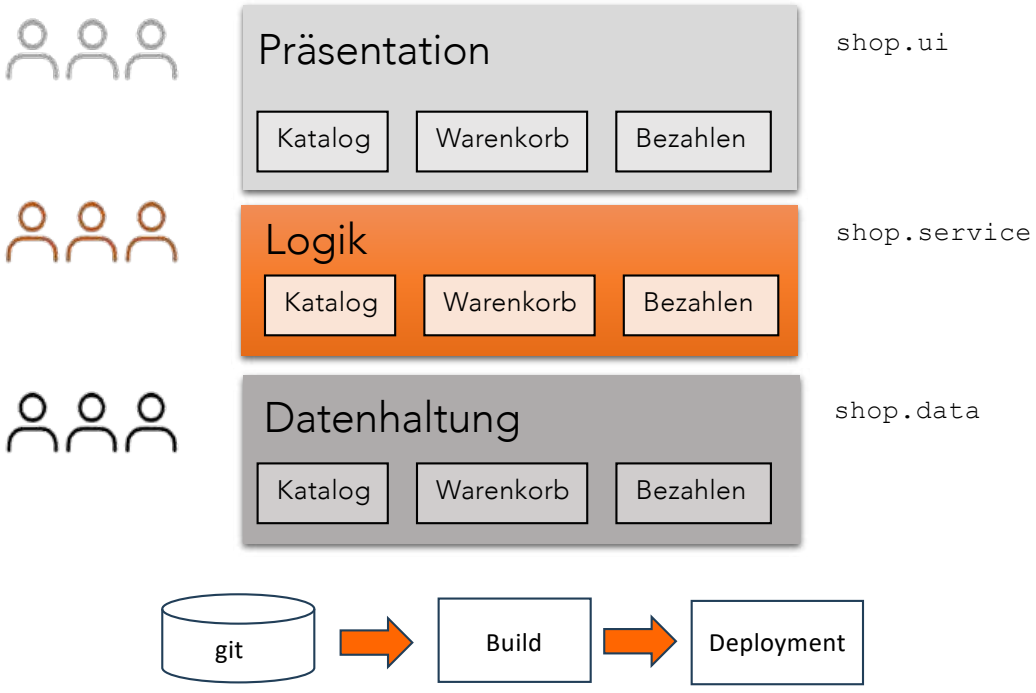
© Prof. Dr. Sabine Sachweh

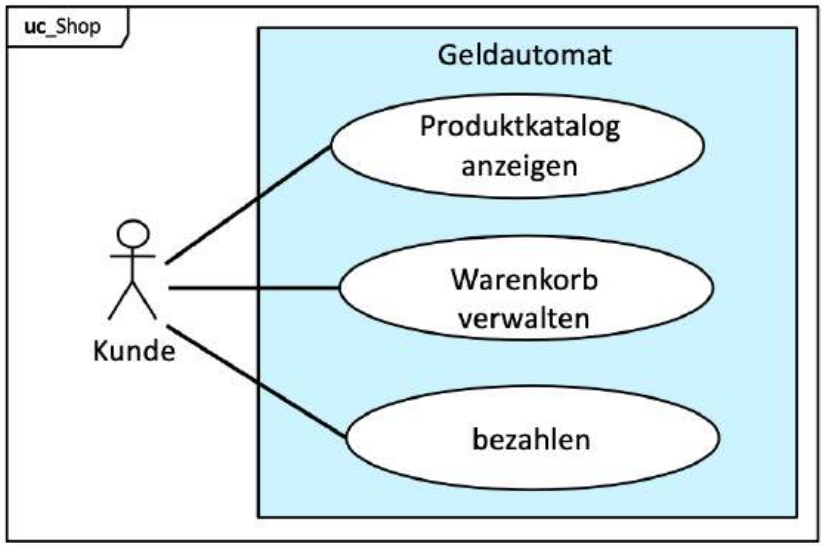
Beispiel - Shop



© Prof. Dr. Sabine Sachweh

Rückblick: Monolith -Schichtarchitektur

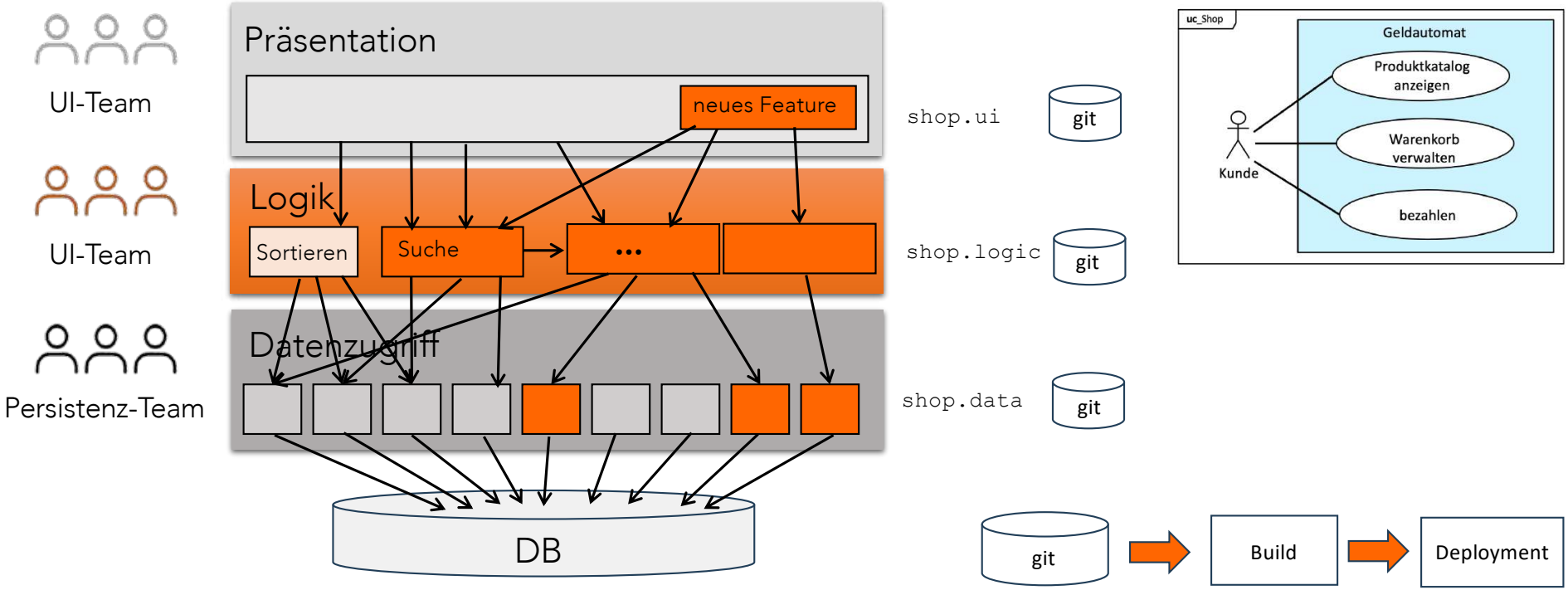




© Prof. Dr. Sabine Sachweh

Rückblick: Monolith – Neues Feature

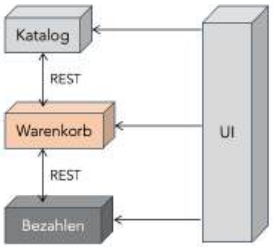
we focus on students

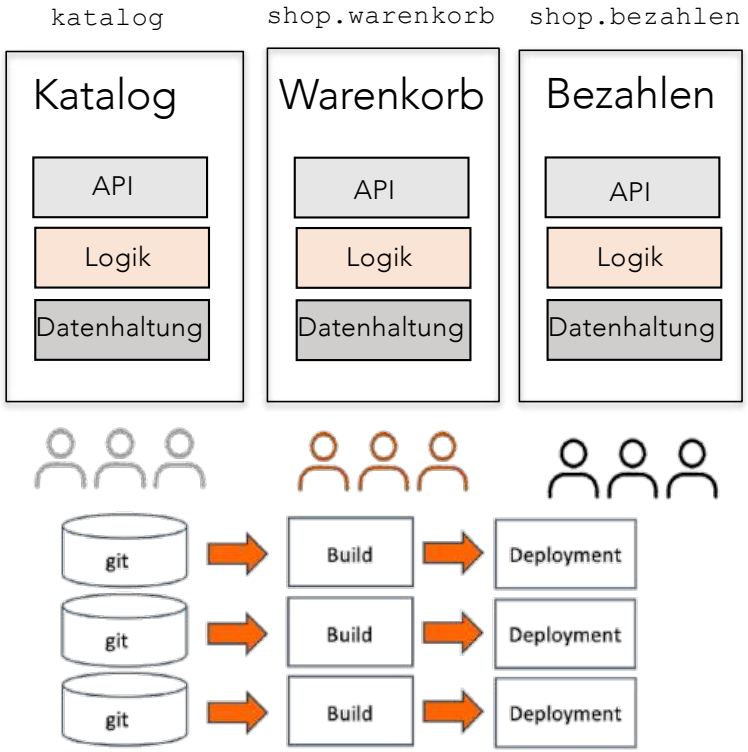


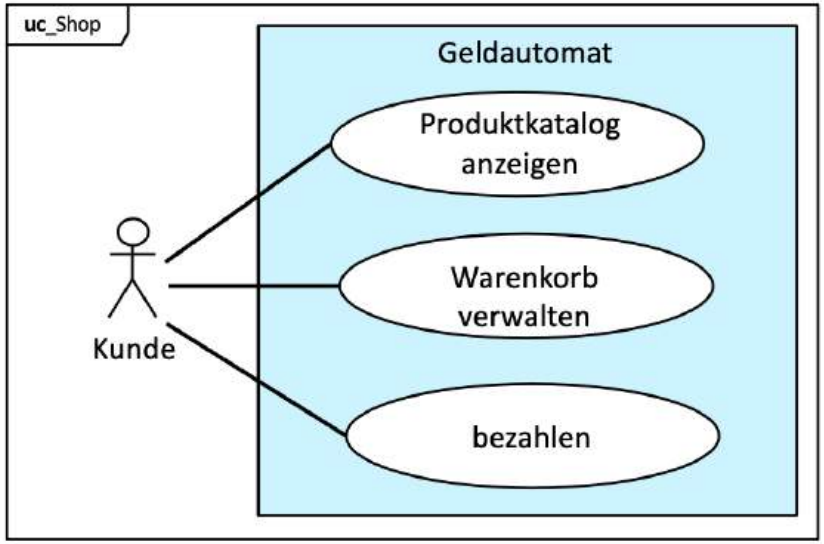
© Prof. Dr. Sabine Sachweh

Rückblick: Microservices

we focus on students







© Prof. Dr. Sabine Sachweh

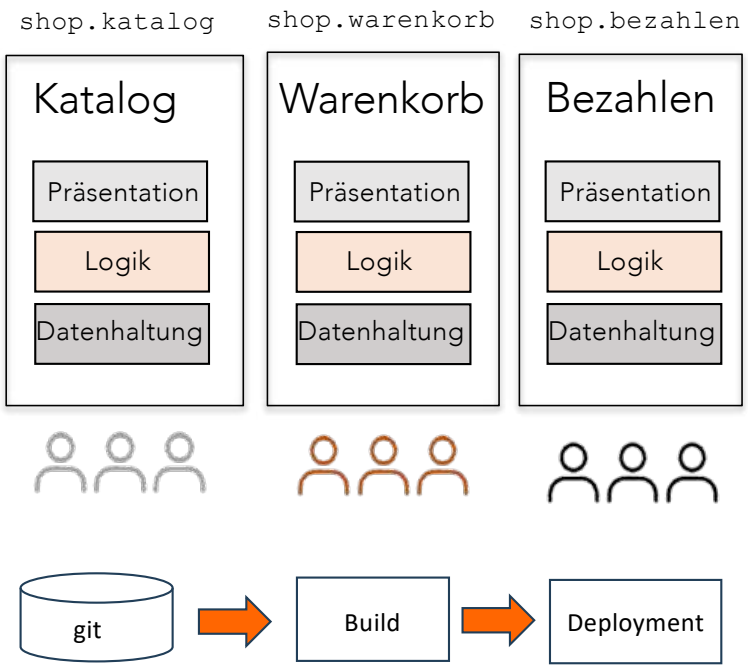
Monolith – Vertical Slicing Architecture

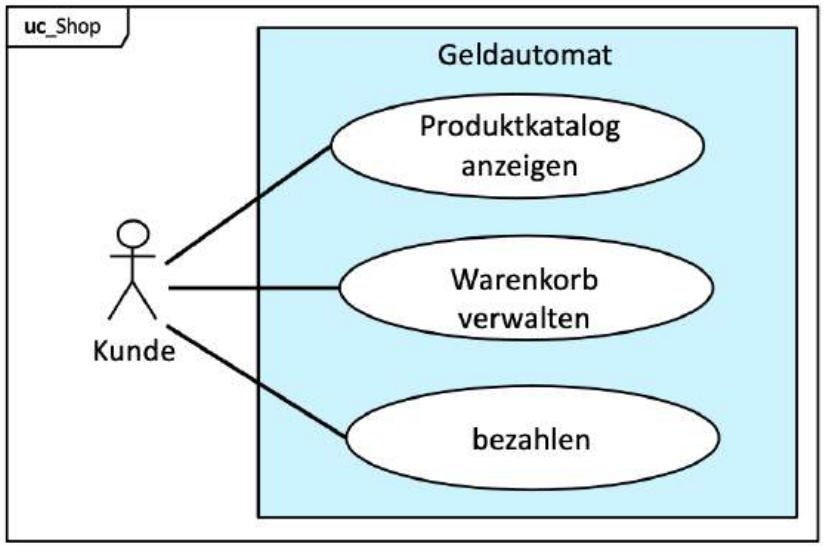
we focus on students

`shop.katalog.ui`

`shop.katalog.service`

`shop.katalog.data`





© Prof. Dr. Sabine Sachweh

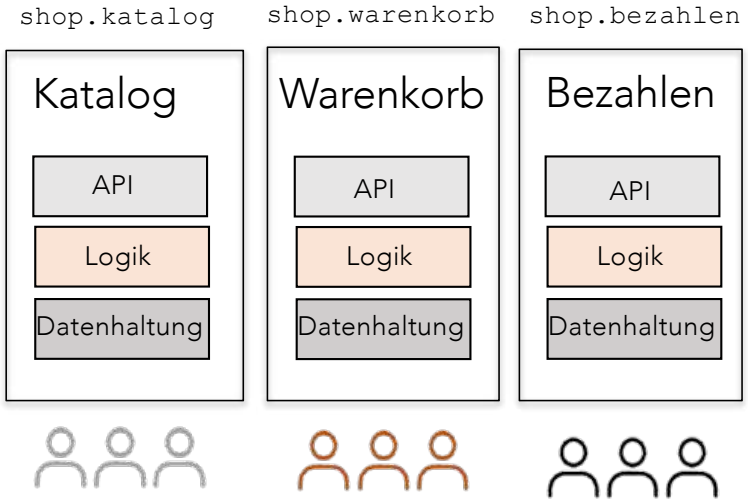
Module und Interfaces/Fassaden

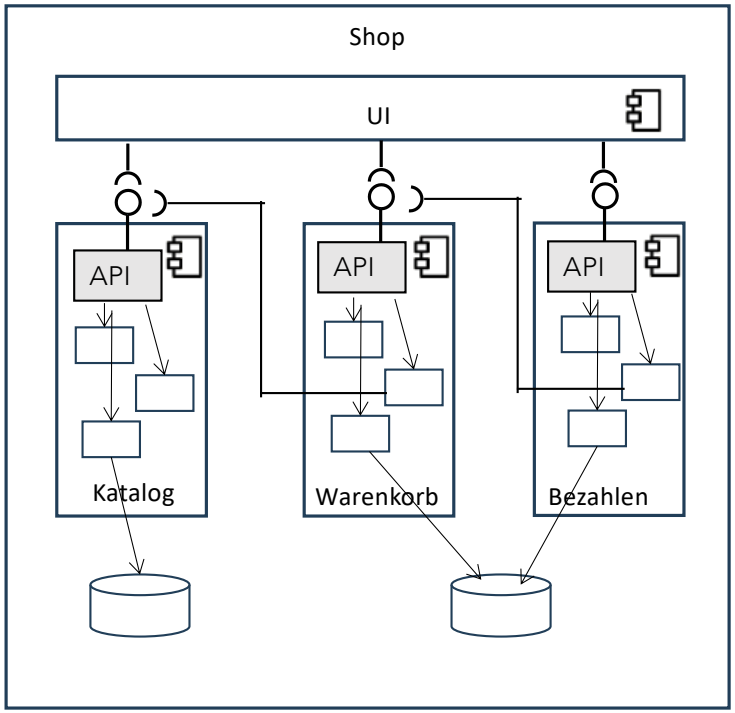
we focus on students

shop.katalog.facade

shop.katalog.service

shop.katalog.data





© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Microservices versus Modulith

we focus on students

Aspekt	Microservices	Modulith
Architektur	Verteiltes System aus vielen Services	Eine Anwendung mit klar getrennten Modulen
Komplexität	Hoch (verteilte Systeme, Infrastruktur)	Niedrig bis mittel
Entkopplung	Stark technisch und organisatorisch	Fachlich gut, technisch geringer
Deployment	Unabhängig pro Service	Gemeinsames Deployment
Skalierbarkeit	Fein granular (pro Service)	Global (gesamte Anwendung)
Performance	Netzwerkabhängig, potenziell langsamer	Sehr schnell (In-Process)
Datenhaltung	Dezentral, je Service	Zentral
Transaktionen	Komplex	Einfach (ACID möglich)
Fehlertoleranz	Teilisolierung möglich	Fehler wirken eher global
Wartbarkeit	Gut bei klaren Services, sonst fragmentiert	Gut bei sauberer Modularisierung
Testbarkeit	Komplex	Einfacher

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Microservices versus Modulith

we focus on students

Aspekt	Microservices	Modulith
Technologie	Heterogen möglich	Einheitlicher Tech-Stack
Betrieb	Aufwendig (DevOps, Monitoring, etc.)	Einfach
Kosten	Hoch	Niedriger
Teamstruktur	Viele autonome Teams	Kleine bis mittlere Teams
Einführung	Hoher Initialaufwand	Schneller Start
Flexibilität	Sehr hoch	Mittel
Typische Nutzung	Große, skalierende Systeme	Kleine bis große Fachanwendungen

Fazit

- **Microservices**: maximale Flexibilität und Skalierung, aber hohe Komplexität
- **Modulith**: geringere Komplexität, dafür weniger technische Unabhängigkeit

© Prof. Dr. Sabine Sachweh

Ein Kompromiss –Hybrider Ansatz

- Basis-System als Modulith realisieren.
- Bei hoher Nachfrage nach bestimmten Services, diese als Microservice implementieren und einbinden.
  → **Skalierbare Services**
- Bei bestimmten technologischen Anforderungen, den Service mit der bestmöglichen Technologie implementieren.
  → **Technische Beschränkung überwinden**

© Prof. Dr. Sabine Sachweh

Abschlussbemerkung

Lohnt es sich einen Sportwagen zu kaufen, um damit morgens zum 300m entfernten Bäcker zu fahren?

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Architekturstile im Zeitverlauf II

Microservice Architecture (MSA)

Self-contained Systems (SCS)

Modulith

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Versionskontrolle mit git und GitHub

Andreas Diepenbrock

07.04.2025

Motivation

- Moderne Softwaresysteme werden immer größer und komplexer
  - zunehmende Abhängigkeiten / Integrationen zu anderen Systemen
  - dynamischere Umgebungen

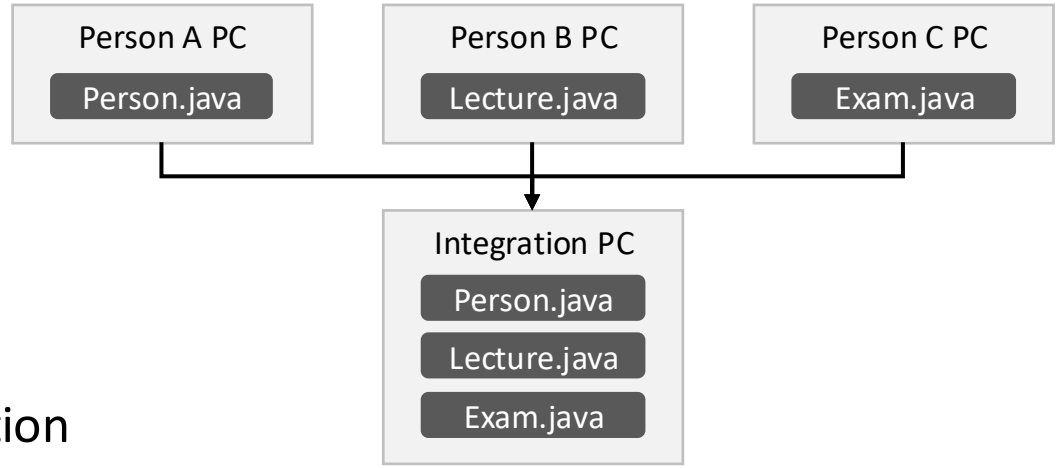
- Software Engineering wird …
  - zunehmend zu einer interdisziplinären Disziplin
  - in Teams entwickelt

Entwicklung im Team

1980er

- Software wird in (atomare) Einheiten aufgeteilt, z.B. Komponenten oder einzelne Dateien
- **Eine** Person ist für **eine** Einheit **alleine** zuständig

- Extensive Planung der Architektur, APIs, etc. im vorhinein nötig, um spätere Änderungen zu vermeiden
- Am Ende des Entwicklungszyklus werden die einzelnen Einheiten zusammengeführt → Big Bang Integration



Kommunikationsaufwand

- Komplexe Softwareprojekte können nicht in vollkommen atomare Aufgaben aufgeteilt werden, die ohne Kommunikation zwischen den Teammitgliedern entwickelbar sind
- Riesiger Kommunikationsaufwand bei der Entwicklung von Softwaresystemen im Team

Anzahl der Kommunikationsbeziehungen: $n\,\frac{(n-1)}{2}$

Beispiel

Team mit 50 Entwickler*innen — $50\,\frac{(50-1)}{2}$ = 1.225 Kommunikationskanäle

Tägliches Meeting mit allen und 5 min pro Komponente — $50 * 5$ = 250 min Meetinglänge → ca. 4h Meeting pro Tag

- Aufstockung des Teams um weitere Entwickler*innen, um schneller zum Ziel zu kommen, führt zu noch mehr Kommunikation und noch längere Entwicklungszeit

Quelle: Fred Brooks; The Mythical Man-Month: Essays on Software Engineering; ISBN: 978-0-201-83595-3

Entwicklungszeit

- Insbesondere moderne Softwaresysteme können nicht mehr alleine entwickelt werden

- Fred Brooks gibt in seinem Buch an, dass ein Software Engineer ca. **10 Lines of Code (LoC) pro Tag** produziert

Quelle: Fred Brooks; The Mythical Man-Month: Essays on Software Engineering; ISBN: 978-0-201-83595-3

- Softwaresysteme haben heutzutage häufig mehr als 1 mio. LoC

Software	10 LoC / Tag	100 LoC / Tag
Linux Kernel (ca. 40 mio LoC)	4 mio Tage / 10.958 Jahre	400.000 Tage / 1.095 Jahre
Visual Studio Code (ca. 2,6 mio LoC)	260.000 Tage / 712 Jahre	26.000 Tage / 71 Jahre
Google (ca. 2 mrd LoC)	200 mio Tage / 584.945 Jahre	20 mio Tage / 54.794 Jahre

Stand: 02.04.2025

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Vorgehensmodelle in der Softwareentwicklung

Vorgehensmodelle

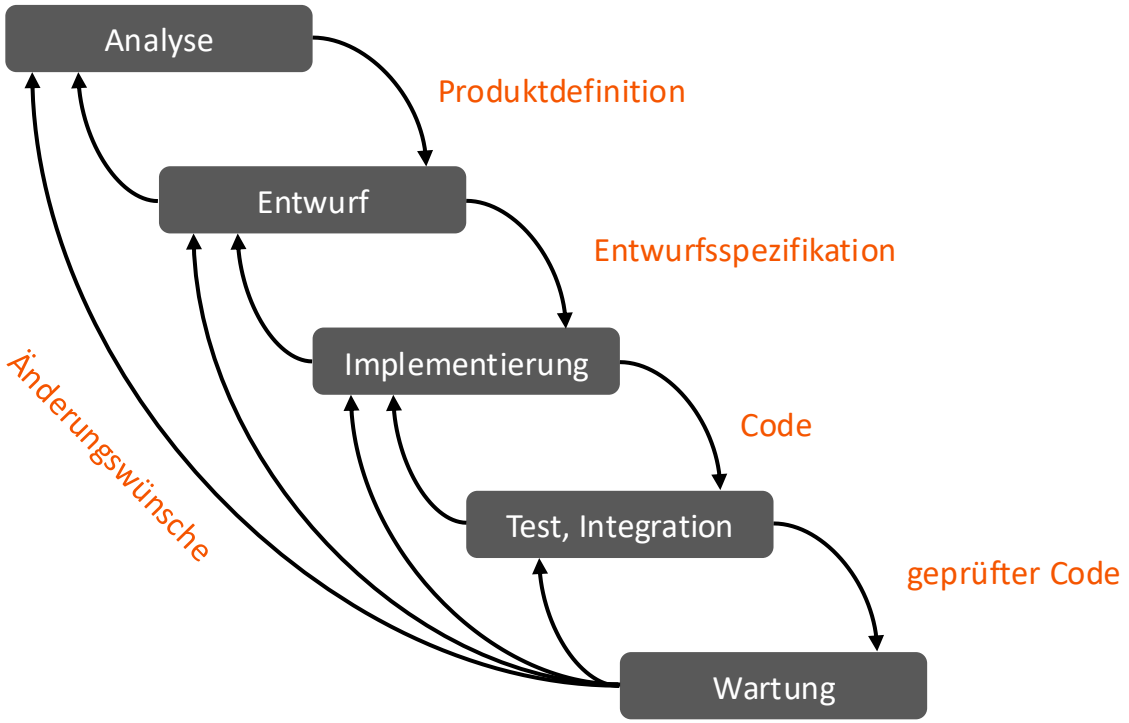
- Organisieren einen Entwicklungsprozess in strukturierte Abläufe
- Sind organisatorische Hilfsmittel, die für konkrete Projekte individuell angepasst werden und in die konkrete Maßnahmenplanung überleiten.

- Können in zwei Klassen unterteilt werden:

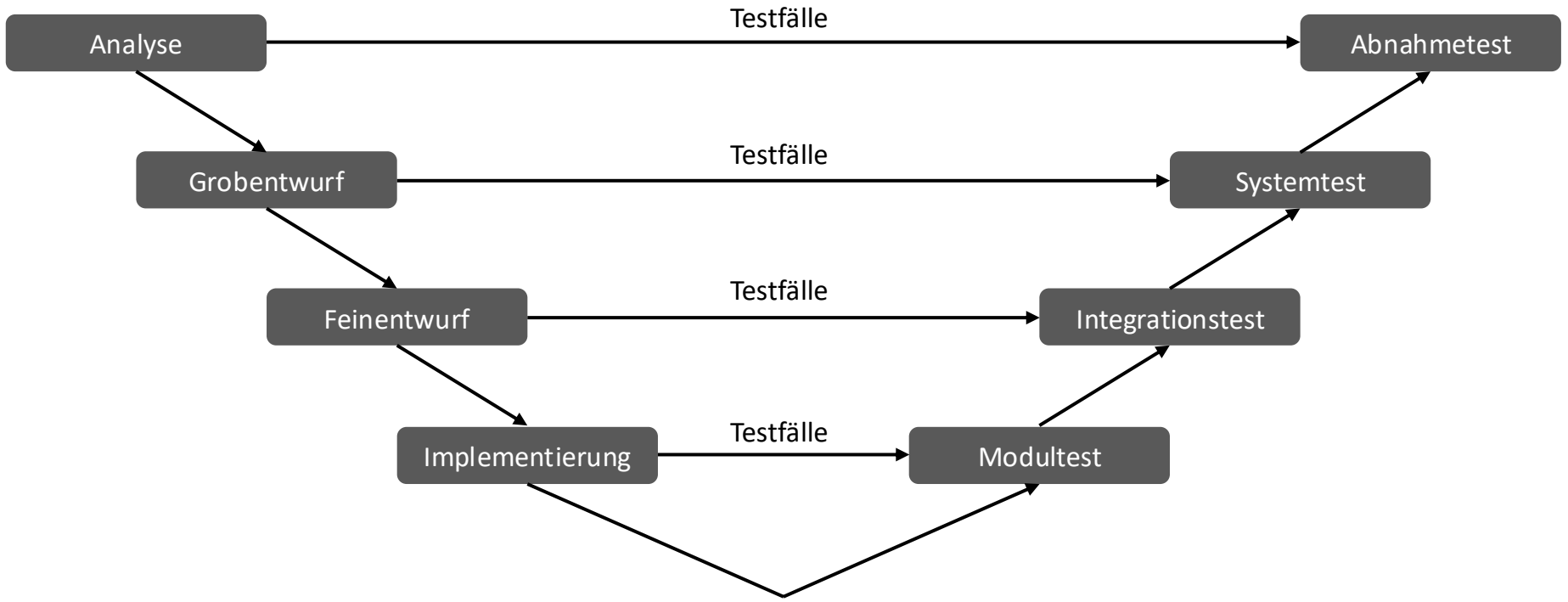
Klassische Vorgehensmodelle
- *Wasserfall*
- *V-Modell*
- Rational Unified Process
- Spiral-Modell
- …

Agile Vorgehensmodelle
- *Extreme Programming*
- Kanban
- *Scrum*
- Adaptive Software Development
- …

Wasserfallmodell



V-Modell


Analyse
Testfälle
Abnahmetest
Grobentwurf
Testfälle
Systemtest
Feinentwurf
Testfälle
Integrationstest
Implementierung
Testfälle
Modultest

Agile Softwareentwicklung

- Eine Bewegung, die diese sehr prozess- und dokumentorientierte Vorgehensweise zur Entwicklung von Softwareprojekten agiler gestalten will, hat ein Manifest zur agilen Softwareentwicklung erstellt
  - **Individuen und Interaktion** > Prozesse und Werkzeuge
  - **Funktionierende Software** > umfassende Dokumentation
  - **Zusammenarbeit mit dem Kunden** > Vertragsverhandlung
  - **Reagieren auf Veränderung** > Befolgen eines Plans

- Dient als Grundlage für viele Vorgehensmodelle, die heutzutage in der Praxis eingesetzt werden

Quelle: https://agilemanifesto.org/

Extreme Programming

- **Evolutionäre** Entwicklung der Software in sehr kleinen Inkrements
- **Tests und Code** sind das Analyseergebnis, Entwurfsdokument und die Dokumentation
- Code wird **permanent lauffähig** gehalten
- Diszipliniertes und automatisiertes Testen als Qualitätssicherung
- Pair-Programming als Qualitätssicherungsmaßnahme
- Einsatz von Standards bzw. Guidelines bei der Entwicklung
- Als **erstes Tests** definieren und dann die Funktionalität dazu implementiert
- Nur für **kleinere Projekte** geeignet

Scrum

Komplexität der Entwicklung wird durch drei Prinzipien reduziert

- **Transparenz** Fortschritt und Probleme werden täglich festgehalten
- **Überprüfung** Funktionen werden iterativ geliefert und beurteilt
- **Anpassung** Flexibel auf Anforderungen reagieren und ggf. ändern

Im Prozess wird zwischen drei Rollen unterschieden:

- **Product Owner** Definition und Priorisierung der Anforderungen im Backlog und Kontakt zu Kunden / Stakeholdern
- **Entwicklungsteam** Entwickelt die Software inkrementell in Selbstverwaltung
- **Scrum Master** Unterstützt das Team, fördert die Einhaltung des Scrum-Prozesses und beseitigt Hindernisse

Scrum

In Scrum gibt es verschiedene Aktivitäten

- **Spring Planning** — Erfolgt vor dem Sprint und erstellt den **Sprint Backlog**
- **Sprint Review** — Team präsentiert Ergebnisse live am Produkt und erhält Feedback für nächsten Sprint
- **Sprint Retrospektive** — Analyse der Zusammenarbeit des Teams zur kontinuierlichen Verbesserung des Teams
- **Daily Scrum** — Auf max. 15 min begrenztes tägliches Meeting mit dem Ziel des Status-Update für alle Rollen

- Alle Aktivitäten in Scrum sind **zeitlich fest** beschränkt (→ timeboxed). Es gibt keine klassischen Meilensteine

Scrum

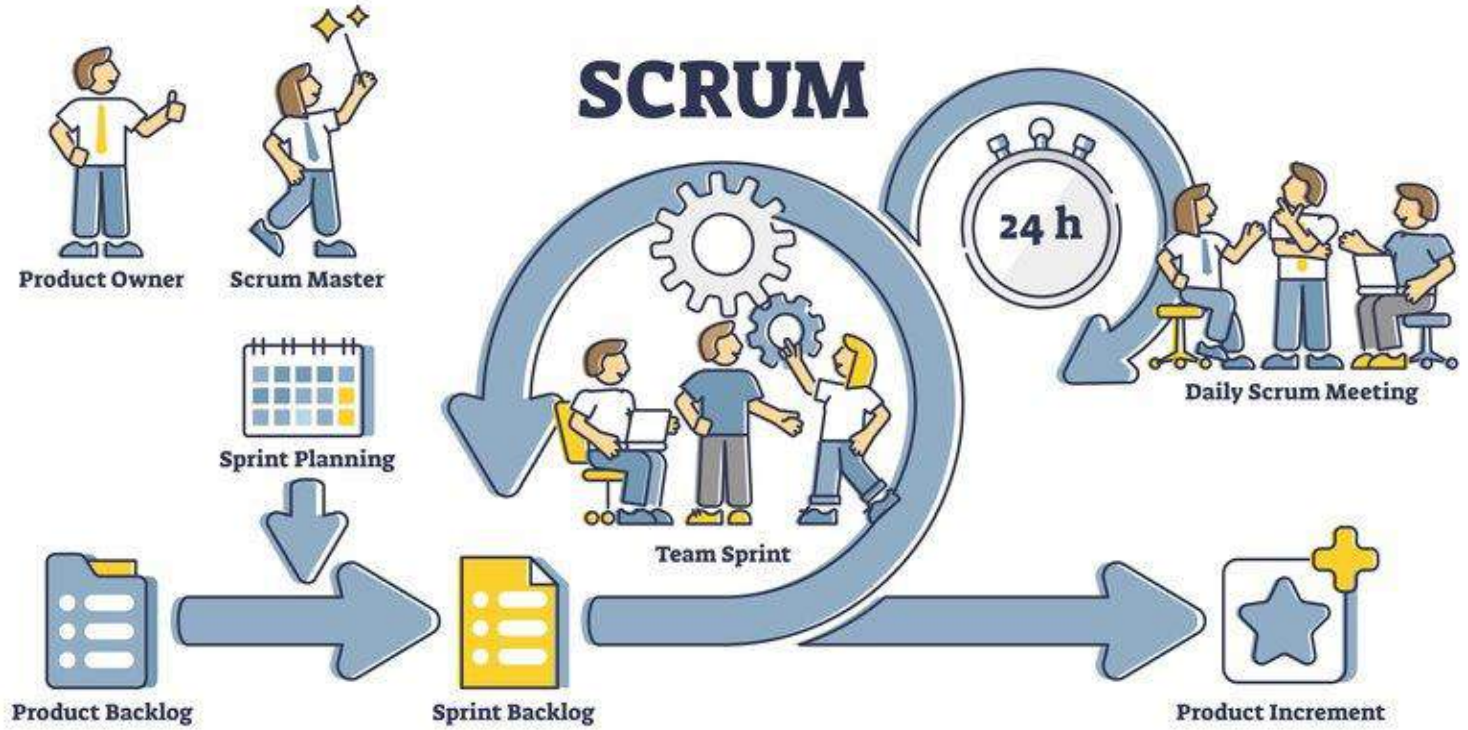
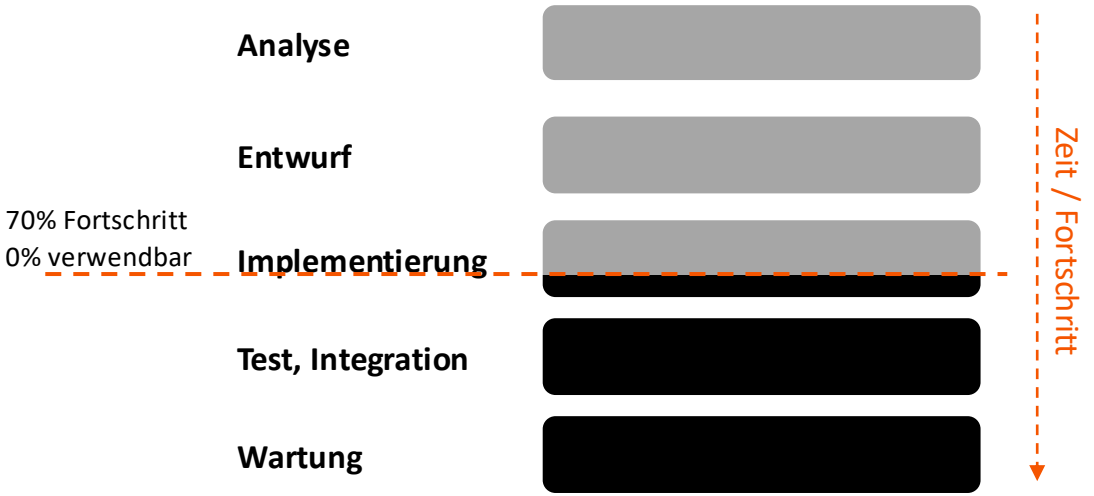


Abbildung von: https://www.managementcircle.de/blog/scrum-mit-agilitaet-zum-projekterfolg.html

Klassische vs. Agile Vorgehensmodelle

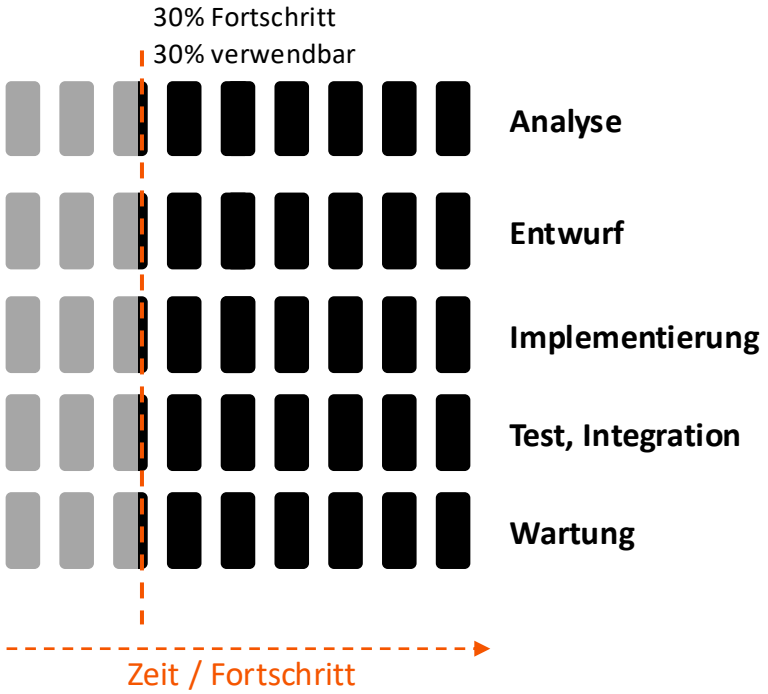
Klassische Vorgehensmodelle

- Aktivitäten sind **chronologisch in Phasen unterteilt**



Agile Vorgehensmodelle

- Aktivitäten sind **kurze Sprints bzw. Intervalle**
- Leben von **vielen kleinen Iterationen**



Fachhochschule
Dortmund
University of Applied Sciences and Arts

Versionskontrolle

Was ist ein Versionskontrollsystem

- Ein Versionskontrollsystem (VCS) speichert
  - … den Inhalt aller Dateien eines Projektes
  - … die gesamte Änderungsgeschichte eines Projektes
- Kann zur Protokollierung des Projektgeschehen genutzt werden:
  wer hat **wann** und **warum**, **welche** Dateien **wie** geändert?

- Bietet i.d.R. die Funktionalitäten
  - … zurücksetzen des Projektes auf einen früheren Stand der Entwicklung
  - … Dokumentation der Projekthistorie und der Beiträge einzelner Entwickler*innen
  - … ermöglicht die parallele Entwicklung an mehreren Entwicklungszweige

Varianten von Versionskontrollsysteme

Lokale Versionskontrollsysteme

z.B.
- Revision Control System
- Source Code Control System

- Das Repository existiert nur auf dem lokalen Computer

Zentrale Versionskontrollsysteme

z.B.
- SVN
- Team Foundation Server

- Es existiert ein zentraler Server, welcher den aktuellen Stand des Repository hat

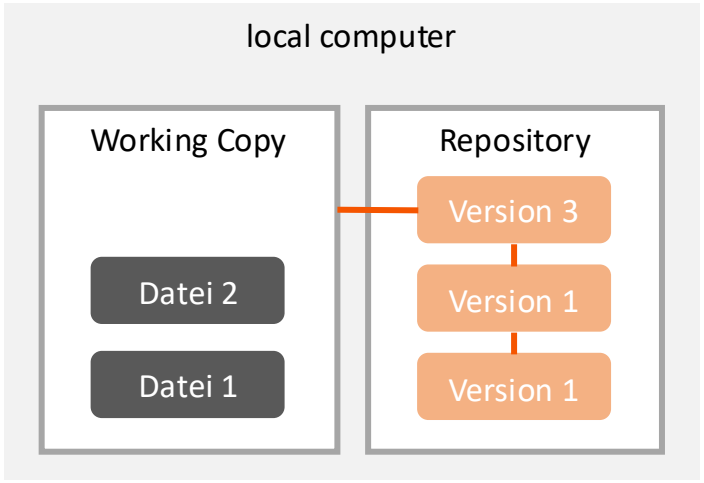
Dezentrale Versionskontrollsysteme

z.B.
- git
- Mercurial

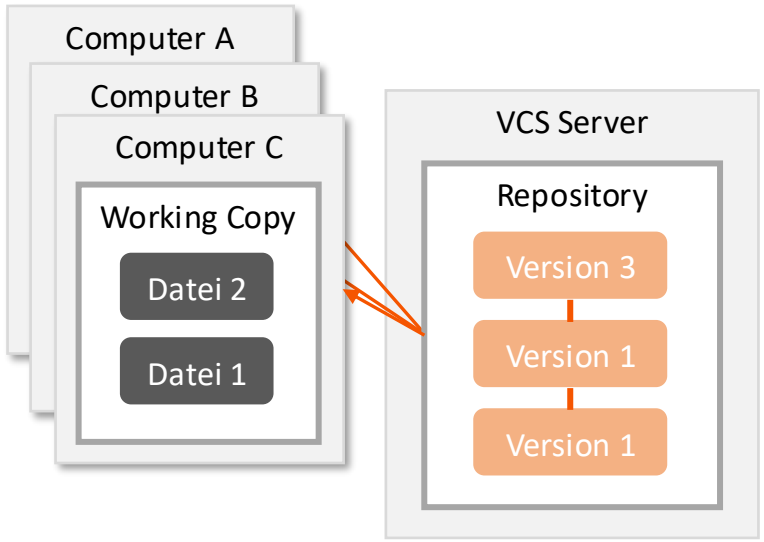
- Jede Person besitzt eine vollständige Kopie des Repository inkl. der History
- Gemeinsame Verwaltung, aber auch lokale Commits und History

Varianten von Versionskontrollsysteme

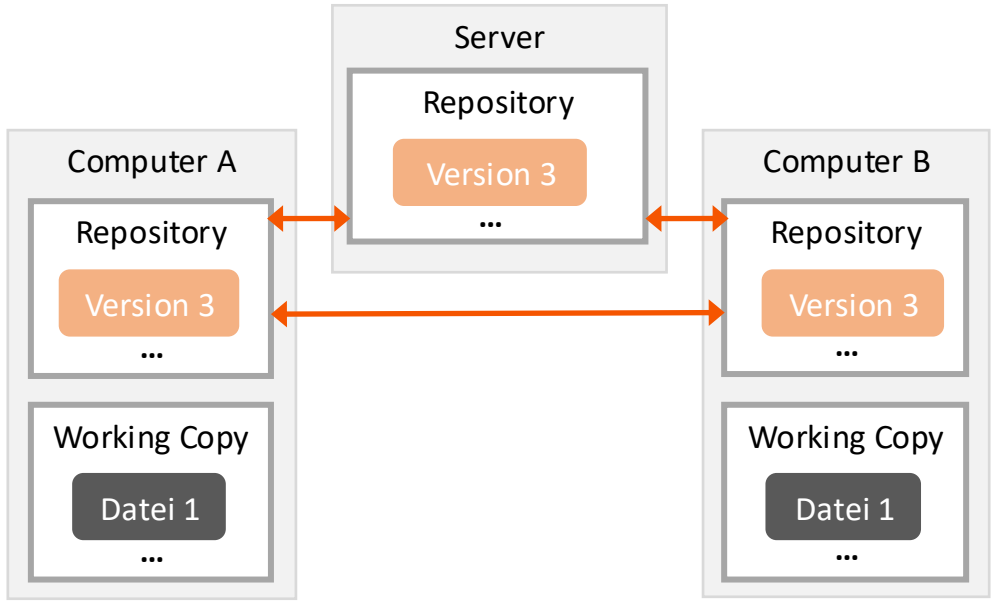
Lokale Versionskontrollsysteme



Zentrale Versionskontrollsysteme



Dezentrale Versionskontrollsysteme



Wichtige allgemeine Begriffe

Repository

- Beinhaltet die gesamte Historie der Entwicklung und der dazugehörigen Dateiinhalte aller Versionen

Arbeitskopie / Working Copy

- Kopie aller Dateien einer spezifischen Version

Commit

- Aktueller Stand der Working Copy, die als neue Version in das Repository übernommen werden soll

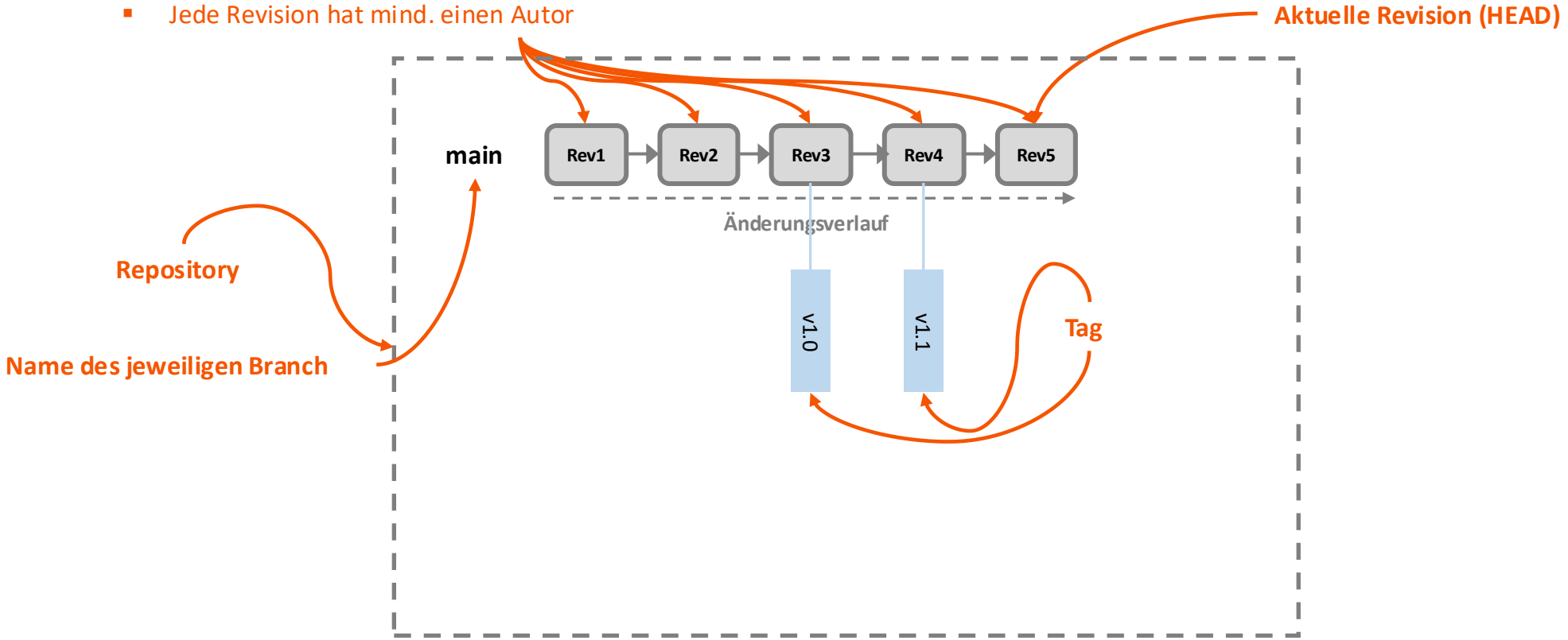
Diff

- Unterschiede der Dateien zwischen Repository und Working Copy oder einer anderen Version

Darstellung eines Repository als Graph

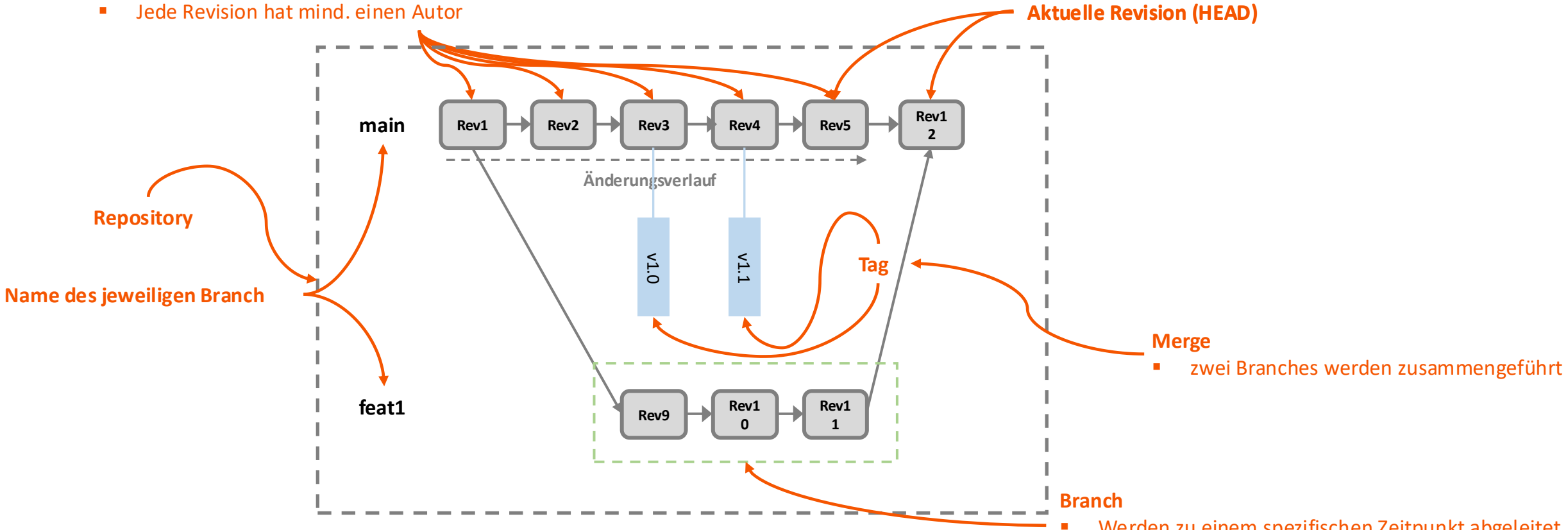
Revisions

- Jede Revision beinhaltet Änderungen
- Jede Revision hat eine textuelle Beschreibung der Änderungen
- Jede Revision hat mind. einen Autor



Darstellung eines Repository als Graph

Revisions
- Jede Revision beinhaltet Änderungen
- Jede Revision hat eine textuelle Beschreibung der Änderungen
- Jede Revision hat mind. einen Autor



Diffs

Version 1

```
int x = 42;
x = x + 1;
System.out.println(x);
```
code

Version 2

```
int x = 42;
x = x - 8;
System.out.print(x);
```
code

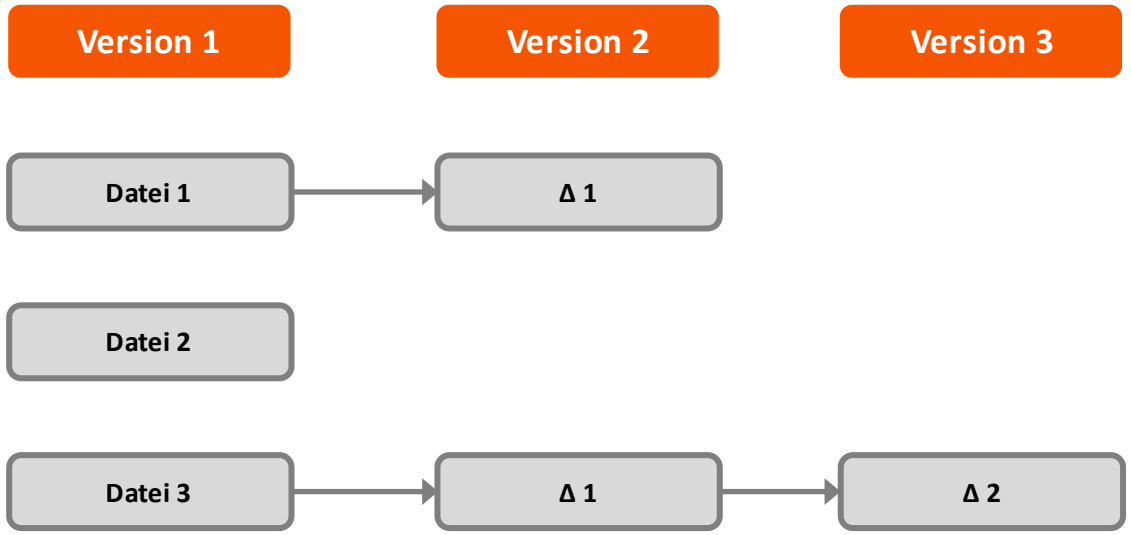
Diff von Version 1 zu Version 2

“alter Stand“

```
int x = 42;
- x = x + 1;
- System.out.println(x);
+ x = x – 8;
+ System.out.print(x)
```
code

neuer Stand / Änderungen
im Vergleich zu Version 1

Diff-basierte Versionskontrollsysteme



Version 1 von Datei 1

```
int x = 42;
x = x + 1;
System.out.println(x);
```

code

Version 2 von Datei 1

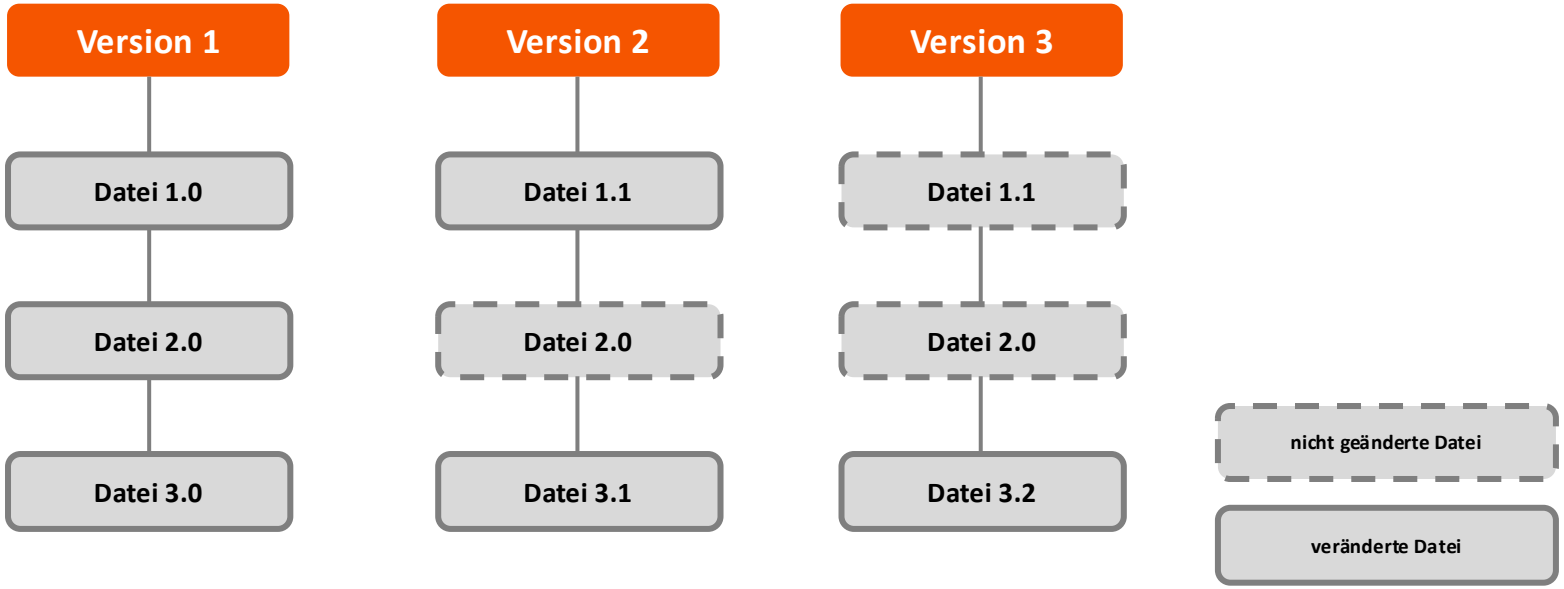
```
int x = 42;
x = x + 1;
System.out.println(x);
System.out.println("Another print");
```

code

Nur die Änderung (Delta) wird gespeichert

Snapshot-basierte Versionskontrollsysteme

- Ein **Snapshot** ist ein **vollständiges Abbild** des **aktuellen Zustands aller verwalteten Dateien** zu einem **spezifischen Zeitpunkt** (z.B. Version)



Version 1 von Datei 1

```
int x = 42;
x = x + 1;
System.out.println(x);
```

code

Version 2 von Datei 1

```
int x = 42;
x = x + 1;
System.out.println(x);
System.out.println("Another print");
```

code

Gesamte Datei wird gespeichert

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Versionskontrollsystem git

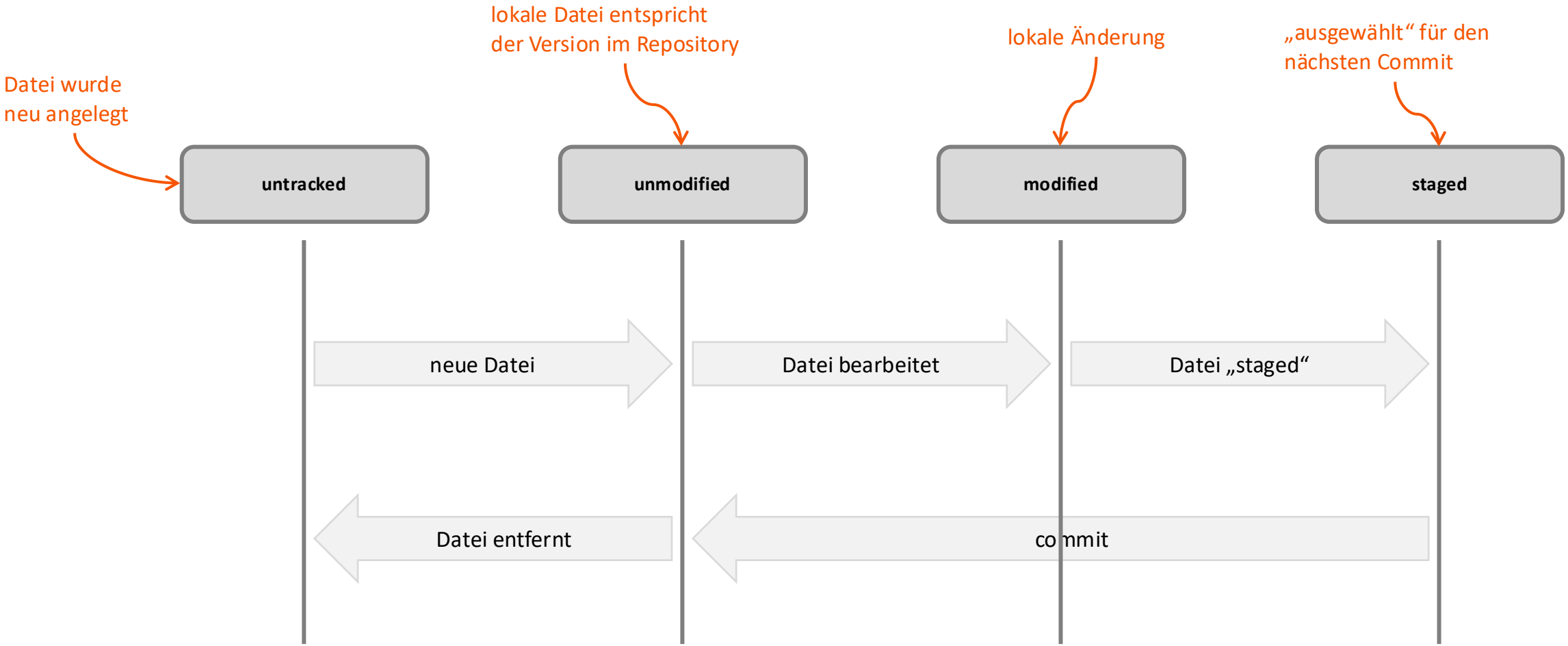
Was ist das Versionskontrollsystem git

- Bei git handelt es sich um ein **dezentrales** und **Snapshot-basiertes Versionskontrollsystem**

- Ursprünglich von Linus Torvalds zur Versionskontrolle des Linux Kernels entwickelt
  - Bis 2005 wurde der Linux Kernel mit Bitkeeper (bis dahin proprietär) verwaltet
  - Keine ausreichenden Alternativen verfügbar, weshalb dann git entwickelt wurde

- git nutzt verschiedene Zustände für die Verwaltung von Änderungen

Zustandsmodell von git



Grundlegende Konzepte in Git

Repository

- Ordner, der alle Dateien und die Historie beinhaltet

Commit

- Speichert Änderungen im Projekt dauerhaft
- Hat i.d.R. eine aussagekräftige Nachricht, was im Commit geändert wurde

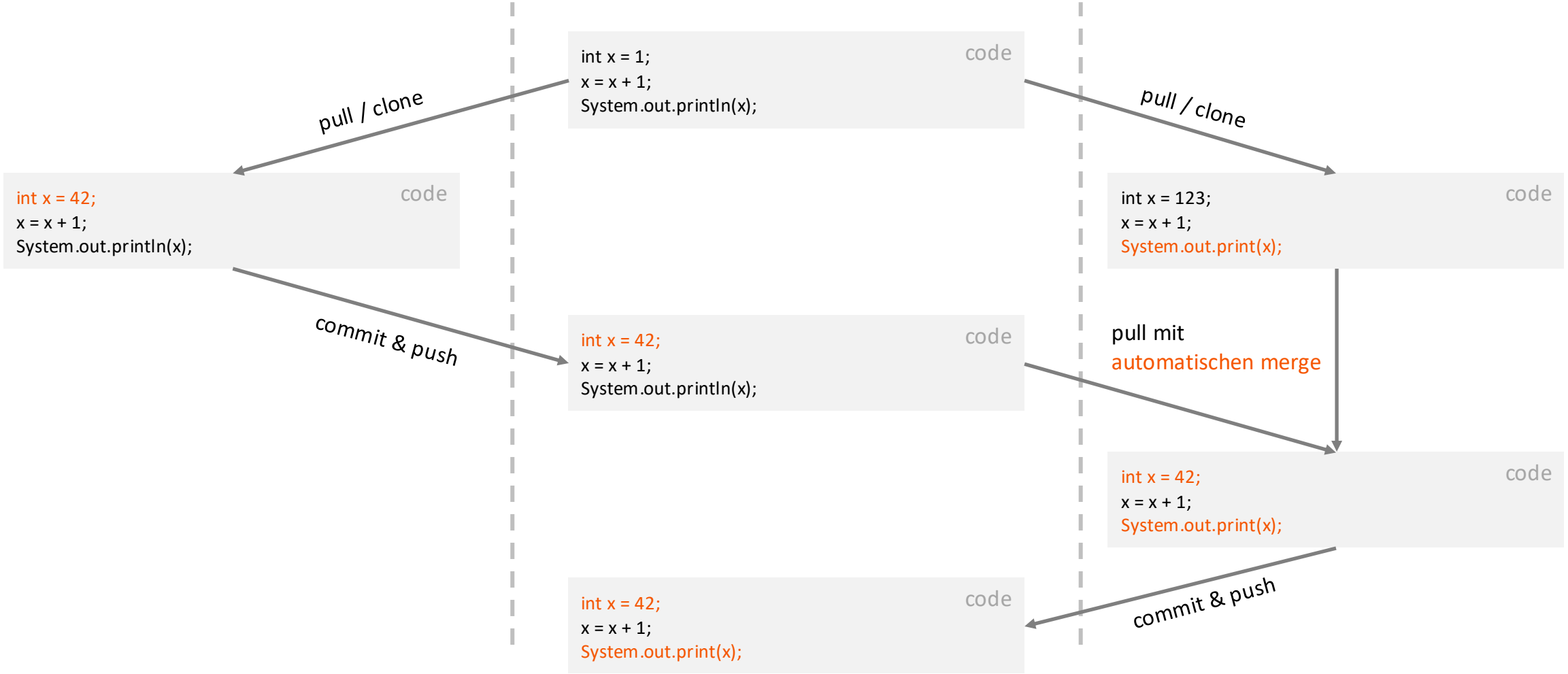
Branch

- Unabhängige Arbeitszweige für Features o. Ä.
- Ein Repository kann mehrere Branches haben, an denen gearbeitet wird

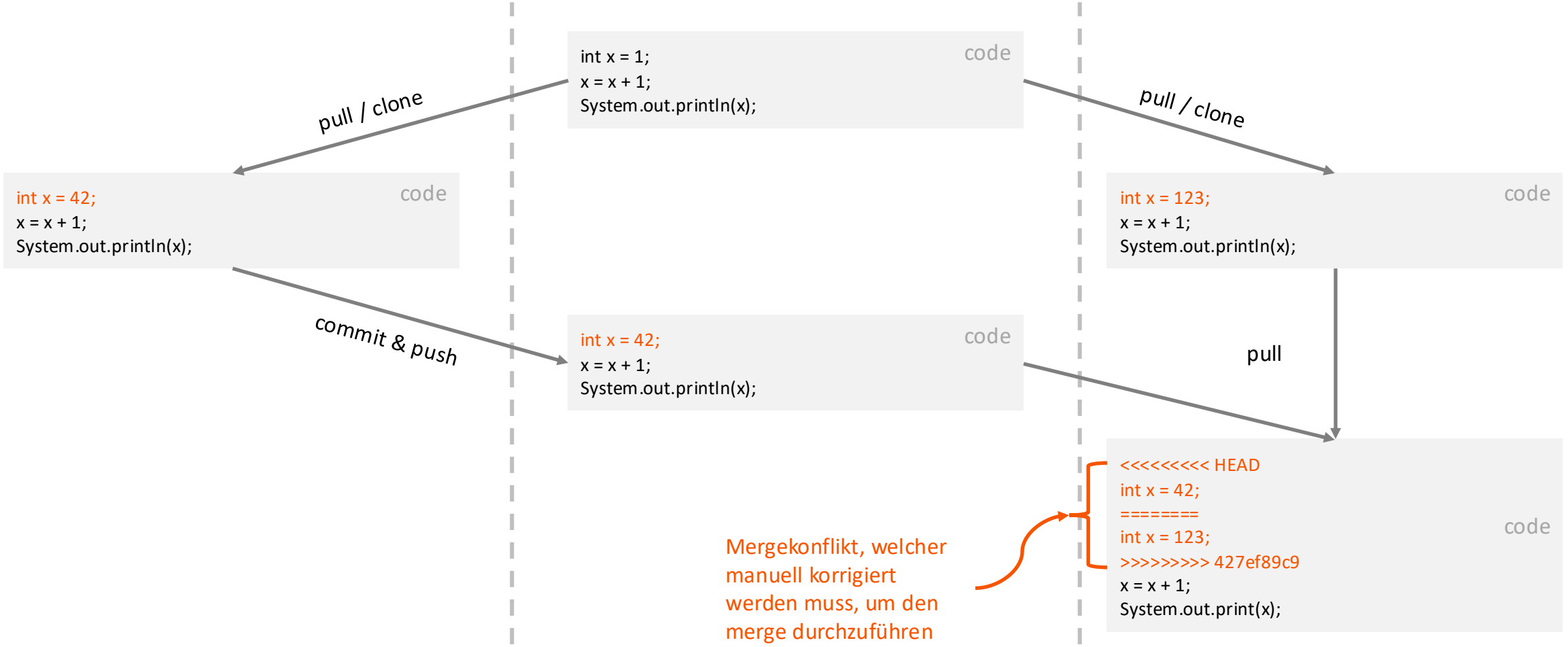
Merge

- Führt unterschiedliche Branches zusammen
- Identifiziert potenzielle Konflikte zwischen den Versionen und behebt diese ggf. automatisch

Merge mit automatischen merge



Merge mit Mergekonflikt



Installation und Einrichtung

- Die Versionsverwaltung git kann unter https://git-scm.com heruntergeladen werden (oder über den Packagemanager des Betriebssystems)

- Nach der Installation können Sie über das Terminal prüfen, ob git erfolgreich installiert wurde, bspw. durch den Befehl:

```bash
> git --version
git version 2.49.0
```

- Nachdem die Installation erfolgreich durchgeführt und getestet wurde, sollten als Erstes zwei wichtige Einstellungen vorgenommen werden:

```bash
> git config --global user.name "Dein Name"
> git config --global user.email "deine.email@beispiel.de"
```

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Nutzung von git

Erstellen eines neuen git Repository

Optional; Wenn nicht angegeben, dann im aktuellen Verzeichnis

```bash
> git init [name]
Initialized empty Git repository in /path/to/git/repo/foo/.git
```

Dieser Ordner ist bei den meisten Betriebssystemen versteckt

- Erstellt ein neues (lokales) git Repository im Ordner mit dem Namen name bzw. im aktuellen Ordner
- Im Ordner des Repository wird ein Ordner .git erstellt, welches das Repository darstellt

Dieser Ordner sollte nicht gelöscht werden (zumindest solange das Repository nicht irgendwo anders vorhanden ist)

```bash
> ls .git
HEAD        description  info        refs        config      hooks       objects
```

Hinzufügen einer Änderung

```bash
> git add <datei> [datei2 … ]
```

- Fügt neue Dateien bzw. Änderungen der **Staging Area** hinzu

Entfernen von Dateien

```bash
> git rm <datei>
```

- Entfernt die angegebene Datei von der **Staging Area und der Working Copy**

Commiten von Änderungen

```bash
> git commit [-m message]
```

- Erstellt einen neuen Commit basierend auf dem aktuellen Stand der Staging Area

Zurücksetzen der Working Copy

```bash
> git checkout -- <pfad>
```

- Entfernt alle Änderungen der Staging Area befindlichen Dateien oder Verzeichnisse

Erstellen eines neuen Branches

```bash
> git checkout –b <name> [ starting_point ]
```

- Erstellt ein neues (lokales) git Repository im Ordner mit dem Namen name bzw. im aktuellen Ordner
- Im Ordner des Repository wird ein Ordner .git erstellt, welches das Repository darstellt

Wechseln in einen anderen Branch

```bash
> git switch <branch_name>
```

- Wechselt in den Branch mit dem angegebenen Namen, sofern dieser existiert

Zusammenführen von Branches

```bash
> git switch <branch_name>
> git merge <branch_name>
```

- Zuerst in den Zielbranch wechseln, wo der andere Branch integriert werden soll
- Mittels git merge wird der angegebene Branch in den aktuellen Branch überführt

Löschen eines Branches

```bash
> git branch –d <name>
```

- Löscht den Branch mit dem angegebenen Namen (lokal)

Lokales Repository mit Remote Repository verbinden

bash

```
> git remote add <name> <url>
```

- Fügt ein Remote Repository zum aktuellen lokalen Repository hinzu
- Jedes Remote Repository besitzt einen eindeutigen Namen und eine URL zum Remote Repository
- In der Regel wird das Remote Repository **origin** genannt, kann jedoch auch anders genannt werden

Klonen eines Remote-Repository

```bash
> git clone <repository url> [ name ]
```

- Erstellt ein neues lokales Repository basierend auf dem angegebenen Remote Repository
- Optional kann ein Name angegeben werden, unter dem das lokale Repository gespeichert wird

Änderungen vom Remote Repository beziehen

```bash
> git pull
```

- Lädt die neusten Änderungen, die nicht bereits lokal vorliegen, vom Remote Repository herunter

Nutzung von git

Änderungen zum Remote Repository senden


```bash
> git push [branch_name]
```

- Veröffentlicht die Änderungen vom lokalen Repository auf dem Remote Repository
- Wenn ein anderer Branch als der aktuelle veröffentlicht werden soll, kann dies über die Angabe des Namen erfolgen

- Wenn auf einem spezifischen Remote Repository etwas veröffentlicht werden soll, muss dieser upstream manuell angegeben werden

```bash
> git push -u repository ref
```

Commit Messages (Allgemein)

- Ein Commit sollte **eine** klar **definierbare** und **atomare** Änderung umfassen

```bash
> git commit [-m message]
```

Empfehlung von git selbst, aber keine festgeschriebene Regel und kann abweichen

```
Kurze Erklärung als Überschrift (max. 50 Zeichen)

Optional: weitere Beschreibung der Änderungen oder andere Informationen
```

Conventional Commit Messages

- Eine Spezifikation, wie Commit Messages aufgebaut sein sollen, um bspw. automatisierte Tools einzusetzen (z.B. automatische Changelogs, Releases, etc.)

In **SemVer** vergleichbar mit einem **Patch Release**

- Ein Commit wird anhand eines Type klassifiziert
  - fix Kennzeichnet die Behebung eines Fehlers im Code

In **SemVer** vergleichbar mit einem **Minor Release**

  - feat Kennzeichnet die Einführung eines neuen Features
  - weitere Typen sind ebenfalls möglich, wie z. B. build, chore, docs, test, etc.

In **SemVer** vergleichbar mit einem **Major Release**

- Ein Breaking Change wird entweder durch BREAKING CHANGE: im Footer oder durch ein Ausrufezeichen (!) hinter dem Type / Scope gekennzeichnet

vollständige Spezifikation: https://www.conventionalcommits.org/

Conventional Commit Messages

- Der Aufbau einer Conventional Commit Message sieht wie folgt aus:

```
<type>[optional scope]: <description>
[optional body]
[optional footer(s)]
```

- Commit Message mit einer Beschreibung und einem Breaking Change:

```
feat: allow provided config object to extend other configs

BREAKING CHANGE: `extends` key in config file is now used for extending other config files
```

- Commit Message mit einer Erklärung und mehreren Footern

Kann alternativ auch über Projektmanagementsoftware verwaltet werden

Referenziert das Ticket mit der ID 123

```
fix(api): prevent racing of requests

Introduce a request id and a reference to latest request. Dismiss incoming responses other than from latest request.

Remove timeouts which were used to mitigate the racing issue but are obsolete now.

Reviewed-by: John Doe
Refs: #123
```

End of Day Commits

- Sofern alleine an einem Branch gearbeitet wird, können End of Day Commits eingesetzt werden, um ggf. Datenverlust zu verhindern.

- Am Ende des Arbeitstags bzw. wenn aufgehört wird an einem Branch gearbeitet wird, ein Commit mit der Nachricht „WIP“ oder „EoD“ o. Ä. anlegen

Bevor der Branch in einen anderen Branch überführt wird, unbedingt den Branch rebasen und sinnvolle Commit(-nachrichten) erstellen!

gitignore

- Git berücksichtigt bei der Versionskontrolle grundsätzliche alle Dateien
- Manche Dateien sollen jedoch nicht berücksichtigt werden und können von der Versionskontrolle über ein sog. gitignore File spezifiziert werden
- Dieses gitignore File sollte im Root-Verzeichnis platziert sein und .gitignore genannt werden
- Es dient nutzt ein Blacklisting-Verfahren

- Die Spezifikation von den Regeln, welche Dateien ausgeschlossen werden sollen, kann als glob Regeln vorgenommen werden

Eine Zeichenfolge, die zum Abgleichen von Dateipfaden verwendet wird

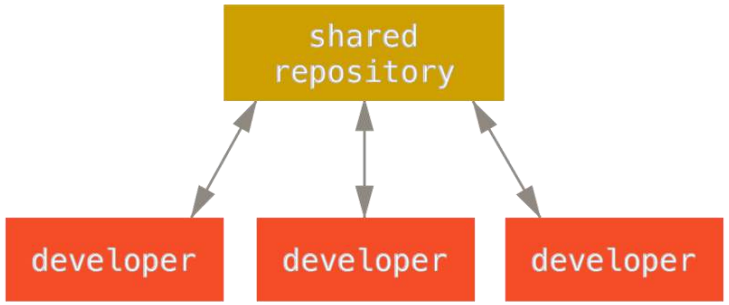
Niemals Code commiten (und insb. ins Remote-Repository pushen) von unvollständigen bzw. nicht lauffähigen Code

Fachhochschule
Dortmund
University of Applied Sciences and Arts

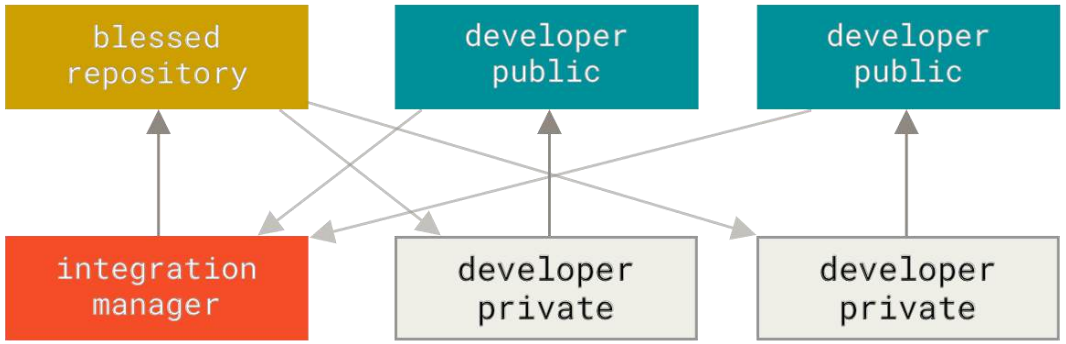
Git Workflows

Allgemeine Workflows

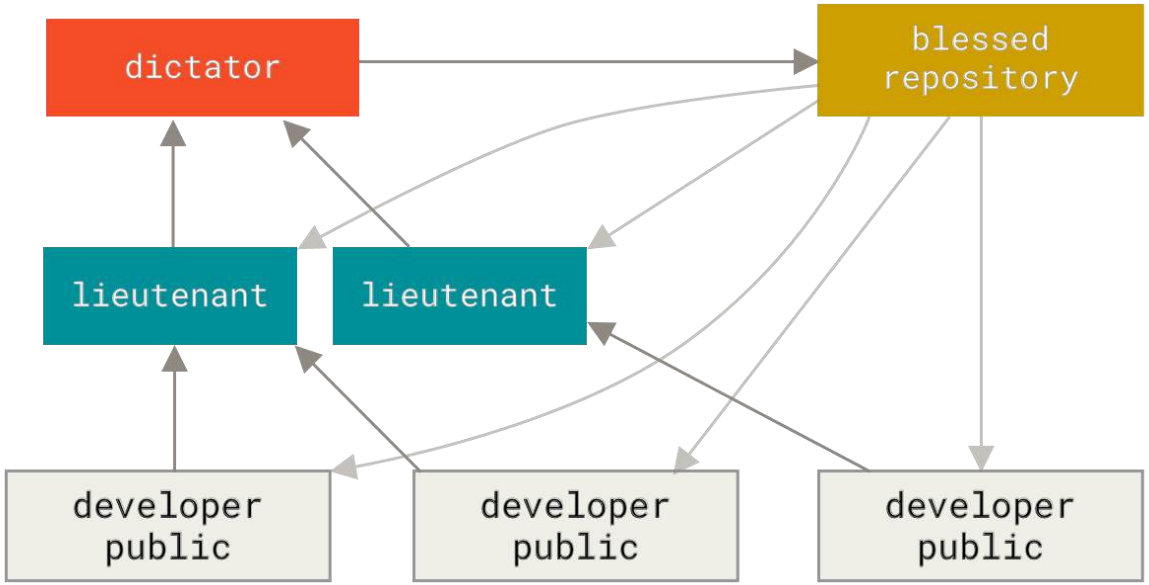
Zentralisierter Workflow



Integration-Manager Workflow

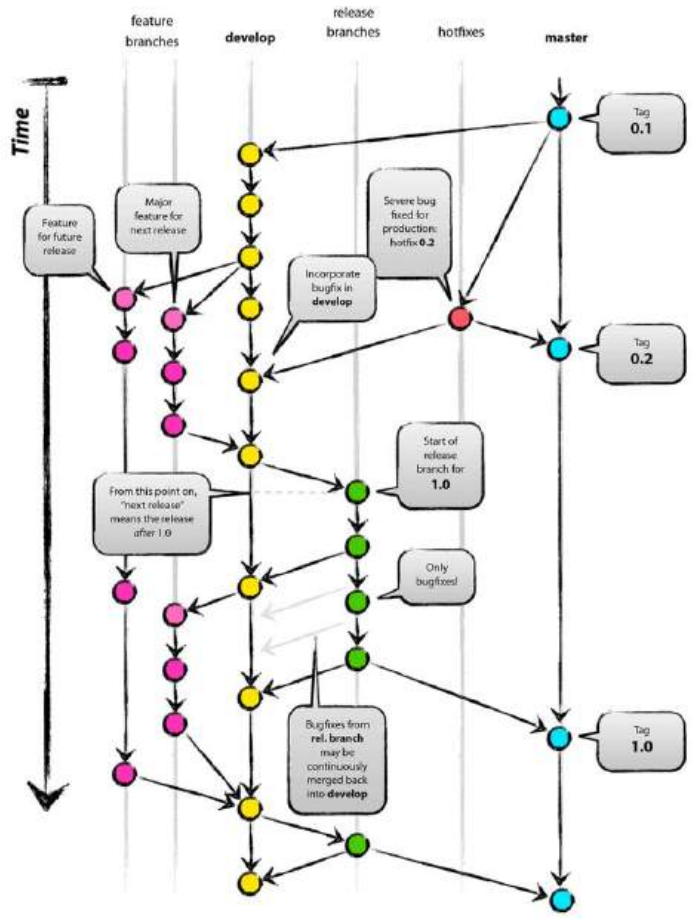


Dictator und Lieutnants Workflow



Quelle: https://git-scm.com/book/en/v2/Distributed-Git-Distributed-Workflows

Git Flow Workflow

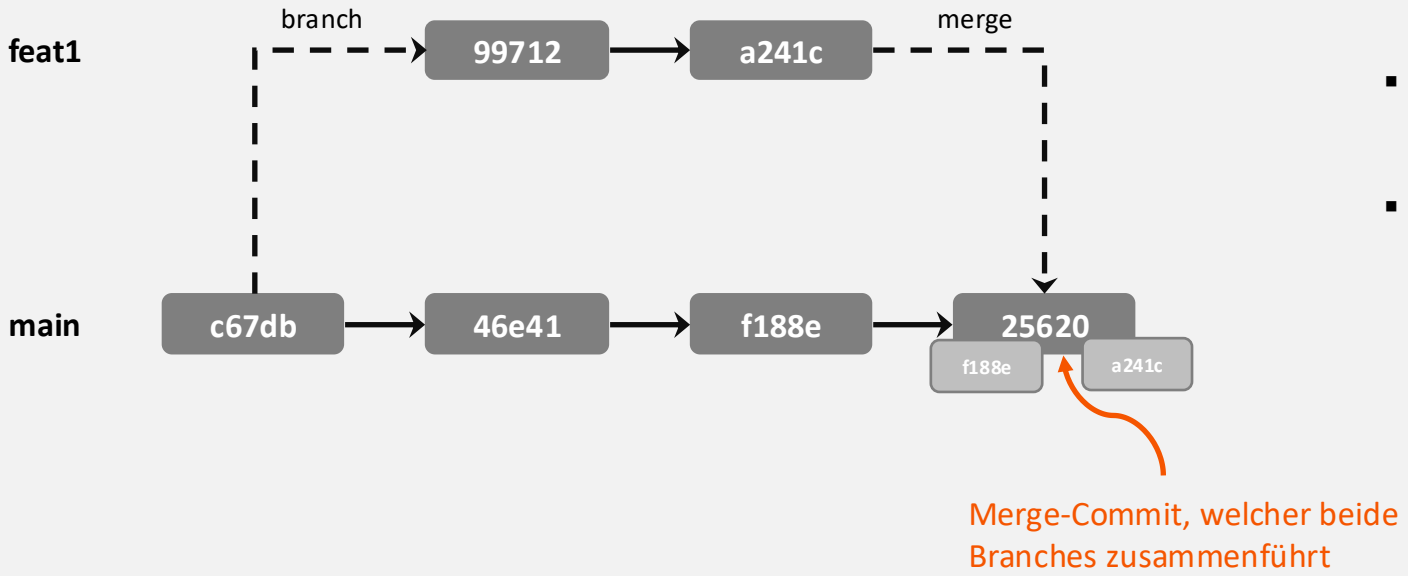


Feature Branch Workflow

- Ähnlich wie der Git Flow Workflow, jedoch wird auf zusätzliche Ebenen, wie z.B. Release- und Hotfix-Branches verzichtet
- Benötigt ein zentrales / geteiltes Repository, wenn im Team gearbeitet wird
- Der main Branch stellt dabei die einzige Änderungshistorie des gesamten Projektes dar
- Die Feature-Branches sollten einen beschreibenden Namen haben, z.B.:
  - feat/animated-login-window
  - feat/1028-integration-postgresql

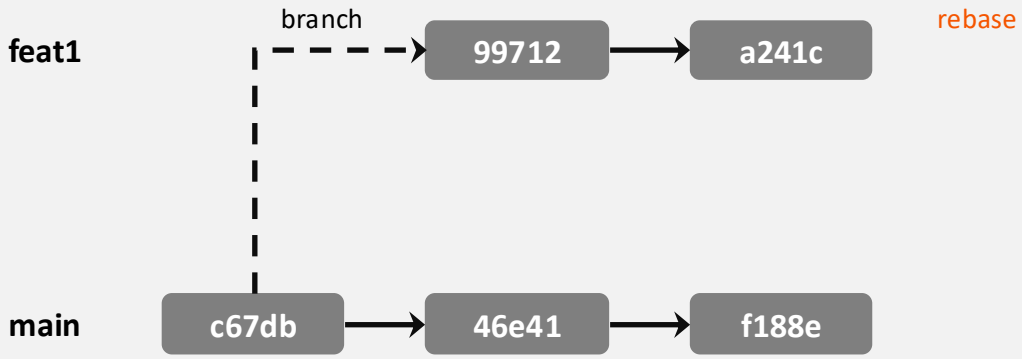
Stellt die ID des Tickets dar, welches in diesem Branch bearbeitet wird (optional)

Feature Branch Workflow

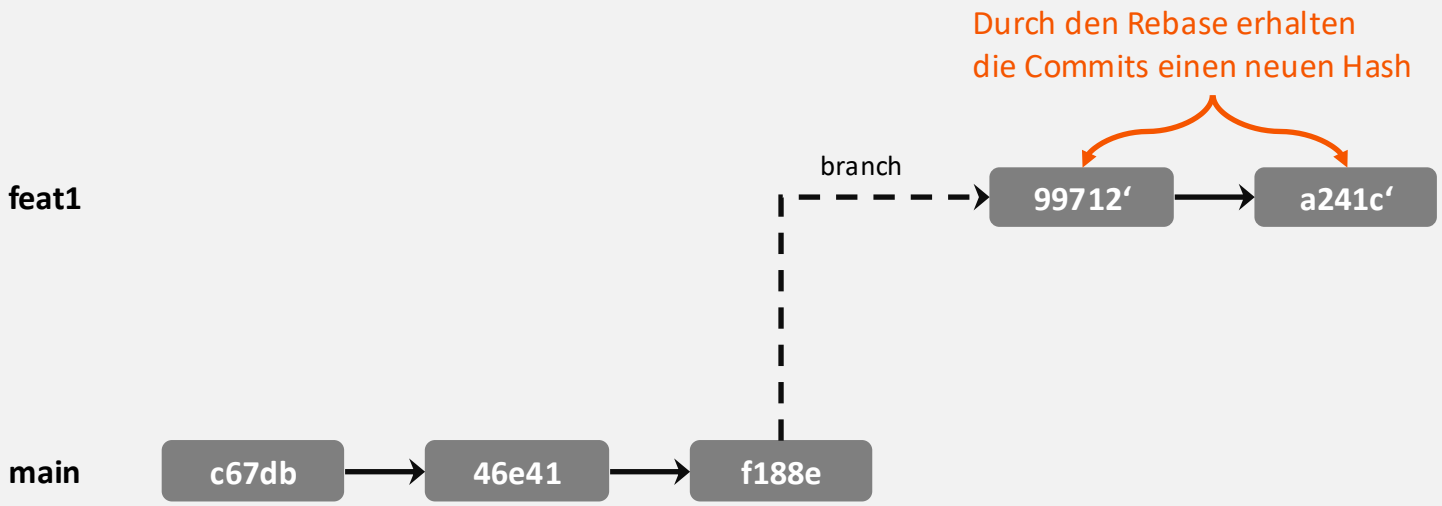


- Es wird ein neuer Merge-Commit erstellt, welcher beide Branches zusammenführt
- Die Person die nicht für das neue Feature zuständig ist, muss Konflikte auflösen, obwohl dies die Aufgabe des Entwicklers sein sollte

Feature Branch Workflow



Feature Branch Workflow



- Die Historie der Commits ist organisiert und “beschreibt“ die Änderungen des Projektes ohne Unterbrechungen
  → keine Merge-Commits
- Auflösung von Merge-Conflicts muss die zuständige Person / Entwickler erledigen
- **Niemals geteilte Branches rebasen!**

Fachhochschule
Dortmund
University of Applied Sciences and Arts

GitHub

GitHub

- Ist gemeinsam mit GitLab im Bereich Open Source der Industriestandard
- Bietet neben der Versionskontrolle weitere Funktionen an
  - Ticketing
  - Continuous Integration
  - Dokumentation
  - Meilensteine und Roadmaps
  - Releases und Package Registries
- Der Kern ist das Git Repository und alle weiteren Funktionen interagieren damit
- Im Internet unter https://www.github.com erreichbar

Ein Account für jedes Teammitglied ist für das Praktikum notwendig!

Issues und Pull Requests

- Issues und Pull Requests sind im Grunde das Gleiche:
  Beschreibung einer (gewünschten) Änderung oder eines Fehlers
  - Issues eher die Beschreibung von gewünschten Änderungen oder Fehler
  - Pull Requests eher die Beschreibung der vorgenommenen Änderungen (ggf. mit Referenz zu Issues)
- Können in Markdown geschrieben werden
- In Commit Messages können Issues referenziert werden und erscheinen in GitHub bei dem jeweiligen Issue
- Pull Requests haben die Extrafunktionalität von sog. Code Reviews
  - Vor der Überführung in den Zielbranch können Anmerkungen zum Code gegeben werden, welche vorher bearbeitet werden sollten
  - In der Praxis ist häufig eine andere Person als die, die es entwickelt hat, als Reviewer tätig

Continuous Integration (GitHub Actions)

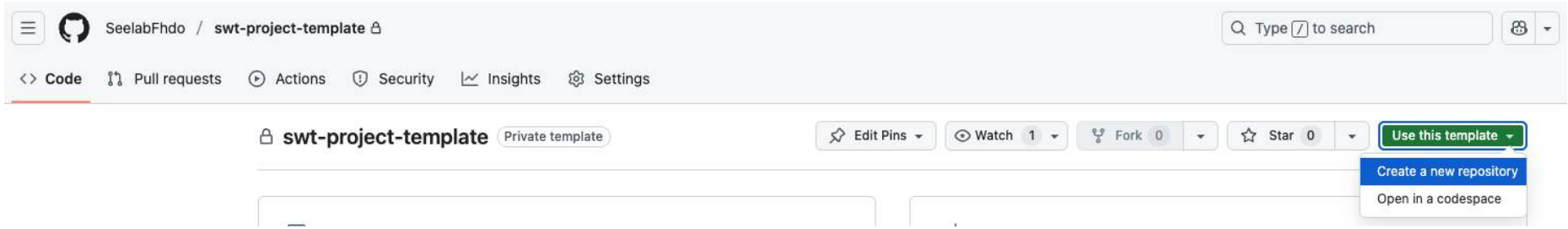
- GitHub bietet ebenfalls die Möglichkeit von Continuous Integration, um bspw. automatische Tests oder Builds (→ nächste Vorlesung) des Source Code durchzuführen
- Werden über yaml Dateien konfiguriert
- Es gibt einen Marketplace für sog. Actions, um den Funktionsumfang zu erweitern
- Im kostenlosen Account sind 2000 GitHub Actions Minuten pro Monat inkl.

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Informationen für das Praktikum

Template Repository für das Praktikum

- Für das Praktikum gibt es ein Template Repository auf GitHub unter: https://github.com/SeelabFhdo/swt-project-template
  - GitHub Actions für Java Projekte vorkonfiguriert
  - Templates für User Stories, Bug Reports und Pull Requests konfiguriert
  - Dockerfile für Java
- Der Gruppenleiter sollte dieses Template nutzen und ein Repository für die Gruppe anlegen und alle Teammitglieder, Dozenten und Tutoren als Collaborators hinzufügen



Template Repository wichtige Informationen

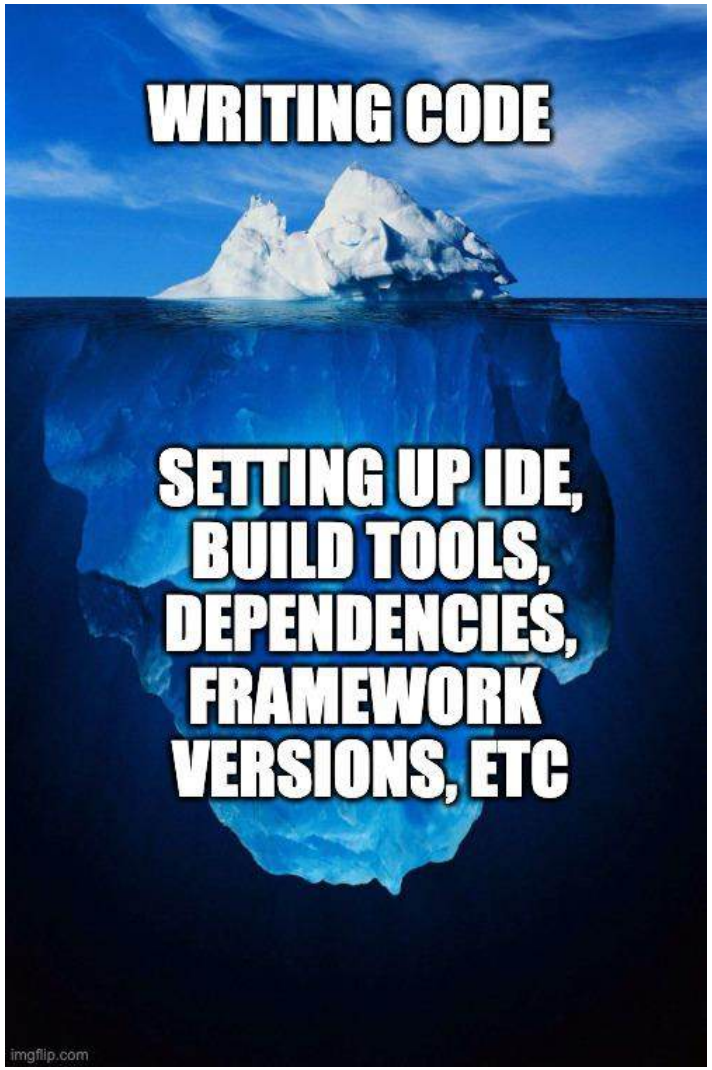
- Wenn Sie ein neues Projekt hinzufügen, dann nicht direkt im Root-Verzeichnis, sondern in einem eigenen Unterverzeichnis
- Der Name des Unterverzeichnisses bzw. des Sub-Module muss in der Datei pom.xml unter Liste “modules“ eingetragen werden
- Sie sollten im Root-Verzeichnis die README.md Datei anpassen, um das Projekt bzw. die Gruppe zu präsentieren und wichtige Informationen zum Betreiben der Software dort zu beschreiben
- Teamleiter: Fügen Sie die Teammitglieder, Dozenten und Tutoren als Collaborators zu dem Projekt hinzu

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Build & Dependency Management

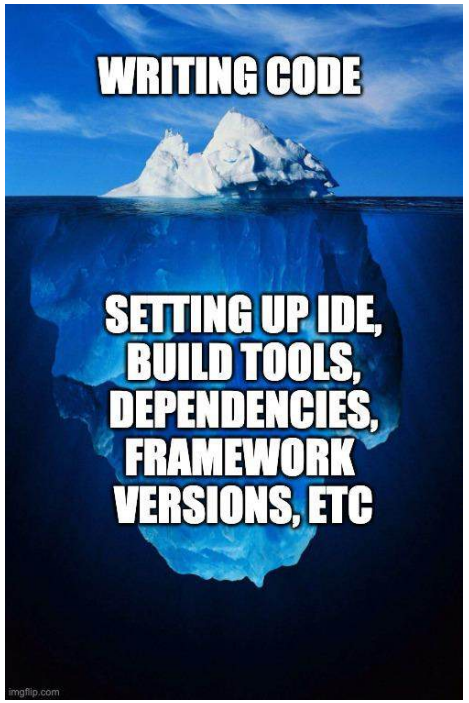
Andreas Diepenbrock

07.04.2025


WRITING CODE
SETTING UP IDE, BUILD TOOLS, DEPENDENCIES, FRAMEWORK VERSIONS, ETC
imgflip.com

Quelle

Problem & Motivation



- Software-Entwicklung
  - Design und Implementierung
  - Build, Test & Deployment
- ”Writing code isn’t everything”
  - Wie gewährleisten wir die **Stabilität** unserer Anwendung? (Über alle Umgebungen hinweg)
  - Wie prüfen wir die **Lauffähigkeit** unserer Anwendung?
  - Wie stellen wir die Anwendung zur **Verfügung**?

Build Management

- Warum Build Management, Eine Java-Datei zu *bauen* ist doch einfach?

```bash
javac src/main/java/de/fh-dortmund/swt2/App.java
```

- Erweiterung um eine weitere Java-Datei mit einer Klasse *Calculator*

```bash
javac src/main/java/de/fh-dortmund/swt2/App.java
➢ src/main/java/de/fhdortmund/swt2/App.java:3: error: cannot find symbol
➢ import de.fhdortmund.swt2.Calculator;
➢ ^
➢ symbol: class Calculator
➢ location: package de.fhdortmund.swt2
➢
➢ src/main/java/de/fhdortmund/swt2/App.java:11: error: cannot find symbol
➢ System.out.println(Calculator.add(5, 3));
➢ ^
➢ symbol: variable Calculator
➢ location: class App
➢ 2 errors
```

- Wir müssen die andere Datei mit *bauen*

```bash
javac src/main/java/de/fhdortmund/swt2/App.java \
    src/main/java/de/fhdortmund/swt2/Calculator.java
```

Build Management

- Verwendung von Bibliotheken?

```bash
javac -d ./target \
  -cp lombok.jar \
  src/main/java/de/fhdortmund/swt2/App.java \
  src/main/java/de/fhdortmund/swt2/utils/Calculator.java
```

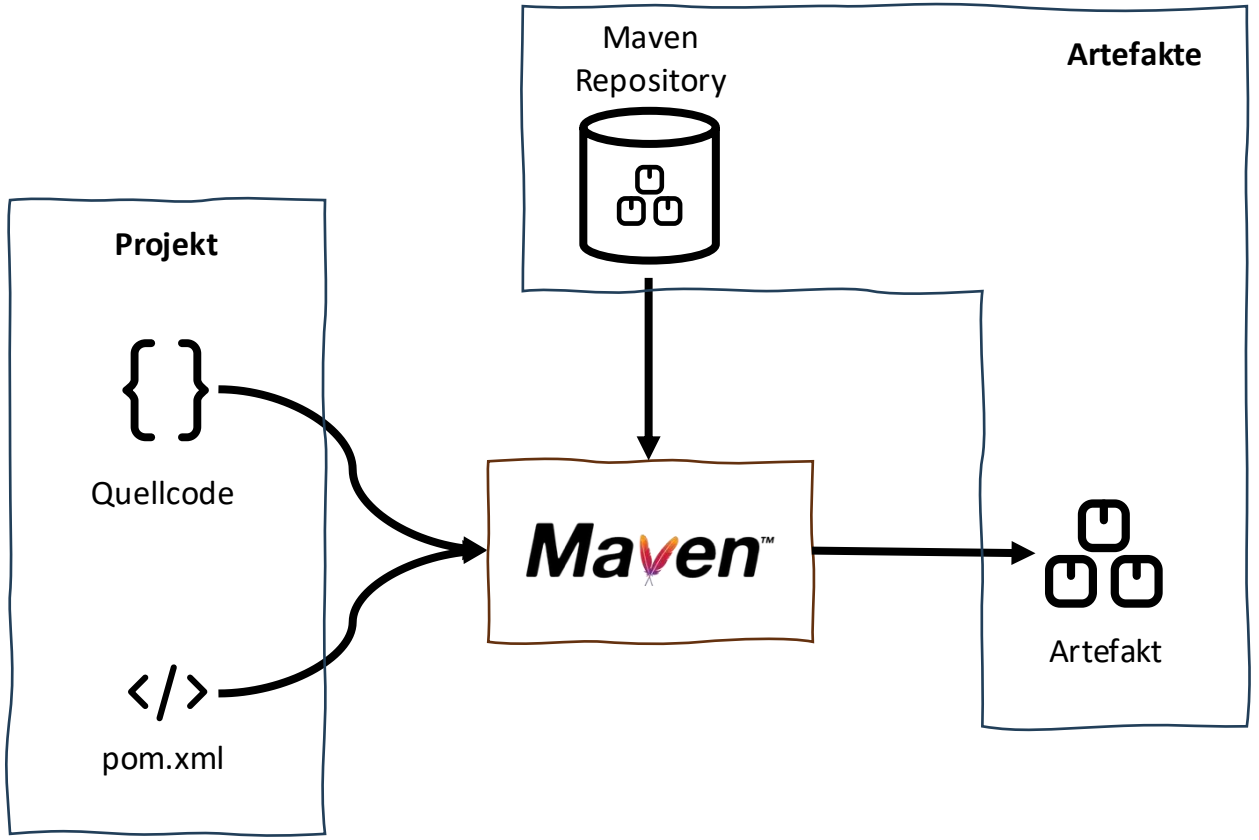
- Und was ist mit Spring?
  Es geht, ist aber deutlich komplizierter!
- Mit wachsender Komplexität des Projekts steigt gleichzeitig die Komplexität des Builds
- Große Software-Projekte erfordern komplizierte Build-Prozesse
- **→ Deshalb verwenden wir Tools für die Unterstützung beim Bauen unserer Anwendung**

Apache Maven Project

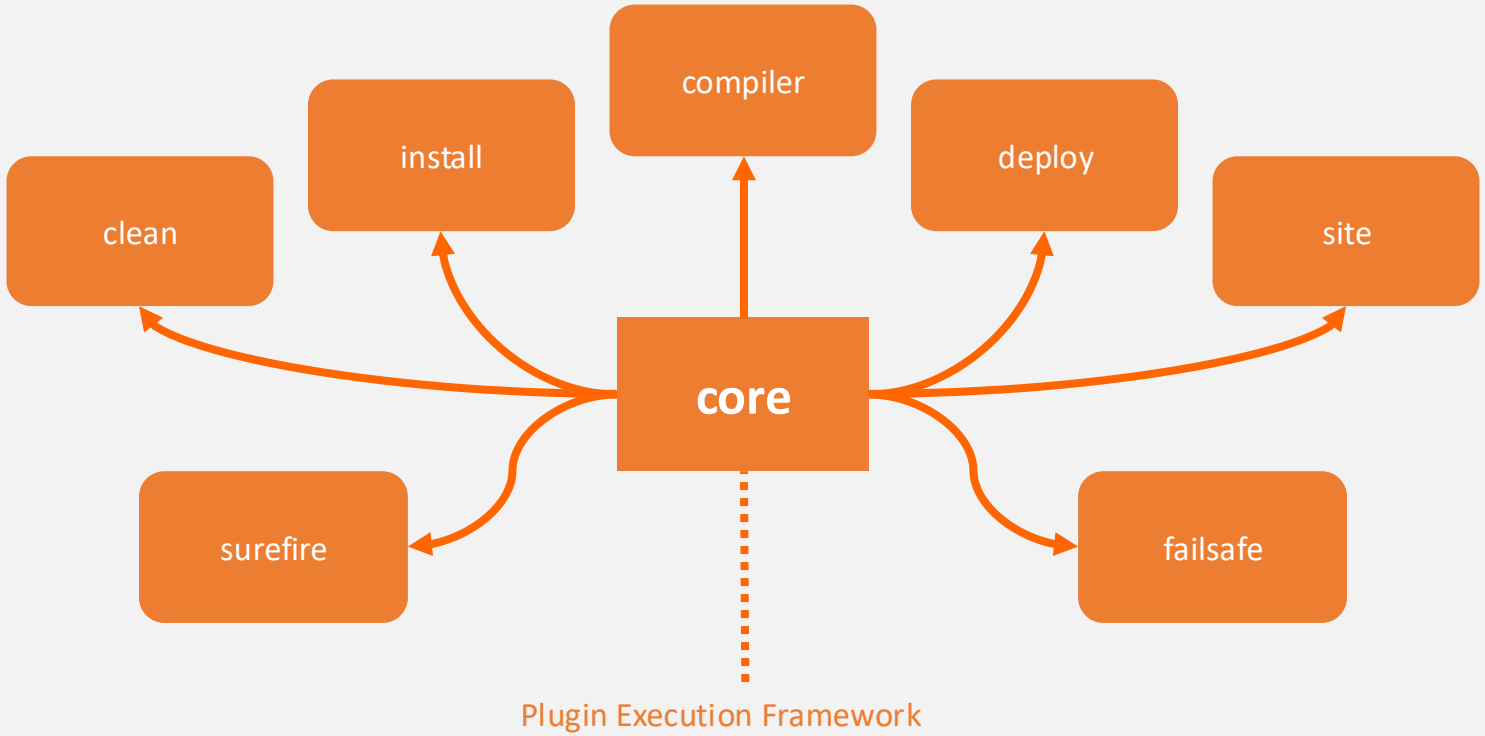
https://maven.apache.org

- Eigenständige Software mit einheitlicher Schnittstelle für Build & Dependency Management
- Deklaratives ***Dependency Management***
- Bereits eingebettet in einige IDEs oder als Plugin installierbar
- „Convention over configuration“
  - Konventionen als *default* verwendet
- Automatisierung
  - Bauen der Anwendung
  - Qualitätssicherung
  - Dependencies
  - Generierung von Dokumentation
  - …

Projekte und Artefakte



Plugin Ecosystem



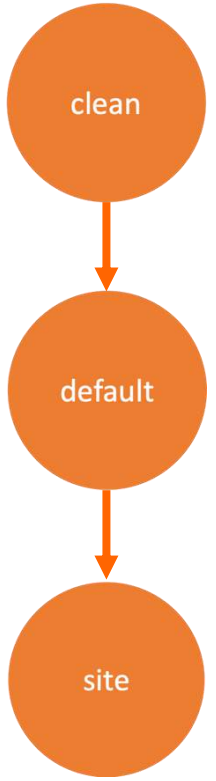
pom.xml

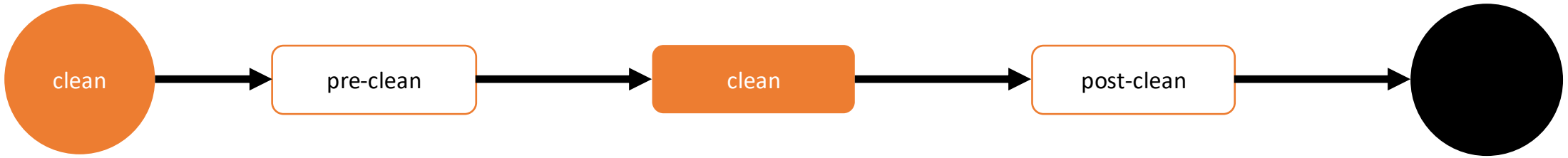
- Enthält Projekt informationen
  - Informationen zur Anwendung
    - **groupId**: *Identifier* der Organisation (*de.fh-dortmund.swt2)*
    - **artifactId**: *Identifier* des zu erzeugenden Artefakts (*toller-service)*
    - **version**: Version des Artefakts (*0.1-SNAPSHOT*)
    - **name**: Anzeigename für die Anwendung (*Toller Service*)
    - **packaging**: Ausgabeformat der Anwendung (Standard: *jar*)
  - **properties** Eigenschaften
    - Plugin-Eigenschaften können gesetzt werden (*<maven.compiler.target>1.8</maven.compiler.target>*)
    - Eigenschaften können wiederverwendet werden (*${hibernate.version}*)
  - **dependencies** Abhängigkeiten
  - **build** Konfiguration des Builds

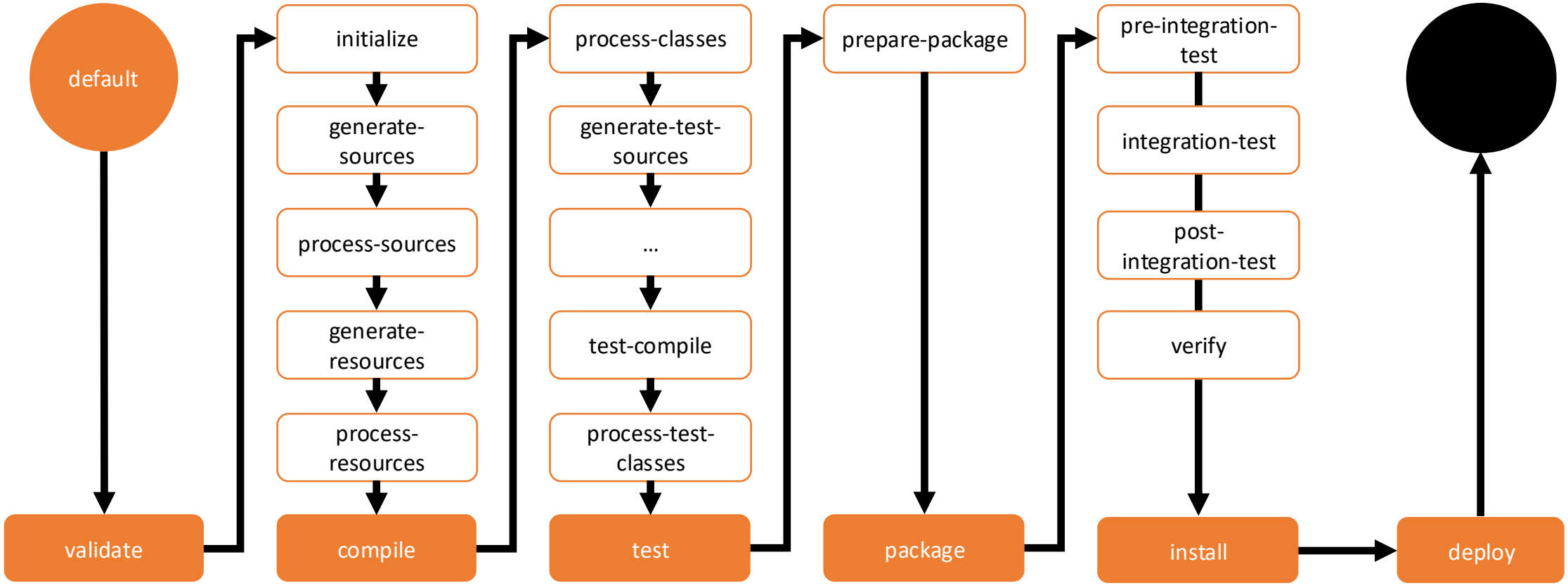
```
<?xml version="1.0" encoding="UTF-8"?>
<project ...>

        <modelVersion>4.0.0</modelVersion>
```
Es existiert nur Version 4.0.0 des POM-Model
```
        <groupId>de.fh-dortmund.swt2</groupId>
        <artifactId>toller-service</artifactId>
        <version>0.0.0-SNAPSHOT</version>
        <name>Toller Service</name>

        <properties>...</properties>

        <dependencies>...</dependencies>

        <build>...</build>
</project>
```

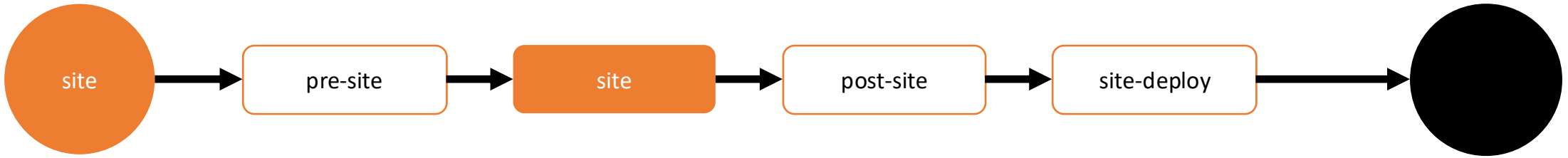
Lifecycles

- Gruppierung von *Phasen*
- Phasen
  - Bauen aufeinander auf
  - Jede Phase beinhaltet alle vorherigen Phasen
  - Abstrakter Schritt im Lifecycle
- Lifecycle
  - clean: Aufräumen
  - default: Bauen und Bereitstellen
  - site: Dokumentation

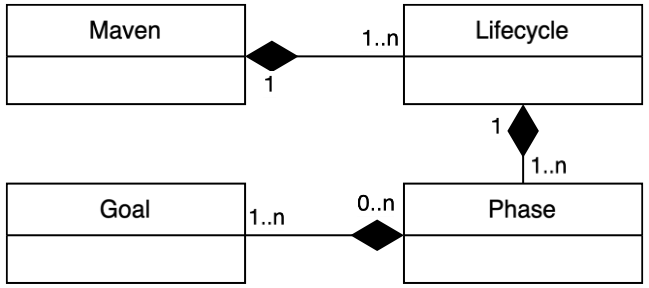



clean
pre-clean
clean
post-clean


default
validate
initialize
generate-sources
process-sources
generate-resources
process-resources
compile
process-classes
generate-test-sources
...
test-compile
process-test-classes
test
prepare-package
package
pre-integration-test
integration-test
post-integration-test
verify
install
deploy


site
pre-site
site
post-site
site-deploy

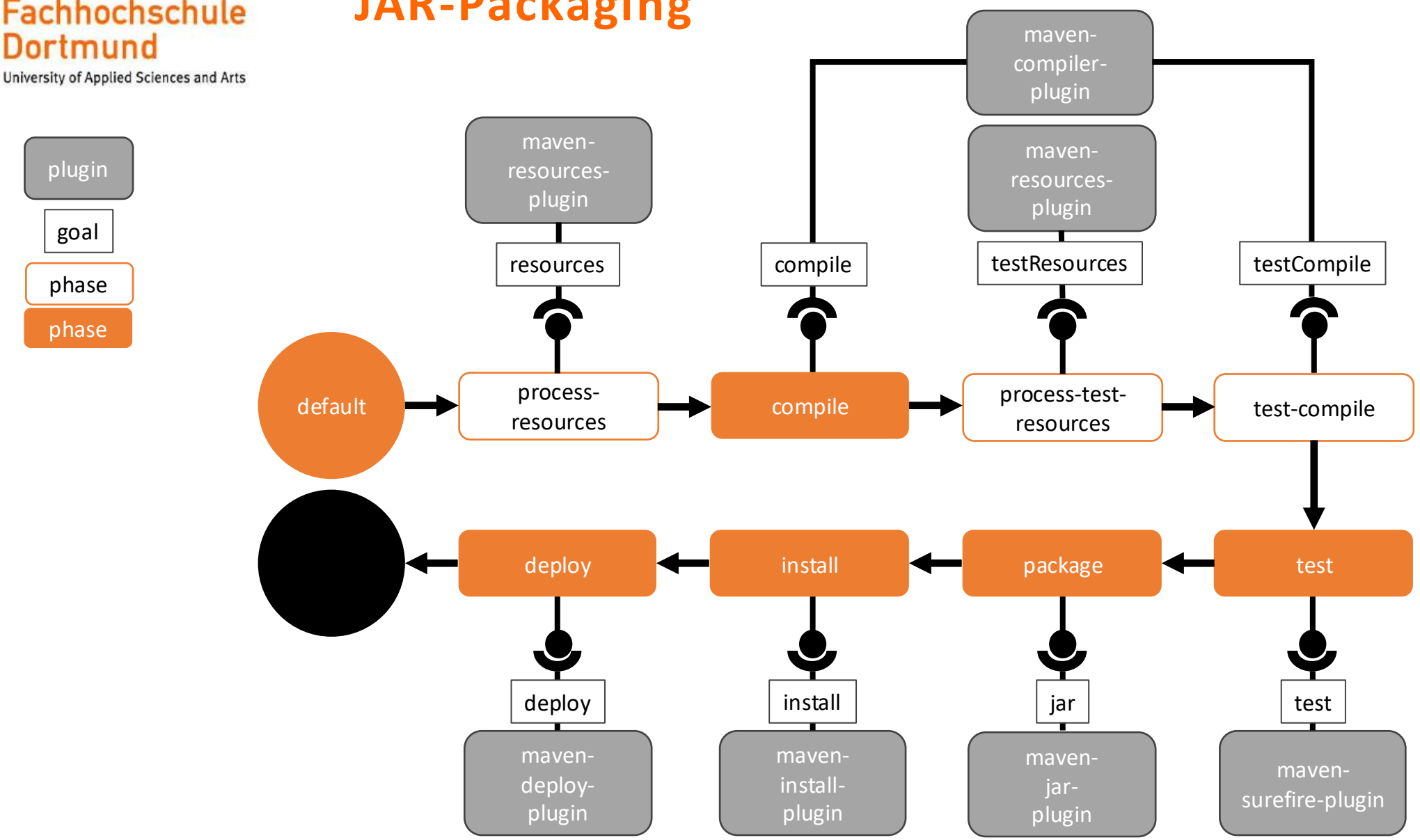
Goals



- ***Phasen*** sind abstrakt und dienen der Strukturierung des Build-Prozesses
- Zur Anreicherung des Build-Prozesses werden ***Goals*** verwendet
- ***Goals*** werden von ***Plugins*** definiert und an ***Phasen*** gebunden
- „Convention over configuration“
  - Maven definiert Standard ***Goals*** abhängig von gewählter Paketierung

JAR-Packaging

we focus on students



Ausführung

- Maven Command Line

```bash
mvn --help

usage: mvn [options] [<goal(s)>] [<phase(s)>]
```

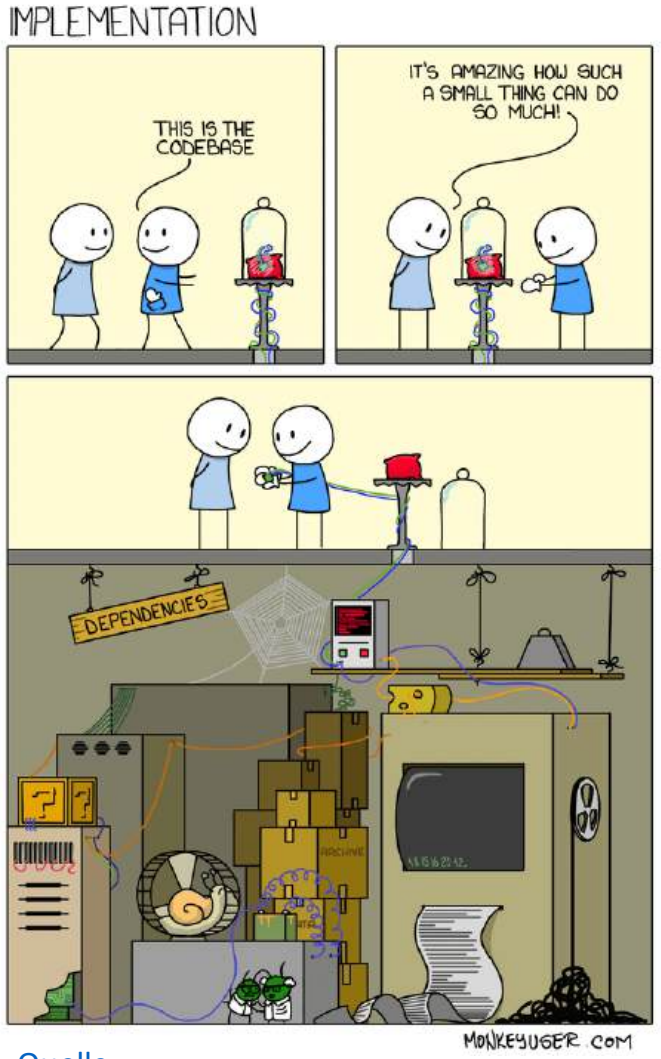
- Ausführung der Phasen clean und install aus den Lifecycles clean und default

```bash
mvn clean install
```

- Ausführung eines Goals direkt ist auch möglich

```bash
mvn compiler:compile compiler:testCompile
```

Dependency Management



Quelle

Dependency Management

- Verwalten der Abhängigkeiten und genutzter Versionen
  Auch die Bekanntmachung der notwendigen Abhängigkeiten ist wichtig!
- Automatische Verwaltung transitiver Abhängigkeiten
- Automatischer Download von Abhängigkeiten
- *Dependency Scoping*
  → Einschränken des Gültigkeitsbereiches von Abhängigkeiten

- Versions Semantik
  Semantic Versioning

Dependency Management in Maven

- Deklarative Abhängigkeitsverwaltung in *pom.xml*
  <dependencies>
- Auch Installation über CLI möglich

```bash
mvn install:install-file -Dfile=pom.xml -DgroupId="org.projectlombok" -DartifactId="lombok"
-Dversion="1.18.38" -Dpackaging="jar"
```

- Zentrales Repository für Maven Artefakte
- Installation von Abhängigkeit
  - Maven prüft ob die Abhängigkeit im lokalen Repository liegt
    - Falls nein: Maven lädt die Abhängigkeit ins lokale Repository herunter
  - Maven nutzt lokales Repository für Build-Vorgänge

- Hinweis: Auch private Repositories können genutzt werden werden aber im Rahmen dieser Vorlesung nicht betrachtet. Bei Interesse:
  https://www.baeldung.com/maven-several-repositories

Dependencies

- Abhängigkeiten werden identifiziert über *Maven Coordinates*
  - *groupId*
  - *artifactId*
  - *version*
- Maven Central verfügbar unter: https://mvnrepository.com
- Bietet Snippets für verschiedene Build-Tools an (Maven, Gradle, …):

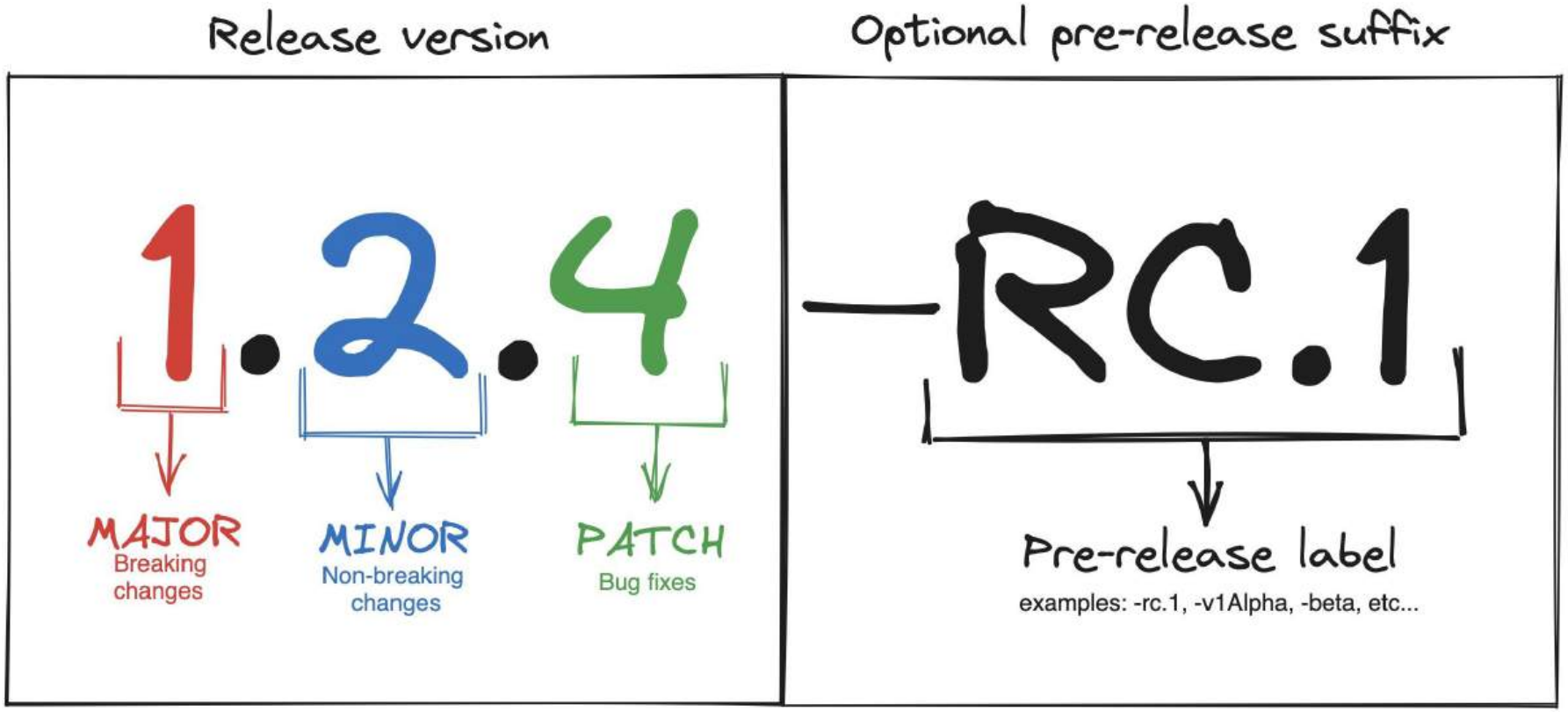
```
<!-- https://mvnrepository.com/artifact/org.projectlombok/lombok -->
<dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.38</version>
</dependency>
```

Transitive Abhängigkeiten

```
[INFO] --- dependency:3.8.1:tree (default-cli) @ toller-service ---
[INFO] de.fhdortmund.swt2:toller-service:jar:0.0.1-SNAPSHOT
[INFO] +- org.springframework.boot:spring-boot-starter:jar:3.4.4:compile
[INFO] |  +- org.springframework.boot:spring-boot:jar:3.4.4:compile
[INFO] |  |  \- org.springframework:spring-context:jar:6.2.5:compile
[INFO] |  |     +- org.springframework:spring-aop:jar:6.2.5:compile
[INFO] |  |     +- org.springframework:spring-beans:jar:6.2.5:compile
[INFO] |  |     +- org.springframework:spring-expression:jar:6.2.5:compile
[INFO] |  |     \- io.micrometer:micrometer-observation:jar:1.14.5:compile
[INFO] |  |        \- io.micrometer:micrometer-commons:jar:1.14.5:compile
[INFO] |  +- org.springframework.boot:spring-boot-autoconfigure:jar:3.4.4:compile
[INFO] |  +- org.springframework.boot:spring-boot-starter-logging:jar:3.4.4:compile
[INFO] |  |  +- ch.qos.logback:logback-classic:jar:1.5.18:compile
[INFO] |  |  |  \- ch.qos.logback:logback-core:jar:1.5.18:compile
[INFO] |  |  +- org.apache.logging.log4j:log4j-to-slf4j:jar:2.24.3:compile
[INFO] |  |  |  \- org.apache.logging.log4j:log4j-api:jar:2.24.3:compile
[INFO] |  |  \- org.slf4j:jul-to-slf4j:jar:2.0.17:compile
[INFO] |  +- jakarta.annotation:jakarta.annotation-api:jar:2.1.1:compile
[INFO] |  +- org.springframework:spring-core:jar:6.2.5:compile
[INFO] |  |  \- org.springframework:spring-jcl:jar:6.2.5:compile
[INFO] |  \- org.yaml:snakeyaml:jar:2.3:compile
[INFO] \- org.springframework.boot:spring-boot-starter-test:jar:3.4.4:test
[INFO]    +- org.springframework.boot:spring-boot-test:jar:3.4.4:test
[INFO]    +- org.springframework.boot:spring-boot-test-autoconfigure:jar:3.4.4:test
[INFO]    +- com.jayway.jsonpath:json-path:jar:2.9.0:test
[INFO]    |  \- org.slf4j:slf4j-api:jar:2.0.17:compile
[INFO]    +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.2:test
[INFO]    |  \- jakarta.activation:jakarta.activation-api:jar:2.1.3:test
[INFO]    +- net.minidev:json-smart:jar:2.5.2:test
[INFO]    |  \- net.minidev:accessors-smart:jar:2.5.2:test
[INFO]    |     \- org.ow2.asm:asm:jar:9.7.1:test
[INFO]    +- org.assertj:assertj-core:jar:3.26.3:test
[INFO]    |  \- net.bytebuddy:byte-buddy:jar:1.15.11:test
[INFO]    +- org.awaitility:awaitility:jar:4.2.2:test
[INFO]    +- org.hamcrest:hamcrest:jar:2.2:test
[INFO]    +- org.junit.jupiter:junit-jupiter:jar:5.11.4:test
[INFO]    |  +- org.junit.jupiter:junit-jupiter-api:jar:5.11.4:test
[INFO]    |  |  +- org.opentest4j:opentest4j:jar:1.3.0:test
[INFO]    |  |  +- org.junit.platform:junit-platform-commons:jar:1.11.4:test
[INFO]    |  |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO]    |  +- org.junit.jupiter:junit-jupiter-params:jar:5.11.4:test
[INFO]    |  \- org.junit.jupiter:junit-jupiter-engine:jar:5.11.4:test
[INFO]    |     \- org.junit.platform:junit-platform-engine:jar:1.11.4:test
[INFO]    +- org.mockito:mockito-core:jar:5.14.2:test
[INFO]    |  +- net.bytebuddy:byte-buddy-agent:jar:1.15.11:test
[INFO]    |  \- org.objenesis:objenesis:jar:3.3:test
[INFO]    +- org.mockito:mockito-junit-jupiter:jar:5.14.2:test
[INFO]    +- org.skyscreamer:jsonassert:jar:1.5.3:test
[INFO]    |  \- com.vaadin.external.google:android-json:jar:0.0.20131108.vaadin1:test
[INFO]    +- org.springframework:spring-test:jar:6.2.5:test
[INFO]    \- org.xmlunit:xmlunit-core:jar:2.10.0:test
```

- Transitive Abhängigkeiten sind Abhängigkeiten der Abhängigkeiten
- Direkte und transitive Abhängigkeiten => *Dependency Tree*
  - Anzeige mit dem Befehl
    mvn dependency:tree
  - Syntax des *Dependency Trees*
    <group>:<artifactId>:<version>:<packaging>:<scope>
- Konfliktauflösung: Nächste Abhängigkeit zum Wurzel-Element
- Gruppierung von Abhängigkeiten
  - spring-boot-starter: Reduzierung der *manuellen* Abhängigkeiten
  - Achtung: Gruppierung führt zu neuen Ebenen im Baum

Semantic Versioning



Quelle

Projektstruktur

- Projektstruktur folgt Konventionen
  - Vorhersehbare Struktur für Plugins
  - Erleichtert Einarbeitung von Entwicklern
- Standard Struktur
  - ./src/main/java: Java Quellcode
  - ./src/main/resources: Ressourcen wie bspw. Bilder oder Konfigurationsdateien,
  - ./src/test/java: Java Quellcode für Tests (Uni-Tests, Integrations-Tests)
  - ./src/test/resources: Ressourcen für Tests
  - ./target: Ausgabe für Build-Artefakte

Multi-Modul Projekt

```
Parent Project
        |-- Module 1
        |         |
        |         `-- pom.xml
        |
        |-- Module 2
        |         |
        |         `-- pom.xml
        |-- Module 3
        |         |
        |         `-- pom.xml
        |
        |
        `-- pom.xml
```

- Strukturierungsmöglichkeit für größere Projekte
- Modularisierung auf Basis von bspw. Domänen oder technische Belangen
- Definition von gemeinsamen Abhängigkeiten oder Plugin-Konfigurationen
- Abwägung *höheren* Komplexität gegenüber dem Mehrwert der singulären Konfiguration
- Bauen der Anwendung im Dockerfile umständlicher, da Parent-POM notwendig ist

Continous *

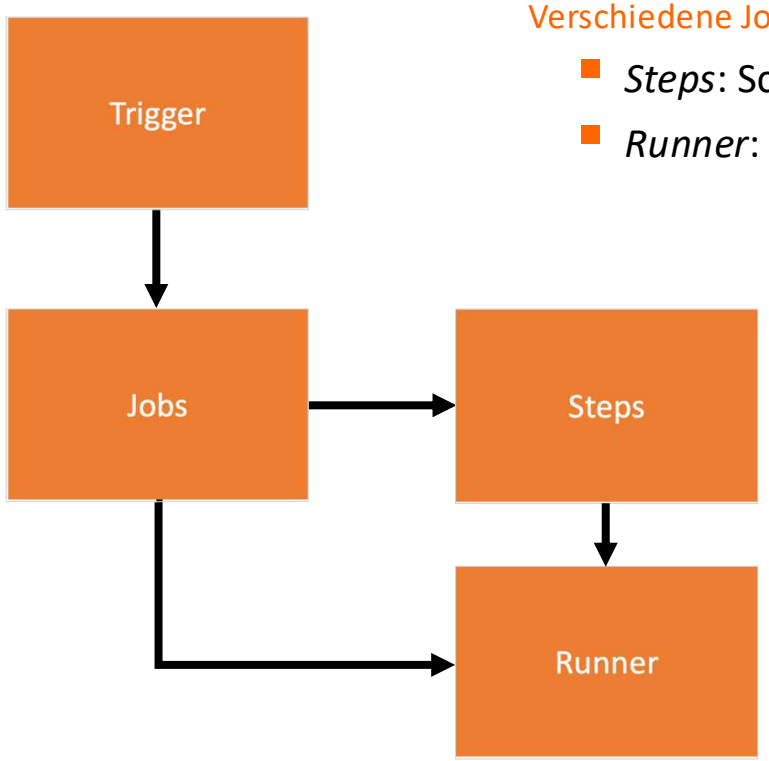
- Continous
  - Integration → Zusammenführung der Bestandteile einer Anwendung
    Nach jedem Commit/Push wird der Code getestet und zusammengeführt.
  - Delivery → Erstellung von Versionen die Verfügbar gemacht werden
    Jeder „Zustand“ eines Systems wird bereitgestellt in einer „Dev“-Umgebung
  - Deployment → Erstellung von Produktiv-Versionen
    Nachdem Fachtests in „Dev“ erfolgt sind können Versionen den Kunden bereitgestellt werden
- Geht häufig einher mit Microservices & Containerisierung
- Für die Unterstützung von Continous * → GitHub Actions

Github Actions

- *Workflow Automation*
  - Continous *
  - Benutzerdefinierte Aufgaben
- → Umfangreiche Automatisierungs-Plattform
- GitHub Actions sind
  - Workflow Engine
  - Workflow Schritte
- Workflow-Definition in YAML
  Hinweis: Einrückungen sind wichtig in YAML!

Komponenten

- *Workflow*: Das umschließende Konzept für
  - *Trigger*: Ein Event wann ein Workflow ausgeführt werden soll
  - *Jobs*: Gruppiert Schritte, die zusammen ausgeführt werden sollen
    Verschiedene Jobs können parallel ausgeführt werden oder voneinander abhängig sein
    - *Steps*: Schritte eines Jobs
    - *Runner*: Ausführungsumgebung eines Jobs



Trigger

- Eingeleitet durch *on* Keyword

- *push*: Ausführung bei push ins Repository
  Bauen, Testen, Linten. Ggf. auf bestimmten Branches sogar Integrations-Tests ausführen
- *pull_request*: Ausführung bei Aktivität auf Pull-Requests
  C*-Aufgaben ausführen, Container bauen, Testabdeckung prüfen
- *schedule*: Ausführung von regelmäßig anfallenden Aufgaben
  Abhängigkeitsprüfung (Sicherheitslücken)

- *workflow_call:* Erlaubt die Ausführung des Workflows von anderen
- *workflow_dispatch:* Erlaubt manuelle Ausführung eines Workflows
- *workflow_run:* Erlaubt „Workflow-Chaining“

```
on:
  <trigger>:
    <parameter>
```

Jobs

- Eingeleitet durch *jobs* Keyword
- Ein Job
  - enthält für die Ausführung notwendige Schritte
  - kann Abhängig von anderen Jobs sein
  - ist in sich geschlossen
- Jobs werden, wenn keine Abhängigkeiten blockieren, parallel ausgeführt
- Abhängigkeit mittles *job-id* in Job-Beschreibung

  needs: <job.id>

```
jobs:
  <job-id>:
    <parameters>
```

Steps

- Eingeleitet durch *steps* Keyword
- Ein *Step* ist
  - Shell-Skript (run: <command>)
  - GitHub-Action
- *Steps* werden nacheinander in der vorgegebenen Reihenfolge, in einer gemeinsamen Umgebung ausgeführt

```
jobs:
  steps:
  - name: Checkout
    uses: actions/checkout@v4
  - name: Build with Maven
    run:  mvn --batch-mode --update-snapshots verify
```

GitHub-Action

Shell-Skript

Runner

- Bildet die *Workflow-Runtime* ab
- Unterscheidung
  - GitHub-hosted
  - Self-hosted
- Muss im Job referenziert werden
  runs-on: [<tags>]
- Runner auf die **alle** Tags zutreffen werden verwendet
  runs-on: [ubuntu-latest]
  runs-on: [self-hosted, linux, x64, gpu]

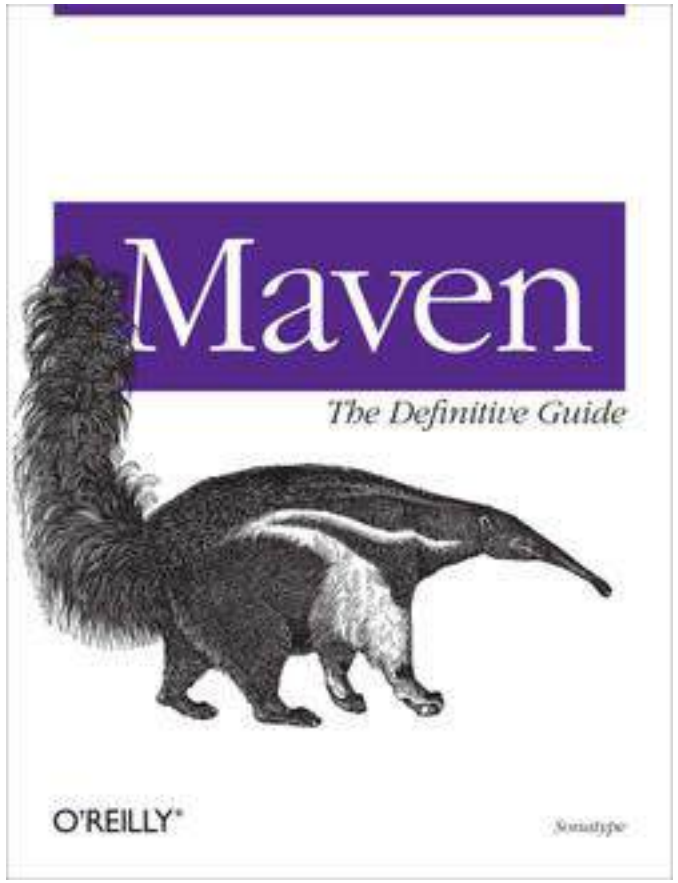
Parametrisierung von Workflows

- Den gleichen Workflow mehrfach ausführen mit unterschiedlichen Parametern
- Matrix-Strategie → Alle möglichen Variationen
- Referenzieren der Parameter in *Jobs* mittels ${{matrix.<name>}}

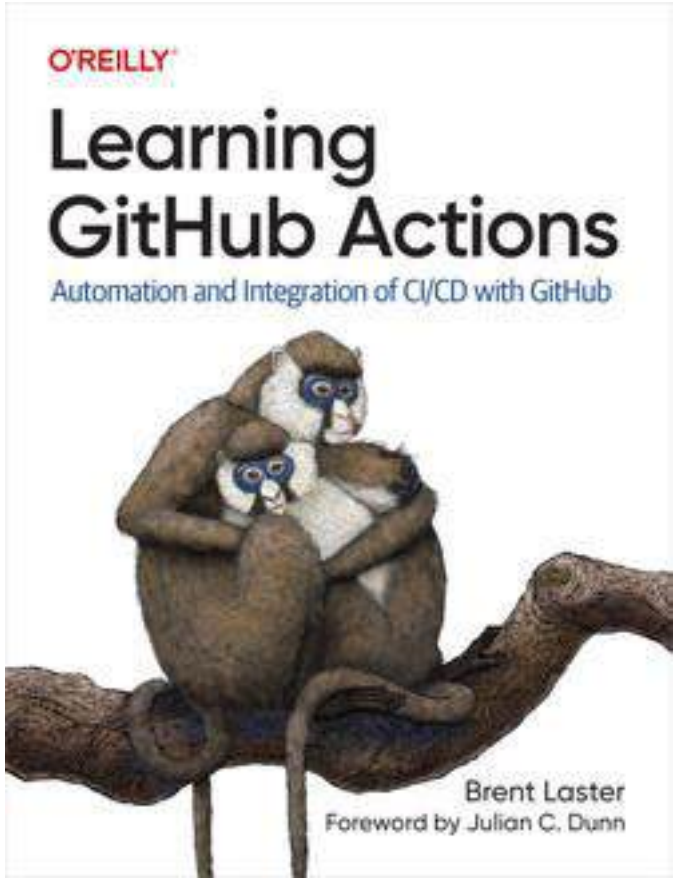
Run	matrix.os	matrix.jdk
1	ubuntu-latest	20
2	windows-latest	20
3	ubuntu-latest	22
4	windows-latest	22

```yaml
jobs:
  strategy:
    matrix:
      os: [ubuntu-latest, windows-latest]
      jdk: [20, 22]
  runs-on: ${{ matrix.os }}
  steps:
  - name: Set up JDK
    uses: actions/setup-java@v4.5.0
    with:
      java-version: '${{ matrix.jdk }}'
      distribution: 'temurin'
      cache: maven
```

Literatur



https://www.oreilly.com/library/view/maven-the-definitive/9780596517335/



https://www.oreilly.com/library/view/learning-github-actions/9781098131067/

we focus on students

Softwaretechnik 2

Architekturmodellierung

Fachhochschule
Dortmund
University of Applied Sciences and Arts

© Prof. Dr. Sabine Sachweh

Struktur

1. Einführung

2. Architekturmodellierung

2.1 Begriffe

2.2 (Software-)Architekturmodellierung

2.3 Verteilungsdiagramm

© Prof. Dr. Sabine Sachweh

Begriffe

Wie wird technisch gebaut?

„[When talking architecture] … we're talking something that's important" [Fowler 2003]

Bedeutung der Architekturmodellierung

- Wird ein Projekt basierend auf einer **unzureichenden Planung/Modellierung** durchgeführt, führt es nicht selten zu einem **nicht zufriedenstellenden Produkt**.
- Ziel ist es ein System in **funktional getrennte Blöcke** zu teilen. → **Modularisierung**
- Die Modularisierung ist notwendig, um ein System entwerfen zu können, das die **Grundanforderungen an eine Software** – Skalierbarkeit, Wartbarkeit – erfüllen kann.

© Prof. Dr. Sabine Sachweh

Begriffe

Begriffsverwirrung

- Architektur kann eine sehr **grobe Form der Darstellung** der Systemteile bedeuten, die dazu dient einen raschen Überblick zu erhalten.
- Architektur kann aber auch den **technisch detaillierten Aufbau** eines konkreten Systemteils bedeuten.

Verwendung der Begriffe „Architektur“ und „Entwurf/Design“ für diese Veranstaltung

- **(Software)Architektur** als Architektur im „Großen“
- **Entwurf/Design** als Architektur im „Kleinen“

© Prof. Dr. Sabine Sachweh

Ziele und Aufgabe der Architekturmodellierung

- Beschreibung eines Systems, das die **Anforderungen** des Kunden berücksichtigt
- Beschreibung des Systems auf **technischer Ebene**
- **Ausgangspunkt für Implementierung**
- **Kommunikationsmittel** zwischen den Projekt-Stakeholdern
- Ausdruck des **Architekturstils**

© Prof. Dr. Sabine Sachweh

Struktur

1. Einführung

2. Architekturmodellierung

2.1 Begriffe

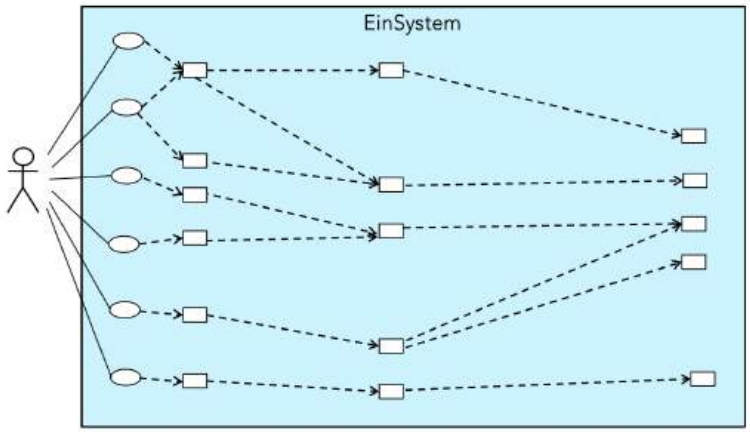
2.2 (Software-)Architekturmodellierung

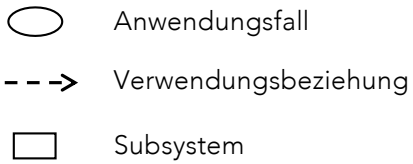
2.3 Verteilungsdiagramm

© Prof. Dr. Sabine Sachweh

Beispiel: Vom Analysemodell zur Systemdekomposition

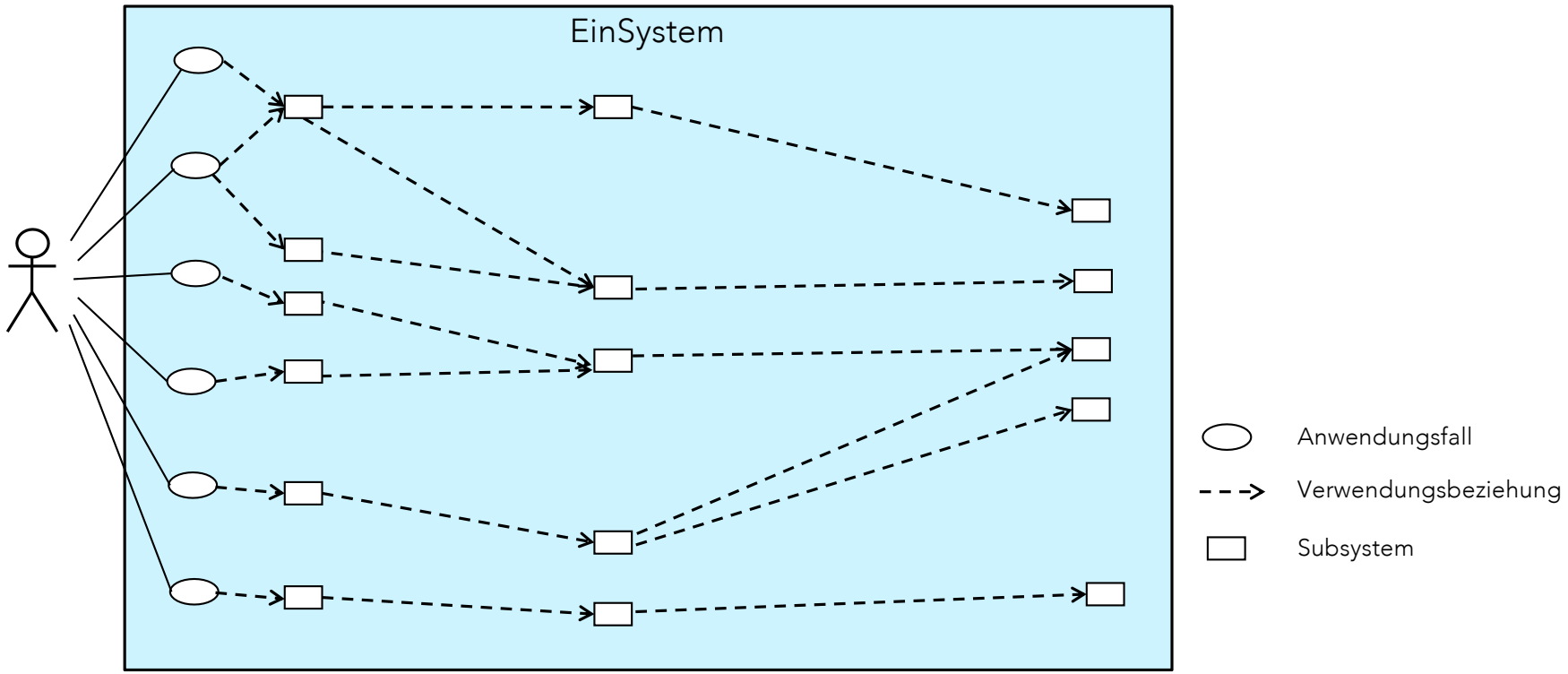
- Gruppieren nach ähnlichen Funktionalitäten (Komponenten bilden)
- Angebotene und genutzte Dienste identifizieren (Schnittstellen)





© Prof. Dr. Sabine Sachweh

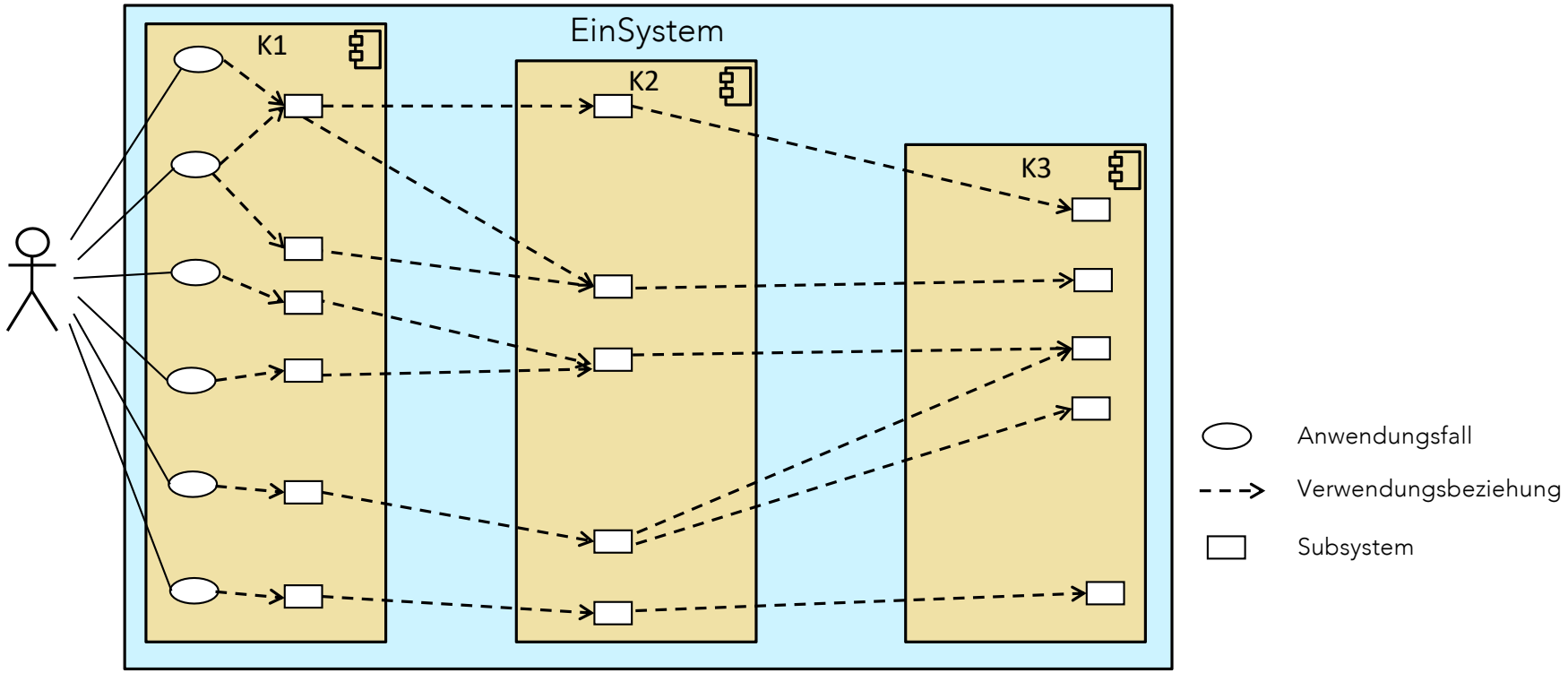
Architekturmodellierung



© Prof. Dr. Sabine Sachweh

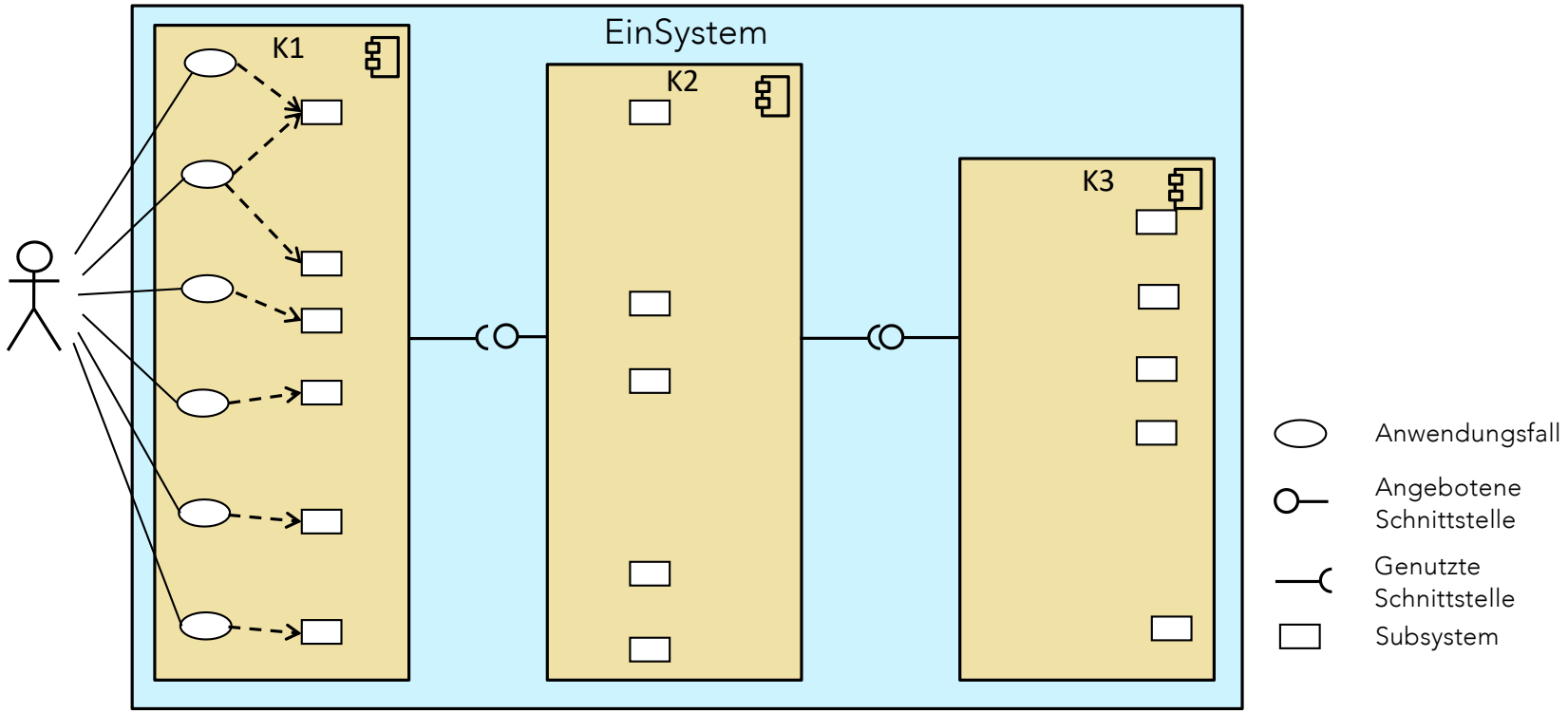
Architekturmodellierung

we focus on students



© Prof. Dr. Sabine Sachweh

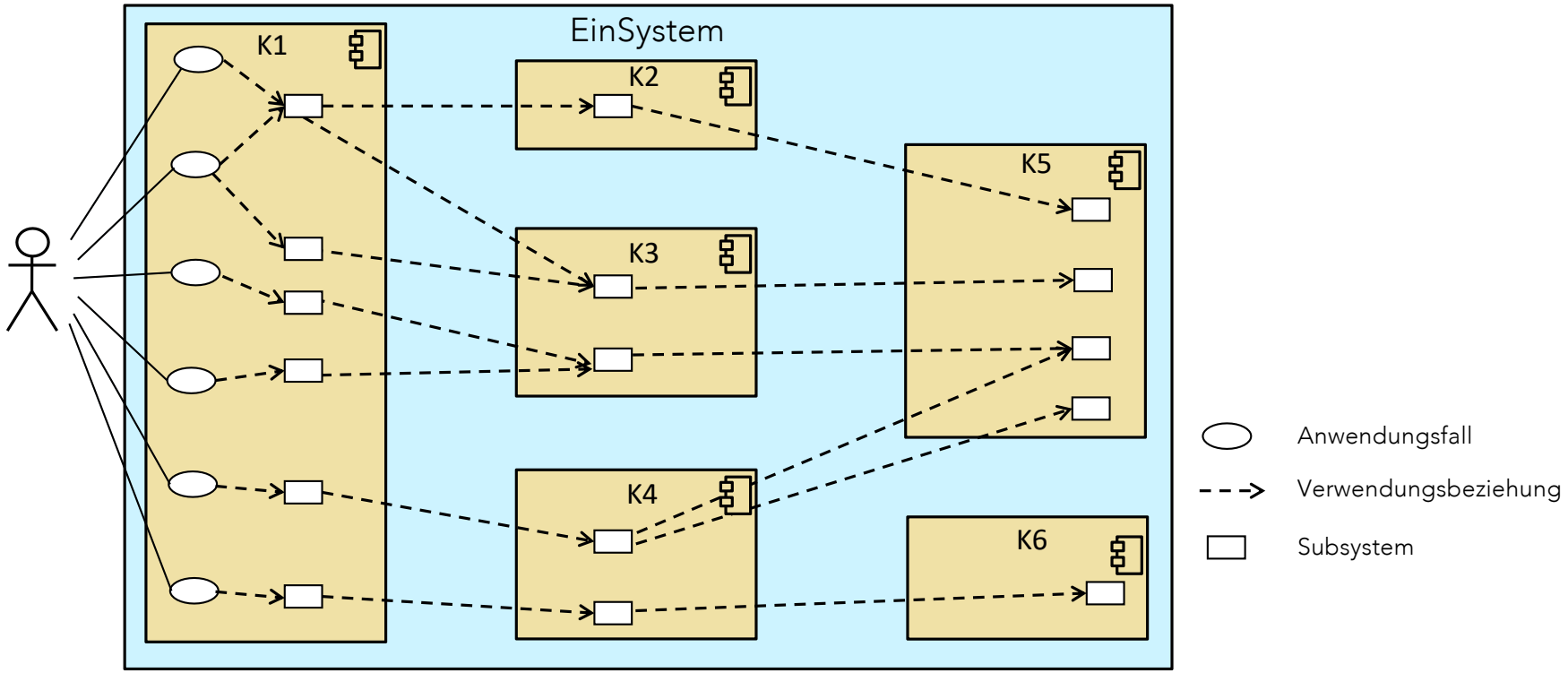
Architekturmodellierung



© Prof. Dr. Sabine Sachweh

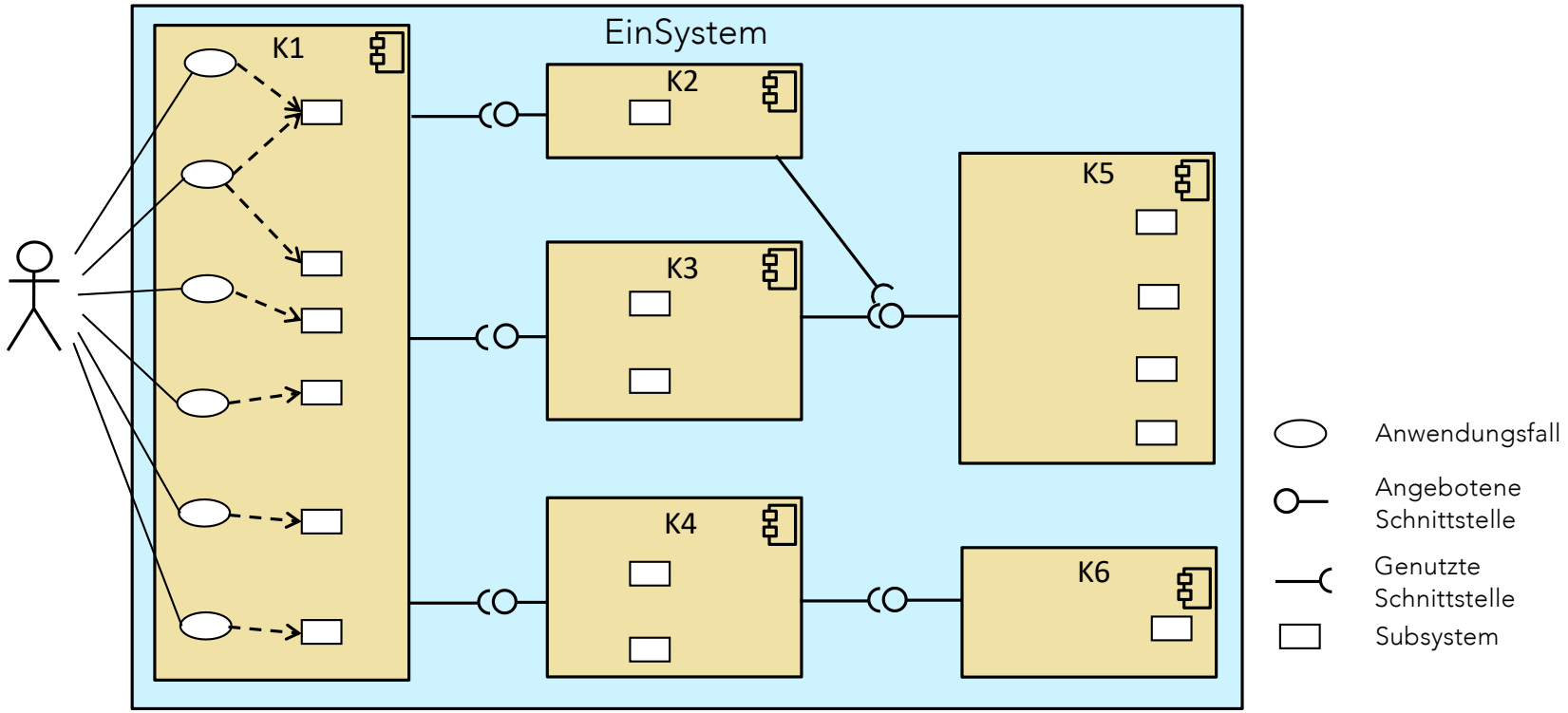
Architekturmodellierung

we focus on students



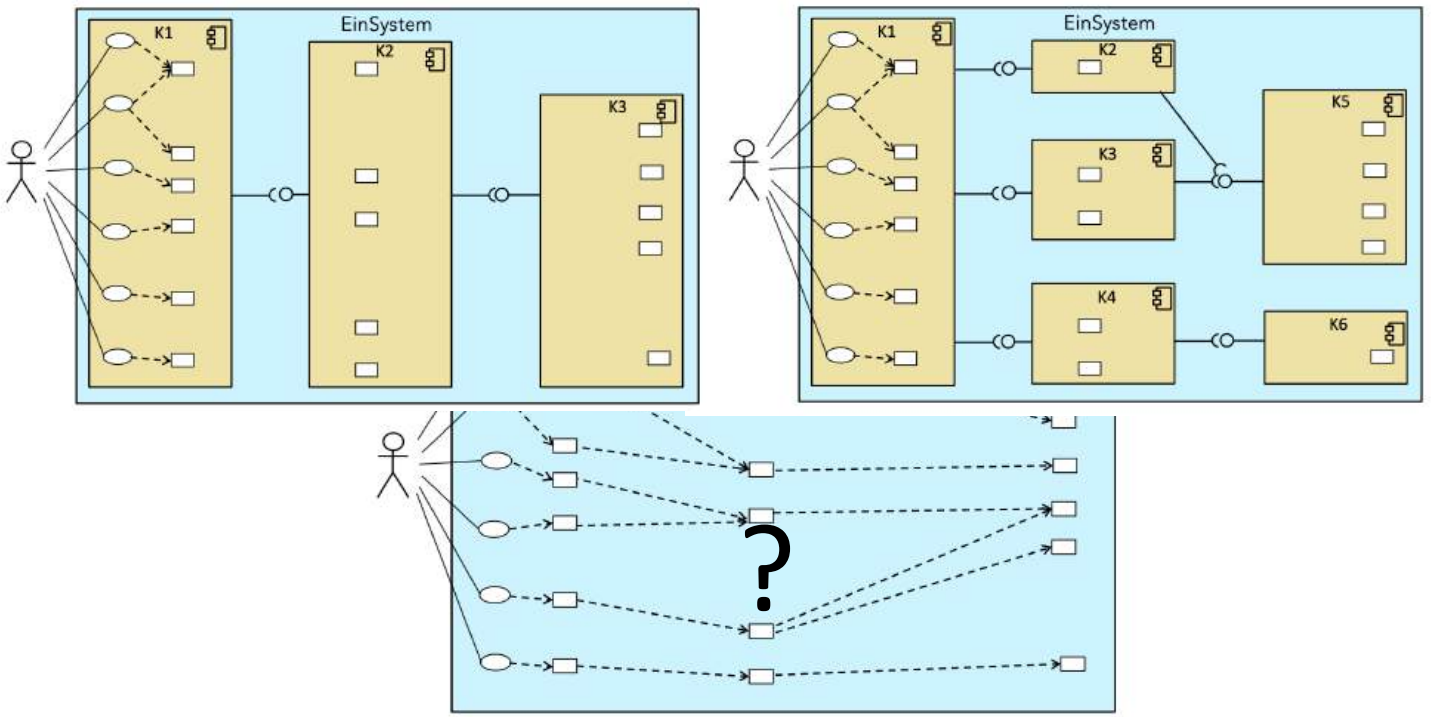
© Prof. Dr. Sabine Sachweh

Architekturmodellierung



© Prof. Dr. Sabine Sachweh

Architekturmodellierung



Anwendungsfall

Angebotene Schnittstelle

Genutzte Schnittstelle

Subsystem

© Prof. Dr. Sabine Sachweh

Struktur

1. Einführung

2. Architekturmodellierung

2.1 Begriffe

2.2 (Software-)Architekturmodellierung

2.3 Verteilungsdiagramm

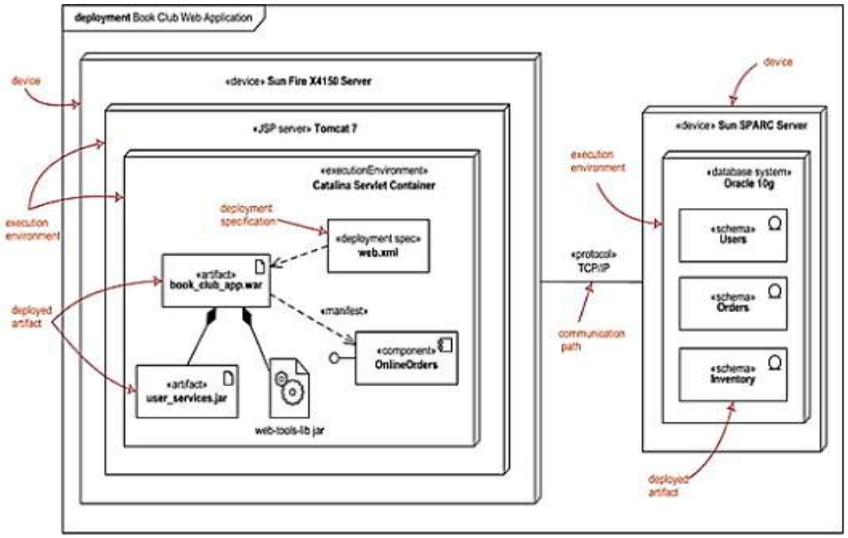
© Prof. Dr. Sabine Sachweh

Verteilungsdiagramm

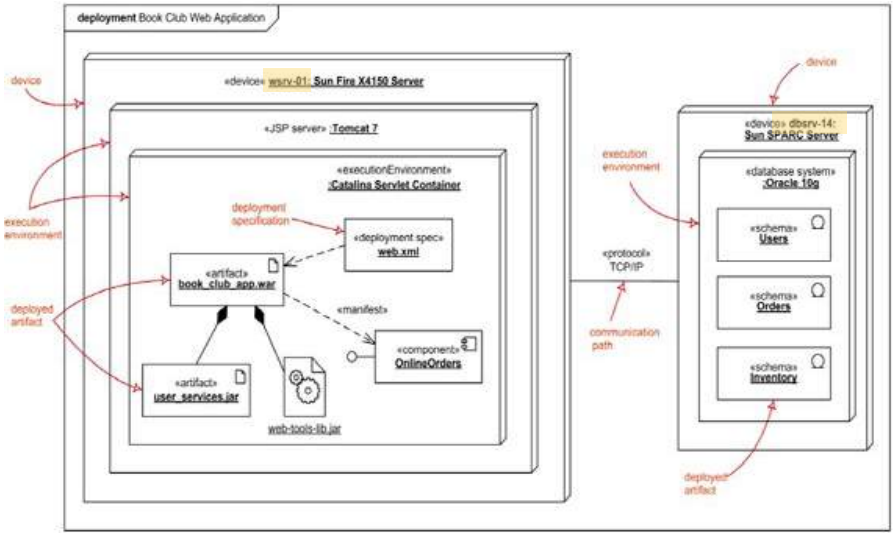
- Liefert **Antwort auf die Fragen**:
  a) „Auf welcher **Hardware läuft die Software,**
  b) auf welchem **Technologiestack und**
  c) über welche **Verbindungen** läuft die Kommunikation?“

- Wesentliche Modellierungselemente
  - Modellierung der eingesetzten **Hardware- und Softwaretopologie**
  - Modellierung des zugeordneten **»Laufzeitsystems«**
  - Verteilungsdiagramm kann auf **Typ- oder Instanzebene** modelliert werden

Verteilungsdiagramm



Typebene



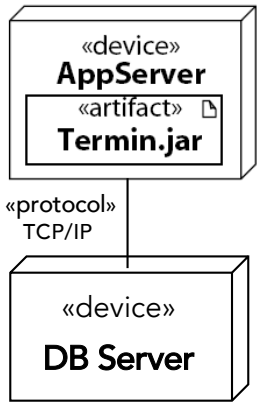
Instanzebene

Die Modellierung auf Instanzebene kann beispielsweise verwendet werden, um Unterschiede zwischen Entwicklungs-, oder Produktionsumgebungen über den Namen bzw. die ID bestimmter Server anzuzeigen. Im obigen Beispiel wird die Webanwendung auf dem Anwendungsserver **wsrv-01** und mehrere Datenbankschemata auf dem Datenbankserver **dbsrv-14** bereitgestellt .

© Prof. Dr. Sabine Sachweh

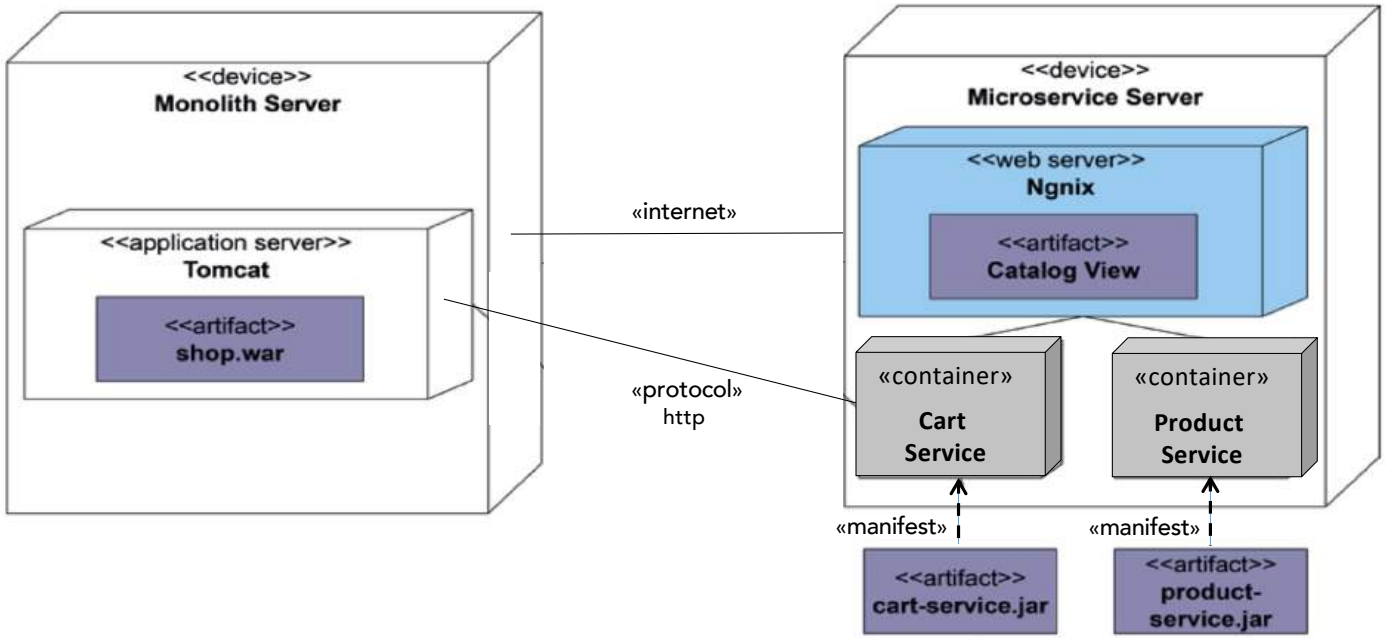
Verteilungsdiagramm

- Knoten
  repräsentieren Geräte oder Ausführungsumgebungen, auf die Artefakte verteilt werden
  - Gerät **«device»** repräsentiert Hardware und besitzt Rechenkapazität
  - Ausführungsumgebung **«execution env»** repräsentiert die notwendige Software, um das Laufzeitsystem auszuführen
  - **Hierarchische Knoten** können durch Schachtelung oder durch interne Strukturen erstellt werden ⍰ Technologiestack
  - **Kommunikationsbeziehungen** sind spezielle Assoziationen und können durch **Stereotypen** näher definiert werden



© Prof. Dr. Sabine Sachweh

Verteilungsdiagramm



© Prof. Dr. Sabine Sachweh

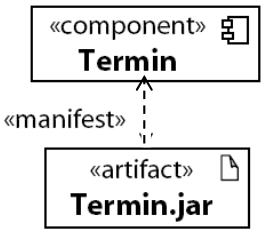
Serverbasierte Architekturbausteine

- Vielen Systeme sind nach dem Client/Server-Konzept strukturiert.
- Typische serverbasierte Architekturbausteine sind u.a.:
  - Webserver
  - Portalserver
  - Applikationsserver
  - Datenbankserver
  - Nachrichtenserver (Message Queue), Kommunikationshub
  - Dokumentations-Management-System (DMS)
  - Transaktionsserver
  - Verzeichnisserver (Directory)
  - Stapelverarbeitung (Batch Processing)
  - Public Key Infrastructure (PKI)

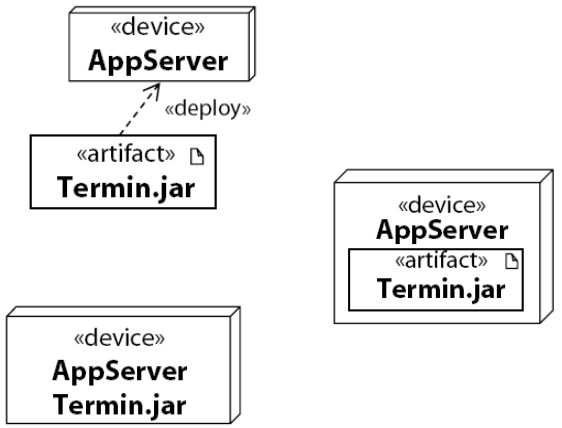
© Prof. Dr. Sabine Sachweh

Artefakte und ihre Verteilung

we focus on students



- Artefakte stellen die **»Objekte« der Verteilung** dar
- unter Verteilung wird die **Zuweisung von Artefakten** an Knoten verstanden
  - ein Artefakt kann auf mehrere Knoten verteilt werden
- **3 Notationsvarianten**
  - explizite Modellierung als Abhängigkeit mit Schlüsselwort «deploy»
  - grafische Schachtelung von Artefakten auf Knoten
  - Notation der Artefaktnamen im Knotensymbol

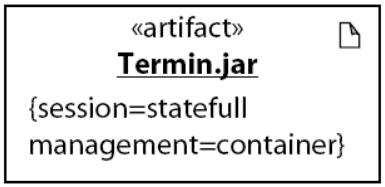


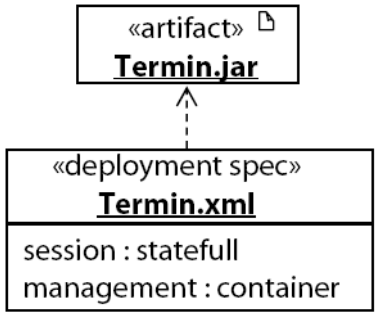
© Prof. Dr. Sabine Sachweh

Einsatzspezifikation

- Verteilungsdetails eines Artefakts durch Parameter in **Einsatzspezifikation** (deployment specification) definierbar
  - z.B. Modellierung von Konfigurationsdateien
  - für eine Verteilung können verschiedene Einsatzspezifikationen definiert werden

- Notationsalternativen

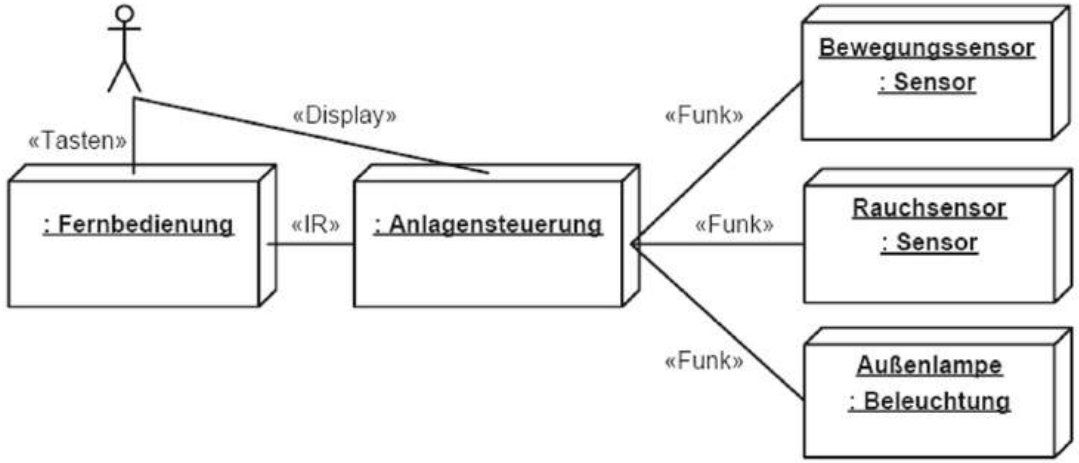




© Prof. Dr. Sabine Sachweh

Sonstiger Einsatz

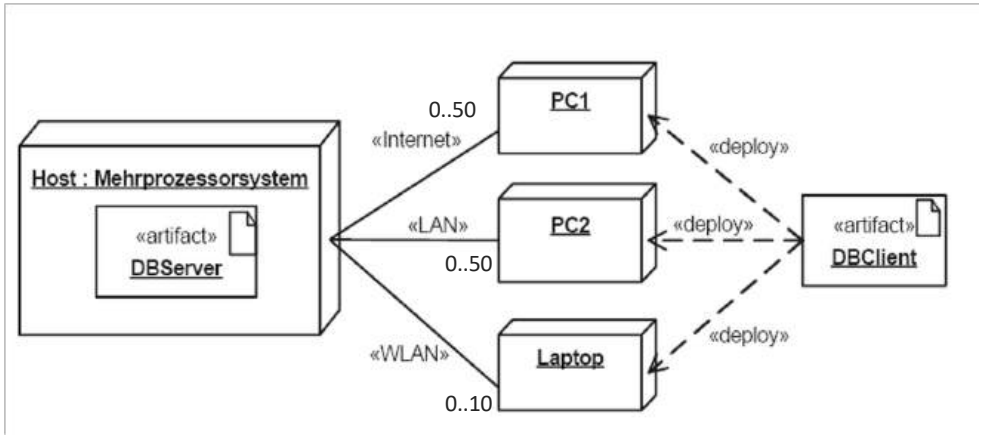
- **Physikalische Kontextabgrenzung** des Systems, wenn Nachbarsysteme existent
  - Blackbox-Modellierung und Darstellung von System und Nachbarsystem



© Prof. Dr. Sabine Sachweh

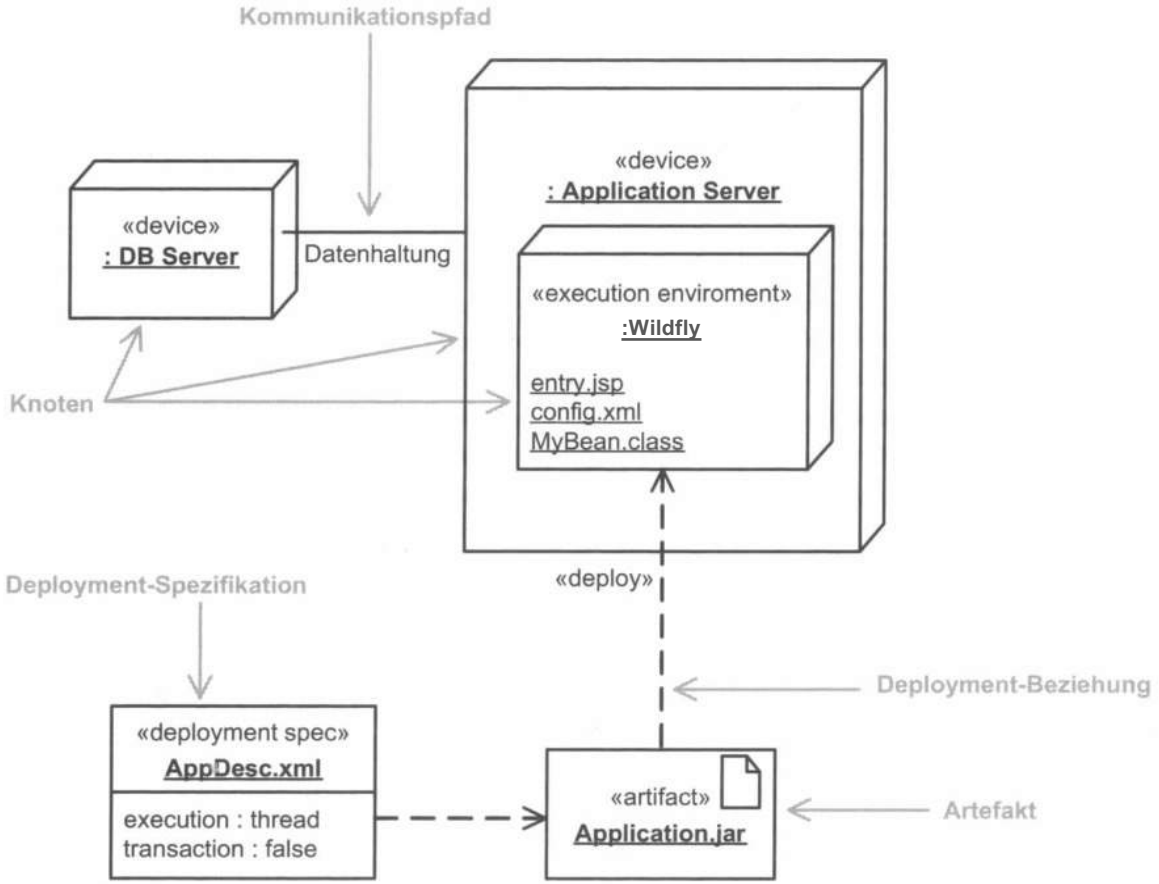
Sonstiger Einsatz

- **Dokumentation von Hardwarevorgaben** in visualisierter Form ist besser als rein textuelle Darstellung



© Prof. Dr. Sabine Sachweh

Verteilungsdiagramm – Syntax im Überblick



© Prof. Dr. Sabine Sachweh

Weitere Fragen

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

we
focus
on
students

Softwaretechnik 2

Das Spring Framework und Spring Boot

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Das Spring Framework und Spring Boot

Spring Framework

Spring Boot

Beispielanwendung

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Spring Framework

Spring Framework

https://docs.spring.io/spring-framework/reference/

we focus on students

- Spring ist **eines der beliebtesten Java-Frameworks** für die Entwicklung von Unternehmens-softwarelösungen
- Es ist entstanden als **Java EE** (Enterprise Edition) heute Jarkata den Markt beherrschte. → Erste Version 2002
- Spring basiert auf den Sprachen **Java, Kotlin und Groovy.**
- Zentrale Konzepte:
  - Inversion of Control (IoC)
  - Dependency Injection (DI)
  - Aspect-Oriented Programming (AOP)
  - Ökosystem von Modulen

© Prof. Dr. Sabine Sachweh

Inversion of Control (IoC)

- **Designprinzip**
  - Die Kontrolle über Programmablauf und Objekterstellung wird ausgelagert.
- **Klassisch:**
  - Entwickler-Code steuert Ablauf und erstellt Objekte.
- **Mit Inversion of Control:**
  - Das Framework steuert den Ablauf und ruft Entwickler-Code auf.
  - Objekterstellung und Abhängigkeiten werden extern verwaltet.
- **Funktionsweise**
  - IoC-Container od. -Kontext
    - erstellt Objekte
    - verwaltet deren Lebenszyklus
    - verbindet Abhängigkeiten
  - Komponenten bekommen benötigte Objekte von außen bereitgestellt

Vorteile:
Flexiblere, leichter wartbare und testbare Software

Hollywood-Prinzip:
„Don't call us, we call you"

© Prof. Dr. Sabine Sachweh

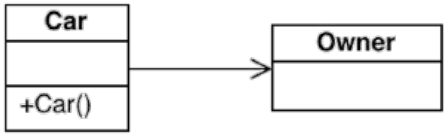
Fachhochschule Dortmund
University of Applied Sciences and Arts

Dependency Injection

we focus on students

- **Definition:**
  Technik, bei der Objekte ihre Abhängigkeiten nicht selbst erstellen, sondern von außen erhalten
- **Ziel:**
  - Lose Kopplung zwischen Klassen
  - Bessere Wartbarkeit, Flexibilität und Testbarkeit
- **Beteiligte Rollen:**
  - Service → wird verwendet (Abhängigkeit)
  - Client → nutzt den Service
  - Injector → übergibt die Abhängigkeit
- **Vorteile:**
  - Klassen sind unabhängiger voneinander
  - Einfacheres Unit Testing (z. B. mit Mocks)
  - Bessere Wiederverwendbarkeit

Ohne Dependcy Injection




```
public class Car {
    private Owner owner;

    public Car() { // feste Kopplung
        owner = new Owner();
    }
}
```


© Prof. Dr. Sabine Sachweh

Dependency Injection

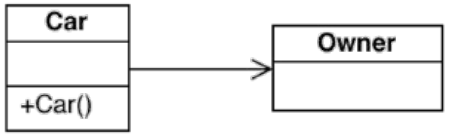
we focus on students

Risiko zyklischer Abhängigkeiten!

Arten von Dependency Injection:

- **Constructor Injection (empfohlen)**
  - Abhängigkeiten werden über den Konstruktor übergeben
  - Pflicht-Abhängigkeiten sind direkt vorhanden
- **Setter Injection**
  - Übergabe über Setter-Methoden
  - Für optionale Abhängigkeiten geeignet, die vernünftige Standardwerte haben sollten
- **Field Injection**
  - Direktes Einfügen in Felder (z. B. mit `@Autowired`)
  - Einfach, aber schlechter testbar

Mit Constructor Injection



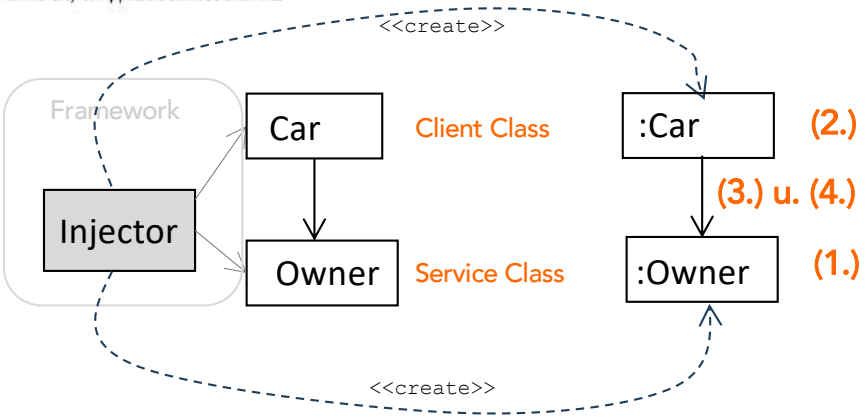

```java
public class Car {
    private Owner owner;

    public Car(Owner owner) {  // lose
        this.owner = owner;    // Kopplung
    }
}
```


© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Dependency Injection

we focus on students



Ablauf:

1. Spring erstellt das Service-Objekt (→ Owner)
2. Spring erstellt das Client-Objekt (→ Car)
3. Spring übergibt das Service-Objekt an den Client (z. B. über den Konstruktor)
4. Das Client-Objekt nutzt das Service-Objekt

Mit Constructor Injection

```
public class Car {
    private Owner owner;

    public Car() { // feste Kopplung
        owner = new Owner();
    }
}
```

Mit Constructor Injection

```
public class Car {
    private Owner owner;

    public Car(Owner owner) {  // lose
        this.owner = owner;    // Kopplung
    }
}
```

Merksatz:

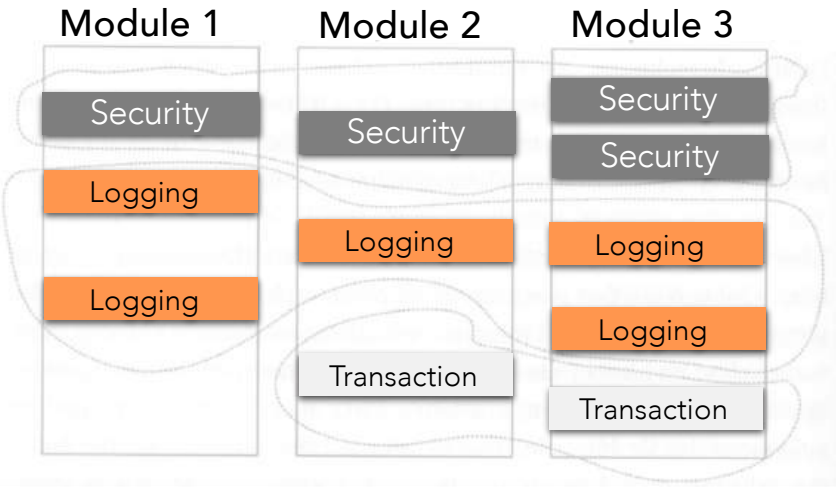
Spring verdrahtet Klassen/Objekte automatisch. – Entwickler definieren nur, was sie brauchen!

© Prof. Dr. Sabine Sachweh

Aspektorientierung (AOP)

we focus on students

- **Grundidee:**
  - DRY (Don't repeat yourself)
  - Querschnittsfunktionalitäten (engl. Crosscutting Concerns)
- **Typische Beispiele:**
  Logging, Sicherheit, Transaktionen
- **Ziel**:
  sauberer, modularer Code ohne Wiederholungen
- **Umsetzung in Spring:**
  - Spring nutzt **Proxy-Objekte**, die Methodenaufrufe abfangen
  - Diese Proxies „umhüllen" die eigentlichen Objekte (Beans)
  - Dadurch kann zusätzlicher Code **vor, nach oder um Methoden herum** ausgeführt werden.



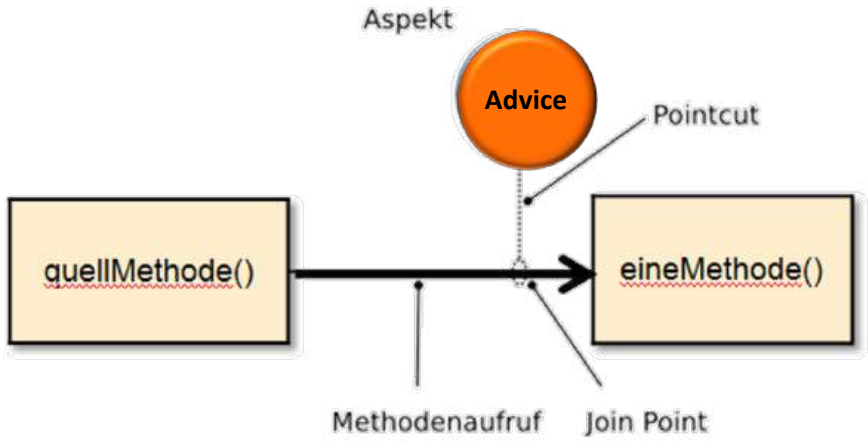
AOP ⇔
„zusätzlicher Code um Methoden herum, ohne sie zu verändern".

© Prof. Dr. Sabine Sachweh

Aspektorientierung (AOP)

we focus on students

- **Aspekt**
  Bündelung von Pointcut und Advice in einer Klasse.
- **Advice**
  Konkreter Code, der am Join Point ausgeführt wird (Before/After/Around).
- **Join Point**
  „Rein-Spring"-Punkt im Programm. Stelle im Kontrollfluss (i.d.R. ein Methodenaufruf).



- **Pointcut**
  Ausdruck, der Join Points auswählt, bspw. alle Methoden einer Klasse umfassen, die mit "xyz" beginnen.
- **Weaving**
  Prozess, bei dem Aspect und Zielobjekt verbunden werden (bei Spring: zur Laufzeit per Porxy).

© Prof. Dr. Sabine Sachweh

Aspektorientierung (AOP)

we focus on students

Arten von Advices

Spring unterstützt 5 wichtige Typen:

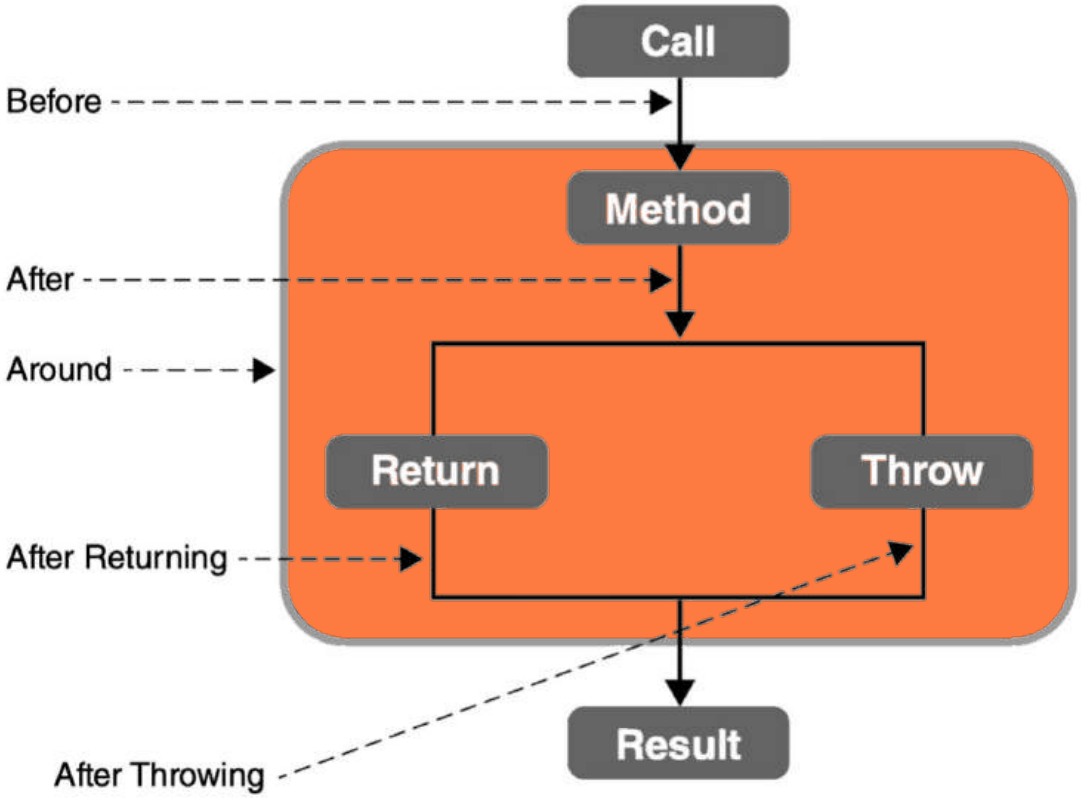
`@Before`
Vor der Methode (z. B. Security-Check)

`@AfterReturning`
Nach erfolgreicher Ausführung

`@AfterThrowing`
Bei Fehlern (Exception Handling)

`@After(finally)`
Immer nach der Methode
(z. B. Ressourcen freigeben)

`@Around`
Um die Methode herum
(mächtigste Variante)



© Prof. Dr. Sabine Sachweh

Vor- und Nachteile des Spring Frameworks

+ VORTEILE

- Flexibilität und Modularität
- Aspektorientierte Programmierung (AOP)
- Leichte Integration
- Eine einheitliche Methode für den Zugriff auf verschiedene Datenquellen
- Sicherheit und Authentifizierung
- Geringes Gewicht und architektonische Flexibilität
- Starke Unterstützung durch die Community
- Verringerung von sich wiederholendem Code
- Vereinfachtes Testen

– NACHTEILE

- Komplexität der Anfangskonfiguration
- Hoher Schwierigkeitsgrad
- XML-Konfiguration in dreistelliger Zeilenzahl
- Manuelles Deployment auf Anwendungsserver
- Inkonsistente Library-Versionen („JAR Hell")
- Lange Feedback-Zyklen
- Schwergewichtige Container

DIE ANTWORT: „Just run it." → Spring Boot

`java -jar app.jar` startet eine vollständige Anwendung — inklusive Server.

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Spring Boot (Framework)

Vom Spring Framework zu Spring Boot

- **2002**
  Rod Johnson publiziert Expert One-on-One J2EE Design and Development — Grundsteine für leichtgewichtige Container.

- **2003**
  Start des Spring Frameworks als Antwort auf die Komplexität von EJB 2.x.

- **2014**
  Erstrelease Spring Boot 1.0 — Auto-Konfiguration, Embedded Tomcat, Starter-POMs.

- **2018**
  Spring Boot 2.0 mit Reactive Stack (WebFlux) und Java 8 Baseline.

- **2022**
  Spring Boot 3.0 — Java 17 Baseline, Jakarta EE 9, native Bilder via GraalVM.

- **Ende 2025**
  Spring Boot 4.0 mit Spring Framework 7.0

[Hinkula 2023, Vorwort & Kap. 1.]

© Prof. Dr. Sabine Sachweh

Spring Framework 7.0 und Spring Boot 4.0

Spring Boot 4.0 und Spring Framework 7.0 beinhalten deutliche Neuerungen

- deutlicher **Modernisierungsschub** statt nur kleiner (inkrementeller) Verbesserungen
- Fokus auf **langfristige Wartbarkeit**, **Robustheit** und **Effizienz**
- Neue Basis:
  - Jakarta EE 11
  - Java 17+
  - Servlet 6.1
  - GraalVM 25

- Ziele:
  - Nutzung **moderner Plattformen** und **aktueller Standards**
  - **Bessere Unterstützung für Cloud- und verteilte Systeme**

© Prof. Dr. Sabine Sachweh

Was ist Spring Boot?

Es ist ein **opinionated Framework** auf Basis des Spring Frameworks, das

- Java-Backends produktionsreif startklar macht —
- mit eingebettetem Anwendungsserver,
- Auto-Konfiguration und
- Dependency-Management.

Opinionated Framework ⇔
ist ein Framework, das Entwicklern eine ganz bestimmte, Weise vorgibt, wie eine Anwendung strukturiert und entwickelt werden soll.

STANDALONE
Embedded Server

Web- und Servlet-Container (Tomcat, Jetty oder Undertow) werden direkt in die Java-Applikation eingebettet — kein separater Anwendungsserver nötig.

OPINIONATED
Starter-Dependencies

Kuratierte Abhängigkeiten (spring-boot-starter-*) bündeln passende Bibliotheken in geprüften Versionen.

AUTO-CONFIGURED
Konvention vor Konfiguration

Bei vorhandenen Bibliotheken konfiguriert Spring Boot

Beans, Datenquellen und Web-Schicht automatisch.

[Hinkula 2023, Kap. 1: Spring Boot is a modern Java-based backend framework that makes development faster.]

© Prof. Dr. Sabine Sachweh

Standards und Spezifikationen in Spring Boot

Spring implementiert oder integriert eine Reihe etablierter Java-Standards.
⇒ Anwendungscode bleibt damit weitgehend portabel.

- **Jakarta EE 9+ —** `jakarta.*` Namespace ersetzt `javax.*` (Spring Boot 3).
- **JPA (Jakarta Persistence API)** — ORM-Spezifikation, Implementierung über Hibernate.
- **JSR-330** — `@Inject, @Named, @Singleton` als DI-Standardannotationen.
- **JSR-380 / Bean Validation 3.0** — `@NotNull, @Size, @Email.`
- **Servlet API 6.0 —** Grundlage für die Web-Schicht (Tomcat 10).
- **JSON Web Token (RFC 7519) —** Token-basierte Authentifizierung.
- **OAuth 2.1 & OpenID Connect —** Autorisierung über externe Provider.
- **OpenAPI 3 —** maschinenlesbare REST-Schnittstellenbeschreibung.

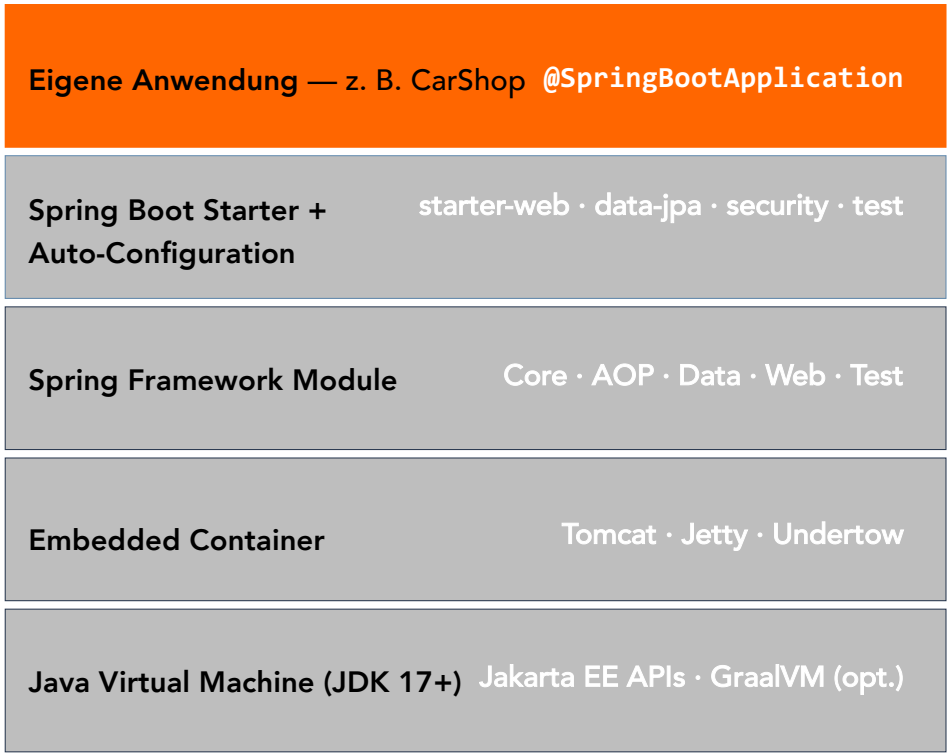
Spring **liefert pro Standard eine eigene Abstraktionsschicht** — austauschbare Implementierungen sind ausdrücklicher Designgrundsatz.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Architektonische Sicht: Spring Boot Stack
(Schichtenmodell)

we focus on students



Ergänzung durch Spring Boot gegenüber dem reinen Spring Framework:

- Auto-Konfiguration,
- Starter-POMs,
- Actuator,
- Embedded Server.

KONSEQUENZ

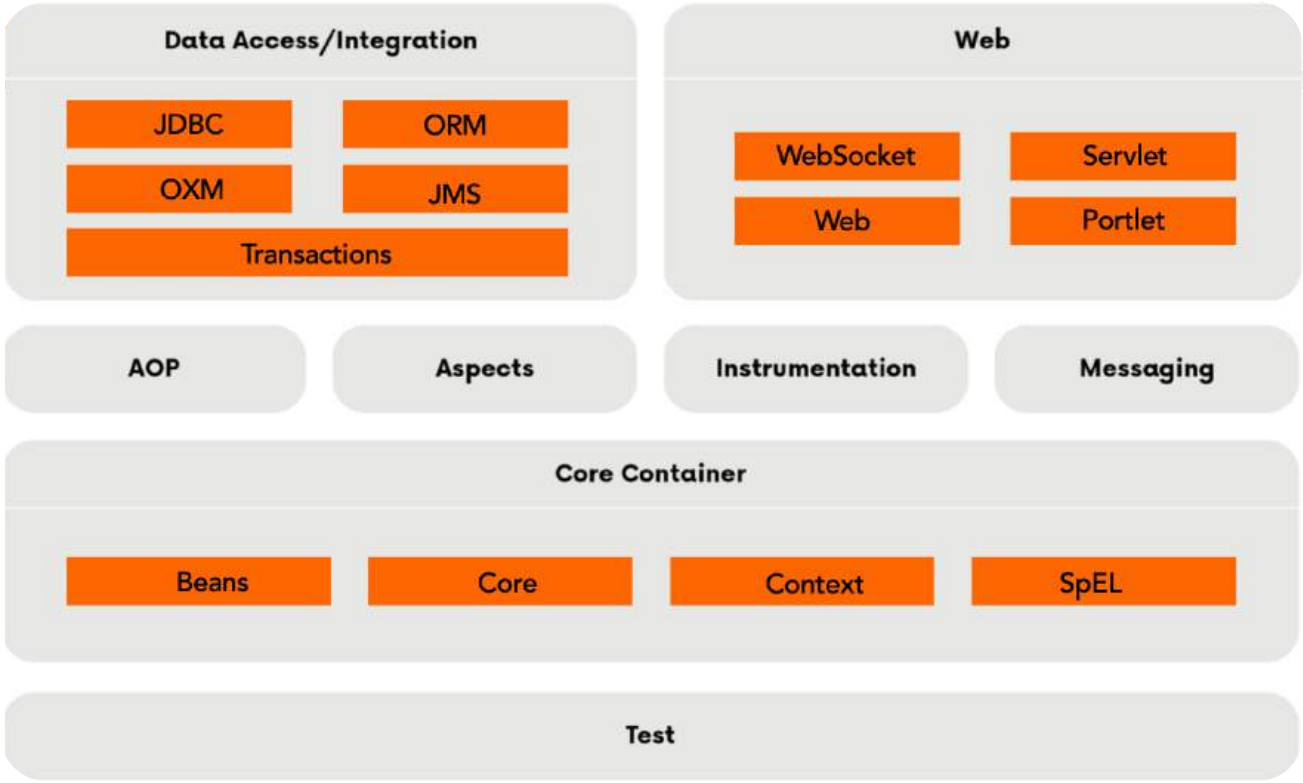
Konfiguration verschwindet

Eine Web-Anwendung mit JPA, Security und REST entsteht aus 4 Starter-Dependencies und einer Hauptklasse mit `main()`.

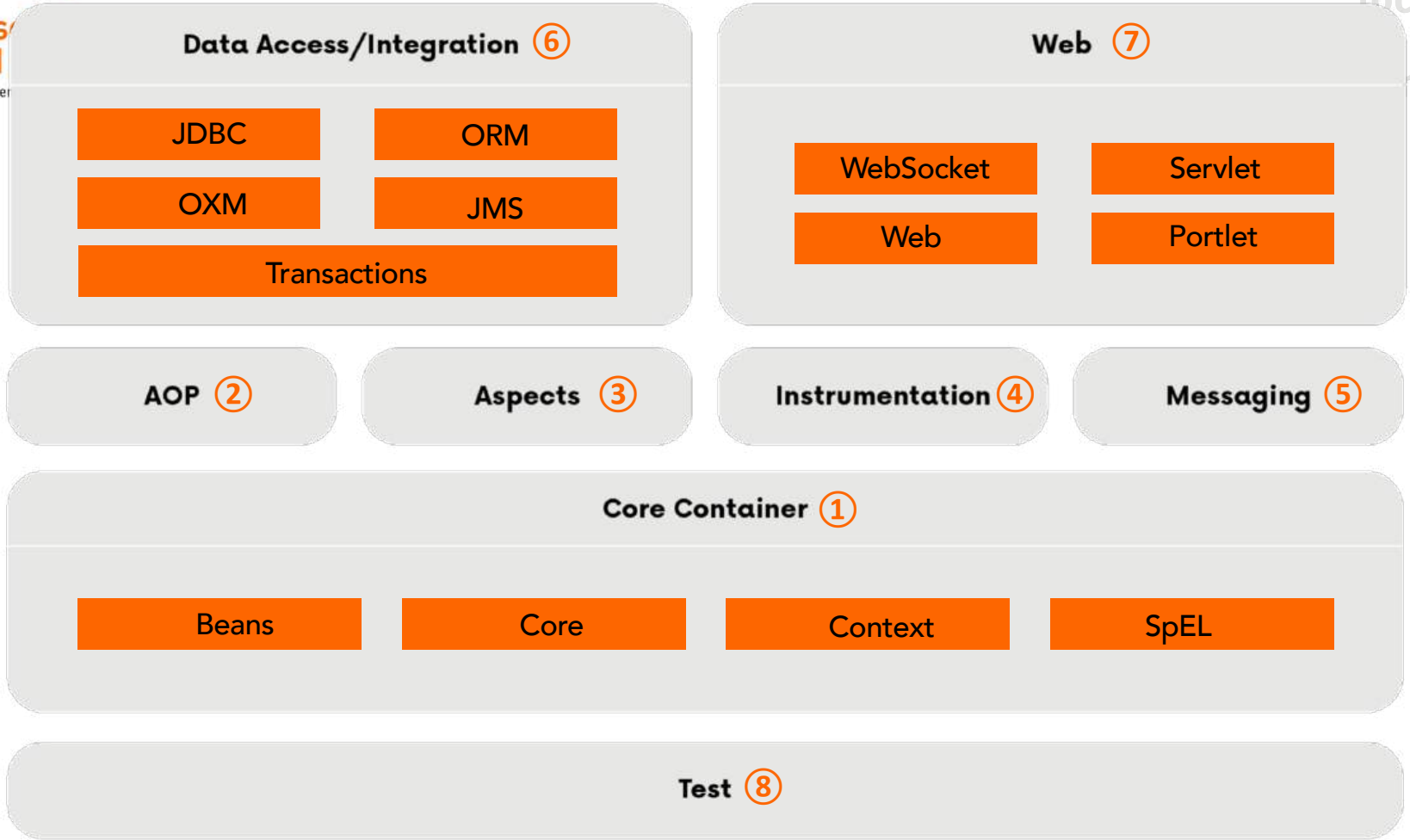
© Prof. Dr. Sabine Sachweh

Moduleübersicht – die acht Säulen des Frameworks

we focus on students



© Prof. Dr. Sabine Sachweh



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Core Container

we focus on students

Herzstück: hier befinden sich der IoC-Container, das Bean-Lifecycle-Management und die Konfigurations-Infrastruktur des gesamten Frameworks.

- **spring-beans** — `BeanFactory` als Basis-Container; verwaltet Bean-Definitionen und Lifecycle.
- **spring-context** — `ApplicationContext` erweitert die BeanFactory um Internationalisierung, Events und Resource-Loading.
- **spring-expression (SpEL)** — Ausdruckssprache zur Laufzeitauswertung in Annotationen, z. B. `@Value("#{systemProperties['user.region']}")`.
- **spring-core** — Hilfsklassen, Resource-Abstraktion, Type-Conversion.

KERNKONZEPT

Inversion of Control

Nicht das Objekt sucht seine Abhängigkeiten — der Container injiziert sie. Resultat:

lose Kopplung,

Testbarkeit, Austauschbarkeit.

```
@Configuration
public class AppConfig {
  @Bean
public CarService carService() {
    return new CarService();
  }
}
```


© Prof. Dr. Sabine Sachweh

Aspect oriented Programming (AOP)

AOP strukturiert **Cross-Cutting Concerns** — also Belange, die quer durch viele Klassen schneiden: Logging, Sicherheit, Transaktionen, Caching.

- **Joinpoint**
  Stelle im Programmfluss (i. d. R. ein Methodenaufruf).
- **Pointcut**
  Ausdruck, der Joinpoints auswählt.
- **Advice**
  der Code, der am Joinpoint ausgeführt wird (Before/After/Around).
- **Aspect**
  Bündelung von Pointcut + Advice in einer Klasse.
- **Weaving**
  Verknüpfung von Aspect und Zielobjekt (in Spring zur Laufzeit per Proxy).

```java
@Aspect
@Component
public class LoggingAspect {

  @Before("execution(* com.shop.service.*.*(..))")
  public void logEntry(JoinPoint jp) {
    log.info("→ {}", jp.getSignature());
  }
}
```

Spring **nutzt AOP intern** für `@Transactional`, `@Async`, `@Cacheable` und `Spring Security`.

© Prof. Dr. Sabine Sachweh

Aspects – ApectJ Integration

Während **Spring Aop** einen einfachen, **proxy-basierten** Ansatz nutzt, integriert **Spring Aspects** AspectJ, was mächtigere Pointcut-Ausdrücke und Compile-Time-Weaving ermöglicht.

SPRING AOP

Proxy-basiert · zur Laufzeit

- Nur öffentliche Methoden auf Spring Beans
- Keine Erweiterung des Compilers nötig
- Reicht für 90 % der Anwendungsfälle

PRAXIS-HINWEIS
Standard ist Spring AOP
AspectJ, nur verwenden wenn Features, die ein Proxy nicht umsetzen kann, wirklich benötigt werden.

SPRING ASPECTS

AspectJ komplett · Compile/Load-Time-Weaving

- Funktioniert auch auf privaten Methoden und Konstruktoren
- Getter und Setter können dynamisch erzeugt werden oder ihr Zugriff (über Pointcuts) abgefangen werden
- Erfasst (auch private) Attributzugriffe (`get()/set()`)
- Benötigt `aspectjweaver` oder AJC-Compiler

© Prof. Dr. Sabine Sachweh

Instumentation – Class Loader Tools

we focus on students

Das Modul **Spring Instumentation** stellt Java-Agents bereit, um Klassen beim Laden zu transformieren.
Dies ist Voraussetzung für Load-Time-Weaving (LTW) bei AspectJ und für JPA-Tools, die den Code von Entity-Klassen modifizieren.

- **Spring Instrumentation** — Java-Agent, einbindbar via `javaagent:spring-instrument-*.jar`.
- **InstrumentationLoadTimeWeaver** — Implementierung des Load-Time-Weavings für AspectJ.
- **Anwendungsfälle:**
  - Lazy-Loading von JPA-Entities
  - Dynamische Aspekte ohne Build-Tool-Eingriff

PRAXIS-HINWEIS

Wird selten direkt verwendet

In typischen Spring-Boot-Anwendungen ist Instrumentation kein Thema — relevant erst bei tiefen Integrationen (z. B. Hibernate-Bytecode-Enhancement) oder bei expliziter LTW-Nutzung.

© Prof. Dr. Sabine Sachweh

Messaging

Das Modul **Spring Messaging** unterstützt eine asynchrone, lose gekoppelte Kommunikation zwischen Anwendungen, meist über Message Broker wie RabbitMQ, Apache Kafka oder JMS. Es nutzt vorkonfigurierte Vorlagen (Templates) (`Message<T>`-Modell) zum Senden und Annotationen (z.B. `@JmsListener`, `@RabbitListener`) zum Empfangen von Nachrichten, was die Integration stark vereinfacht.

STOMP

Streaming Text

Sub-Protokoll für WebSocket — Pub/Sub-Modell für Browser-Clients.

AMQP

RabbitMQ

Asynchrone Verarbeitung über Message-Broker — entkoppelt Producer und Consumer.

KAFKA

Event-Streaming

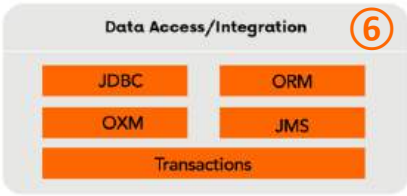
Hoch-durchsatzfähige Event-Logs für Microservices-Architekturen.

```java
@MessageMapping("/orders")
@SendTo("/topic/orders")
public OrderEvent create(OrderRequest r) { return new OrderEvent(r.id()); }
```


© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Data Access und Integration



we focus on students

Sammelmodul für alles, was mit **persistenten Daten und externen Systemen** zu tun hat. Das Modul bildet eine Abstraktionsschicht für Datenzugriffstechnologien.

JDBC

spring-jdbc

JdbcTemplate — Boilerplate-freier Zugriff auf **relationale DBs**.
Eine `DataSource` und ein `JdbcTemplate` werden automatisch konfiguriert.
Das Template ermöglicht SQL-Abfragen und Repositories vereinfachen CRUD-Operationen.

ORM

spring-orm

Integration für Hibernate, JPA, JDO — **EntityManager** als zentrale Schnittstelle zur Interaktion mit der Datenbank.
Er verwaltet u.a.

- CRUD-Operationen
- Lebenszyklus der Entities
- arbeitet in Transaktionen

OXM

spring-oxm

Object/XML-Mapping über beliebte XML-Binding-Technologien (wie JAXB, od. XStream)
ermöglicht die einfache Konvertierung von Java-Objekten in XML-Dokumente und umgekehrt.
— heute weniger relevant.

JMS (Java Message Service)

spring-jms

Ermöglicht einfache asynchrone Kommunikation mit Hilfe des `JmsTemplates`.
Entwickler referenzieren `spring-boot-starter-activemq` hinzu für die Broker-Verbindung.
`@JmsListener` zum Empfangen und das `JmsTemplate` zum Senden von Nachrichten.

TX (Transaktionsmanagement)

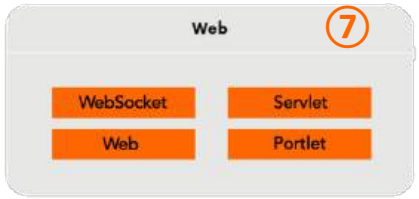
spring-tx

Deklarative Transaktionen via `@Transactional`.
Annotation wird auf Service-Klassen oder Methoden angewendet. Spring Boot erstellt im Hintergrund einen Proxy zum Management der Transaktion

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Web – die HTTP-Schicht



we focus on students

Submodule für unterschiedliche HTTP-Programmiermodelle — vom klassischen Servlet-MVC über WebSocket bis zum (historischen) Portlet-Container.

SPRING-WEB

Web-Basis

Zentrales Starter-Paket, um
- Webanwendungen,
- RESTful APIs und
- Spring MVC-Anwend.

zu entwickeln.

Nutzt standardmäßig Tomcat als eingebetteten Webserver.

Vereinfacht die Arbeit mit Spring MVC und REST erheblich, indem es die Konfiguration automatisiert und einen Server mitliefert.

SPRING-WEBMVC

Servlet-MVC

Herzstück der Entwicklung von Java-Webanwend-ungen und RESTful Web Services – basiert auf Servlet-API.

Nutzt MVC--Muster.

Konvertiert automatisch HTTP-Anfrageparameter in Java-Objekte (POJOs).

Für RESTful-APIs werden mit `@RestController` Daten automatisch in JSON oder XML um-gewandelt.

SPRING-WEBSOCKET

WebSocket

Baut eine **langlebige, bidirektionale Verbindung** zwischen Client (z. B. Browser) und Server auf, über die Daten in beide Richtungen gesendet werden können, ohne dass der Client ständig neue HTTP-Anfragen stellen muss.

Grundlage für Live-Updates und Push.

Integriert sich nahtlos in Spring-WebMVC.

WebSocket-ähnliche Funktionalität auch in Browsern, die WebSockets nicht unterstützen.

SPRING-WEBFLUX

Non-Blocking Web-FW

Nicht-blockierendes, reaktives Webframework für Java.

Ermöglicht die Entwicklung hochperformanter, ressour-ceneffizienter Anwendungen (Microservices) mittels Event-Loops.

Ideal für hohe Nebenläufig-keit, bei denen wenige Threads viele Anfragen bearbeiten.

→ Reakor Pattern

SPRING-PORTLET

Portlet API

Eingebettete Web-Komponenten in Portalen (JSR-168/286)

Heute kaum genutzt.

LEGACY

© Prof. Dr. Sabine Sachweh

Test — integriertes Test-Framework

we focus on students

Das **Spring-Test-Modul** stellt Werkzeuge bereit, mit denen Beans, Web-Schicht und Datenzugriffsschicht in voll- oder partiell hochgefahrenen Anwendungskontexten getestet werden können.

- **`@SpringBootTest`**
  voller **Anwendungskontext** für Integrationstests.
- **`@WebMvcTest`**
  schmaler Slice nur für **Controller-Schicht**.
- **`@DataJpaTest`**
  Persistenz-Schicht mit eingebetteter H2.
- **`MockMvc`**
  HTTP-Aufrufe ohne echten Servlet-Container.
- **`TestRestTemplate/ WebTestClient`**
  Anwendung wird für die Dauer des Tests in einem eingebetteten Webserver vollständig hochgefahren und bekommt dabei einen zufälligen, freien Port aus dem verfügbaren Netzwerkbereich zugewiesen.

```
@SpringBootTest
@AutoConfigureMockMvc
class CarControllerTest {
  @Autowired MockMvc mvc;

  @Test
void getAllReturns200() throws Exception {
    mvc.perform(get("/api/cars"))
       .andExpect(status().isOk());
  }
}
```


© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Weitere Module interessante Module

we focus on students

- **Spring Security**
  Ein leistungsstarkes, anpassbares Authentifizierungs- und Zugriffskontroll-Framework, das als De-facto-Standard für die Absicherung von Java-Spring-Anwendungen gilt.
  - überprüft die Anmeldeinformationen anhand einer Datenquelle und
  - verifiziert die Benutzeridentität
  - Gängige Methoden:
    - Benutzername/Passwort
    - Token-basierte Authentifizierung
    - OAuth2.
  - Speichert Benutzerdaten im `SecurityContext` nach erfolgreicher Authentifizierung.
- **Spring Cloud**
  bietet Werkzeuge für die Entwicklung verteilter Systeme und Microservices auf Basis von Spring Boot. Es erleichtert Aufgaben wie
    - Service-Discovery
    - zentralisierte Konfiguration
    - Lastverteilung (Load Balancing)
    - Ausfallsicherheit (Circuit Breakers)
    - intelligentes Routing

© Prof. Dr. Sabine Sachweh

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

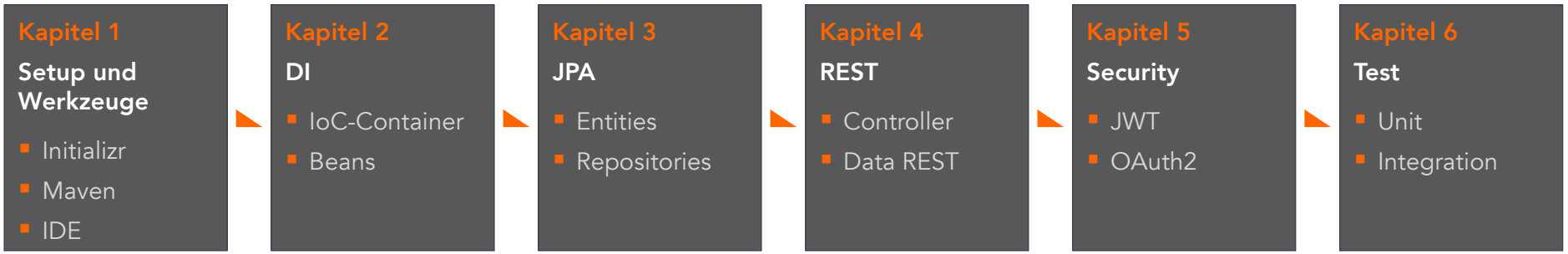
Lehrbuch: Hinkula, J. (2023). Full Stack Development with Spring Boot 3 and React, 4. Edition, Packt Publishing.

Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

we focus on students

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

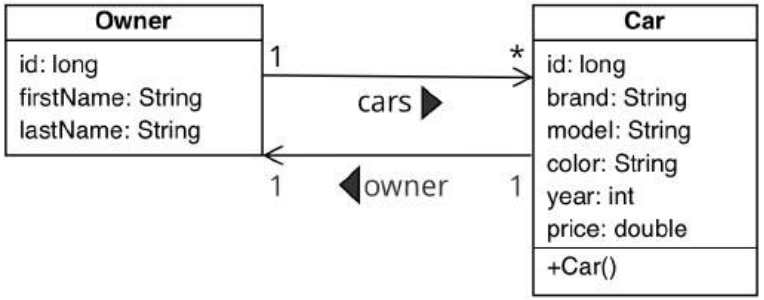
DURCHGÄNGIGES CODE-BEISPIEL

Die Anwendung: CarShop

we focus on students

Es wird eine kleine **Fahrzeug-Verwaltung** entwickelt —
fachlich überschaubar, aber komplex genug, um Beans, JPA, REST, Security und Tests sauber durchzuspielen.

Domäne	Fahrzeuge (`Car`) und ihre Eigentümer (`Owner`) inkl. 1:N-Beziehung.
Schichten	`Controller → Service → Repository → DB`
Persistenz	H2 für Entwicklung, MariaDB für Produktion.
API	REST + Spring Data REST + OpenAPI 3.
Security	JWT-basierter Login, Rollen `USER` / `ADMIN`.




```java
// Domain-Modell
class Owner {
  Long id;
  String firstName, lastName;
  List<Car> cars;
}

class Car {
  Long id;
  String brand, model, color;
  int year;
  double price;
  Owner owner;
}
```


© Prof. Dr. Sabine Sachweh

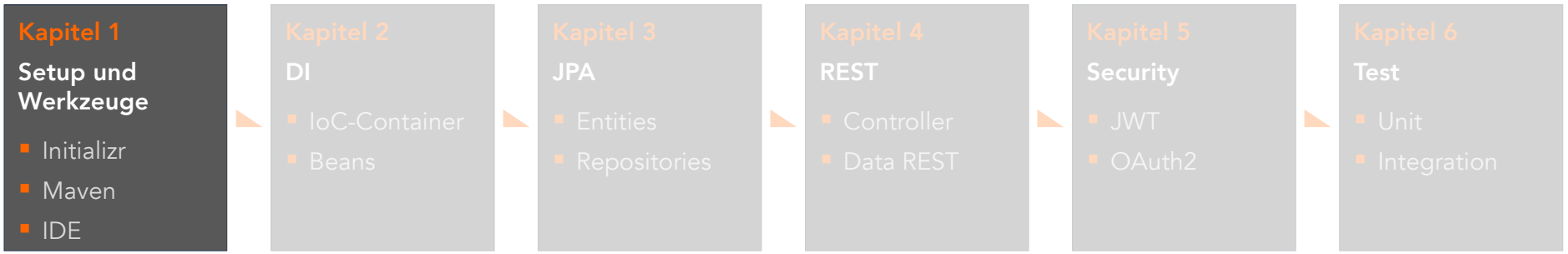
Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

we focus on students

01

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.



Setup und Werkzeuge
Spring Initializr, Maven, IDE — Einrichtung der Arbeitsumgebung und Starten des ersten Projekts

© Prof. Dr. Sabine Sachweh

Werkzeuge im **Backend-Stack**

- **JDK 17+**
  Spring Boot 3 verwendet Records, Sealed Classes, Pattern Matching
- **Eclipse / IntelliJ / VS Code**
  gewünschte IDE verwenden
  (im Buch wird Eclipse IDE for Enterprise Java verwendet)
- **Gradle / Maven**
  deklaratives Build-Tool, sehr flexibel (basiert auf DSL statt auf starrem XML), kombiniert die besten Ansätze von Ant und Maven
- **Spring Initializr**
  webbasiertes Tool, das die Erstellung von Spring Boot-Projekten automatisiert; generiert Projekt-Grundgerüst (ZIP-Datei)
- **MariaDB**
  relationales Open-Source-DBMS, weitgehend kompatibel mit MySQL, oft schnellere Abfragen und bessere Performance unter hoher Last
- **Postman / cURL** (Client for URLs)
  Postman ⇔ umfassende REST-API-Entwicklungsplattform zum Erstellen, Testen, Dokumentieren und Teilen von APIs sowie zur Test-Automatisierung
  cURL ⇔ leistungsfähiges Kommandozeilen-Tool,
  um REST-Endpunkte abzufragen und Antworten zu analysieren

BEFEHLE

Erste Schritte in der Shell

```
# Java-Version prüfen
java --version

# Gradle-Wrapper ausführen (im Projekt)
./gradlew bootRun

# Anwendung verpacken
./gradlew build
java -jar build/libs/carshop-0.0.1.jar
```

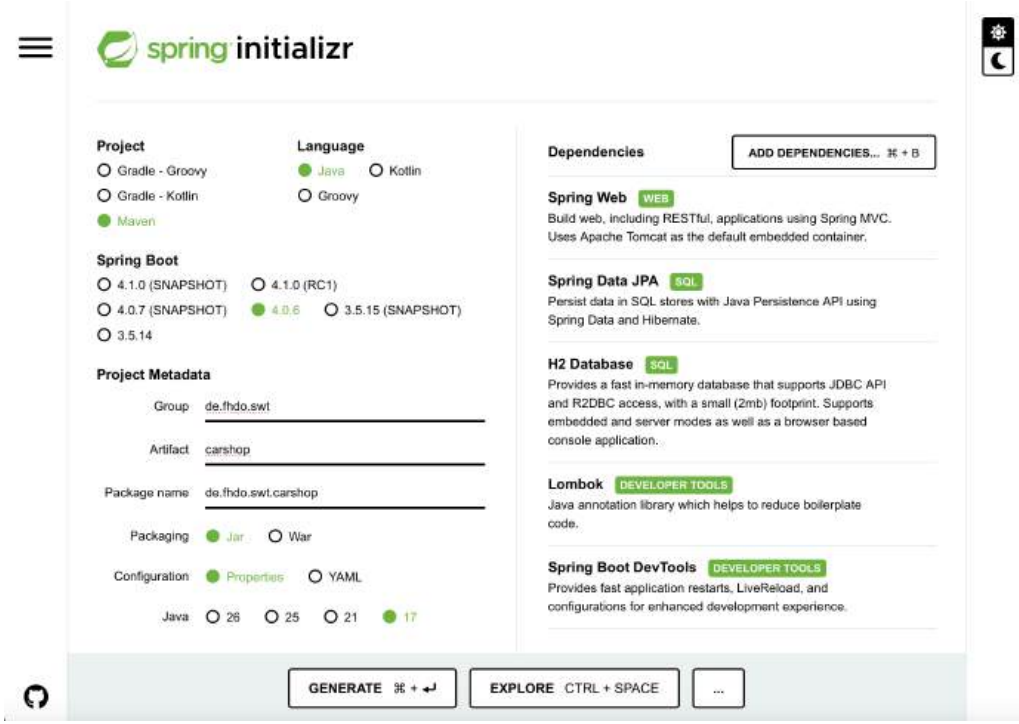
[Hinkula 2023, Kap. 1 — Setting Up the Environment and Tools – Backend.]

© Prof. Dr. Sabine Sachweh

Spring Initializr

Der **Initializr** (https://start.spring.io) erzeugt ein Maven-Skelett mit den gewählten Dependencies.
Ergebnis: ein lauffähiges Projekt in unter zwei Minuten.

Project	Maven
Language	Java
Spring Boot	4.0.6
Group	de.fhdo.swt
Artifact	carshop
Packaging	Jar
Java	17
Dependencies	Spring Web, Spring Data JPA, H2, Lombok, DevTools



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Maven-Standard-**Projektstruktur**

we focus on students

carshop/

```
carshop/
├── build.gradle
├── settings.gradle
├── gradlew
├── src/
│   ├── main/
│   │   ├── java/de/fhdo/swt/carshop/
│   │   │   ├── CarshopApplication.java
│   │   │   ├── domain/    // Entities
│   │   │   ├── repo/      // Repositories
│   │   │   ├── web/       // Controller
│   │   │   └── service/
│   │   └── resources/
│   │       ├── application.properties
│   │       └── static/
│   └── test/
│       └── java/de/fhdo/swt/carshop/
└── build/
```

- **CarshopApplication**
  Hauptklasse mit @SpringBootApplication und main().
- **domain/**
  JPA-Entities mit Annotation-driven Mapping.
- **repo/**
  Repository-Interfaces, von Spring Data implementiert.
- **web/**
  REST-Controller, ggf. DTOs.
- **service/**
  Geschäftslogik, Transaktionsgrenze.

- **application.properties**
  externe Konfiguration der Anwendung wie Server-Ports, DB-Verbindungen, etc.

© Prof. Dr. Sabine Sachweh

Erster Lauf und Console-Logs


```java
@SpringBootApplication
public class CarshopApplication {
  public static void main(String[] args) {
    SpringApplication.run(
      CarshopApplication.class, args);
  }
}

// === Log-Auszug beim Start ===
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v4.0.6)

INFO  Starting CarshopApplication using Java 17.0.19 with …
INFO  Devtools property defaults active!
INFO  Bootstrapping Spring Data JPA repositories in DEFAULT mode.
INFO  H2 console available at '/h2-console'
INFO  Tomcat initialized with port(s): 8080 (http)
INFO  JPA EntityManagerFactory for persistence unit 'default'
INFO  Started CarshopApplication in 1.384 seconds
```

- **Banner**
  Spring-typische ASCII-Ausgabe; via `spring.main.banner-mode=off` (in `application.properties`) deaktivierbar.
- **DevTools** — Auto-Reload bei Code-Änderungen im Klassenpfad. Deaktivierbar indem `spring.devtools.add-properties=false` (in `application.properties`) gesetzt wird.
- **Embedded Tomcat**
  Port 8080 standardmäßig, konfigurierbar über `server.port` (in `application.properties`).
- **EntityManagerFactory**
  EntityManagerFactory für Default-Persitenz-Unit gestartet.

[Hinkula 2023, Kap. 1 — Logs and problem-solving.]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework **zur Anwendung**

we focus on students

02

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.

Kapitel 1	Kapitel 2	Kapitel 3	Kapitel 4	Kapitel 5	Kapitel 6
Setup und Werkzeuge	**DI**	**JPA**	**REST**	**Security**	**Test**
▪ Initializr	▪ IoC-Container	▪ Entities	▪ Controller	▪ JWT	▪ Unit
▪ Maven	▪ Beans	▪ Repositories	▪ Data REST	▪ OAuth2	▪ Integration
▪ IDE					

Dependency Injection (DI)

Lose Kopplung durch IoC: Service · Client · Injector — und wie Spring Boot diese Rolle übernimmt.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Dependency Injection

we focus on students

Dependency Injection ist eine Entwurfstechnik, die Objekte erschafft, die von anderen Objekten abhängen — ohne dass die Klasse selbst ihre Abhängigkeiten konstruiert.

SERVICE

Die genutzte Klasse

Eine Klasse, die genutzt wird wie z. B. eine Klasse `Owner`.

CLIENT

Die nutzende Klasse

Eine Klasse, die den Service ***nutzt*** wie z. B. eine Klasse `Car`.

INJECTOR

Der Vermittler

Eine Komponente, die den Service in den Client ***injiziert.***

In Spring: der `ApplicationContext`.

```
// ❌ ohne DI: enge Kopplung
class Car {
  private Owner owner;
  Car() {
    owner = new Owner();
  }
}
```

```
// ✓ mit DI: lose Kopplung
class Car {
  private final Owner owner;
  Car(Owner owner) {
    this.owner = owner;
  }
}
```

[Hinkula 2023, Kap. 2]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Dependency Injection Arten

Constructor · Setter · Field

Dependency Injection ist eine Entwurfstechnik, die Objekte erschafft, die von anderen Objekten abhängen — ohne dass die Klasse selbst ihre Abhängigkeiten konstruiert.

EMPFOHLEN

Constructor Injection

- Pflicht-Abhängigkeiten als Konstruktor-Parameter übergeben.
- Objekt ist nach Konstruktion vollständig.
- Felder sind `final`, leichter Test, klare Lebenszyklen.

```
// Constructor Injection
class Car {
  private final Owner owner;
  Car(Owner owner) {
    this.owner = owner;
  }
}
```

OPTIONAL

Setter Injection

- Für nicht zwingend erforderliche (optionale) Abhängigkeiten.
- Abhängigkeiten können so nach der Objekterstellung jederzeit geändert oder neu gesetzt werden.
- Mit `@Autowired(required = true)` über dem Setter kann sie erzwungen werden.

```
class Car {
  private Owner owner;
  @Autowired   // Setter Injection
  public void setOwner(Owner owner) {
    this.owner = owner;
  }
}
```

`@Autowired` meist explizit erforderlich, damit Spring die Methode nach der Konstruktion aufruft.

NICHT EMPFOHLEN

Field Injection

`@Autowired` direkt am Attribut ⇒ Probleme:

- Erlaubt keine `final`-Felder ⇒ Veränderung der Abhängigkeit später möglich.
- Macht Testen und Debuggen schwierig.
- Außerhalb von Spring sind die Felder bei der Instanziierung `null`.

```
class Car {
  // Field Injection
  @Autowired
  private Owner owner;
  . . .
}
```


© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

DI in Spring Boot — Stereotype Annotationen

we focus on students

Beim Classpath-Scan registriert Spring automatisch alle **Klassen mit Stereotype-Annotationen als Beans** im ApplicationContext. Eine Bean ist ein Java-Objekt, das vom Spring IoC-Container instanziiert, konfiguriert und verwaltet wird. Anstatt Objekte manuell mit new zu erstellen, übernimmt Spring deren Lebenszyklus-Management.

- **`@Component`** — generische Bean.
  `@Service`, `@Repository` und `@Controller` sind spezialisierte Formen von `@Component`
- **`@Service`**
  Geschäftslogik-Schicht (semantisch).
- **`@Repository`**
  Datenzugriffsschicht; übersetzt DB-Exceptions.
- **`@Controller`/ `@RestController`**
  Web-Schicht.
- **`@Configuration`** (an Klasse) + **`@Bean`** (an Methode)
  Bean-Definition für Drittklassen (externer Klassen).

HINWEIS: Spring Boot scannt standardmäßig das Package der `@SpringBootApplication`-Klasse und alle Unterpackages.

Wenn zwei Klassen (z. B. Cat und Dog) das gleiche Interface (Animal) implementieren, weiß Spring nicht, welche zu injizieren ist.

- **`@Autowired`**
  Markiert Injektionspunkt (in Konstruktor optional).
- **`@Qualifier("name")`**
  Zur Auflösung von Mehrdeutigkeiten bei mehreren Kandidaten.
- **`@Primary`**
  Default-Bean wenn mehrere Beans desselben Typs im Spring-Kontext existieren.
- **`@Scope("prototype")`**
  Sorgt dafür, dass bei jeder Anforderung einer Bean eine neue Instanz erstellt wird.
  Standard ist **„Singleton“,** dabei wird dieselbe Instanz geliefert.
- **`@Inject`**
  JSR-330-Alternative zu **`@Autowired`**.

© Prof. Dr. Sabine Sachweh

KAP. 2 · CODE-BEISPIEL

Beispiel: CarService

we focus on students

CarRepository.java

```java
@Repository
public interface CarRepository
    extends JpaRepository<Car, Long> {
  List<Car> findByBrand(String brand);
}


@Service
public class CarService {
  private final CarRepository cars;

  public CarService(CarRepository cars) {
    this.cars = cars;                    // ← injiziert
  }

  public List<Car> byBrand(String b) {
    return cars.findByBrand(b);
  }
}
```

- Kein `new CarRepository()` — Spring stellt die Bean.
- `final`-Felder garantieren Unveränderbarkeit nach Konstruktion.
- Testbar:
  Ein Mock kann in Unit-Tests einfach in den Konstruktor gegeben werden.
- Single Responsibility:
  Service kennt nur das Interface, nicht die Implementierung.

WAS PASSIERT BEIM START?

Bean-Verdrahtung

→ Spring scannt Pakete
→ registriert Beans
→ analysiert Konstruktor-Signaturen
→ instantiiert in topologischer Reihenfolge
→ injiziert.

[Hinkula 2023, Kap. 2 — Using dependency injection in Spring Boot.]

© Prof. Dr. Sabine Sachweh

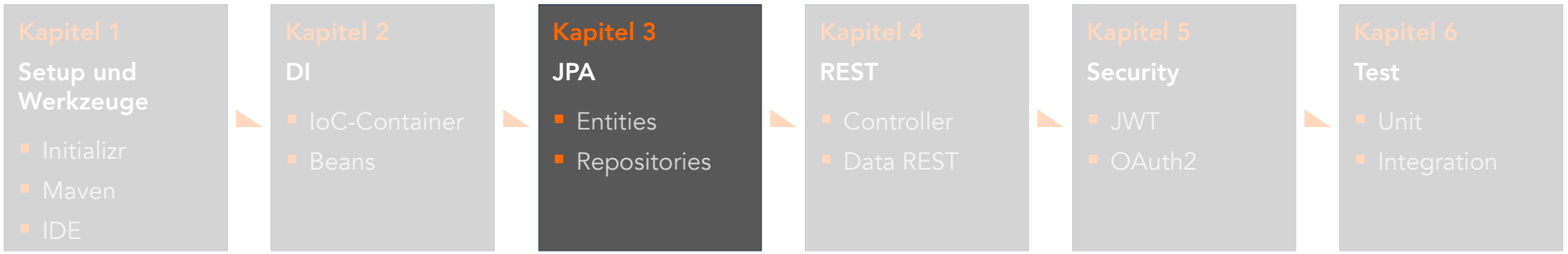
Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

we focus on students

03

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.



JPA und Datenbank
Entities, Repositories, Relationen — vom Java-Objekt zur relationalen Tabelle.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

ORM · JPA · Hibernate

we focus on students

KONZEPT

ORM

Object-Relational Mapping — Brücke zwischen objektorientiertem Code und relationalen Datenbanken.

Entwickler arbeiten mit Objekten statt SQL.

SPEZIFIKATION

JPA

Jakarta Persistence API — definiert

- Annotationen (`@Entity`, `@Id`, `@OneToMany`) und den
- `EntityManager`

Anbieter-neutral.

IMPLEMENTIERUNG

Hibernate

Etablierter JPA-Provider ⇔ Standard-Wahl in Spring Boot.

Generiert SQL, verwaltet Caches, Lazy-Loading.

[Hinkula 2023, Kap. 3 — *Basics of ORM, JPA, and Hibernate.*]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Entity-Klassen — die Persistenzbasis

we focus on students

Eine Entity-Klasse ist ein einfaches POJO mit `@Entity`, JavaBean-Konventionen (Getter/Setter), parameterlosem Konstruktor und einem Identifier.

- `@Entity`
  markiert die Klasse als persistent.
- `@Table(name = "...")`
  definiert abweichenden Tabellennamen.
- `@Id`
  markiert Primärschlüssel-Attribut.
- `@GeneratedValue`
  Schlüsselgenerierung
  - IDENTITY → Auto-Increment-Spalte der DB.
  - SEQUENCE → Sequenz-Objekt mit Werten in der DB.
  - AUTO → Persistenzanbieter (meist Hibernate) wählt automatisch die "beste" Strategie
  - TABLE → Tabelle mit Werten in der DB.
- `@Column(...)`
  Kann Spaltennamen, Länge oder Nullability definieren.
- `@Transient`
  Attribut wird nicht persistiert.

VORGEHEN

Code-First versus DB-First

Das Buch wählt Code-First:
Beim Anwendungsstart wird das Datenbankschema automatisch aus den Java-Entitätsklassen generiert.

Einstellung: `spring.jpa.hibernate.ddl-auto=`

- `none` ⇔ keine DDL-Änderungen ⇒ der sicherste Einstellung für Produktion
- `validate` ⇔ DB prüft, ob Tabellen/Spalten exisitieren – ändert nichts
- `update` ⇔ DB-Schema wird an die aktuellen Entities angepasst
- `create` ⇔ DB-Schema wird beim Start erstellt, bestehende Daten gelöscht
- `create-drop` ⇔ DB-Schema wird beim Start erstellt, und beim Beenden der Anwendung gelöscht ⇒ ut für kurze Tests

In Produktion:
Die Versionierung von Datenbankschemata erfolgt durch die Integration von Migrationstools wie Flyway oder Liquibase, die beim Anwendungs-start SQL-Skripte oder Konfigurationsdateien ausführen.

[Hinkula 2023, Kap. 3 — *Creating the entity classes.*]

© Prof. Dr. Sabine Sachweh

KAP. 3 · CODE-BEISPIEL

Beispiel: Car-Entity

we focus on students

Car.java

```java
package de.fhdo.swt.carshop.domain;

import jakarta.persistence.*;

@Entity
public class Car {

  @Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(nullable = false, length = 80)
  private String brand;

  private String model;
  private String color;
  private int year;
  private double price;

  protected Car() {}
  // + Konstruktor & Getter/Setter
}
```

RESULTIERENDES SQL

Hibernate generiert

```
CREATE TABLE car (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  brand VARCHAR(80) NOT NULL,
  model VARCHAR(255),
  color VARCHAR(255),
  year  INT,
  price DOUBLE
);
```


© Prof. Dr. Sabine Sachweh

CRUD-Repositories — Spring Data JPA

Spring Data generiert Repository-Implementierungen zur Laufzeit aus Interface-Definitionen
⇒ kein Bolierplate für Standard-CRUD.

- **`CrudRepository<T,ID>`** — **`save`**, **`findById`**, **`findAll`**, **`delete`**.
- **`PagingAndSortingRepository`**
  ergänzt Pagination und Sortierung.
- **`JpaRepository`**
  JPA-spezifische Erweiterungen wie **`flush()`**, Batch-Save.
- **Derived Queries** — **`findByBrandAndYearGreaterThan(...)`**
  werden automatisch in SQL übersetzt.
- **`@Query`** — eigene JPQL/SQL-Anweisungen.

```java
@Repository
public interface CarRepository
        extends JpaRepository<Car, Long> {

  List<Car> findByBrand(String brand);

  List<Car> findByYearGreaterThanEqual(int year);

  @Query("select c from Car c where c.price < :max")
  List<Car> affordable(@Param("max") double max);
}
```

[Hinkula 2023, Kap. 3 — *Creating CRUD repositories.*]

© Prof. Dr. Sabine Sachweh

KAP. 3 · CODE-BEISPIEL

Beispiel: Demo-Daten beim Start bereit stellen

we focus on students

CarshopApplication.java (Auszug)

```
@SpringBootApplication
public class CarshopApplication {

  public static void main(String[] args) {
    SpringApplication.run(CarshopApplication.class, args);
  }

  @Bean
CommandLineRunner seed(CarRepository cars) {
    return args -> {
      cars.save(new Car("VW",    "Golf",    "silber", 2020, 21900));
      cars.save(new Car("Tesla", "Model 3", "weiß",   2023, 42990));
      cars.save(new Car("Ford",  "Focus",   "blau",   2018, 12500));

      cars.findByBrand("VW").forEach(System.out::println);
    };
  }
}
```

HINWEIS:
Die `CommandLineRunner`-Bean wird vom Spring-Container nach dem Start des ApplicationContexts ausgeführt — typisches Muster für Demo-Daten.

© Prof. Dr. Sabine Sachweh

Relationen zwischen Tabellen — One-to-Many

we focus on students

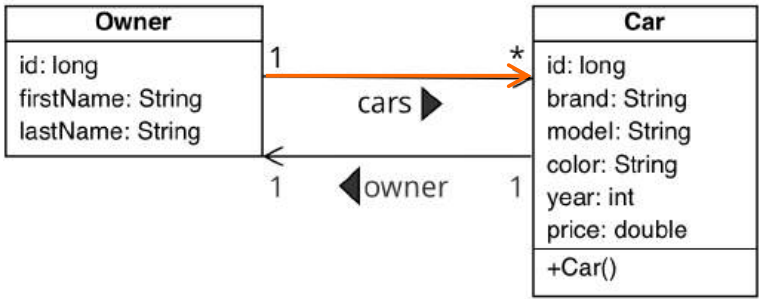
- **`@OneToMany`**
  ein Entity verweist auf viele andere Entities.
- **`@ManyToOne`**
  die andere Seite der Beziehung.
- **`mappedBy`**
  kennzeichnet die inverse Seite (nicht-besitzende) Seite einer bidirektionalen Beziehung (doppelte Fremdschlüssel-Spalten werden vermieden). Besitzende Seite mit **`@JoinColumn`** markiert
- **`Cascade`**
  kaskadierende Operationen (PERSIST, MERGE, REMOVE, ALL). Default ist keine Kaskadierung.
- **`@fetch`**
  `EAGER` (sofort) oder `LAZY` (bei Zugriff).
  Default ist: `FetchType.LAZY`

DESIGNENTSCHEIDUNG

Wer ist *Owner* der Beziehung?

Die Seite *ohne* `mappedBy` hält den Fremdschlüssel.

Im CarShop:
`Car` hat ein `owner`-Attribut und in der DB eine `(owner_id)`-Spalte.
`Owner` bekommt eine Java-Liste, aber in der dazugehörigen DB-Tabelle keine eigene Spalte.



[Hinkula 2023, Kap. 3 — *Adding relationships between tables.*]

© Prof. Dr. Sabine Sachweh

KAP. 3 · CODE-BEISPIEL

Beispiel: Owner ↔ Car

we focus on students

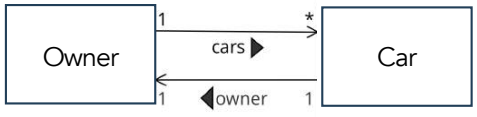
Owner.java

```java
@Entity
public class Owner {
  @Id @GeneratedValue(strategy = IDENTITY)
  private Long id;

  private String firstName, lastName;

  @OneToMany(
      cascade = CascadeType.ALL,
      mappedBy = "owner",
      fetch = FetchType.LAZY)
  private List<Car> cars = new ArrayList<>();

  // + getter/setter
}
```

Car.java (ergänzt)

```java
@Entity
public class Car {
  // id, brand, model, color, year, price …
@ManyToOne
@JoinColumn(name = "owner_id")
  private Owner owner;

  public void setOwner(Owner o) {
    this.owner = o;
    if (o != null) o.getCars().add(this); // Konsistenz }
}
```

HINWEIS:
Resultat im Schema:
- Tabelle `car` mit Spalte `owner_id` als Fremdschlüssel auf `owner.id`.
- Bidirektionale Konsistenz pflegt der `setOwner()`-Helfer.



© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

application.properties — externe Konfiguration

Persistenzwechsel – von H2 zu MariaDB

we focus on students

H2

In-Memory DB zur Entwicklung

- Schneller Start
- Keine externe Installation
- Web-Konsole unter `/h2-console`
- Daten verschwinden beim Neustart

H2 (dev)

```
# In-Memory H2
spring.datasource.url=jdbc:h2:mem:carshop
spring.datasource.username=sa
spring.datasource.password=

spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

- **Profile** via `application-dev.properties` / `application-prod.properties` getrennt halten.
- **Zugangsdaten** niemals einchecken — `${DB_PWD}` aus Umgebungsvariablen ziehen.

MARIADB

Relationale DB für Produktion

- Persistente Speicherung
- Zugriff über Standard-Port 3306
- JDBC-Treiber `mariadb-java-client`
- Identische JPA-Schicht — nur Properties ändern sich

MariaDB (prod)

```
# MariaDB
spring.datasource.url=\
  jdbc:mariadb://localhost:3306/carshop
spring.datasource.username=carshop
spring.datasource.password=${DB_PWD}

spring.jpa.database-platform=\
  org.hibernate.dialect.MariaDBDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
```

- **`ddl-auto`-Werte** vgl. Folie zu Entity-Klassen
- **In Produktion: `validate`**
  Schema-Änderungen explizit per Migrationstool.

© Prof. Dr. Sabine Sachweh

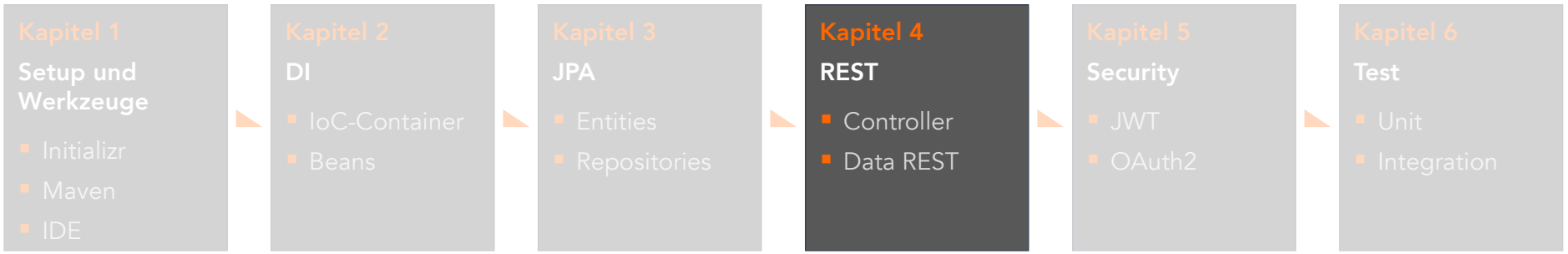
Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

we focus on students

04

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.



RESTful Web Services
Vom Controller zur dokumentierten Schnittstelle — REST, Spring Data REST und OpenAPI 3.

© Prof. Dr. Sabine Sachweh

REST — die sechs Prinzipien nach Fielding

we focus on students

01
Zustandslosigkeit

Server speichert keinen Sitzungszustand. Jede Anfrage enthält alle nötigen Informationen.

⇒ einfache Skalierung
⇒ weniger Server-Komplexität

02
Client-Server

Saubere Trennung von Client und Server. Beide Seiten entwickeln sich unabhängig voneinander.

⇒ bessere Skalierbarkeit
⇒ bessere Wartbarkeit

03
Cachebarkeit

Antworten können als cachebar markiert werden. Clients oder Zwischenserver dürfen Daten zwischenspeichern.

⇒ bessere Performance
⇒ weniger Serverlast

04
Einheitliche Schnittstelle

- Ressourcen über URIs identifizieren
- Standardisierte HTTP-Methoden
- Repräsentationen (z. B. JSON)
- HATEOAS (Links für nächste Aktionen)

⇒ einfache und einheitliche Nutzung

05
Schichtenarchitektur

System kann aus mehreren Schichten bestehen: Load Balancer, Proxy, Server. Der Client merkt das nicht. Komponenten sind austauschbar.

⇒ bessere Skalierbarkeit
⇒ bessere Sicherheit

06
Code on Demand (optional)

Server kann Code an Client senden z. B. JavaScript.

⇒ flexible Erweiterung des Clients

Wichtig: Wird selten genutzt!

[Hinkula 2023, Kap. 4 — *Basics of REST*.]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Restful Controller

we focus on students

Mit `@RestController` wird eine Klasse zur Web-Schnittstelle. Spring konvertiert Rückgaben automatisch mit Hilfe der Jackson-Bibliothek zu JSON.

markiert die Klasse als Spring-Controller

Sorgt dafür, dass Rückgabewerte direkt als HTTP-Response gesendet werden.

- **`@RestController` = `@Controller` + `@ResponseBody`**
  für jede Methode.
- **`@RequestMapping("/api/cars")`**
  Basis-Pfad der Klasse.
- **`@GetMapping` · `@PostMapping` · `@PutMapping` · `@DeleteMapping`**
  s.o. Pfad für die entsprechenden HTTP-Methoden.
- **`@PathVariable`**
  um Variablen aus der URL als Methodenparameter zu übernehmen.
- **`@RequestParam`**
  Liest Werte hinter dem „?“ in der URL (Query-Parameter) aus, um diese als Methodenparameter zu übernehmen.
- **`@RequestBody`**
  wird verwendet, um den Inhalt (Body) einer HTTP-Anfrage direkt in ein Java-Objekt umzuwandeln.

```
@GetMapping ("/cars/{id}")
public String getCar(@PathVariable int id) {
      return "Car ID: " + id;
}
```

```
@GetMapping("/cars")
public String getCar(@RequestParam String brand) {
      return "Brand: " + brand;
}
```

STATUS-CODES

Sprechende Antworten

- Standardmäßig: 200 OK.
- Bei `POST`: 201 Created
- Bei nichts gefunden: 404 Not Found.

Über `ResponseEntity<T>` präzise steuerbar.

© Prof. Dr. Sabine Sachweh

Beispiel: CarController

CarController.java

```java
@RestController                                          // Klasse als Web-Schnittstelle
@RequestMapping("/api/cars")                             // Basis-Pfad der Klasse.
public class CarController {

  private final CarRepository cars;

  public CarController(CarRepository cars) { this.cars = cars; }

  @GetMapping                                            // Bearbeitung GET all-Request
  public List<Car> all() { return cars.findAll(); }

  @GetMapping("/{id}")                                   // Bearbeitung GET-Request
  public Car one(@PathVariable Long id) {                // Variable id als Parameter
    return cars.findById(id)
        .orElseThrow(() -> new ResponseStatusException(NOT_FOUND));
  }

  @PostMapping                                           // Bearbeitung Post-Request
  @ResponseStatus(HttpStatus.CREATED)                    // Rückgabe-Status: 201 Created
  public Car create(@RequestBody Car car) { return cars.save(car); }   // Wandel RequestBody in ein
                                                         // Java-Object
  @DeleteMapping("/{id}")
  public void delete(@PathVariable Long id) { cars.deleteById(id); }   // Variable id als Parameter
}
```

HINWEIS:

Den Controller klein halten: er übernimmt nur Mapping und Validierung.

Geschäftslogik gehört in einen `@Service`, an den der Controller delegieren kann.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

HTTP-Methoden und Status-Codes für REST

we focus on students

METHODE	PFAD-BEISPIEL	WIRKUNG	IDEMPOTENT	ERFOLGS-STATUS
GET	/api/cars	Alle Fahrzeuge auflisten	ja	200 OK
GET	/api/cars/{id}	Einzelnes Fahrzeug	ja	200 / 404
POST	/api/cars	Neues Fahrzeug erzeugen	nein	201 Created
PUT	/api/cars/{id}	Fahrzeug ersetzen	ja	200 / 204
PATCH	/api/cars/{id}	Teil-Aktualisierung	nein	200 / 204
DELETE	/api/cars/{id}	Fahrzeug löschen	ja	204 No Content

HINWEIS:

Idempotenz

Mehrfaches Ausführen einer Operation hat dieselbe Wirkung wie eine einmalige Ausführung. Wichtig für Retry-Sicherheit in verteilten Systemen.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Spring Data REST — Repository als HTTP-Endpunkt

we focus on students

Spring Data REST macht Repositories automatisch über REST-Endpunkte zugreifbar. Es wird eine vollständige CRUD-API ohne selbstgeschriebenen Controller erzeugt inkl. HATEOAS-Links (Aufruf weiterer Methoden) und Pagination (Seitenverwaltung).

build.maven

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
// fertig – kein Code mehr nötig.
```

```
public interface CarRepository
                    extends JpaRepository<Car, Long> {
}
// liefert automatisch:
// GET    /cars        → alle Autos
// GET    /cars/1      → ein Auto
// POST   /cars        → neues Auto
// PUT    /cars/1      → Auto ändern
// DELETE /cars/1      → Auto löschen
```

- **Endpunkt:** `/cars` automatisch aus `CarRepository`
- **Methoden:** GET, POST, PUT, PATCH, DELETE.
- **Pagination:** `?page=0&size=10&sort=year,desc`.
  `page=0` → erste Seite, `size=10` → 10 Einträge pro Seite,
  `sort=year,desc` → absteigend sortiert nach Jahr
- **Konfigurierbar:**
  `@RepositoryRestResource(exported=false)` als Annotation am Repository deaktiviert die auto-generierte REST-API.
- **Was im Hintergrund passiert:**
  - Spring erkennt dein Repository
  - Erstellt automatisch: Controller, Endpunkte, JSON-Ausgaben
  - Nutzt oft: HATEOAS (Links in Responses)

[Hinkula 2023, Kap. 4 — *Using Spring Data REST*.]

© Prof. Dr. Sabine Sachweh

Generierte CRUD-Endpunkte in Aktion

HATEOAS ⇔ Hypermedia As The Engine Of Application State.
Liefert neben Daten auch nächstmögliche Aktionen als Links.

cURL

```
# Alle Fahrzeuge
$ curl http://localhost:8080/cars

# JSON-Antwort mit HATEOAS
{
  "_embedded": {
    "cars": [
      { "brand": "VW", "model": "Golf", "year": 2020,
        "_links": { "self": { "href": "http://localhost:8080/cars/1" } }
    ]
  },
  "page": { "size": 20, "totalElements": 3, "totalPages": 1 }
}

# Neues Fahrzeug anlegen
$ curl -X POST -H "Content-Type: application/json" \
       -d '{"brand":"BMW","model":"i4","year":2024,"price":58000}' \
       http://localhost:8080/cars
```

HINWEIS
cURL ist ein Kommandozeilen-Tool für **HTTP-Anfragen.**

- Sendet eine **GET-Anfrage**.
- **Antwort** liefert ein Objekt (VW, Golf, 2020)
- **Embedded** enthält die eigentlichen Daten (hier: cars)
- **Link** (self) im `_links-Bereich` verweist auf den aktuellen Datensatz
- **next / prev / first / last** erscheinen z.B. im `_links-Bereich` und ermöglichen zwischen Seiten zu navigieren.
- **Paging-Information** besagt,
  - es werden bis zu 20 Datensätze pro Seite angezeigt,
  - es gibt insgesamt 3 Einträge,
  - alle Einträge passen auf eine Seite

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

OpenAPI 3 — Schnittstellen dokumentieren

we focus on students

OpenAPI ist der Industriestandard zur maschinenlesbaren Beschreibung von REST-APIs.
Und Grundlage für interaktive Doku (Swagger UI), Code-Generierung und Vertragstests.

HINWEIS

Eine **interaktive Doku** einer REST-API ist ein Tool/ eine Webseite, zum Visualisieren, Verstehen und direktem Testen der API.

Dazu wird benötigt:

- `springdoc-openapi-starter-webmvc-ui` als einzige Dependency.
- **REST-Controller** – OpenAPI erkennt Endpunkte automatisch
- **(Optionale) Annotationen** für schönere und genauere API-Dokumentationen: `@Operation`, `@Parameter`, `@Schema`
- **Interaktive-Doku-Tools**
  - Der Enpunkt /v3/api-docs liefert eine automatisch gen. API-Beschreibung im JSON-Format nach dem OpenAPI-3-Standard.
  - Swagger UI unter /swagger-ui.html
- **Was im Hintergrund passiert**
  - Endpunkte werden analysiert
  - OpenAPII-Dokument wird erzeugt
  - Swagger UI wird bereit gestellt

```
@Operation(summary = "Alle Autos abrufen")
@ApiResponses({
  @ApiResponse(responseCode = "200",
    description = "Liste aller Autos"),
  @ApiResponse(responseCode = "401",
    description = "Nicht authentifiziert")
})
@GetMapping
public List<Car> all() { ... }
```

[Hinkula 2023, Kap. 4 — *Documenting a RESTful API.*]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

MANUELLER TEST

API-Tests mit Postman und cURL

we focus on students

POSTMAN

GUI-basiert

- Collections für wiederkehrende Aufrufe.
- Variablen für Umgebungen (dev/prod).
- Tests in JavaScript für Antworten.

cURL

Skriptbar und CI-tauglich

- Eingebaut in jedem Unix-System.
- Ideal für Smoke-Tests in Shell-Skripten und CI-Pipelines.

Smoke-Test (bash)

```
BASE=http://localhost:8080/api/cars

curl -s -o /dev/null -w "%{http_code}\n" $BASE              # 200
curl -s -X POST -H "Content-Type: application/json" \
     -d '{"brand":"Audi","model":"A4","year":2022,"price":34000}' \
     -w "\n%{http_code}\n" $BASE                            # 201
curl -s $BASE/1 | jq '.brand'
```

HINWEIS:

Smoke-Test

Sehr einfacher Test, der prüft, ob die Anwendung grundsätzlich startet und funktioniert. Ziel: schnell erkennen, ob „alles noch lebt".

HINWEIS:

Manuelle vs. Automatisierte tests

Manuelle Tests sind ein erster Schritt — automatisierte Tests folgen in Schritt 6.

© Prof. Dr. Sabine Sachweh

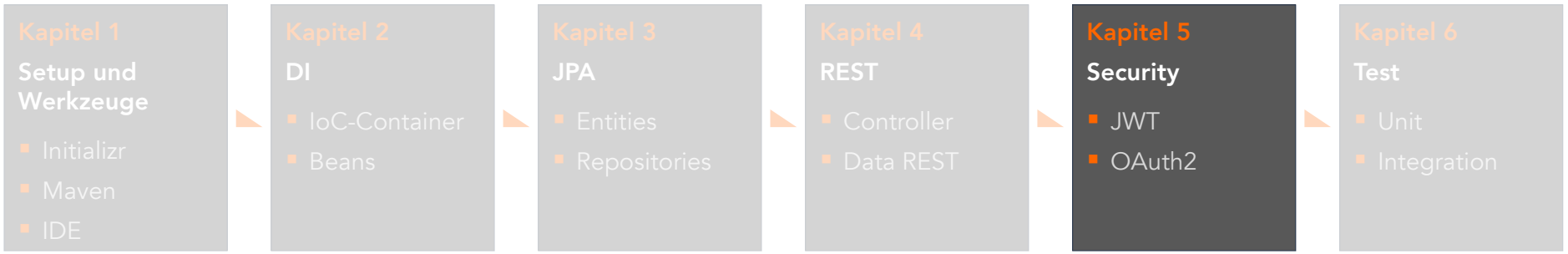
Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

we focus on students

05

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.



Security im Backend

Spring Security, JWT, Rollen, OAuth2 — Authentifizierung und Autorisierung als Crosscuting Concern.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Spring Security — Defaults und Architektur

we focus on students

Mit der Dependency `spring-boot-starter-security` sind alle Endpunkte sofort geschützt — Default-User user, Passwort im Konsolen-Log.

- **AuthenticationManager**
  prüft Credentials.
- **UserDetailsService**
  lädt Benutzerdaten (DB / LDAP / In-Memory).
- **PasswordEncoder**
  heute BCrypt; nie Klartext.
- **SecurityFilterChain**
  Kette aus Servlet-Filtern; jeder Request läuft durch.
- **SecurityContextHolder**
  ThreadLocal mit aktueller Authentifizierung.

AKTIV AB WERK

Schutzfunktionen

- HTTP-Basic-Authentifizierung
- Auto-generierte Login-Page
- CSRF-Schutz
- HSTS · XSS-Header
- Session-Fixation-Schutz

[Hinkula 2023, Kap. 5 — *Understanding Spring Security.*]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Eigene SecurityFilterChain

we focus on students

SecurityConfig.java

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

  @Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable())                    // stateless API
      .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/login", "/register").permitAll()
          .requestMatchers(HttpMethod.GET, "/api/cars/**").permitAll()
          .requestMatchers("/api/admin/**").hasRole("ADMIN")
          .anyRequest().authenticated())
      .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
  }

  @Bean PasswordEncoder encoder() { return new BCryptPasswordEncoder(); }
}
```

HINWEIS

Stateless-Modus + JWT-Filter ersetzen die klassische Session-basierte Authentifizierung.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

JSON Web Tokens — kompakt und signiert

we focus on students

Ein JWT ist ein **Base64URL**-kodiertes JSON, das aus drei Teilen besteht — getrennt durch Punkte:

```
eyJhbGciOi….eyJzdWIiOi….SflKxw…
```

- **Header** — Algorithmus und Token-Typ.
- **Payload** — Claims (Subject, Rollen, Ablauf).
- **Signature** — HMAC oder RSA über Header+Payload.

VORTEILE

Stateless

- Server muss keine Sessions halten
- Skaliert horizontal
- Frontend nutzt Token im `Authorization: Bearer …`-Header
- Selbstbeschreibend (Claims)

WICHTIG

Tokens sind nicht verschlüsselt, nur signiert. Niemals sensible Daten im Payload — und kurze Lebensdauer (15 Min) wählen.

© Prof. Dr. Sabine Sachweh

Beispiel: Token-Erzeugung

we
focus
on
students

JwtService.java

```java
@Service
public class JwtService {

  private static final long EXPIRATION_MS = 900_000;   // 15 Minuten
private final SecretKey key = Keys.hmacShaKeyFor(secret.getBytes());

  public String generate(UserDetails user) {
    return Jwts.builder()
        .subject(user.getUsername())
        .claim("roles", user.getAuthorities())
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))
        .signWith(key, Jwts.SIG.HS256)
        .compact();
  }

  public String extractUsername(String token) {
    return Jwts.parser().verifyWith(key).build()
              .parseSignedClaims(token).getPayload().getSubject();
  }
}
```

HINWEIS

Bibliothek:
`io.jsonwebtoken:jjwt` *oder*
`com.auth0:java-jwt`.

Secrets in `application.properties` *+ Umgebungsvariable.*

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Ablauf - Login-Flow und JwtFilter

we focus on students

SCHRITT 01
Login-Request
POST /login mit username + password

SCHRITT 02
Authentifizierung
AuthenticationManager prüft gegen UserDetailsService + Encoder

SCHRITT 03
Token-Ausgabe
JwtService erzeugt Token; Server antwortet mit JSON

SCHRITT 04
Folge-Requests
Client sendet Authorization: Bearer …

SCHRITT 05
JwtFilter
Validiert Signatur, setzt SecurityContext

AuthController

```
@PostMapping("/login")
public TokenResponse login(@RequestBody LoginRequest r) {
  Authentication auth = authManager.authenticate(
      new UsernamePasswordAuthenticationToken(
          r.username(), r.password()));
  return new TokenResponse(
      jwt.generate((UserDetails) auth.getPrincipal()));
}
```

JwtFilter

```
protected void doFilterInternal(HttpServletRequest req,
    HttpServletResponse res, FilterChain chain) {
  String hdr = req.getHeader("Authorization");
  if (hdr != null && hdr.startsWith("Bearer ")) {
    String user = jwt.extractUsername(hdr.substring(7));
    // → SecurityContextHolder befüllen
  }
  chain.doFilter(req, res);
}
```


© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Rollen-basierte Sicherheit

we focus on students

Authentifizierung beantwortet „*Wer bist du*?" — Autorisierung beantwortet „*Was darfst du*?".
Spring Security trennt diese sauber.

- **Rollen** sind Strings mit Präfix `ROLE_`
  (z. B. `ROLE_ADMIN`).
- **URL-basiert:**
  in `SecurityFilterChain` via `.hasRole("ADMIN")`.
- **Methoden-basiert:**
  `@PreAuthorize("hasRole('ADMIN')")`
  auf Service- oder Controller-Methoden.
- **Aktivierung:**
  `@EnableMethodSecurity` auf einer Konfigurations-Klasse.
- **Authorities** als feinkörnige Alternative
  (z. B. `car:write`).

```java
@RestController
public class AdminController {

  @PreAuthorize("hasRole('ADMIN')")
  @DeleteMapping("/api/cars/{id}")
  public void remove(@PathVariable Long id) {
    cars.deleteById(id);
  }

  @PreAuthorize(
      "hasRole('USER') and #id == authentication.principal.id")
  @GetMapping("/api/owners/{id}")
  public Owner profile(@PathVariable Long id) { ... }
}
```


© Prof. Dr. Sabine Sachweh

OAuth 2.1 — Login über Identity Provider

OAuth verlagert Authentifizierung an einen externen Provider (Google, GitHub, Keycloak). Die Anwendung sieht nur noch validierte Tokens.

RESOURCE OWNER

Endbenutzer

Besitzt das Konto, willigt ein.

AUTHORIZATION SERVER

Identity Provider

Stellt Access- und ID-Tokens aus (z. B. Keycloak).

RESOURCE SERVER

Unsere API

Validiert Tokens, exponiert geschützte Ressourcen.

application.properties

```
spring.security.oauth2.client.registration.google.client-id=…
spring.security.oauth2.client.registration.google.client-secret=…
spring.security.oauth2.client.registration.google.scope=openid,profile,email

spring.security.oauth2.resourceserver.jwt.issuer-uri=https://accounts.google.com
```

[Hinkula 2023, Kap. 5 — *Using OAuth2 with Spring Boot.*]

© Prof. Dr. Sabine Sachweh

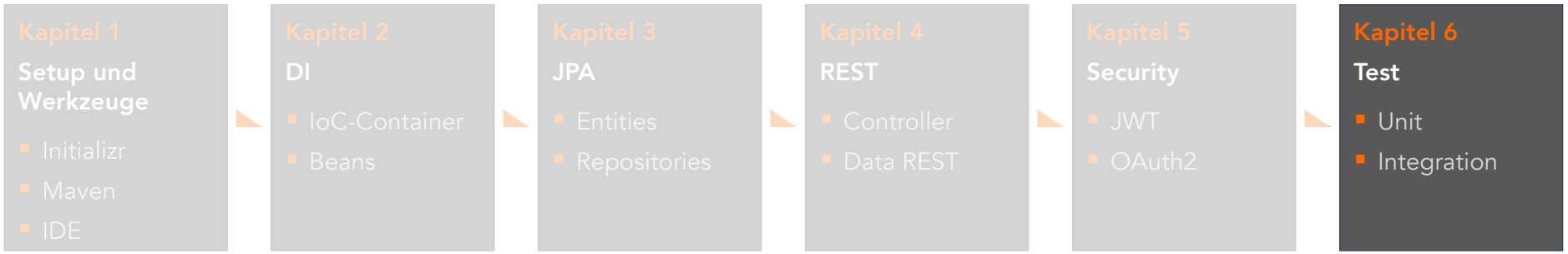
Fachhochschule Dortmund
University of Applied Sciences and Arts

Vom Framework zur Anwendung

we focus on students

06

Beispielanwendung in Teilschritten auf Basis des **I.-Teils** des o.g. Buchs von Hinkula.



Testing im Backend
JUnit 5, Mockito, MockMvc — vom Unit-Test bis zum Integrationstest mit hochgefahrenem Context.

© Prof. Dr. Sabine Sachweh

Test-Pyramide und Spring-Test-Kit

- **Unit-Tests**
  kleinste Einheiten isoliert; Abhängigkeiten gemockt.
- **Integration**
  Zusammenspiel mehrerer Komponenten.
- **Funktional / End-To-End**
  vollständiger Anwendungspfad über HTTP.

Faustregel

Viele Unit-Tests, einige Integrations-Tests, wenige End-to-End-Tests.

SPRING-BOOT-STARTER-TEST

Bündelt alles Nötige

- **JUnit 5 (Jupiter)** — Test-Framework für Java
- **Mockito** — Mocking-Tool
- **AssertJ** — lesbare, fließende (fluent) Assertions
- **Hamcrest** — Matcher
- **Spring TestContext Framework**
- **JsonPath** — Antwort-Inspektion

[Hinkula 2023, Kap. 5 — *Testing im Spring Boot.*]

© Prof. Dr. Sabine Sachweh

Unit-Tests mit JUnit 5 und Mockito

Faustregel

Für schnelle Tests, kein ApplicationContext-Start — unabhängig von DB oder externen Systemen testen.
Ideal für **Test-Driven-Development (TDD)**-Zyklen im Sekundenbereich.

CarServiceTest.java

```
@ExtendWith(MockitoExtension.class)                    // Junit-Annotation zur Integration von Mockito
class CarServiceTest {

  @Mock CarRepository repo;                            // repo ist Mock-Objekt (simuliertes Repo)
  @InjectMocks CarService service;                     // Erstellt Instanz der zu testenden CarService-
                                                       // Klasse u. injiziert alle Mock-Objekte
  @Test                                                // Junit-Annotation: markiert Methode als ausführbaren Test
void byBrand_returnsMatches() {
    Car golf = new Car("VW", "Golf", "silber", 2020, 21900);
    when(repo.findByBrand("VW")).thenReturn(List.of(golf));   // Ausdruck aus dem Mocking-Framework Mockito
                                                              // das echte Repository wird nicht benutzt –
                                                              // Antwort wird simuliert
    List<Car> result = service.byBrand("VW");                 // Eigentlicher Service-Aufruf, der getestet wird

    assertThat(result).hasSize(1)                             // Prüft, ob Methodenauf das richtige Ergbnis liefert.
        .first().extracting(Car::getModel).isEqualTo("Golf");
    verify(repo).findByBrand("VW");                           // Mockito-Ausdruck, der prüft, ob Methode mindestens einmal
                                                              // aufgerufen wurde u. ob die richtigen Parameter verwendet
  }                                                           // wurden
}
```


© Prof. Dr. Sabine Sachweh

Integrationstest mit @SpringBootTest

CarApiIT.java

```java
@SpringBootTest(webEnvironment = RANDOM_PORT)
@AutoConfigureMockMvc
class CarApiIT {

  @Autowired MockMvc mvc;
  @Autowired CarRepository repo;

  @BeforeEach
void seed() {
    repo.save(new Car("BMW", "i4", "blau", 2024, 58000));
  }

  @Test
void getAll_returns200_andContent() throws Exception {
    mvc.perform(get("/api/cars"))
       .andExpect(status().isOk())
       .andExpect(jsonPath("$[0].brand").value("BMW"));
  }
}
```

HINWEIS

Der gesamte ApplicationContext wird hochgefahren inklusive eingebetteter H2 (über Test-Profil).
MockMvc spricht das DispatcherServlet an, ohne eine echte Webserver-Umgebung (wie Tomcat) zu starten.

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Repository-Test mit @DataJpaTest

Slice-Test - Mittelding zwischen Unit- und Integrationstest

we focus on students

CarRepositoryTest.java

```java
@DataJpaTest
class CarRepositoryTest {

  @Autowired TestEntityManager em;
  @Autowired CarRepository cars;

  @Test
void findByBrand_returnsMatches() {
    em.persist(new Car("Audi", "A4", "grau", 2022, 34000));
    em.persist(new Car("VW",   "Golf","rot",  2020, 21000));
    em.flush();    // schreibt Änderung direkt in die DB

    assertThat(cars.findByBrand("Audi"))
        .extracting(Car::getModel)
        .containsExactly("A4");
  }
}
```

- **Slice-Tests** laden nur einen Teil des Kontextes.
  ⇒ Schneller als `@SpringBootTest`.
- `@DataJpaTest` liefert Repositories, Test-EntityManager und eingebettete DB mit.
- `@WebMvcTest` als Pendant für Controller-Schicht.
- **Transaktionen** werden am Ende jedes Tests automatisch zurückgerollt, damit die DB nach jedem Test wieder im ursprünglichen Zustand ist.
  ⇒ Keine Test-Reihenfolge-Probleme.

ABLAUF

→ Test startet
→ Transaktion beginnt
→ Daten werden gespeichert
→ Test prüft Ergebnis
→ Transaktion wird zurückgerollt
→ Datenbank ist wieder sauber

© Prof. Dr. Sabine Sachweh

Test-Driven Development mit Spring Boot

SCHRITT 01
RED – Test schreiben
- Test für neue Fkt. schreiben.
- Schlägt fehlt, weil Fkt. noch nicht existiert

Ziel: Anforderungen klar definieren.

SCHRITT 02
GREEN – Code schreiben
Minimaler Code, um den Test zu bestehen —

Fokus: Test bestehen, nicht perfekter Code.

SCHRITT 03
REFACTOR
Code aufräumen, Duplikate entfernen — Tests bleiben grün.

Der Test beschreibt, was der Code tun soll, bevor der Code implementiert wird.
- Was soll die Funktion genau machen?
- Welche Eingaben gibt es?
- Welches Ergebnis wird erwartet?

- Es wird geprüft, welche Methoden aufgerufen werden,
- Statt zu prüfen, ob das Ergebnis korrekt ist.
- Mocks simulieren Verhalten oft zu einfach.

- **Vorteile:** hohe Testabdeckung, klare Anforderungen, weniger Regressionen.
- **Spring-Sicht:** Kleine Slice-Tests (Mittelding zwischen Unit- und Integrationstest) fokussieren, große `@SpringBootTest`-Tests erst spät hinzu nehmen.
- **Stolpersteine:** zu viele Mocks → testet Implementation statt Verhalten.
- **Testbarer Code** ergibt sich von selbst — DI macht TDD erst praktikabel.

[Hinkula 2023, Kap. 6 — *Test-driven development.*]

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

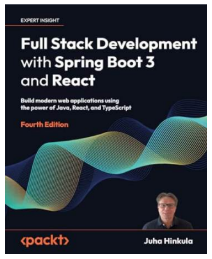
RÜCKBLICK

Zusammenfassung

Anwendungsentwicklung mit Spring Boot

we focus on students

- **Spring Boot** als opinionated Framework verstanden — Embedded Server, Auto-Konfiguration, Starter-Dependencies.
- **Acht Module** des Spring Frameworks betrachtet:
  Core, AOP, Aspects, Instrumentation, Messaging, Data Access, Web, Test.
- **Dependency Injection** als zentrales Designprinzip für lose Kopplung und testbaren Code kennengelernt.
- **Persistenz** mit JPA und Hibernate — Entities, Repositories, Relationen.
- **RESTful APIs** manuell und über Spring Data REST — dokumentiert mit OpenAPI 3.
- **Security** mit JWT-basierter Authentifizierung und rollenbasierter Autorisierung.
- **Testing** auf drei Ebenen — Unit, Slice, Integration.



Struktur des Buchs

- Teil I: Backend-Programmierung mit Spring Boot
- Teil II: Frontend-Programmierung mit React
  ⇔ Verzahnung zur REST-API
- Teil III: Full Stack Entwicklung

© Prof. Dr. Sabine Sachweh

Literatur und Quellen - Weiterführende Materialien

we focus on students

- **Hinkula, J. (2023).** Full Stack Development with Spring Boot 3 and React (4th ed.). Packt Publishing. — *Primäre Quelle dieses Foliensatzes.*
- **Spring Framework Reference Documentation.**
  `docs.spring.io/spring-framework/reference`
- **Spring Boot Reference Documentation.**
  `docs.spring.io/spring-boot/docs/current/reference/htmlsingle`
- **Spring Security Reference.**
  `docs.spring.io/spring-security/reference`
- **Fielding, R. T. (2000).** Architectural Styles and the Design of Network-based Software Architectures (Dissertation).
- **Code-Begleitung (Buch).**
  `github.com/PacktPublishing/Full-Stack-Development-with-Spring-Boot-3-and-React-Fourth-Edition`

© Prof. Dr. Sabine Sachweh

Fachhochschule Dortmund
University of Applied Sciences and Arts

Das Spring Framework und Spring Boot

Spring Framework

Spring Boot

Beispielanwendung

Von OOA hin zu OOD

Softwaretechnik 2

Systemmodelle im Überblick

Statische Modelle

Darstellung von Subsystemen und Komponenten als getrennte Einheiten.

Dynamische Modelle

Beschreiben das Verhalten des Systems zur Laufzeit.

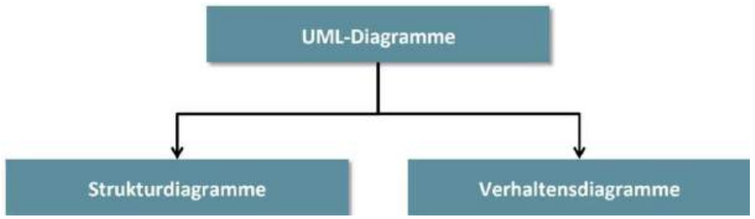
Interface-Modelle

Services, die Objekten durch ein öffentliches Interface zur Verfügung gestellt werden.

Distributions-Modelle

Verteilung von Subsystemen über verschiedene Computer.

UML-Diagrammarten



UML unterscheidet grundlegend zwei Kategorien von Diagrammen:

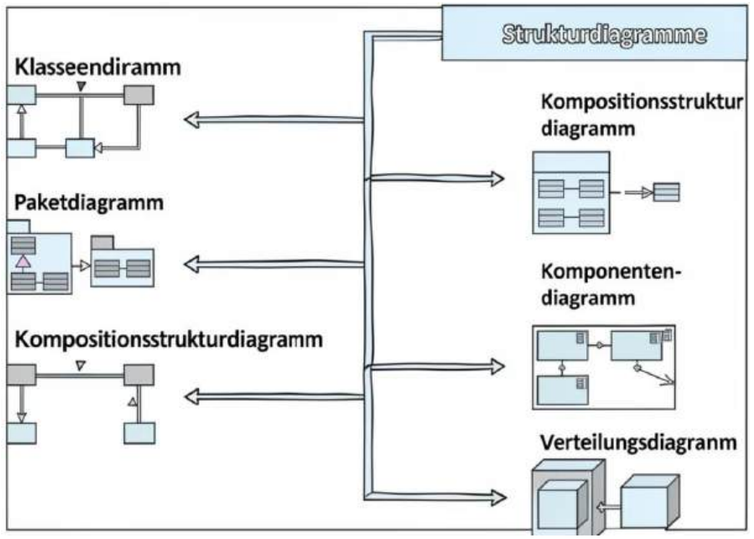
Strukturdiagramme

Zeigen den statischen Aufbau eines Systems – Klassen, Pakete, Komponenten, Verteilung.

Verhaltensdiagramme

Zeigen das dynamische Verhalten – Sequenzen, Aktivitäten, Zustände, Use Cases.

UML-Strukturdiagramme



Klassendiagramm

Klassen, Attribute, Methoden und ihre Beziehungen.

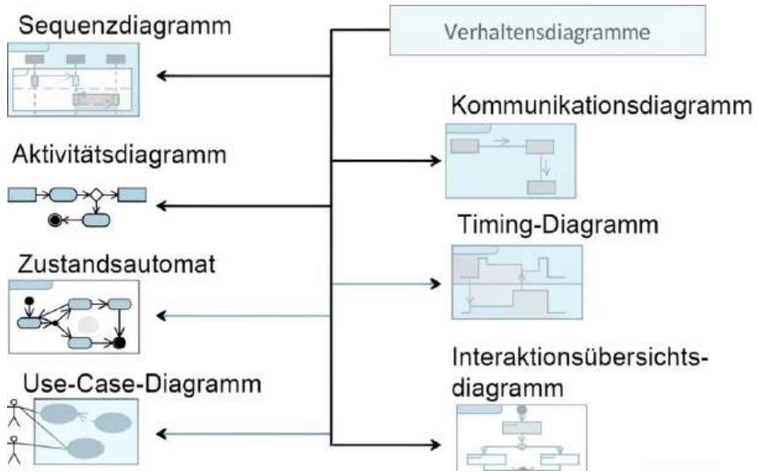
Paket- & Objektdiagramm

Pakete mit Abhängigkeiten bzw. konkrete Objektinstanzen.

Komponenten- & Verteilungsdiagramm

Systemkomponenten und deren physische Verteilung auf Hardware.

UML-Verhaltensdiagramme



Sequenzdiagramm

Nachrichtenfluss zwischen Objekten über die Zeit.

Aktivitätsdiagramm

Ablauf von Aktionen und Kontrollflüssen.

Zustandsautomat

Zustände und Übergänge eines Objekts.

Use-Case-Diagramm

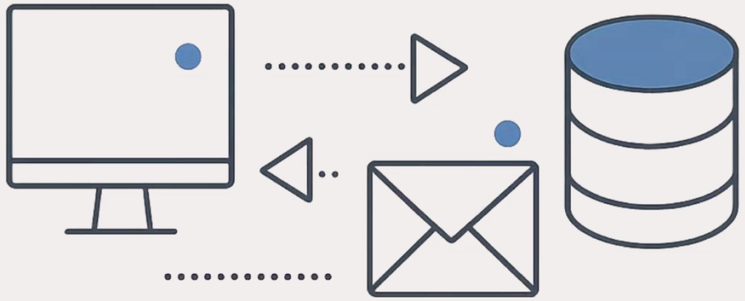
Akteure und ihre Interaktionen mit dem System.

Kommunikations-, Timing- & Interaktionsübersichtsdiagramm

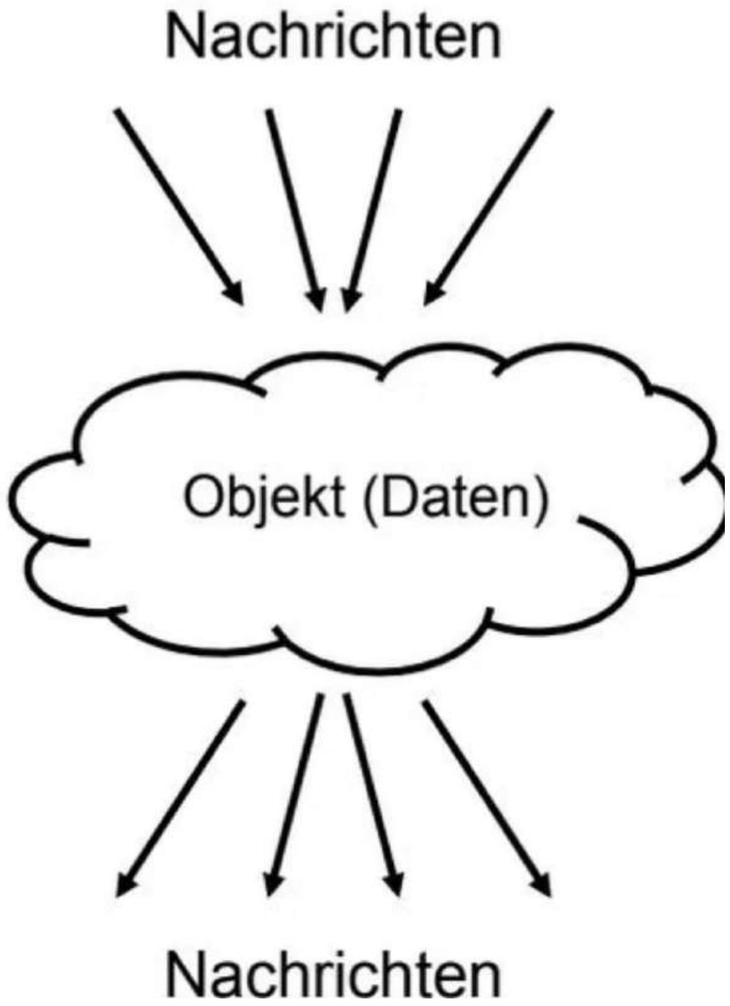
Weitere Interaktionsformen.

Objektorientiertes Design (OOD)

Jedes Ding der Realität wird durch ein Objekt (eine Klasse) im Modell repräsentiert.
Ursprung: **Daten-Abstraktion**. Standardnotation: **UML**.



OOD – Kernkonzepte



Klassen & Objekte

Objekte sind eindeutig identifizierbar, besitzen einen Zustand (Attribute) und Operationen.

Vererbung & Polymorphismus

Wiederverwendung und flexible Erweiterbarkeit von Typen.

Komponenten

Klar abgegrenzte Einheiten mit definierten Schnittstellen.

Ziele des OOD

1 Anforderungen umsetzen

Funktionale und nichtfunktionale Anforderungen realisieren.

2 Architektur & Subsysteme

Entwurfsarchitektur festlegen, System in Subsysteme aufteilen und Schnittstellen definieren.

3 Klassen definieren

Wesentliche Klassen identifizieren und Objektschnittstellen spezifizieren.

Prinzipien des Unified Entwicklungsprozesses [JBR99]: **Architekturzentriert**, **iterativ & inkrementell**, **anwendungsfallgesteuert**.

Jacobson, I., Booch, G., Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley.

Ablauf des OOD

Kontext

Systemzweck und Einsatzumgebung aus der OOA-Phase übernehmen. Use-Case-Diagramme und Szenarien als Grundlage nutzen. Nichtfunktionale Anforderungen wie Antwortzeiten, Datenschutz und Skalierbarkeit explizit dokumentieren.

Entwurfsmodelle

Dynamisches Verhalten mit Sequenzdiagrammen (Nachrichtenfluss zwischen Objekten), Zustandsautomaten (Lebenszyklen von Objekten) und Aktivitätsdiagrammen (Kontrollflüsse und Parallelität) modellieren.

Architektur

Gesamtsystem in Subsysteme und Schichten gliedern (z.B. 3-Schichten: Präsentation, Geschäftslogik, Datenhaltung). Kommunikationswege und Abhängigkeiten zwischen Subsystemen als Komponentendiagramm festhalten. Technologieentscheidungen (Frameworks, Protokolle) treffen.

Objektschnittstellen

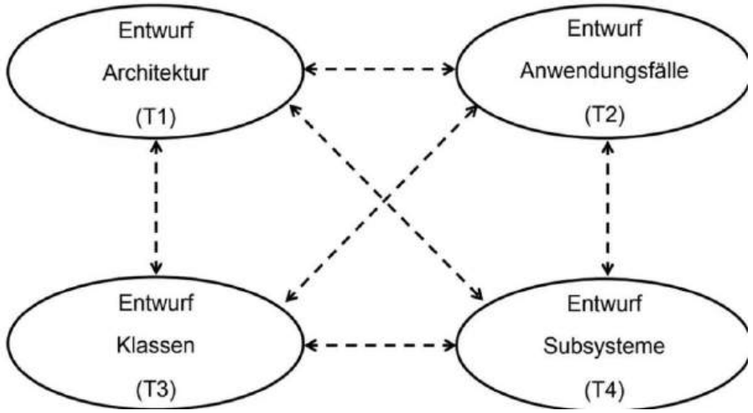
Öffentliche Schnittstellen (Methoden, Parameter, Rückgabetypen) formal spezifizieren. Vor- und Nachbedingungen (Contracts) definieren. Schnittstellen von Implementierung trennen – Clients kennen nur das „Was", nicht das „Wie".

Objekte

Analyseklassen zu Designklassen verfeinern: Attribute typisieren, Methoden mit Signaturen versehen, Sichtbarkeiten (public/private/protected) festlegen. Beziehungen (Assoziation, Vererbung, Komposition, Aggregation) in Klassendiagrammen modellieren. Verantwortlichkeiten nach SRP und DRY zuweisen.

Die fünf Tätigkeiten des OOD bauen aufeinander auf – sind aber in der Praxis iterativ und wechselseitig abhängig.

Tätigkeiten und ihre Abhängigkeiten beim OOD



Alle vier OOD-Tätigkeiten sind wechselseitig voneinander abhängig:

Änderungen in einer Tätigkeit wirken sich auf alle anderen aus.

T1 – Entwurf Architektur

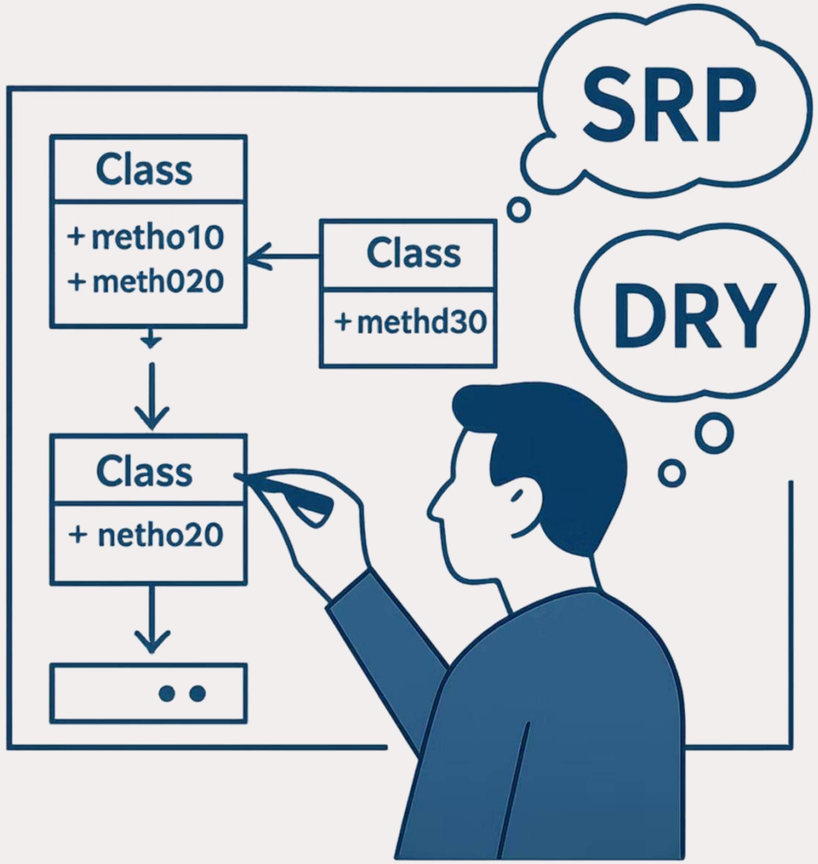
T2 – Entwurf Anwendungsfälle

T3 – Entwurf Klassen

T4 – Entwurf Subsysteme

Feinentwurf: Wie komme ich zu guten Klassen?

Gute Klassen entstehen nicht zufällig – sie folgen bewährten Prinzipien.



SRP & DRY – Grundregeln guter Klassen

Single Responsibility Principle (SRP)

Jedes Objekt hat **eine einzelne Verantwortung**.
Test: „Das Auto *startet* selber" ✓ – „Das Auto *wechselt Reifen* selber" ✗

SRP verletzt? → **Klassen spalten oder zusammenfügen.**

Don't Repeat Yourself (DRY)

Es gibt **keinen doppelten Code** im System.

DRY verletzt? → **Code auslagern** (Klasse oder Methode).

Das Design bestimmt das Bewusstsein!

Wie kommen wir zu einem guten Design? Woran erkenne ich es?

Entwurfsprinzipien im Überblick

Ein *Principle* ist „a basic truth or general law that is used as a basis of reasoning or a guide to action." Diese Prinzipien gelten in allen gängigen Ansätzen als grundlegend.

Abstraktion

Kapselung

Modularisierung ⟶Kopplung, Kohäsion

Offen-Geschlossen-Prinzip

Liskovsches Substitutionsprinzip

Delegation

Design-Prinzip: Abstraktion (1/2)

> „Abziehen oder Herauslösen von Teilgehalten aus einem konkreten Ganzen, in der Absicht, das Gleichbleibende, **Wesentliche** verschiedener Gegenstände zu erkennen." [Meyers Lexikon]

→ Verallgemeinert durch **Vernachlässigen unwesentlicher Eigenschaften**.

→ Was wesentlich ist, hängt von den **Anforderungen** ab.

Design-Prinzip: Abstraktion (2/2)

Komposition

Zusammensetzen aus Teilen.

Benutzung

Nutzung anderer Komponenten.

Generalisierung

Verallgemeinern gemeinsamer Merkmale.

Aspekte

Querschnittliche Belange trennen.

Von konkreten Blumen zur abstrakten Darstellung – Abstraktion in Aktion.

Abstraktion: Wie und wo?

Parametrisierung & Spezifikation

Abstraktion durch Parameter und formale Beschreibung.

Prozedurale & Datenabstraktion

Operationen und Datenstrukturen hinter Schnittstellen verbergen.

Übersicht & Detail

Gewinnung von Übersicht durch Weglassung von Details; systematischer Zusammenhang zwischen Übersichts- und Detailsichten.

Abstraktion = Verstehen durch systematisches Vergessen von Information.

Abstraktion: Vor- und Nachteile

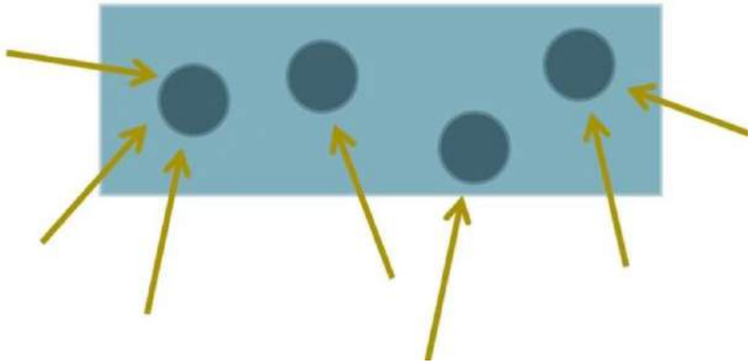
✅ Vorteile

- Systemkomplexität wird reduziert.
- Größere Allgemeingültigkeit und Wiederverwendbarkeit.

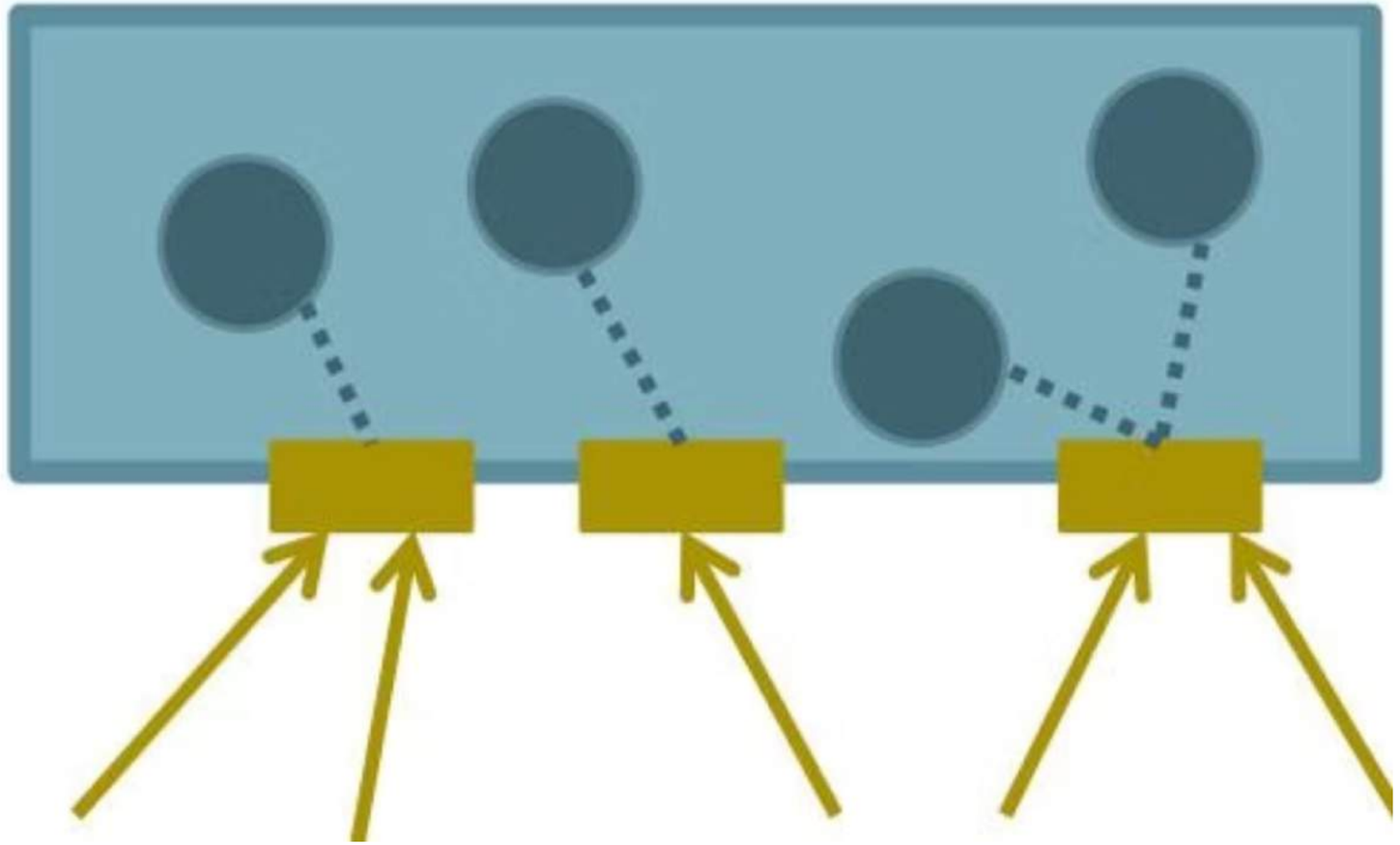
⚠️ Nachteile

- Wichtige Details können vergessen werden.
- Modellbildung setzt Erfahrung und Übung voraus.

Design-Prinzip: Kapselung



Direkter Zugriff auf Daten



Gekapselter Zugriff über Interface

Kapselung – Information Hiding [Par72]

Was – nicht Wie

Außerhalb einer Komponente ist nur bekannt, **was** sie leistet – nicht **wie**.

Vollständige Leistung

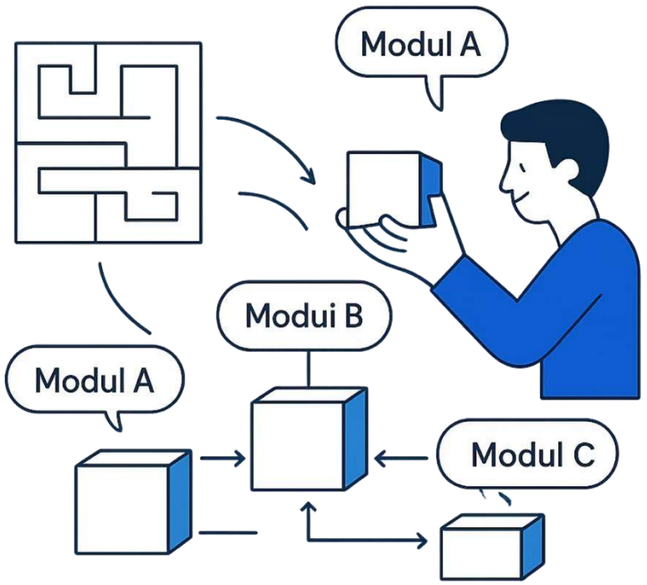
Jede Komponente erbringt eine Leistung (oder eine Gruppe logisch zusammenhängender Leistungen) vollständig.

Fundamentales Prinzip

Liefert gute Modularisierungen und ist grundlegend zur Beherrschung komplexer Systeme.

Fußnote: Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12), 1053–1058.

Design-Prinzip: Modularisierung



Dekomposition

Zerlegen eines großen Systems in kleinere, beherrschbare Einheiten. Verteilen von Funktionalität auf Module.

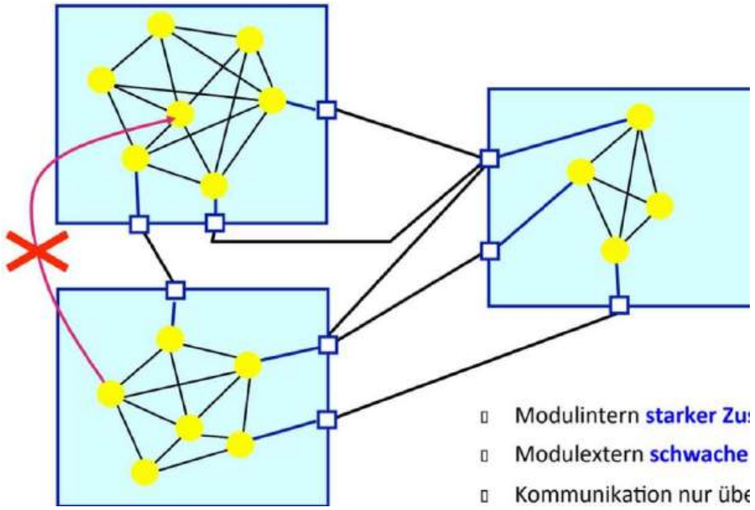
Modul

Benannte, klar abgegrenzte Komponente. Module bilden in sich geschlossene Teillösungen mit **möglichst wenigen, wohldefinierten Interaktionen** zu anderen Modulen.

Vollständigkeit & Primitivität

Komponenten enthalten alles, was zu einer Abstraktion gehört – aber nicht mehr.

Prinzip der Modularisierung



Starker innerer Zusammenhang

Modulintern hohe Kohäsion.

Schwache externe Kopplung

Modulextern minimale Abhängigkeiten.

Kommunikation über Schnittstellen

Kein direkter Zugriff auf Modulinterna.

Warum ist Modularisierung gut?

Nutzung ohne Innenkenntnisse

Die Verwendung eines Moduls erfordert keine Kenntnisse über seinen inneren Aufbau.

Änderungssicherheit

Änderungen im Inneren, die die Schnittstelle unverändert lassen, haben keine Rückwirkungen auf das übrige System.

Isolierte Korrektheit

Die Korrektheit eines Moduls ist ohne Kenntnis seiner Einbettung ins Gesamtsystem nachprüfbar.

Kohäsion & Kopplung

Kohäsion (cohesion)

Maß für den **inneren Zusammenhang** einer Komponente.
Je höher die Kohäsion, desto besser die Modularisierung.

Kopplung (coupling)

Maß für den **Zusammenhang zwischen zwei Komponenten**.
Je geringer die Kopplung, desto besser die Modularisierung.

Kohäsion und Kopplung sind **keine** Designprinzipien, sondern **Maße für die Güte** der Modularisierung.

Metriken: CBO & LCOM

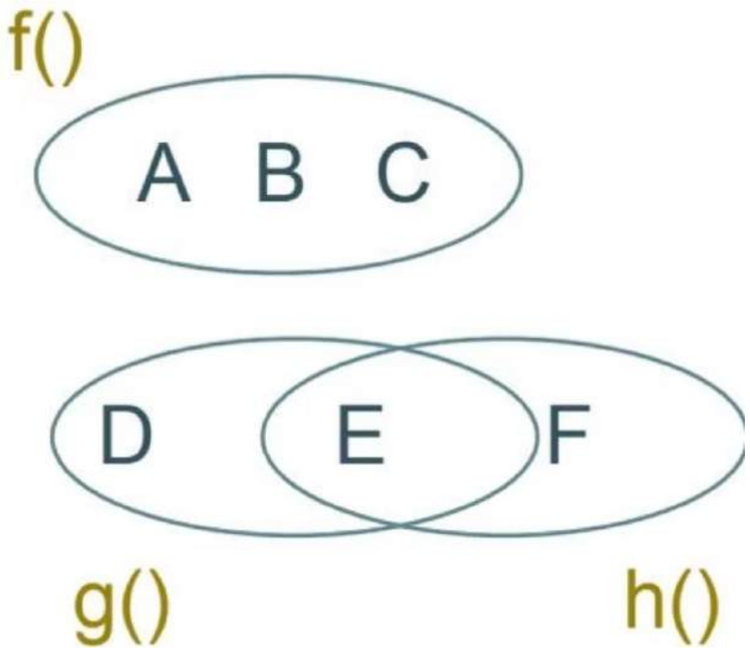
CBO – Coupling Between Object Classes

Eine Klasse ist mit einer anderen gekoppelt, wenn sie deren Methoden/Attribute benutzt oder direkt von ihr erbt. CBO gibt die **Anzahl der gekoppelten Klassen** an.

LCOM – Lack of Cohesion in Methods

Anzahl der Methodenpaare **ohne** gemeinsame Instanzvariablen **minus** Anzahl der Paare **mit** gemeinsamen Instanzvariablen.

LCOM – Beispiel




```
class X {
  int A, B, C, D, E, F;
  void f() { // uses A, B, C
  }
  void g() { // uses D, E
  }
  void h() { // uses E, F
  }
}
```

LCOM = 2 − 1 = 1f() teilt keine Variablen mit g()/h(). g() und h() teilen E. → Klasse hat geringe Kohäsion.

Stufen der Kopplung

Stufe	Beschreibung	Erreichbar für
Einbruch	Code kann verändert werden.	Einbrecher
Volle Öffnung	Auf alle Daten kann zugegriffen werden.	Methoden
Selektive Öffnung	Bestimmte Variablen sind global zugänglich.	Module, Methoden
Methodenkopplung	Kopplung nur durch Parameter oder Funktionen.	Module
Funktionskopplung	Nur durch Werteparameter und Funktionsresultate.	Module
Keine Kopplung	Kein Zugriff syntaktisch möglich.	Module

Stufen der Kohäsion

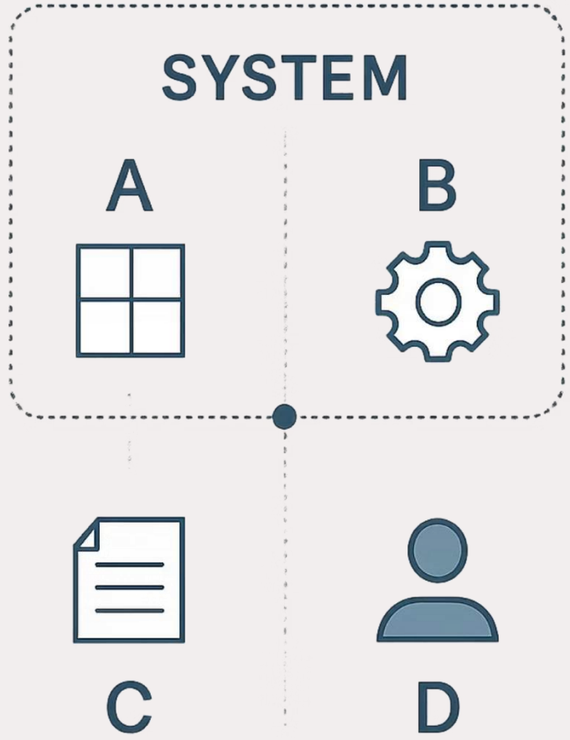
Stufe	Beschreibung	Erreichbar für
Keine Zusammenarbeit	Zufällige Zusammenstellung.	Module
Ähnlichkeit	Teile dienen einem ähnlichen Zweck.	Module
Zeitliche Nähe	Teile werden zur selben Zeit ausgeführt.	Module, Methoden
Gemeinsame Daten	Teile durch gemeinsame Daten verbunden.	Module, Methoden
Hersteller/Verbraucher	Ein Teil erzeugt, was der andere verwendet.	Module, Methoden
Einziges Datum (Kapselung)	Alle Operationen auf eine gekapselte Datenstruktur.	Module, Methoden
Einzig Operation	Nicht mehr zerlegbare Operationen.	Methoden

Separation of Concern

Idee: Jede Komponente ist nur für **einen bestimmten Aufgabenbereich** zuständig.

Separation of Concerns: Prozess, ein Programm in Features aufteilen, die sich in ihrer Funktionalität **so wenig wie möglich überschneiden**.

OO-Entwurf: Zusammenfassung von Daten und Operationen zu einer Klasse.

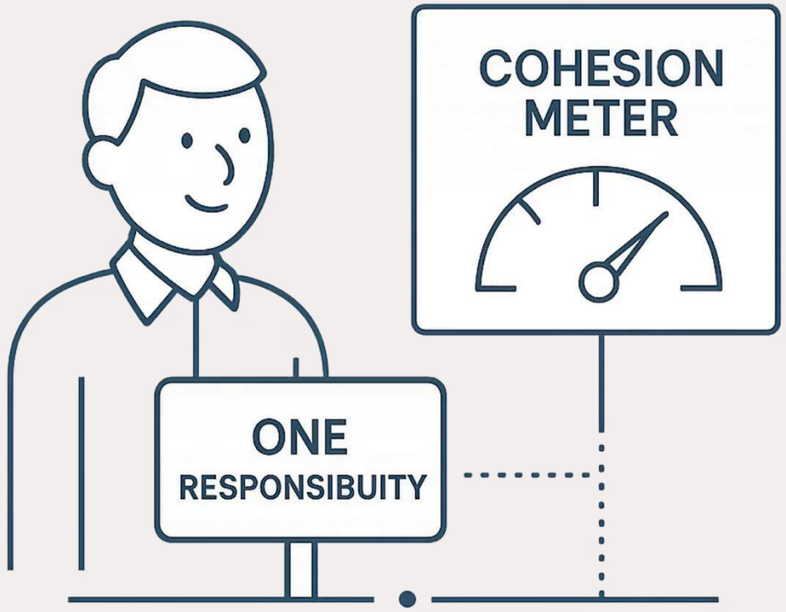


Was hat SRP mit Kohäsion zu tun?

> Wenn die Software einen hohen Kohäsionsgrad besitzt, wurde das SRP (Single Responsibility Principle) befolgt!

Hohe Kohäsion → Klasse hat starken inneren Zusammenhang → SRP wurde eingehalten

Niedrige Kohäsion → Klasse hat zu viele Verantwortlichkeiten → SRP wurde verletzt



Modularisierung & Kapselung – Vorteile

Keine Innenkenntnisse nötig

Verwendung ohne Wissen über den inneren Aufbau.

Fehlersicherheit

Gefahr unbeabsichtigter Fehlmanipulationen sinkt.

Lokalität

Datenstrukturen und Zugriffsoperatoren liegen zusammen – besser verständlich.

Änderungssicherheit

Schnittstellenstabile Änderungen haben keine Rückwirkungen.

Modularisierung & Kapselung – Nachteile

Laufzeit-Overhead – Der Aufruf vermittelnder Operationen (Getter/Setter) kostet zusätzliche Rechenzeit.

Erschwerte Zugriffe – Nicht optimale Schnittstellen können den Datenzugriff unnötig komplizieren. Lösung: Refactoring der Schnittstellen.

Trennung von Schnittstelle und Implementierung

Schnittstelle (Interface)

Beschreibt das **Leistungsangebot** einer Komponente – möglichst realisierungsneutral.

Beschreibungsmittel

Vorbedingungen, Nachbedingungen, Invarianten, Verpflichtungen.

Design by Contract

Bedingungen und Zusicherungen bilden einen **Vertrag** zwischen Modul (Leistungserbringer) und Klient (Leistungsverwender).

Geheimnisprinzip im Softwareentwurf

WAS (Schnittstelle)	WIE (Implementierung)
Modulschnittstelle – nach außen sichtbar.	Entwurfsentscheidungen – nach außen verborgen.

Vier typische verborgene Entwurfsentscheidungen:

- Wie eine Funktion realisiert ist.
- Wie ein Objekt aus dem Anwendungsbereich repräsentiert ist.
- Wie eine Datenstruktur aufgebaut ist.
- Wie Leistungen Dritter realisiert sind.

Offen-Geschlossen-Prinzip (OGP)

Module sollen **stabil benutzbar** (geschlossen/vollständig) und gleichzeitig **offen für Erweiterungen** sein.

OGP – Geschlossen & Offen

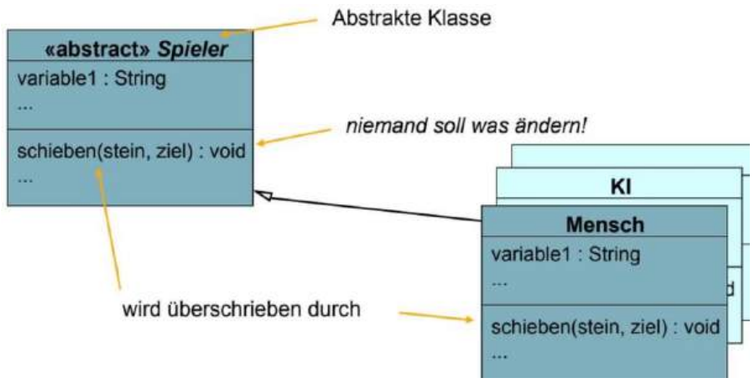
Geschlossen (= vollständig)

- Schnittstelle deckt alle Anforderungen der Clients ab.
- Kann ohne Anpassung verwendet werden.
- Ist stabil.

Offen (= erweiterbar)

- Modul lässt sich ohne Änderung der Client-Module erweitern.
- Richtige Abstraktionen finden.
- Veränderbare Aspekte implizit modellieren.

OGP – Beispiel: Abstrakte Klasse



Klassen sollen für **Erweiterungen offen** und für **Veränderungen geschlossen** sein.

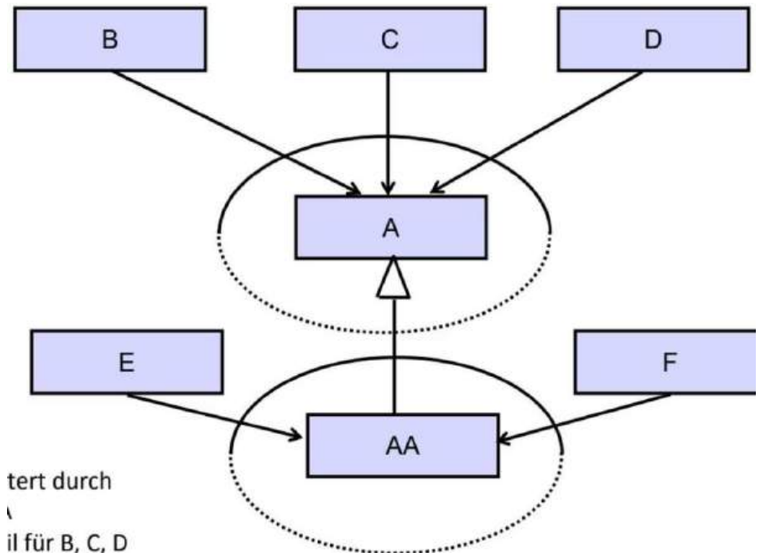
«abstract» Spieler

Niemand soll was ändern.
Methodensignatur ist vorgegeben.

Mensch extends Spieler

Überschreibt `schieben()` – offen für Erweiterung durch Vererbung.

OGP – Erweiterung durch Subklasse



A wird erweitert durch Subklasse AA.

A bleibt stabil für B, C, D.

AA liefert E, F zusätzliche Funktionalität.

OGP-Verletzung – Beispiel

Dieses Beispiel verletzt das Offen-Geschlossen-Prinzip: Jede neue Form erfordert eine Änderung der `draw()`-Methode.

```
public void draw(Form form) {
    if (form.type == CIRCLE) {
        drawCircle(form);
    }
    else if (form.type == SQUARE) {
        drawSquare(form);
    }
}
```

⊗ Lösung: Abstrakte Klasse `Form` mit überschreibbarer Methode `draw()` – jede Unterklasse implementiert ihr eigenes Zeichnen.

Liskovsches Substitutionsprinzip (LSP)

> Sei *q(x)* eine beweisbare Eigenschaft von Objekten *x* des Typs *T*. Dann soll *q(y)* für Objekte *y* des Typs *S* wahr sein, wobei *S* ein Untertyp von *T* ist.

Vereinfacht: **Subklassen müssen ihre Basisklassen ersetzen können** – ohne dass die Basisklasse davon wissen muss. Signaturen kompatibel, Vor-/Nachbedingungen einhalten.

LSP – Verletzung: Rectangle & Square (Code)

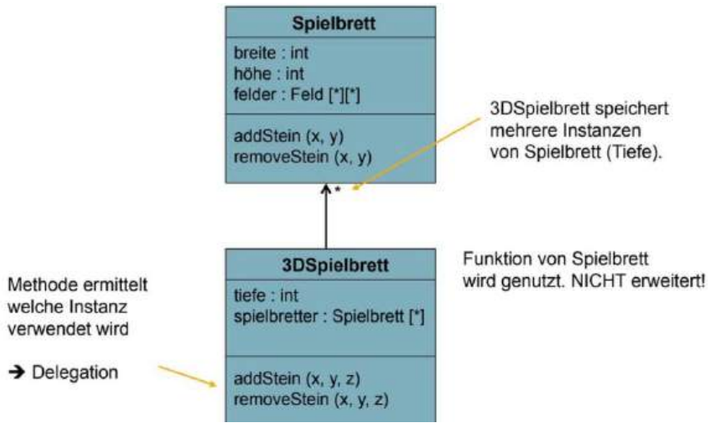
```
// Violation of Likov's Substitution Principle
class Rectangle {
    protected int m_width;
    protected int m_height;

    public void setWidth(int width) {
        m_width = width;
    }

    public void setHeight(int height) {
        m_height = height;
    }

    public int getWidth() {
        return m_width;
    }

    public int getHeight() {
        return m_height;
    }

    public int getArea() {
        return m_width * m_height;
    }
}
```

```
class Square extends Rectangle {
    public void setWidth(int width) {
        m_width = width;
        m_height = width;
    }

    public void setHeight(int height) {
        m_width = height;
        m_height = height;
    }
}
```

```
public class LSPTest {
    private static Rectangle getNewRectangle() {
        // it can be an object returned by some factory
...
        return new Square();
    }

    public static void main(String args[]) {
        Rectangle r = LSPTest.getNewRectangle();

        r.setWidth(5);
        r.setHeight(10);
        // user knows that r it's a rectangle.
        // It's assumed that he's able to set the width
and height independently

        System.out.println(r.getArea());
        // now he's surprised to see that the area is
100 instead of 50
    }
}
```

Ergebnis: `r.setWidth(5); r.setHeight(10); r.getArea()` liefert **100** statt 50 – überraschend für den Nutzer!

Besser: Delegation statt Vererbung. Das Design-Prinzip: Delegation



3DSpielbrett speichert mehrere Instanzen von Spielbrett (Tiefe). Funktion von Spielbrett wird genutzt – **nicht erweitert**!

- Delegation = Verantwortlichkeit für eine Aufgabe wird an eine **andere Klasse übergeben**.
- Geeignet, wenn Funktionalität einer anderen Klasse genutzt werden soll, **ohne sie zu ändern**.
- Typischerweise genutzt mit **Komposition**.

Vererbung vs. Komposition/Aggregation

Vererbung

Wiederverwendung durch Ableitung. Kann LSP verletzen. Geeignet bei echter „ist-ein"-Beziehung.

Komposition

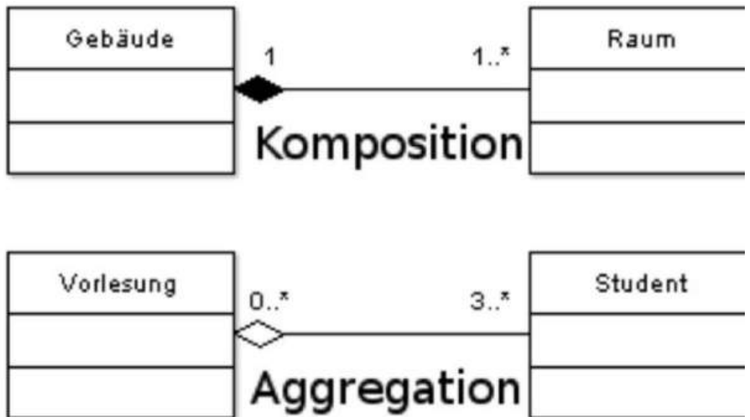
Objekt ist vollständiger Eigentümer. Teile existieren **nicht** außerhalb des Besitzers.

Aggregation

Nutzt Vorteile der Komposition. Teile bestehen **auch außerhalb** ihres Objektes.

Aggregation/Komposition ermöglichen Wiederverwendung **ohne LSP-Verletzung**.

Komposition & Aggregation in UML



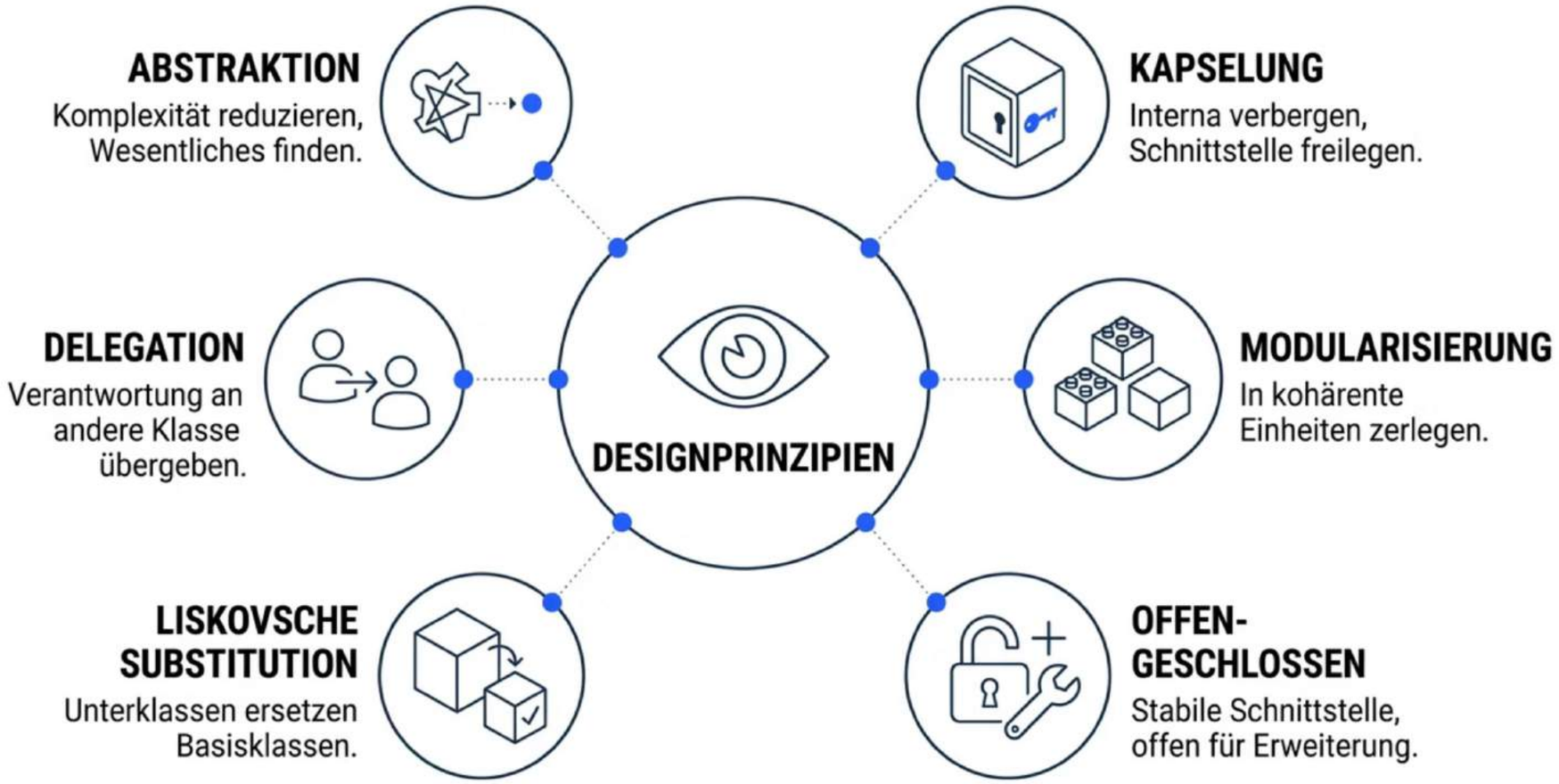
Komposition (ausgefüllte Raute)

Gebäude ◆—— **Raum**Raum existiert nur als Teil des Gebäudes. Multiplizität: 1 zu 1..*

Aggregation (offene Raute)

Vorlesung ◇—— **Student**Student existiert unabhängig von der Vorlesung. Multiplizität: 0..* zu 3..*

Entwurfsprinzipien – Zusammenfassung



Checkliste: Gutes OOD

SRP & DRY einhalten

Jede Klasse hat eine Verantwortung, kein doppelter Code.

Kapseln & Abstrahieren

WAS sichtbar, WIE verborgen. Wesentliches vom Unwesentlichen trennen.

OGP & LSP beachten

Erweiterbar ohne Änderung. Subklassen ersetzen Basisklassen korrekt.

Delegation bevorzugen

Komposition/Aggregation statt Vererbung, wo LSP gefährdet wäre.

Ausblick: Entwurfsmuster (Design Patterns)

Entwurfsmuster sind bewährte, wiederverwendbare Lösungsschablonen für häufig auftretende Entwurfsprobleme. Sie bauen direkt auf den hier gelernten Prinzipien auf.

Erzeugungsmuster

z. B. Factory, Singleton

Strukturmuster

z. B. Adapter, Decorator, Composite

Verhaltensmuster

z. B. Observer, Strategy, Command

Lernziele dieser Vorlesung

UML sicher anwenden

Struktur- und Verhaltensdiagramme gezielt für OOD einsetzen.

Entwurfsprinzipien kennen

Abstraktion, Kapselung, Modularisierung, OGP, LSP, Delegation verstehen und anwenden.

Qualität messen

Kohäsion und Kopplung mit Metriken (CBO, LCOM) bewerten.

Gute Klassen entwerfen

SRP, DRY und Separation of Concerns in der Praxis umsetzen.

Softwaremuster

Design-Patterns sind bewährte Rezepte in der Programmierung – wiederverwendbare Lösungen für wiederkehrende Probleme. Sie verbessern Codequalität, sparen Zeit und erleichtern die Kommunikation zwischen Entwicklern.

Erzeugermuster

Factory Method, Builder, Singleton

Verhaltensmuster

Strategy, Chain of Responsibility, Command, Observer, Iterator

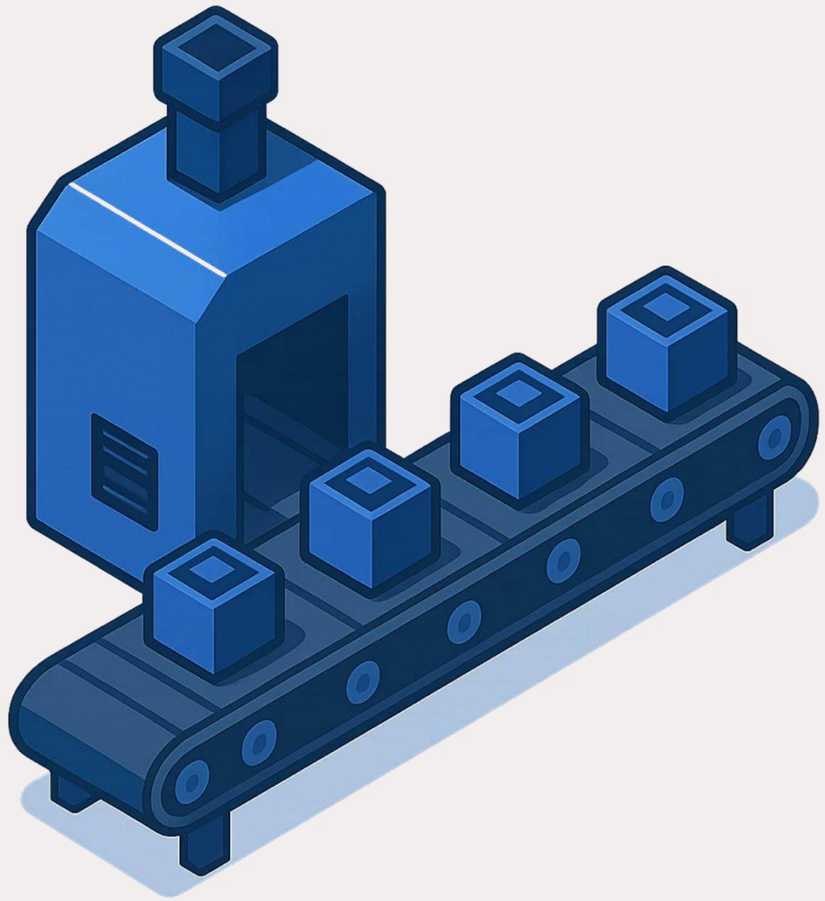
Strukturmuster

Adapter, Composite, Façade, Proxy

GANG OF FOUR · 1994

Entwurfsmuster (Design Patterns)

- Bewährte, generische Lösung für ein immer wiederkehrendes Entwurfsproblem
- Zusammengefasst in einem Standardwerk von 1994: Design Patterns. Elements of Reusable Object-Oriented Software
- Autoren (GoF, Gang of Four): Ralph Johnson, Erich Gamma, Richard Helm und John Vlissides (†2005)



Factory Method / Fabrikmethode

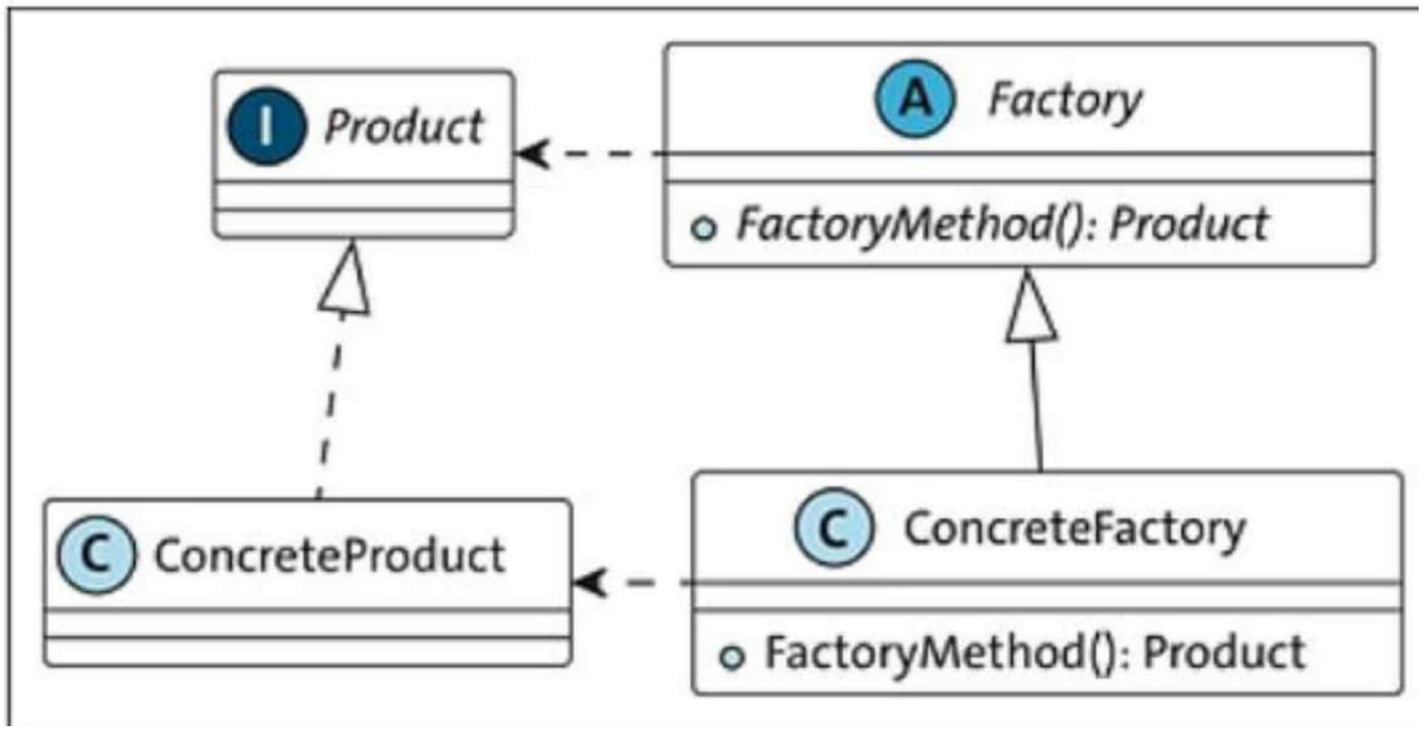
ERZEUGERMUSTER · GANG OF FOUR

Problem: Eine Schulungsverwaltungssoftware muss Räume für verschiedene Schulungstypen (Inhouse, Onsite, Online) buchen. Direkte Instanziierung führt zu unübersichtlichen If-Bedingungen an vielen Codestellen.

```
if (training.isOnsite()) {
    OnsiteLocation onsite = new OnsiteLocation();
} else if (training.isInhouse()) {
    Room room = new Room();
}
```

Lösung: Die Objekterzeugung wird in eine abstrakte *Factory-Methode* ausgelagert. Konkrete Fabriken erzeugen die passenden Produkte – ohne If-Bedingungen im aufrufenden Code.

Factory Method – Struktur



Product

Abstrakte Schnittstelle der zu erzeugenden Objekte

ConcreteProduct

Konkrete Ausprägung des Produkts

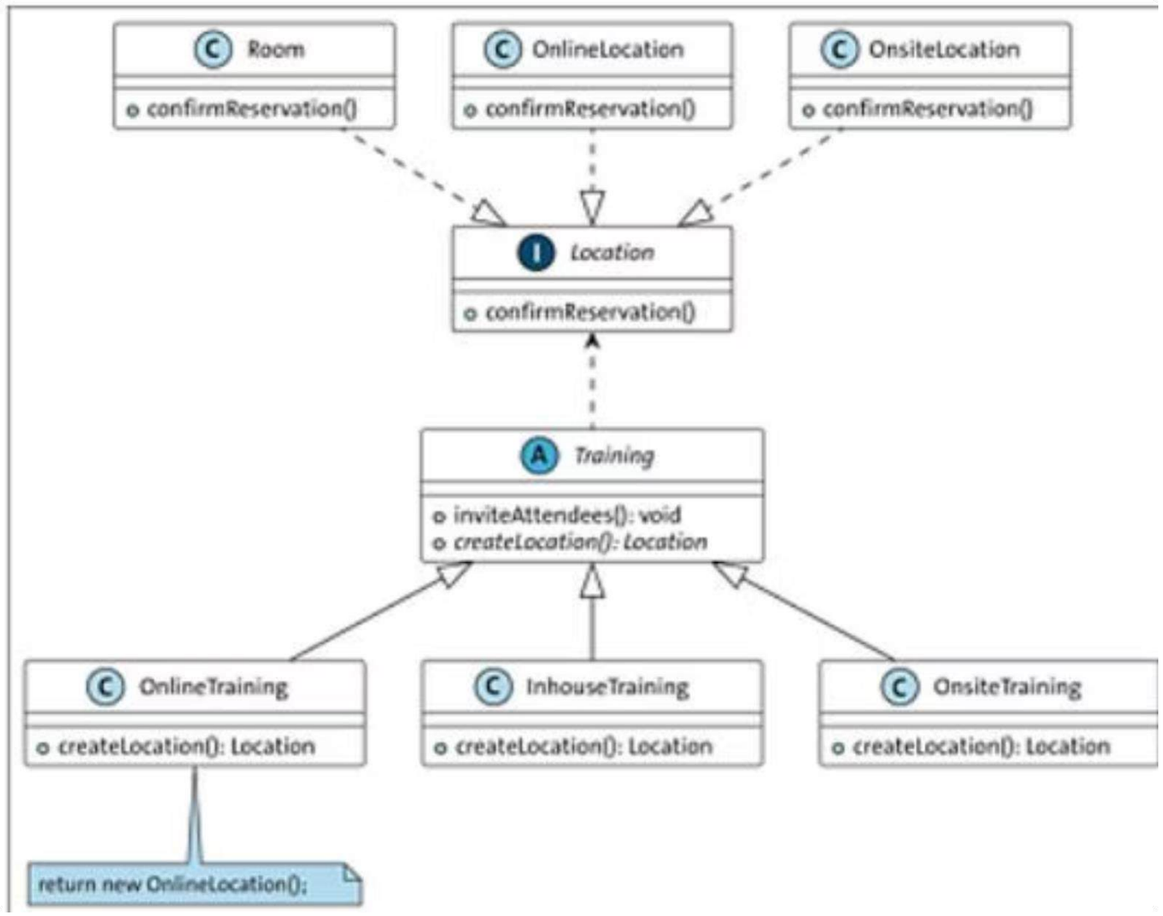
Factory

Abstrakte Klasse mit der Factory-Methode

ConcreteFactory

Erzeugt das konkrete Produkt

Factory Method – Schulungsbeispiel



Konkrete Fabriken

- **InhouseTraining** → erzeugt `Room`
- **OnsiteTraining** → erzeugt `OnsiteLocation`
- **OnlineTraining** → erzeugt `OnlineLocation`

Aufruf ohne If-Bedingungen

```
Training t = new OnlineTraining();
Location l = t.createLocation();
l.confirmReservation();
```

Factory Method – Wann einsetzen & Konsequenzen

Wann einsetzen?

- Exakte Typen der Objekte noch nicht bekannt
- Komplexe oder variable Erzeugungslogik
- Bibliotheken erweiterbar machen
- Caching / Pooling von Produkten

Konsequenzen

- ✅ Entkopplung von Erzeugung und Nutzung
- ✅ Erweiterbarkeit (OCP)
- ✅ Verbesserte Testbarkeit (Mock-Objekte)
- ⚠️ Zusätzlicher Code und Vererbungshierarchien

ⓘ **Real-World-Beispiel:** Apache Commons Compress – `CompressorStreamFactory.createCompressorOutputStream()` erzeugt z. B. einen `GzipCompressorOutputStream`.

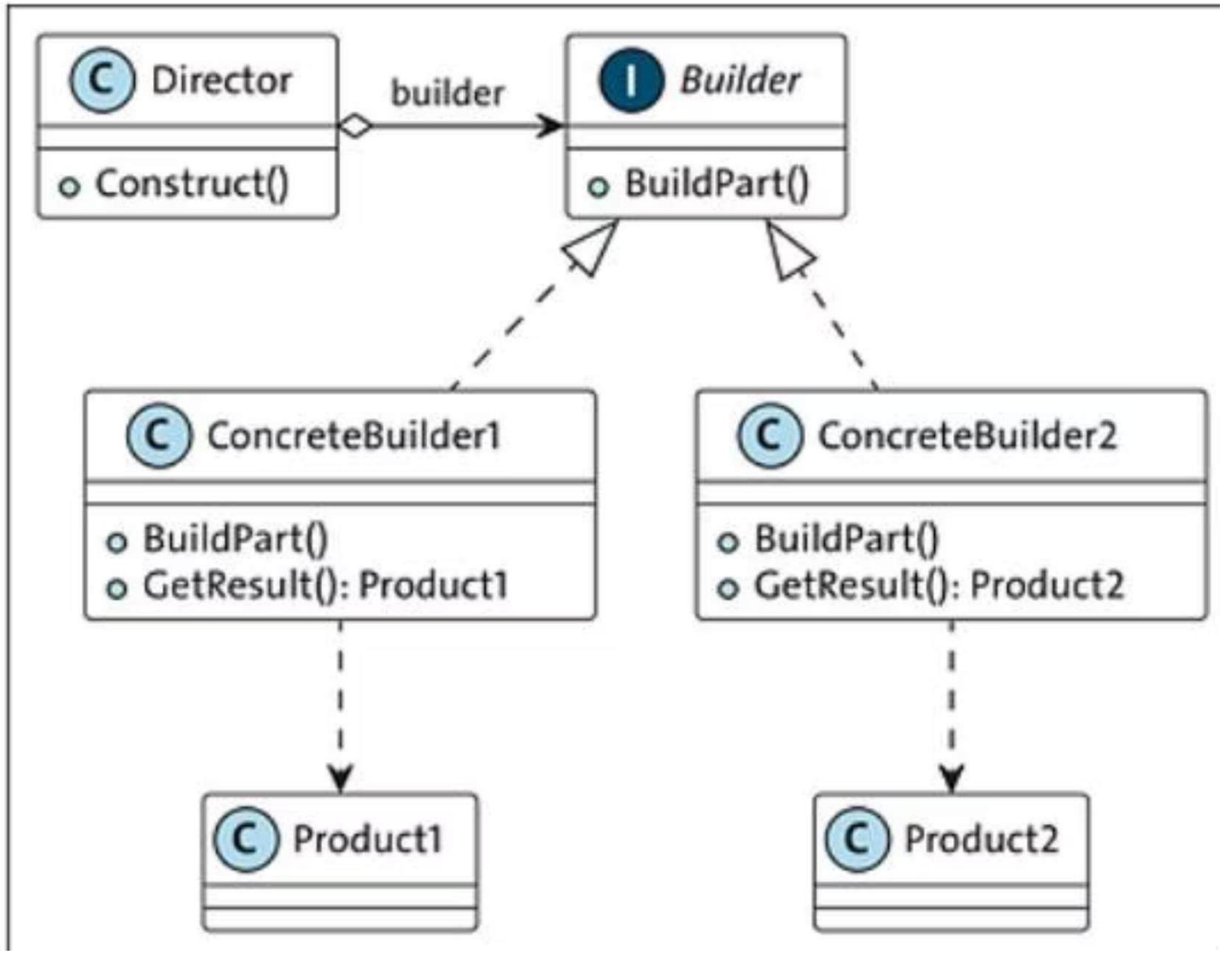
Builder / Erbauer

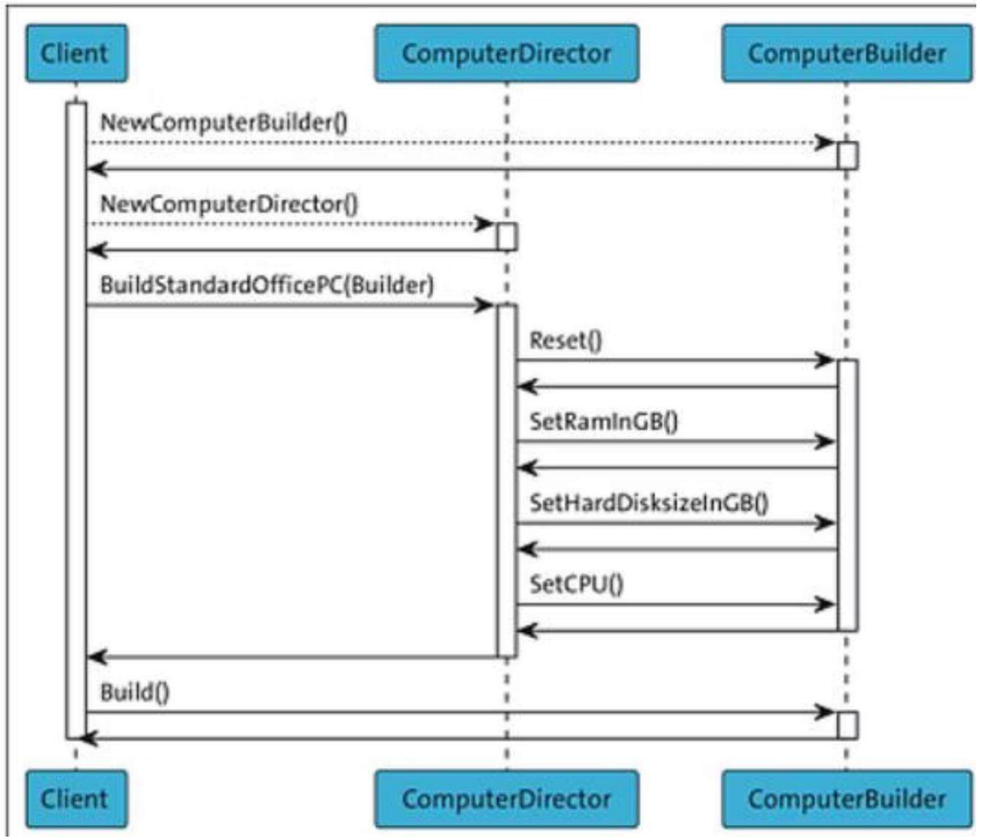
ERZEUGERMUSTER · GANG OF FOUR

Problem: Ein Computer-Hersteller konfiguriert PCs auf Kundenwunsch. Viele optionale Parameter führen zu unübersichtlichen Teleskop-Konstruktoren oder einer explodierenden Vererbungshierarchie.

1. **Director**
   Orchestriert den Bauprozess
2. **Builder**
   Abstrakte Schritte der Erzeugung
3. **ConcreteBuilder**
   Erzeugt das konkrete Produkt
4. **Product**
   Das fertige Objekt

Builder – Struktur & Sequenz





Builder – Wann einsetzen & Konsequenzen

Wann einsetzen?

- Komplexe Objekte mit vielen optionalen Parametern
- Unterschiedliche Repräsentationen desselben Bauprozesses (z. B. Computer + Handbuch)
- Director vereinfacht Schnittstelle für Clients

ⓘ **Fluent API:** Builder-Methoden geben `this` zurück → verkettete Aufrufe: `ship.setName("X").setPosition(...)`

Konsequenzen

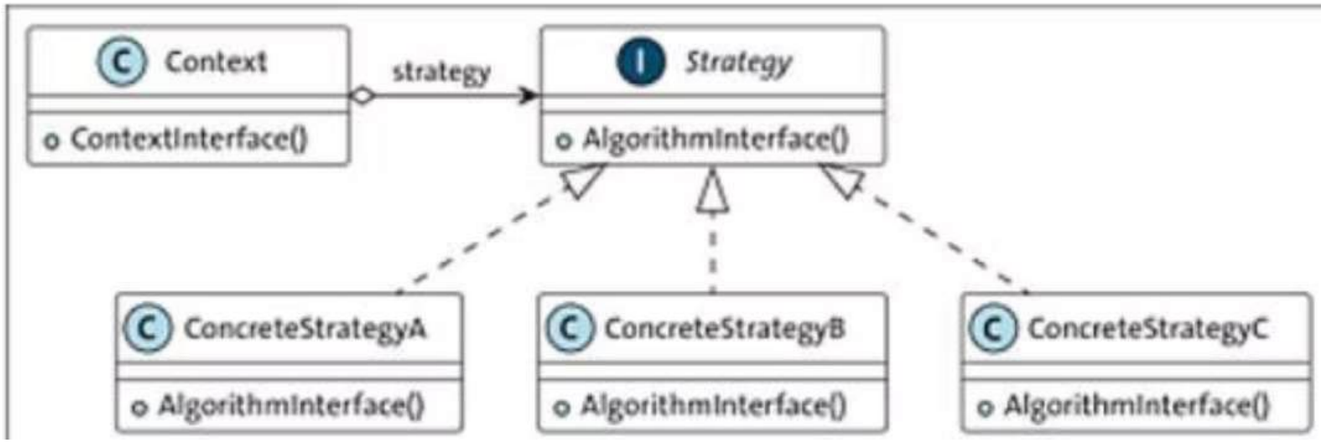
- ✅ Bessere Kontrolle, lesbarer Code
- ✅ Austauschbare Repräsentationen
- ✅ Single-Responsibility-Prinzip
- ⚠️ Zusätzliche Klassen und Methoden
- ⚠️ Mögliche Inkonsistenz bei unvollständiger Konfiguration

Real-World: Apache HttpComponents – `HttpClientBuilder` mit `HttpClients` als Director.

Strategy / Strategie

VERHALTENSMUSTER · GANG OF FOUR

Problem: Ein Webshop integriert immer mehr Bezahlvarianten (Kreditkarte, PayPal, Lastschrift) direkt im Warenkorb. Der Code wird durch viele If-Bedingungen komplex und fehleranfällig.



Strategy – Struktur & Einsatz

Teilnehmer

- **Strategy** – abstrakte Algorithmus-Schnittstelle
- **ConcreteStrategy** – konkrete Implementierung
- **Context** – hält Referenz zur Strategie; Auswahl durch den Client

```
let s = new CreditCardStrategy();
let cart = new ShoppingCart(s);
cart.AddItem(item);
cart.Pay();
```

Wann einsetzen?

- Mehrere Varianten eines Algorithmus
- Klassen unterscheiden sich nur in Teilverhalten
- Viele Code-Verzweigungen bei Verhaltensunterschieden

Konsequenzen

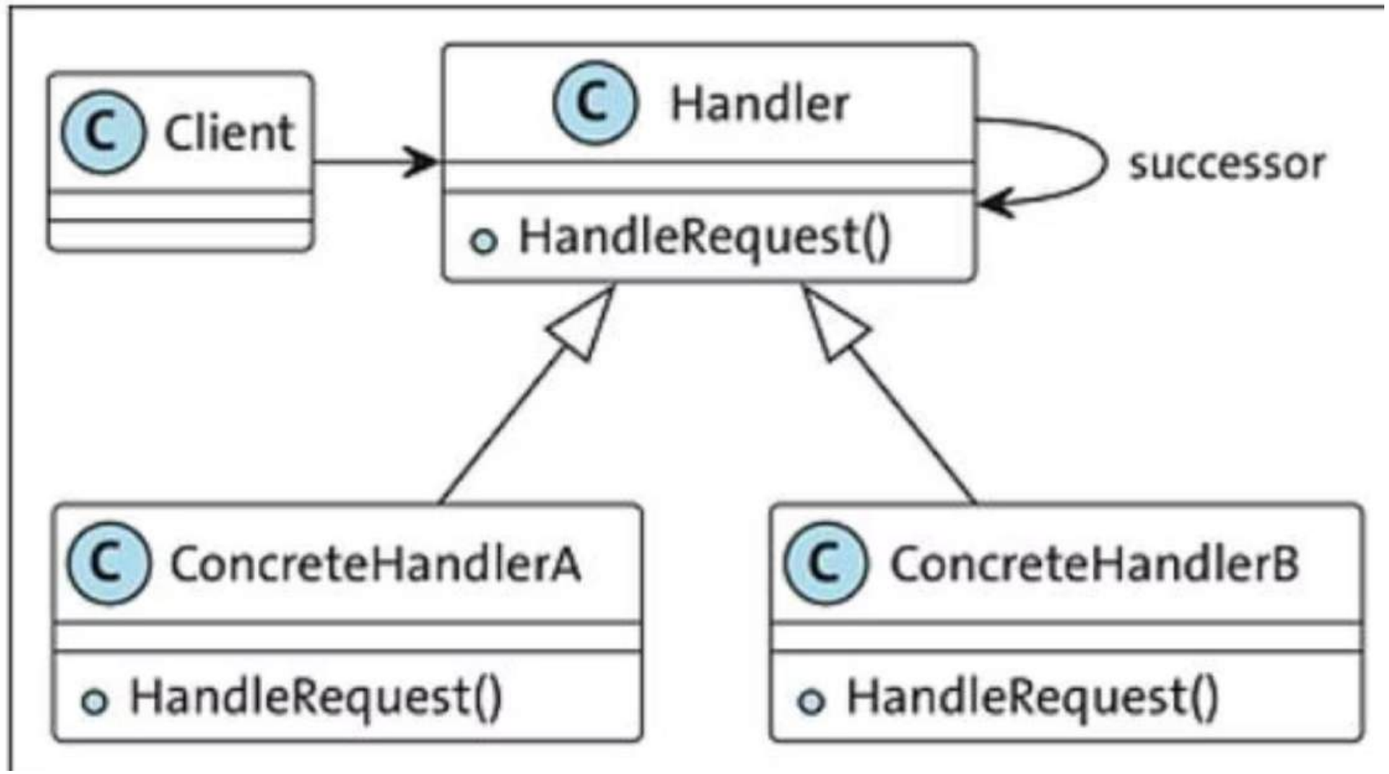
- ✅ Algorithmen austauschbar, auch zur Laufzeit
- ✅ Ersetzt komplexe Vererbungshierarchien
- ⚠️ Client muss Strategien kennen

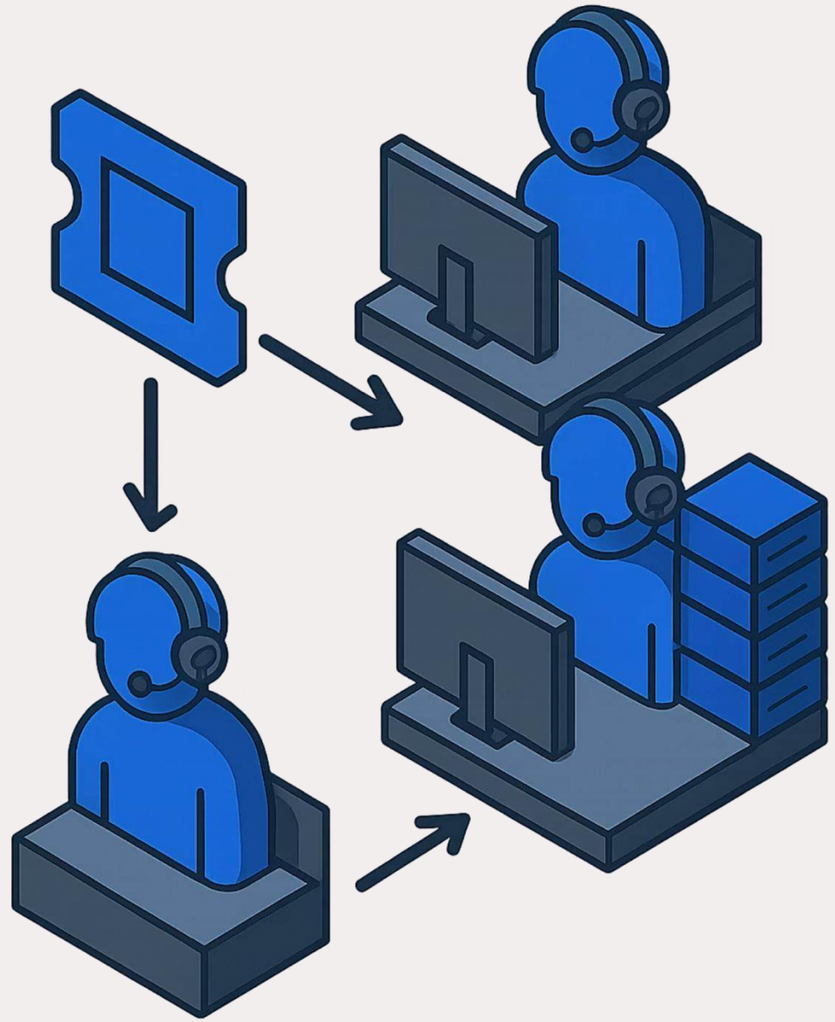
ⓘ **Real-World:** Java Collections – `Collections.sort(list, Comparator.reverseOrder())`

Chain of Responsibility / Zuständigkeitskette

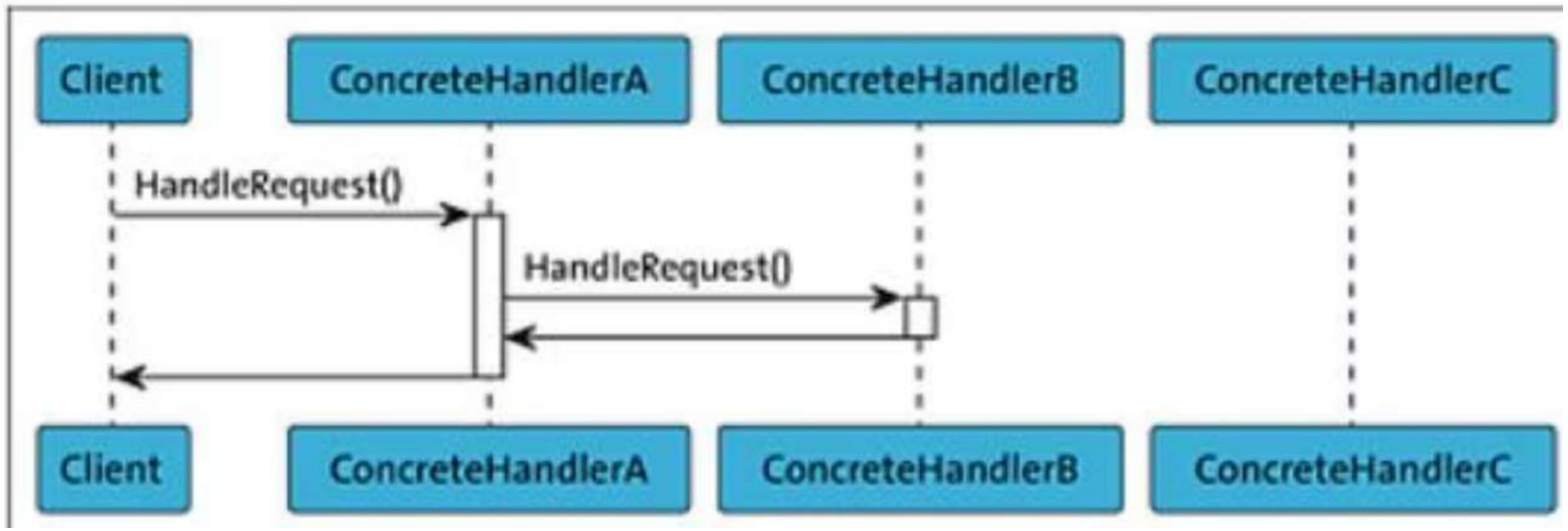
VERHALTENSMUSTER · GANG OF FOUR

Problem: Eine Support-App leitet Anfragen zunächst an Entwickler weiter. Mit wachsender Nutzerzahl kommen KI-Support und eine Support-Gruppe hinzu. Die gesamte Steuerungslogik landet im Webformular – komplex und schlecht wartbar.





Chain of Responsibility – Ablauf & Einsatz



Teilnehmer

- **Handler** – abstrakte Schnittstelle
- **ConcreteHandler** – verarbeitet oder gibt weiter (Successor)
- **Client** – kennt nur das erste Kettenglied

Konsequenzen

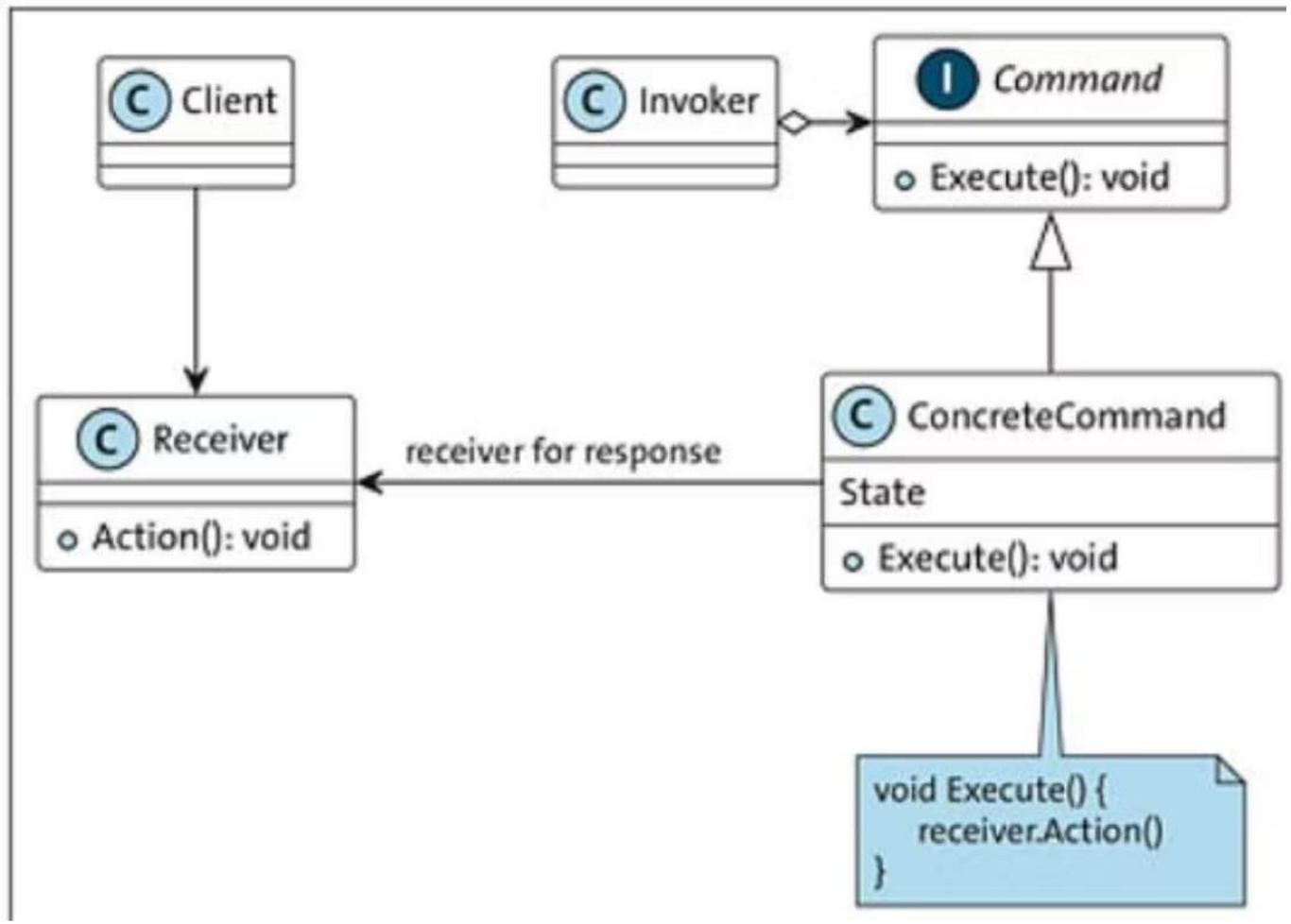
- ✅ Flexible Reihenfolge, dynamisch konfigurierbar
- ✅ Entkopplung Aufrufer / Ausführender
- ⚠️ Keine garantierte Ausführung

ⓘ **Real-World:** DOM Event Bubbling – Events wandern vom innersten zum äußersten Element; `stopPropagation()` bricht die Kette ab.

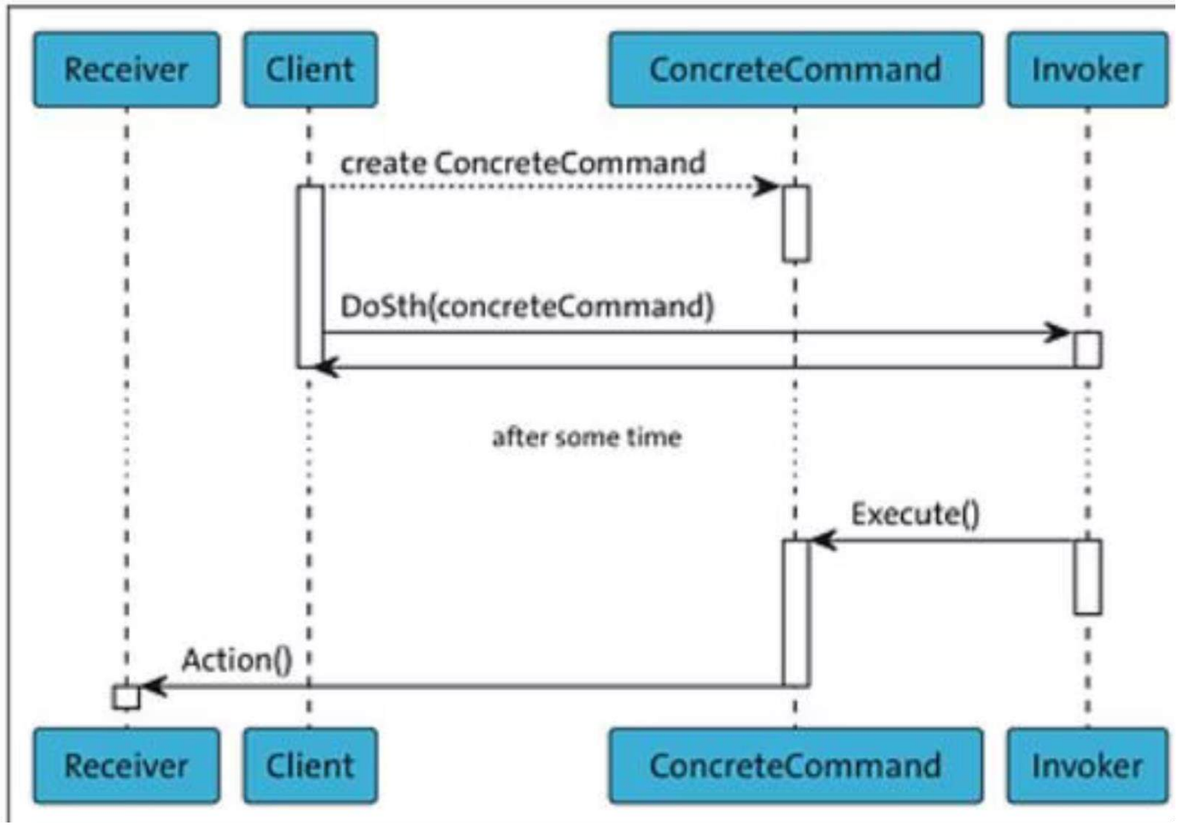
Command / Kommando

VERHALTENSMUSTER · GANG OF FOUR

Problem: WLAN-Schalter (Dash-Buttons) sollen Hausautomations-Aktionen auslösen. Jede Aktion als Subklasse führt zu einer wachsenden Hierarchie und vermischt Kommunikations- mit Aktionslogik.



Command – Sequenz & Einsatz



Teilnehmer

- **Command** – abstrakte Schnittstelle (`Execute()`)
- **ConcreteCommand** – kapselt Aktion am Receiver
- **Invoker** – startet Kommandos
- **Receiver** – führt tatsächliche Aktion aus

Konsequenzen & Einsatz

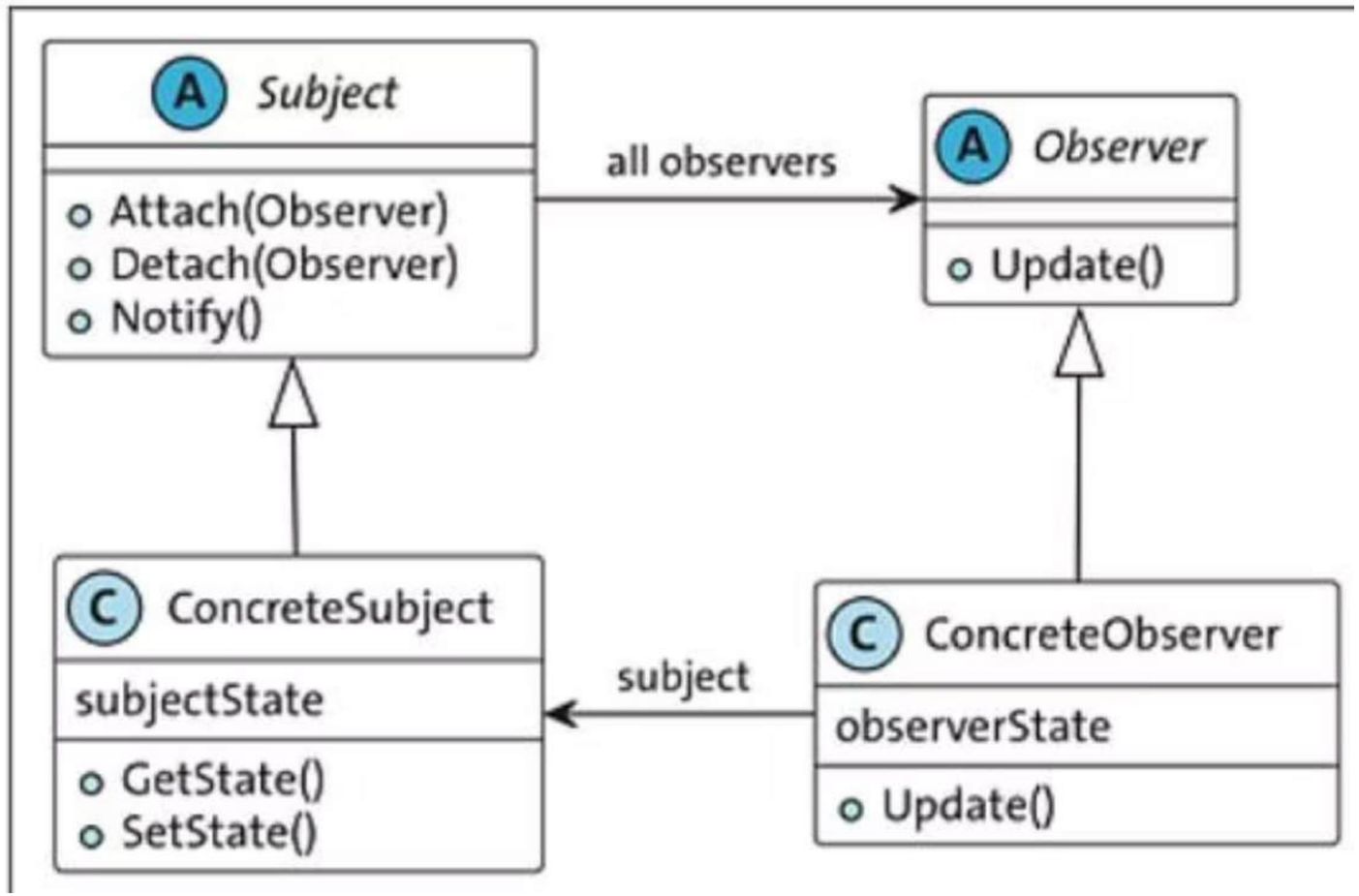
- ✅ Entkopplung Definition / Ausführung
- ✅ Kommandos als First-Class Objects
- ✅ Undo/Redo, Protokollierung, Queues
- ✅ Makrokommandos durch Kombination

ⓘ **Real-World:** Java `java.util.concurrent` – `Executor` (Invoker) + `Runnable` (Command)

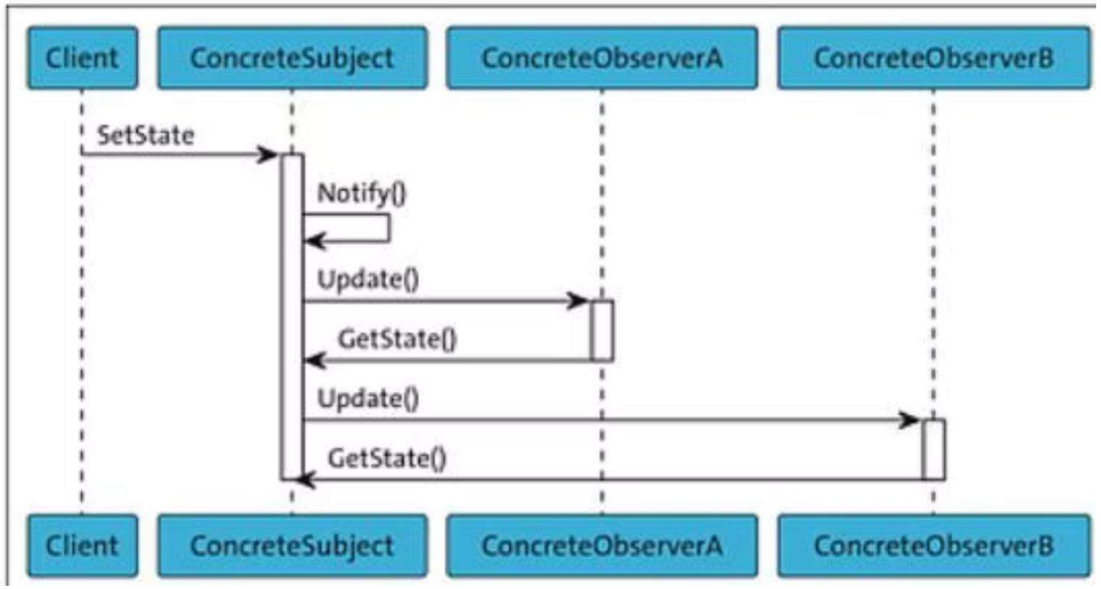
Observer / Beobachter

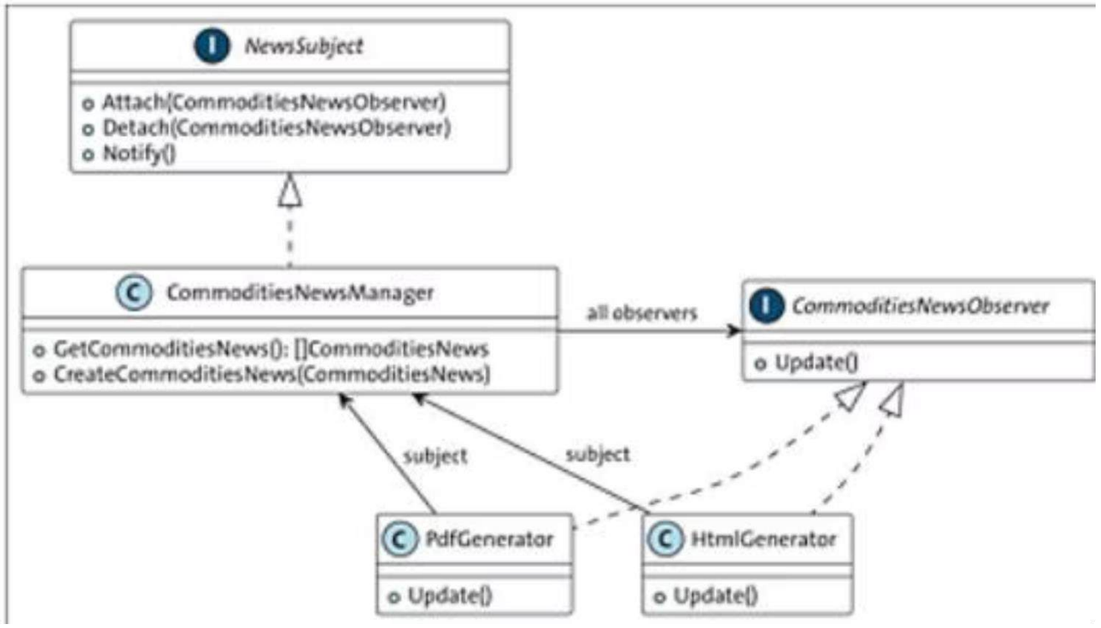
VERHALTENSMUSTER · GANG OF FOUR

Problem: Ein Rohstoff-Nachrichtenportal generiert PDF und EPUB in unterschiedlichen Intervallen. Dokumente enthalten dadurch unterschiedliche Inhalte – Kunden beschweren sich über veraltete Informationen.



Observer – Sequenz & Schulungsbeispiel





Observer – Wann einsetzen & Konsequenzen

Wann einsetzen?

- Zustandsänderungen an mehrere Objekte verteilen
- Push statt Polling (Empfänger registrieren sich)
- Empfänger zu Beginn unbekannt oder variabel

Konsequenzen

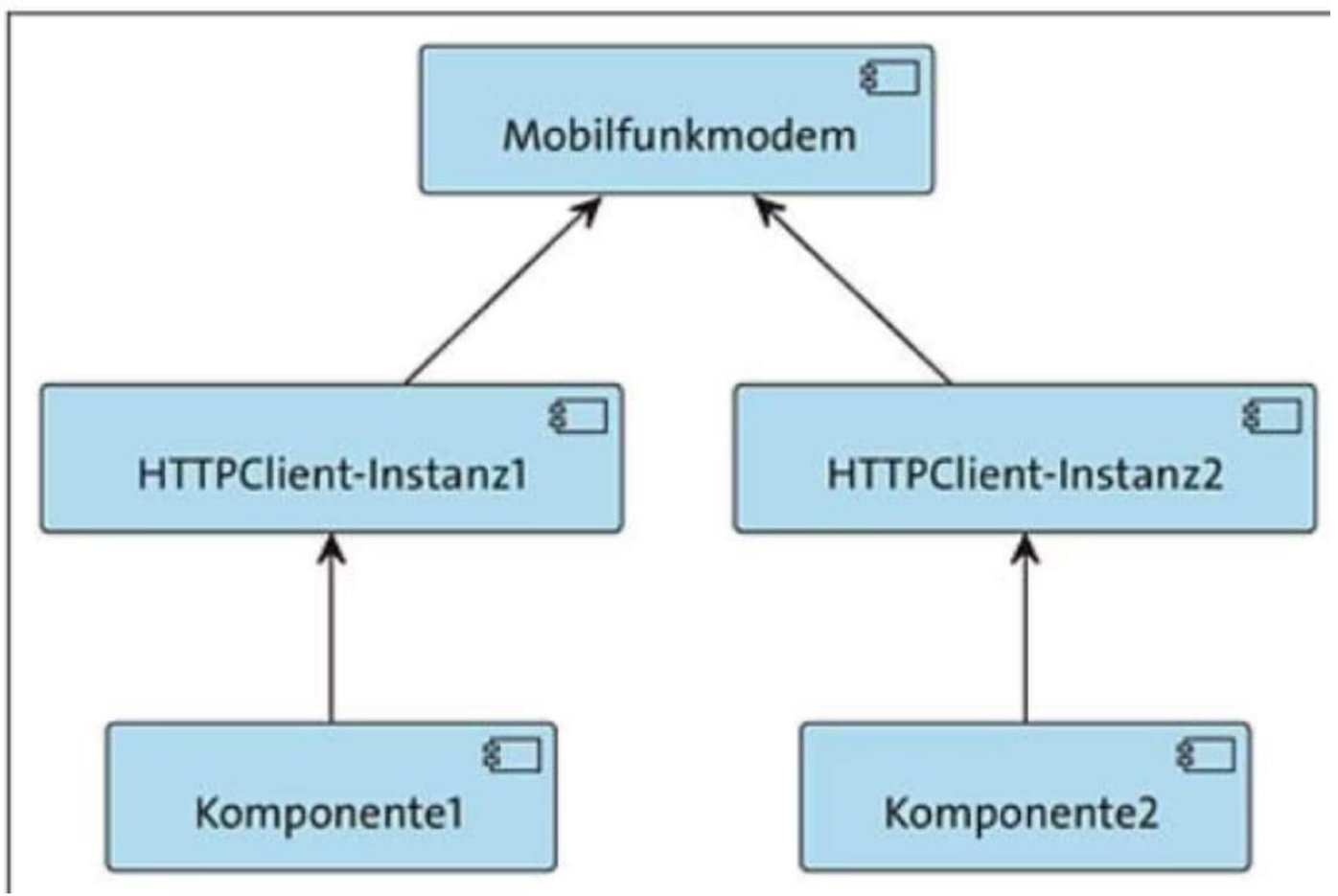
- ✅ Entkopplung Sender / Empfänger
- ✅ Variable Empfängerliste (OCP)
- ⚠️ Keine definierte Aufruf-Reihenfolge
- ⚠️ Ungewollte Kaskaden / Endlosschleifen möglich
- ⚠️ **Lapsed-Listener-Problem**: Observer müssen abgemeldet werden, sonst Memory-Leak!

ⓘ **Real-World:** Spring Framework – `ApplicationListener` empfängt Events des `ApplicationContext` (Start, Stop, Close).

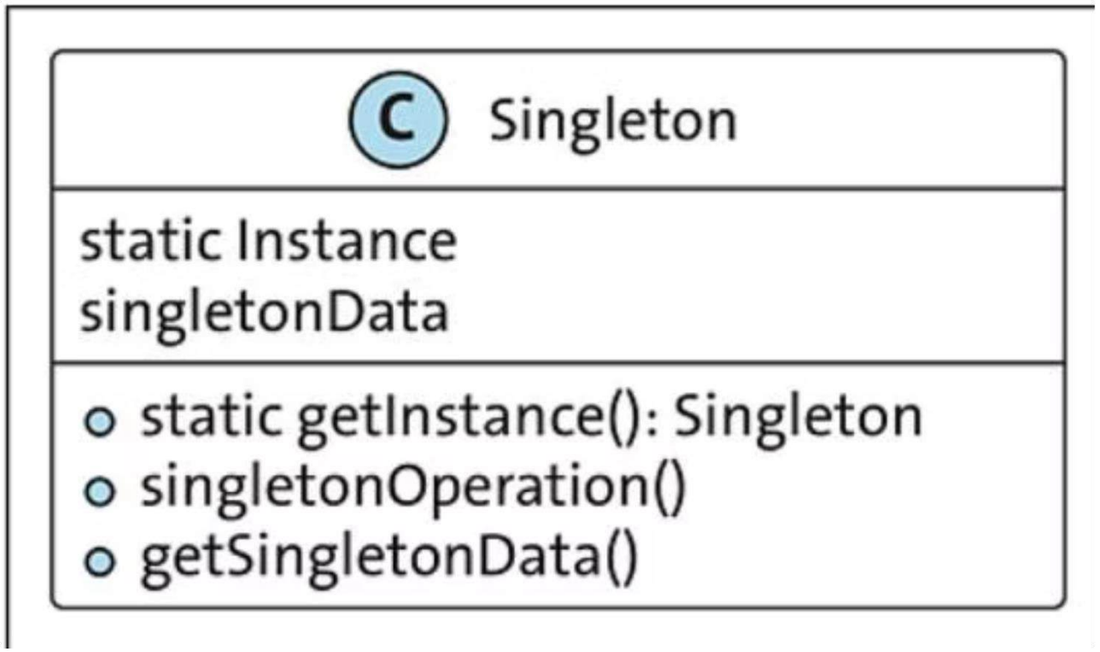
Singleton / Einzelstück

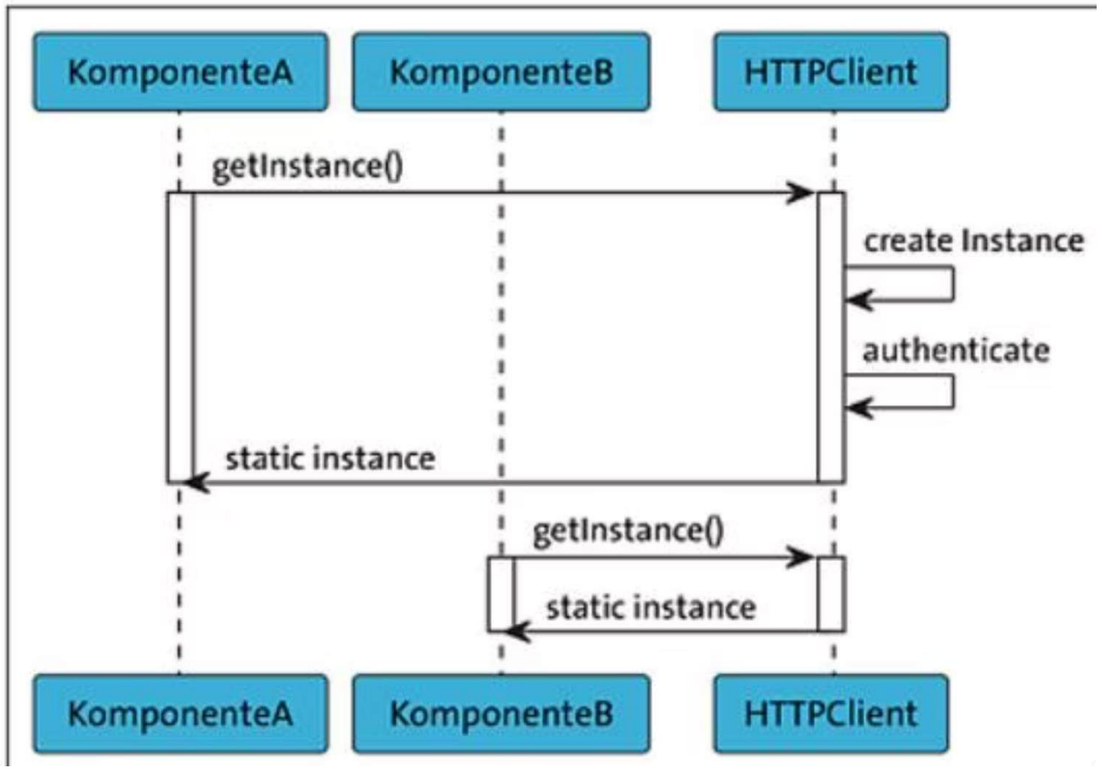
ERZEUGERMUSTER · GANG OF FOUR

Problem: Mehrere Softwarekomponenten auf einem IoT-Gerät erzeugen je eine eigene `HTTPClient`-Instanz. Gleichzeitige Zugriffe auf das Mobilfunkmodem führen zu Fehlern und erhöhten Übertragungskosten.



Singleton – Struktur & Sequenz





Singleton – Wann einsetzen & Konsequenzen

Wann einsetzen?

- Nur eine Instanz eines Typs erlaubt (z. B. DB-Verbindung, Modem)
- Erzeugung von Objekten beschränken
- Abgeleitete Singleton-Klassen einheitlich erzeugen

Lazy Init vs. direkte Erzeugung

```
// Direkt (thread-safe):
static HttpClient instance = new HttpClient();
```

Konsequenzen

- ✅ Kontrollierter Zugriff auf eine Instanz
- ✅ Verzögerte Erzeugung möglich
- ⚠️ **Multithreading:** Synchronisierung nötig (`synchronized` / `sync.Once`)
- ⚠️ Erschwerte Testbarkeit (Mock-Ersatz schwierig)

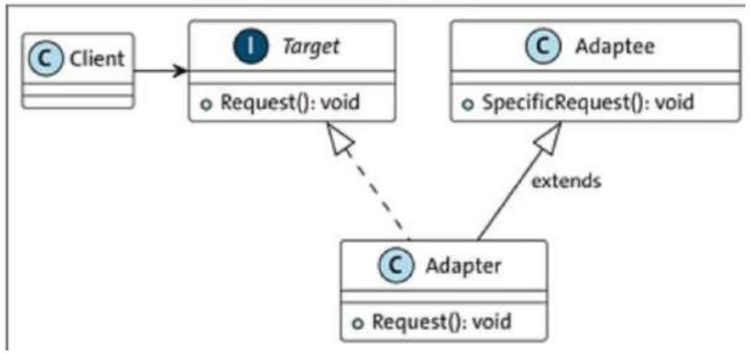
ℹ️ **Real-World:** `java.lang.Runtime.getRuntime()` – klassisches Singleton im JDK

Adapter / Wrapper

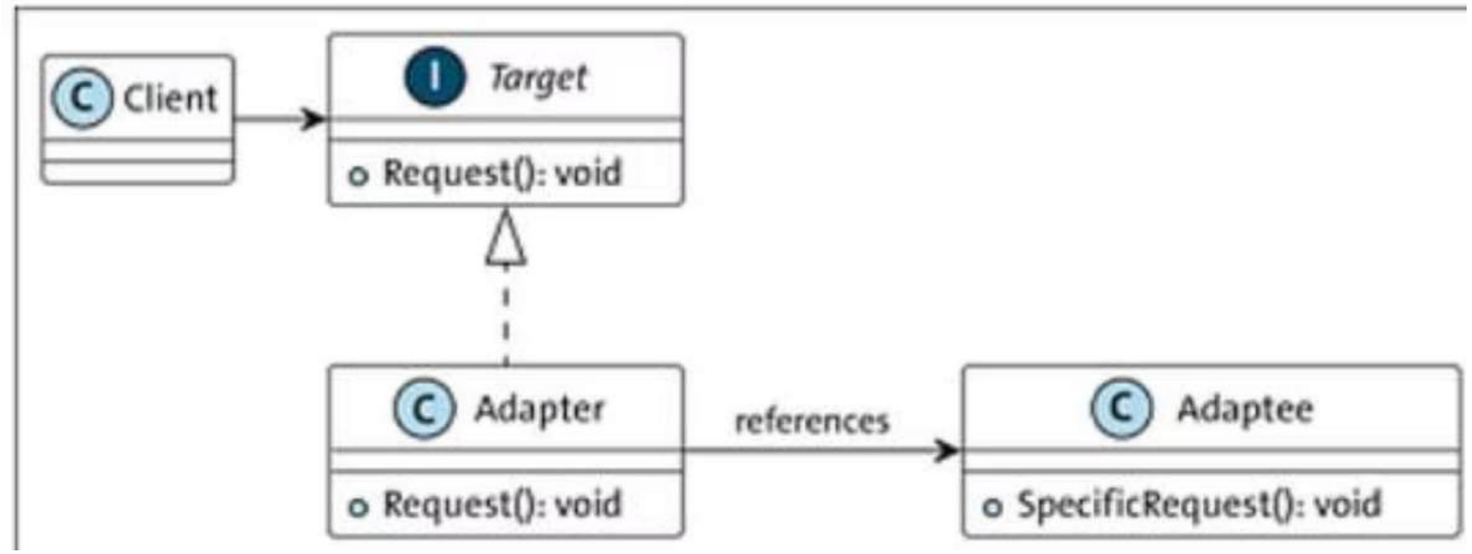
STRUKTURMUSTER · GANG OF FOUR

Problem: Ein Versicherungsportal zeigt Lebensversicherungen aus einer Datenbank. Nun sollen auch Sachversicherungen aus einem externen System angezeigt werden – dessen HTTP-Schnittstelle ist jedoch inkompatibel.

Class-Adapter

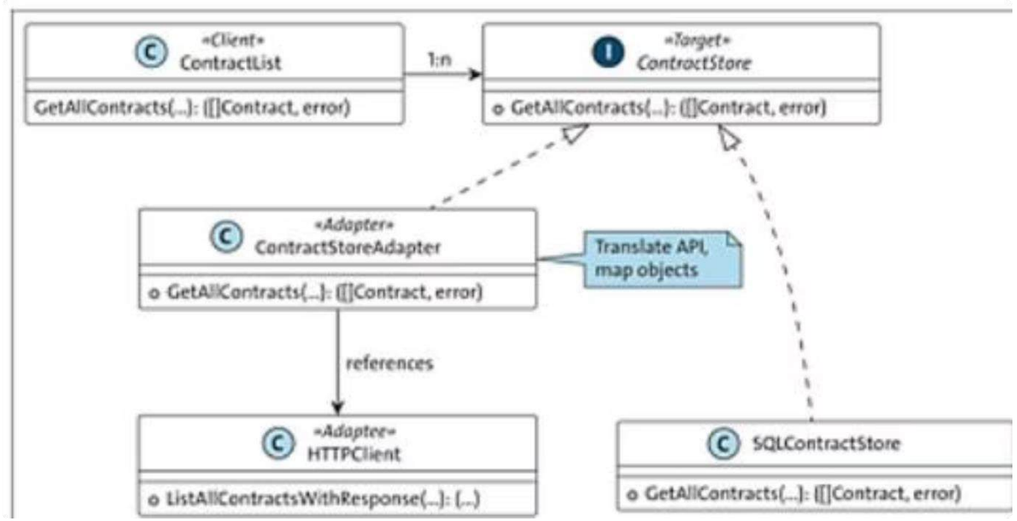


Object-Adapter



Adapter – Lösungsbeispiel & Einsatz

Real-World: JAXB `XmlAdapter` – übersetzt `HashMap` in eine kompakte XML-Struktur beim Serialisieren.



Teilnehmer

- **Target** – vom Client verwendete Schnittstelle
- **Adapter** – übersetzt Target-Aufrufe in Adaptee-Aufrufe
- **Adaptee** – inkompatibles Zielobjekt
- **Client** – kennt nur das Target-Interface

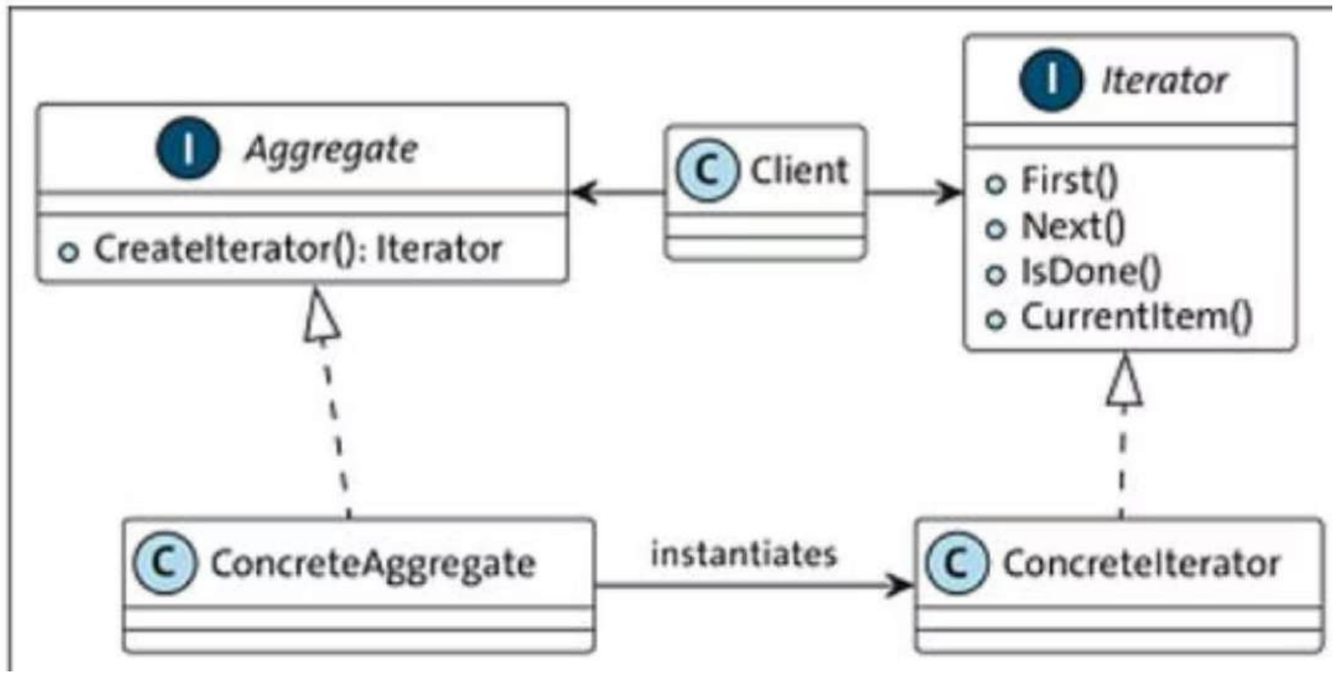
Konsequenzen

- ✅ Transformation von Geschäftslogik getrennt
- ✅ Erweiterung durch neue Adapter (OCP)
- ⚠️ Komplexität nimmt zu

Iterator

VERHALTENSMUSTER · GANG OF FOUR

Problem: Eine Mailing-Software iteriert über Kundenadressen aus einer Datenbank. Nun sollen Filterfunktionen und eine zweite Abteilung mit Daten aus einer Textdatei unterstützt werden – ohne die Versandlogik zu ändern.



Iterator – Einsatz & Konsequenzen

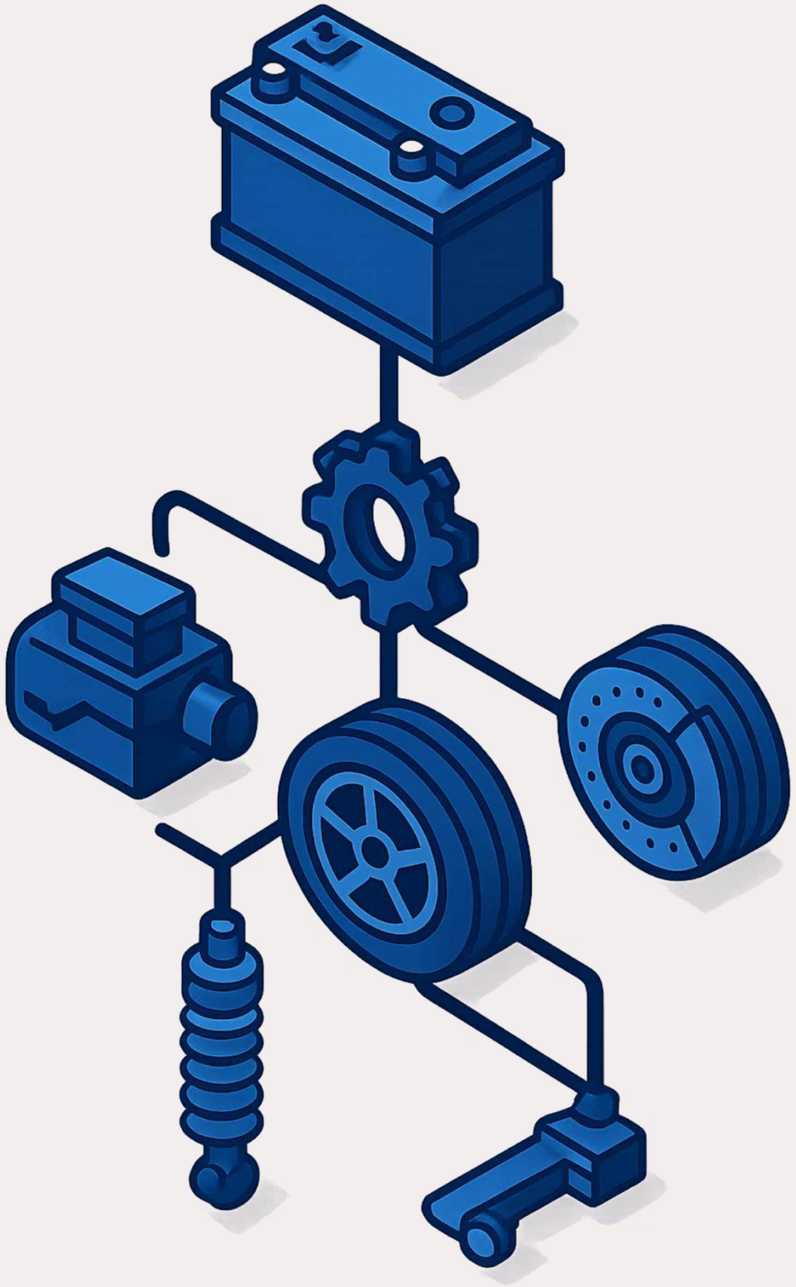
Teilnehmer

- **Iterator** – abstrakte Traversierungs-Schnittstelle (`First`, `Next`, `IsDone`)
- **ConcreteIterator** – hält Position und Filterlogik
- **Aggregate** – erzeugt Iterator-Instanzen
- **ConcreteAggregate** – liefert vorkonfigurierten Iterator

Konsequenzen

- ✅ Einheitlicher Zugriff unabhängig von Datenstruktur
- ✅ Traversierungslogik zentral gekapselt
- ✅ Mehrere Iteratoren gleichzeitig möglich
- ⚠️ Overhead bei einfachen Collections

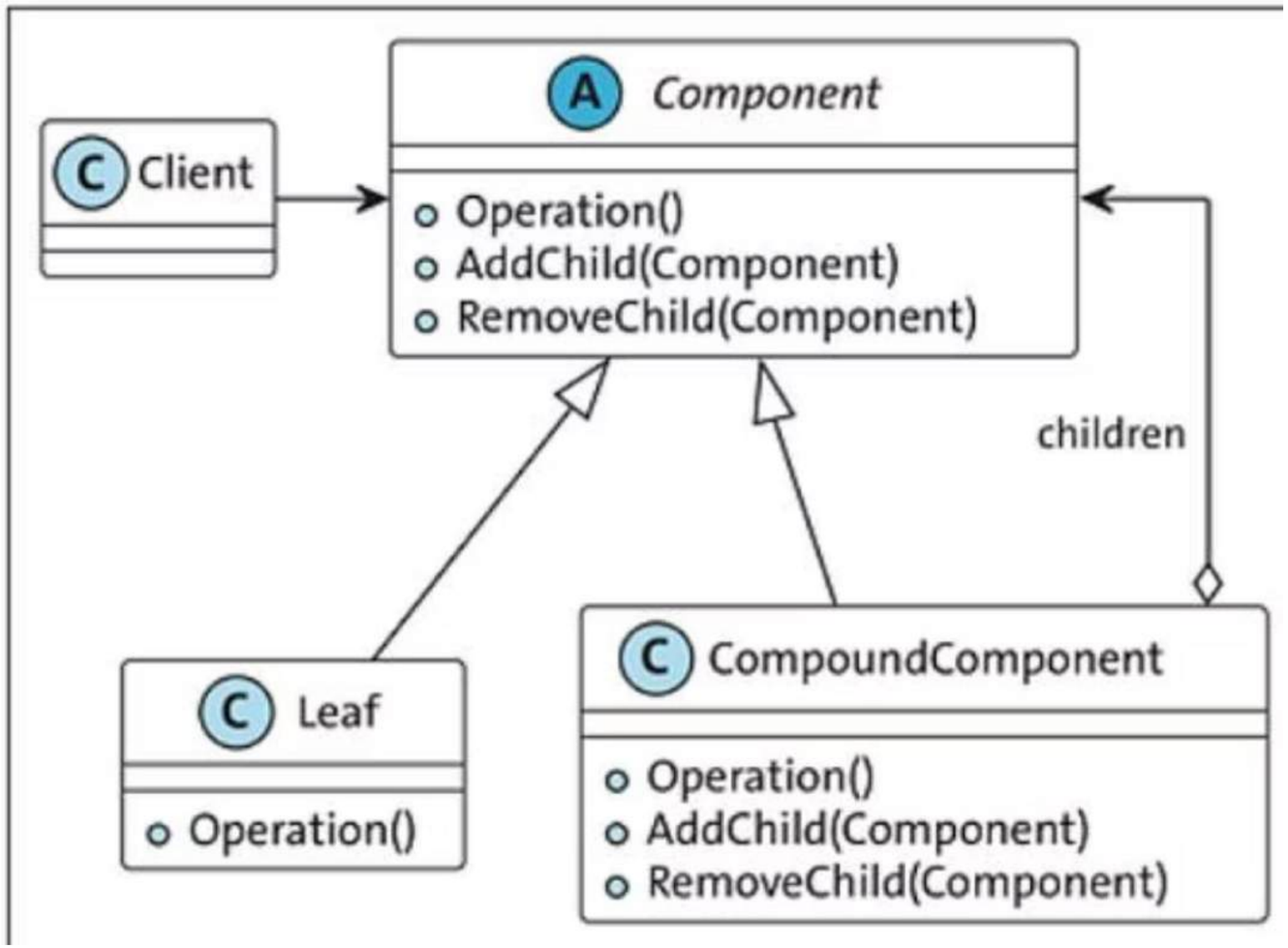
> ℹ️ **Real-World:** Java `java.util.Iterator` – `ArrayList`, `LinkedList` etc. implementieren einheitlich `hasNext()` / `next()`.



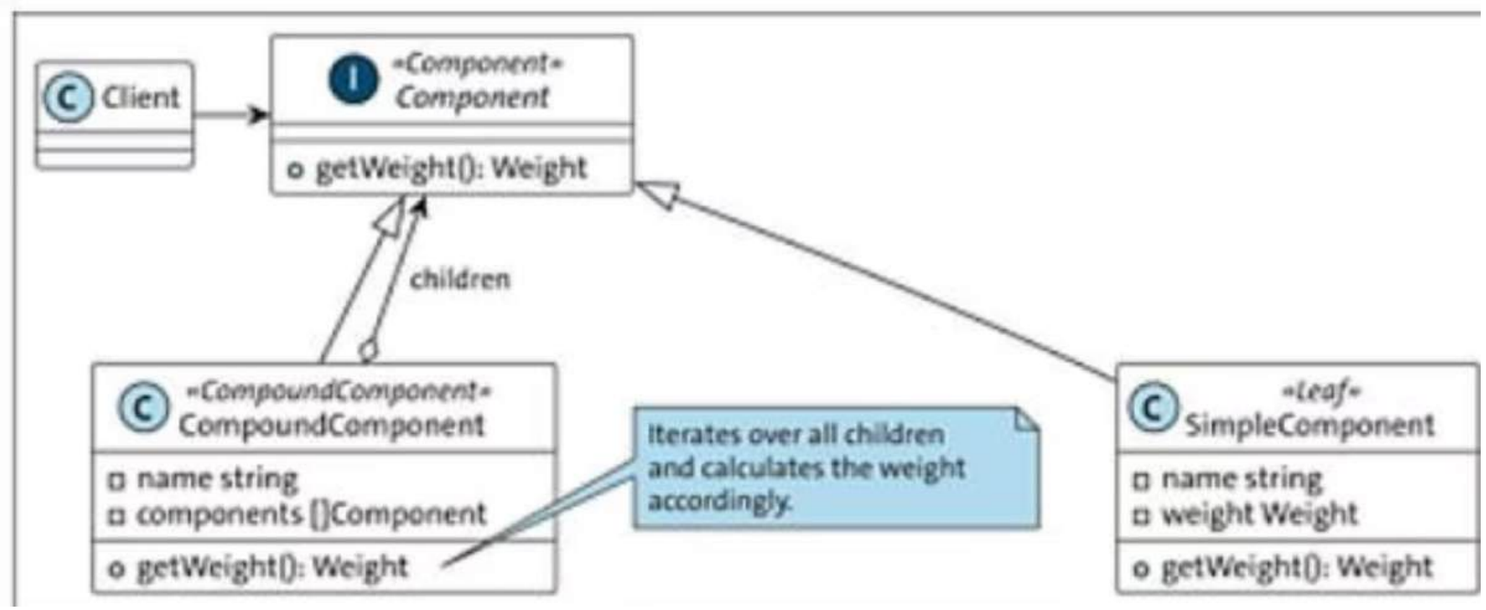
Composite / Kompositum

STRUKTURMUSTER · GANG OF FOUR

Problem: Ein Fahrzeughersteller berechnet das Gewicht eines E-Autos. Baugruppen bestehen aus Einzelteilen oder weiteren Baugruppen. Eine flache Liste verliert die Hierarchieinformation.



Composite – Lösungsbeispiel & Einsatz



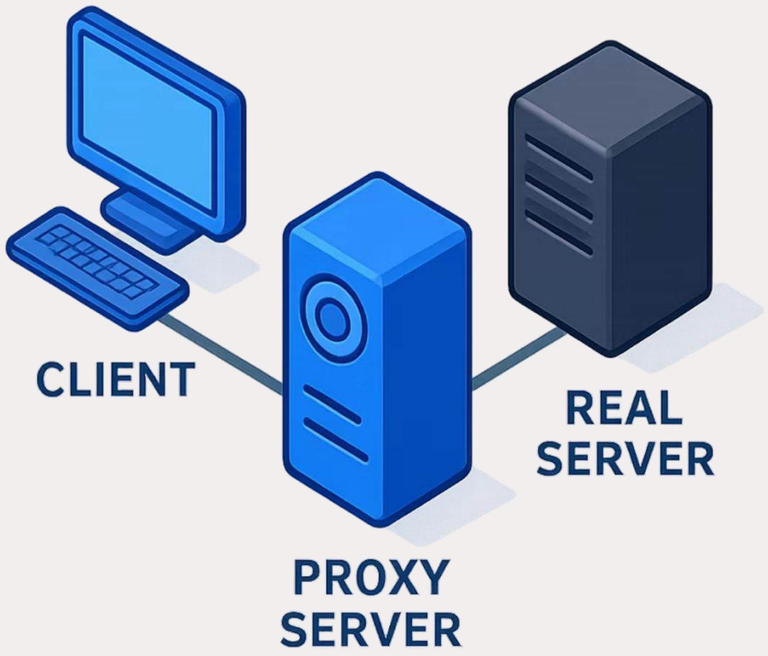
Teilnehmer

- **Component** – einheitliche Schnittstelle für alle Baumelemente
- **Leaf** – Blatt ohne Kinder (enthält eigentliche Logik)
- **Composite** – Container, delegiert an Kinder
- **Client** – arbeitet nur mit Component-Interface

Konsequenzen

- ✅ Einheitlicher Zugriff auf Baum- und Blattknoten
- ✅ Neue Komponenten einfach ergänzbar (OCP)
- ⚠️ Sehr generalisiertes Design – Casts nötig

ⓘ **Real-World:** Go `http.ServeMux` – implementiert selbst `Handler` und kann weitere `Handler` enthalten.



Proxy / Stellvertreter

STRUKTURMUSTER · GANG OF FOUR

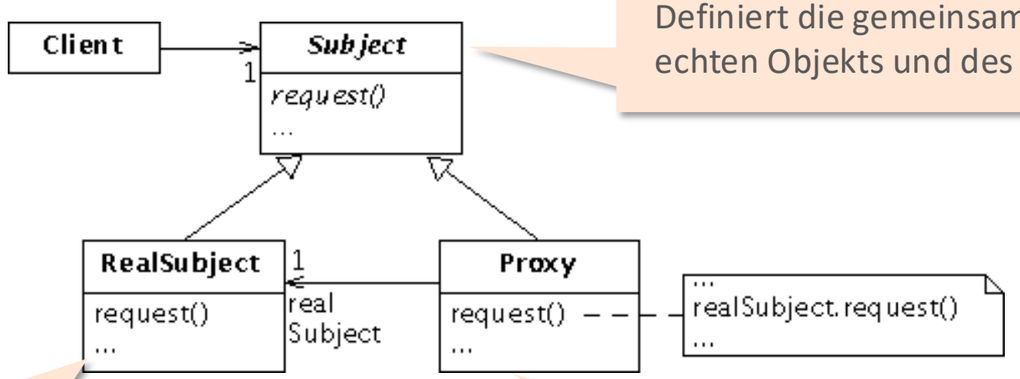
Problem: Ein Client greift direkt auf ein ressourcenintensives Objekt zu (z. B. hochauflösende Bilder, Remote-Dienste). Das führt zu unnötigen Ladezeiten, fehlender Zugriffskontrolle und mangelnder Sicherheit.

Lösung: Ein Proxy-Objekt steht stellvertretend für das echte Objekt. Es kontrolliert den Zugriff, kann Operationen verzögern (Lazy Loading), cachen oder absichern.

Varianten

- **Virtual Proxy:** Verzögertes Laden ressourcenintensiver Objekte.
- **Remote Proxy:** Kapselt den Netzwerkzugriff auf Remote-Objekte.
- **Protection Proxy:** Prüft Zugriffsrechte vor der Ausführung.
- **Cache Proxy:** Speichert Ergebnisse zwischen, um Wiederholungsaufrufe zu optimieren.

Proxy – Struktur



Definiert die gemeinsame Schnittstelle des echten Objekts und des Proxy-Objekts

Definiert das echte Objekt.

Kontrolliert Zugriff auf das eigentliche Objekt und ist dafür zuständig, es zu erzeugen und zu löschen.

Proxy – Wann einsetzen & Konsequenzen

Wann einsetzen?

- Ressourcenintensive Objekte verzögert laden (Virtual Proxy)
- Zugriff auf Remote-Objekte kapseln (Remote Proxy)
- Zugriffsrechte und Sicherheit durchsetzen (Protection Proxy)
- Ergebnisse zwischenspeichern (Cache Proxy)
- Logging, Monitoring oder Transaktionssteuerung ergänzen

Konsequenzen

- ✅ Transparenz: Client merkt keinen Unterschied zum echten Objekt
- ✅ Open/Closed Principle: Neues Verhalten ohne Änderung am RealSubject
- ✅ Lazy Loading reduziert Startzeit und Speicherverbrauch
- ⚠️ Zusätzliche Indirektionsebene kann Latenz erhöhen
- ⚠️ Viele Proxy-Typen können Klassenanzahl stark erhöhen

Abgrenzung

- **Decorator vs. Proxy:** Decorator erweitert Verhalten, Proxy kontrolliert Zugriff
- **Adapter vs. Proxy:** Adapter ändert die Schnittstelle, Proxy behält sie bei

Facade / Fassade

STRUKTURMUSTER · GANG OF FOUR

Problem: Ein Client muss mit einem komplexen Subsystem aus vielen Klassen interagieren (z. B. Compiler-Pipeline, Heimkino-Anlage). Die direkte Nutzung erzeugt starke Kopplung und erfordert tiefes Wissen über interne Abläufe.

Lösung: Eine Fassade bietet eine vereinfachte, einheitliche Schnittstelle zum Subsystem. Der Client kommuniziert nur noch mit der Fassade – die interne Komplexität bleibt verborgen.

- ### Vereinfacht

  Ermöglicht die einfache Nutzung komplexer Subsysteme.

- ### Entkoppelt

  Reduziert die Kopplung zwischen Client und Subsystem (Low Coupling).

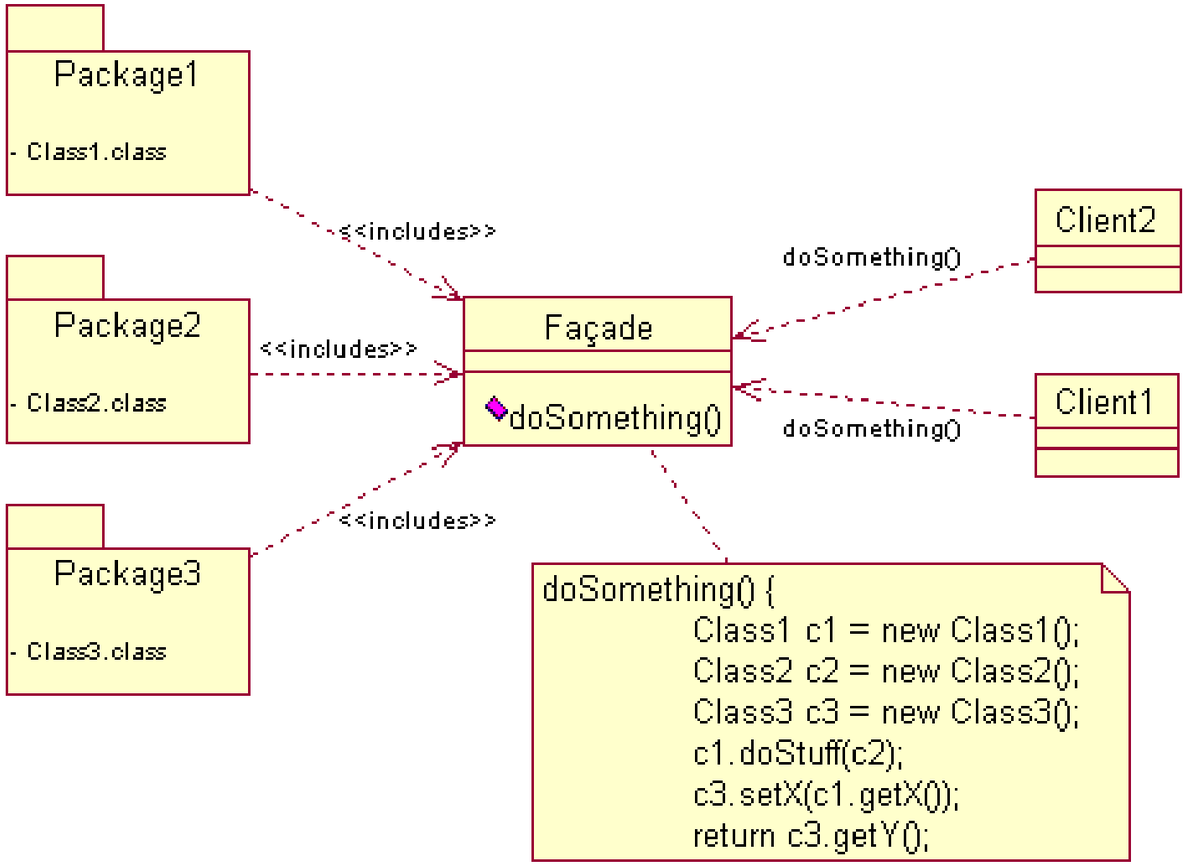
- ### Flexibel

  Das Subsystem bleibt weiterhin direkt nutzbar (keine Einschränkung).

- ### Architektur

  Dient als Einstiegspunkt für Schichten-Architekturen.

Facade – Struktur



Facade – Wann einsetzen & Konsequenzen

Wann einsetzen?

- Komplexe Subsysteme mit vielen Abhängigkeiten vereinfachen
- Schichten-Architektur mit klaren Einstiegspunkten definieren
- Abhängigkeiten zwischen Subsystemen und Clients minimieren
- Legacy-Systeme hinter einer sauberen Schnittstelle kapseln
- Testbarkeit durch Abstraktion des Subsystems verbessern

Konsequenzen

- ✅ Reduziert Kopplung zwischen Client und Subsystem
- ✅ Fördert schwache Abhängigkeiten (Law of Demeter)
- ✅ Vereinfacht Portierung und Austausch von Subsystemen
- ⚠️ Fassade kann zum "God Object" werden, wenn zu viel Logik aufgenommen wird
- ⚠️ Subsystem-Funktionalität kann durch die Fassade verborgen bleiben

Abgrenzung

- **Adapter vs. Facade:** Adapter passt eine Schnittstelle an, Facade vereinfacht mehrere
- **Mediator vs. Facade:** Mediator koordiniert Kommunikation zwischen Objekten, Facade nur nach außen
- **Abstract Factory vs. Facade:** Abstract Factory erzeugt Objekte, Facade kapselt Subsystem-Nutzung

Anti-Patterns

SCHLECHTE PRAXIS ERKENNEN & VERMEIDEN

Anti-Patterns sind wiederkehrende schlechte Lösungsansätze, die kurzfristig funktionieren, aber langfristig zu Problemen führen. Sie zu kennen ist genauso wichtig wie das Wissen über gute Patterns.

Code-Ebene

Schlechte Praktiken im Quellcode einzelner Klassen und Methoden

Architektur-Ebene

Fehlerhafte Strukturentscheidungen auf System- oder Komponentenebene

Prozess-Ebene

Schlechte Praktiken im Entwicklungsprozess und Team

Anti-Patterns – Häufige Beispiele

God Object / Blob

Eine Klasse übernimmt zu viele Verantwortlichkeiten. Verletzt das Single-Responsibility-Prinzip. Schwer testbar und wartbar.

Spaghetti Code

Unstrukturierter Code ohne klare Trennung von Verantwortlichkeiten. Entsteht durch fehlende Planung und wachsende Komplexität ohne Refactoring.

Golden Hammer

Ein bekanntes Werkzeug oder Pattern wird auf jedes Problem angewendet, auch wenn es nicht passt. *„Wenn alles ein Nagel ist, benutzt man immer den Hammer."*

Copy-Paste-Programmierung

Code wird dupliziert statt abstrahiert. Führt zu inkonsistenten Änderungen und schwer wartbarem Code (verletzt DRY – Don't Repeat Yourself).

Premature Optimization

Optimierungen werden zu früh vorgenommen, bevor ein Problem tatsächlich existiert. Erhöht Komplexität ohne messbaren Nutzen.

Magic Numbers / Strings

Hartcodierte Werte ohne erklärenden Kontext. Erschweren Verständnis und Wartung erheblich.

Anti-Patterns – Architektur & Prozess

Architektur-Anti-Patterns

Big Ball of Mud

Keine erkennbare Architektur – alles hängt mit allem zusammen.

Stovepipe System

Isolierte Systeme ohne Wiederverwendung gemeinsamer Komponenten.

Vendor Lock-In

Starke Abhängigkeit von einem Anbieter verhindert Flexibilität.

Prozess-Anti-Patterns

Analysis Paralysis

Endlose Analyse ohne Entscheidung – kein Fortschritt.

Death March

Unrealistische Deadlines und Ressourcen – Team arbeitet bis zum Zusammenbruch.

Cargo Cult Programming

Patterns oder Technologien werden kopiert, ohne deren Zweck zu verstehen.

⚠ Anti-Patterns entstehen oft nicht durch Unwissenheit, sondern durch Zeitdruck, fehlende Kommunikation oder mangelndes Refactoring. Regelmäßige Code-Reviews helfen, sie frühzeitig zu erkennen.

Zusammenfassung

Design-Patterns sind kein Selbstzweck – immer fragen: **Löst das Pattern mein konkretes Problem?** Zu viele Patterns erhöhen unnötig die Komplexität.

Erzeugermuster

Factory Method – flexible Objekterzeugung · **Builder** – schrittweise Konstruktion · **Singleton** – eine Instanz

Strukturmuster

Adapter – inkompatible Schnittstellen überbrücken · **Composite** – Baumstrukturen einheitlich behandeln · **Proxy** – Zugriff kontrollieren und kapseln · **Facade** – Subsysteme vereinfachen

Verhaltensmuster

Strategy – austauschbare Algorithmen · **Chain of Responsibility** – Verarbeitungskette · **Command** – Aktionen kapseln · **Observer** – Zustandsänderungen verteilen · **Iterator** – einheitliches Traversieren

Anti-Patterns

God Object, Spaghetti Code, Golden Hammer, Copy-Paste, Big Ball of Mud – erkennen und vermeiden!

Fachhochschule
Dortmund
University of Applied Sciences and Arts

Compliance im Software Engineering I

Andreas Diepenbrock

11.05.2026

Was hat das mit Softwaretechnik zu tun?

KI generiert

SOFTWARE ENGINEERING
ARCHITEKTUR-MODELLIERUNG
IMPLEMENTIERUNG
BETRIEB
PROJEKT-MANAGEMENT
TEAMARBEIT
COMPLIANCE
KI generiert

ARCHITEKTURMODELLIERUNG

INHALTE

- Anforderungserhebung
- Modellierung (z.B. UML)
- Überführen von Anforderungen in Architektur
- Architekturstile

IM STUDIUM *

- SWT2
- IT-Recht
- Informationssicherheit

*Modulhandbuch 2019 Praktische Informatik

IMPLEMENTIERUNG

INHALTE

- Programmierung
- Continuous Integration
- Konfiguration
- Architekturstile
- (Künstliche Intelligenz)

IM STUDIUM *

- SWT2
- Informationssicherheit
- Datenbanken
- Web-Technologien
- Programmierkurs

*Modulhandbuch 2019 Praktische Informatik

BETRIEB

INHALTE

- Netzwerktechnik
- Security
- Deployment / DevOps

IM STUDIUM *

- SWT2
- Kommunikation- und Rechnernetze
- Rechnerstrukturen und Betriebssysteme
- Informationssicherheit

*Modulhandbuch 2019 Praktische Informatik

PROJEKTMANAGEMENT

INHALTE

- Anforderungserhebung
- Vorgehensmodelle
- Tooling
- Dokumentation

IM STUDIUM *

- SWT2
- Projektmanagement
- Informationssicherheit

*Modulhandbuch 2019 Praktische Informatik

TEAMARBEIT

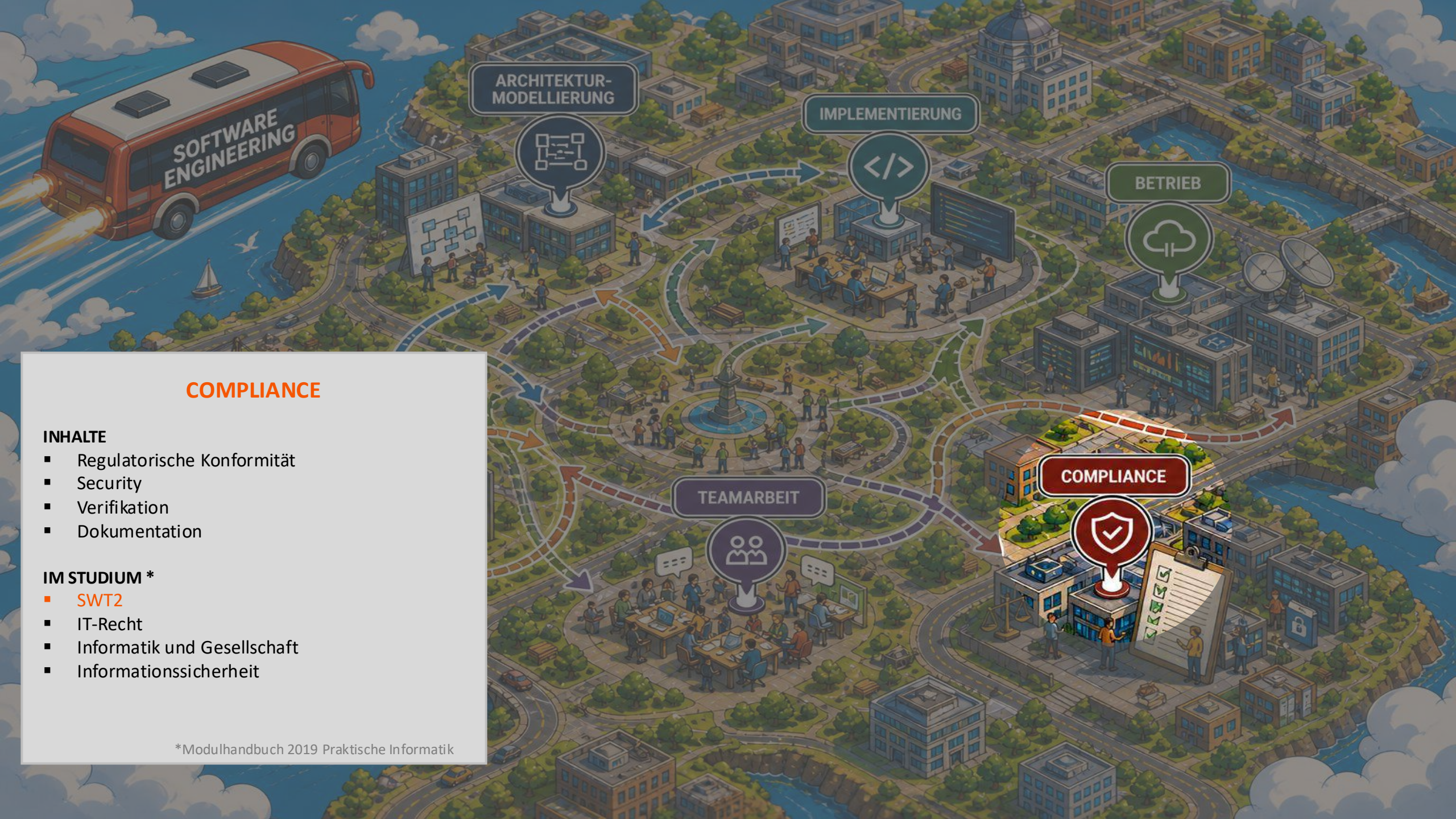
INHALTE

- Versionsverwaltung
- Tooling
- Vorgehensmodelle

IM STUDIUM *

- SWT2
- IT-Recht
- Informatik und Gesellschaft
- Programmierkurs

*Modulhandbuch 2019 Praktische Informatik



COMPLIANCE

INHALTE

- Regulatorische Konformität
- Security
- Verifikation
- Dokumentation

IM STUDIUM *

- SWT2
- IT-Recht
- Informatik und Gesellschaft
- Informationssicherheit

*Modulhandbuch 2019 Praktische Informatik

Compliance im Software Engineering

- Software-Systeme müssen nicht nur funktional korrekt, sondern auch regelkonform entwickelt und betrieben werden.
- Rechtliche, regulatorische und organisatorische Anforderungen müssen in Architektur, Implementierung, Prozesse und Betrieb übersetzt werden.
- Compliance ist damit keine reine Rechtsfrage, sondern eine Engineering-Aufgabe.
- Ziel ist, Regelkonformität technisch umzusetzen und nachvollziehbar nachzuweisen.

📝 **Compliance im Software Engineering bedeutet die Überführung von regulatorischen Regeln in technische und organisatorische Anforderungen an Softwaresysteme**

Arten von Compliance im Unternehmen

In-house	Extern

Zentral

- Eigene Rechtsabteilung für alles
- Jeder kommuniziert direkt mit der Rechtsabteilung

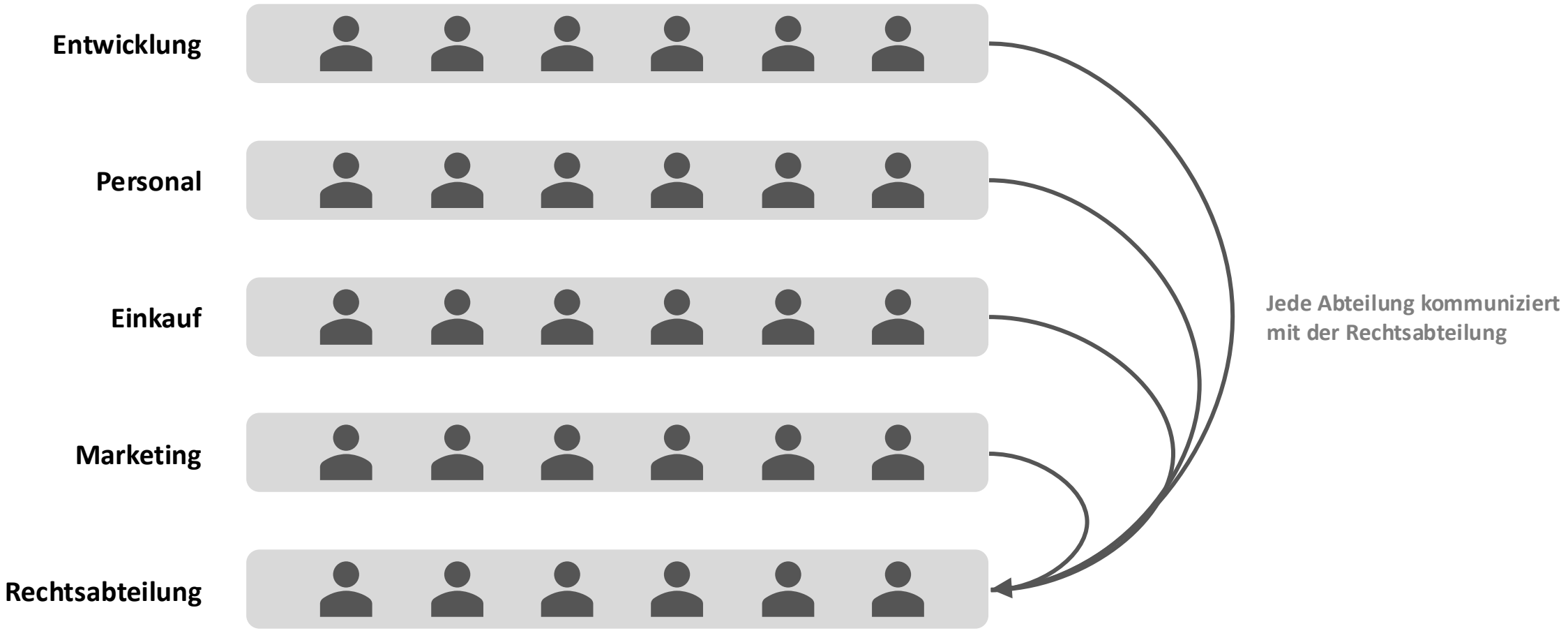
Dezentral

- Zuständige Personen in Teams integriert (→ z.B. Security Champions[1])
- Champions sind Bindeglied zwischen Rechtsabteilung und der fachlichen Abteilung (in kleinen Unternehmen ggf. auch ohne Rechtsabteilung)

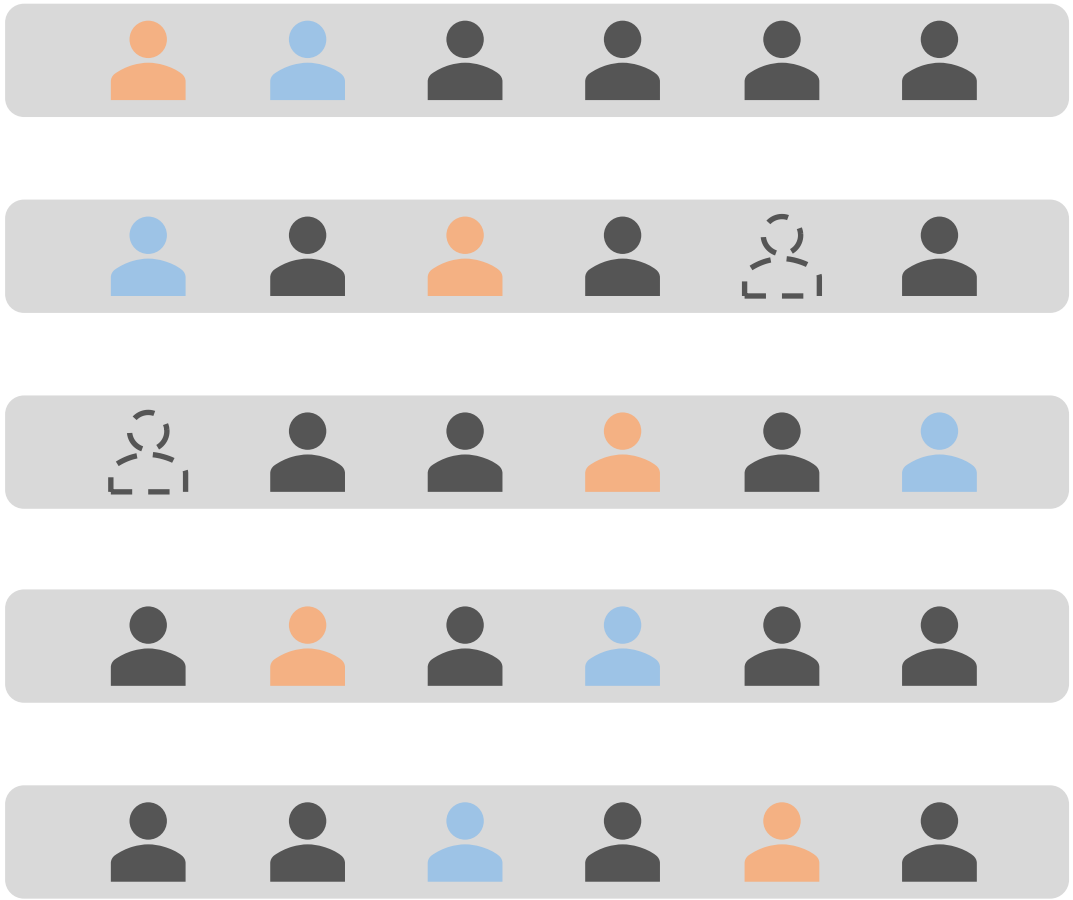
- Externer Dienstleister wird für rechtliche Angelegenheiten beauftragt
- Kommunikation ggf. über Ansprechpartner im Unternehmen

[1] https://owasp.org/www-project-security-culture/v10/4-Security_Champions/

Zentrale Compliance In-House



Dezentrale Compliance In-House



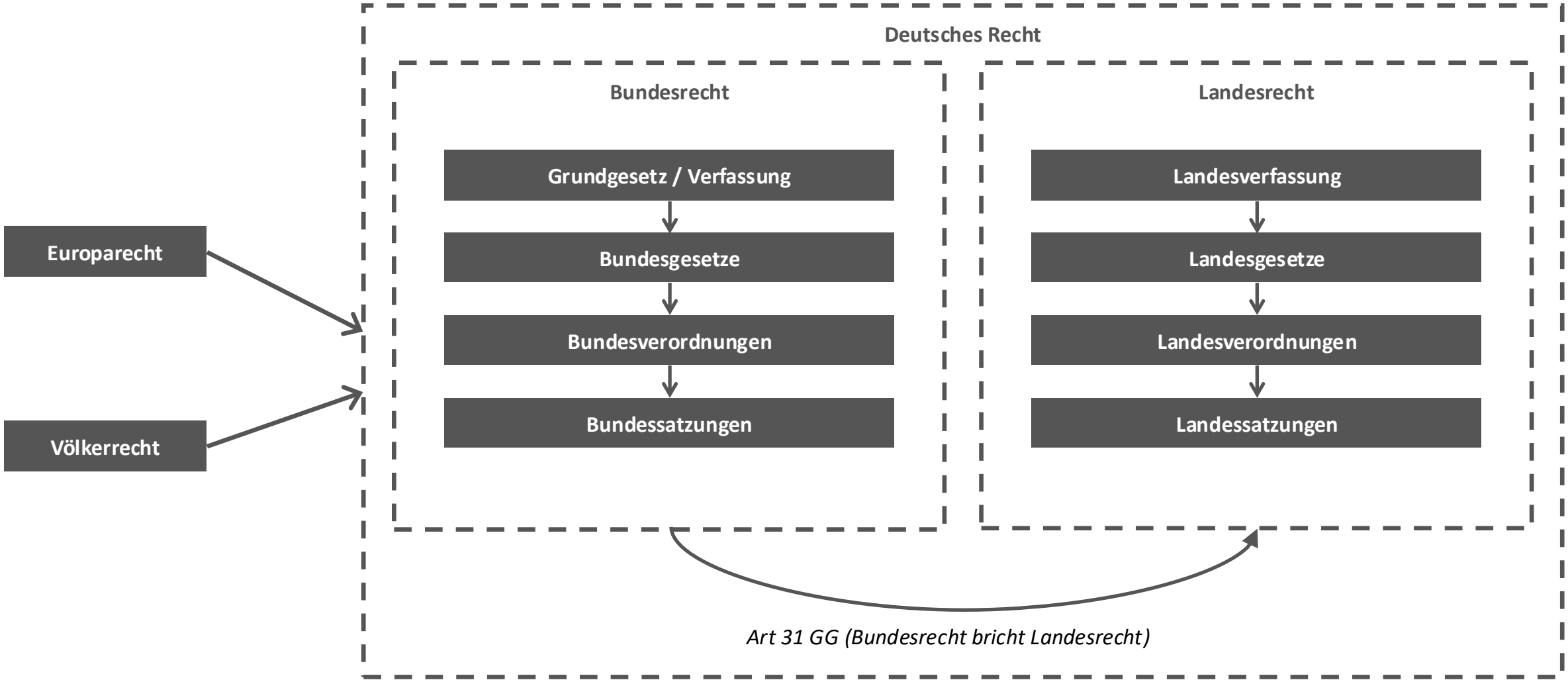
Compliance Champion

Security Champion

… Champion

Champions sind Bindeglied zwischen Abteilung und Rechtsabteilung

(Europäische) Normenhierarchie



Deutsches Gesetz vs. EU-Verordnung

Deutsches Gesetz / deutsche Verordnung

- oft gegliedert in Teil – Abschnitt – Titel – §
- innerhalb des §: Absatz – Satz – Nummer – Buchstabe
- Zitation etwa: § 433 Abs. 1 Satz 1 BGB

EU-Verordnung / Richtlinie

- vor dem Normtext häufig Erwägungsgründe
- dann Artikel
- oft zusätzliche Annexes / Anhänge
- Zitation etwa: Art. 6 Abs. 1 AI Act

Anatomie eines Rechtstextes

- **Titel des Rechtsakts:** Was ist das überhaupt?
- **Einleitender Teil:** Zweck, Geltungsbereich, Kontext
- **Erwägungsgründe (vor allem EU-Recht):** Begründung und Zielsetzung
- **Normtext:** Artikel bzw. §§ mit verbindlichen Regelungen
- **Anhänge / Anlagen:** Tabellen, Listen, technische Details, Formblätter
- **Querverweise:** Verweise auf andere Normen

Erwägungsgründe: Wozu sind sie da?

- Erwägungsgründe erklären Ziele, Motive und systematischen Hintergrund
- i.d.R. vor den Artikeln beschrieben
- sie sollen nach dem gemeinsamen EU-Leitfaden die Gründe für die wesentlichen Regelungen knapp darstellen
- Enthalten keine normativen Regelungen an sich

📝 **Erwägungsgründe erklären das Warum — der Normtext regelt das Was.**

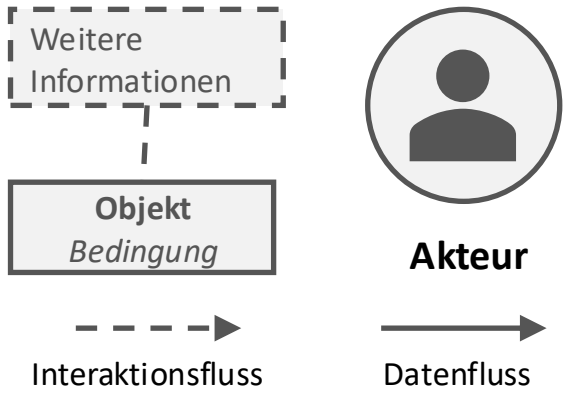
📝 Eine Zuordnung von Erwägungsgründe ←→ Artikeln ist i.d.R. in den offiziellen Dokumenten nicht gegeben, da diese nicht zwingend einem spezifischen Artikel zuteilbar sind.

Elemente in Normen

Element	Leitfrage	Beispiel
Akteur	Wer handelt oder ist Adressat der Norm?	Verantwortlicher
Gegenüber	Wen betrifft die Handlung?	Betroffene Person
Objekt	Worauf bezieht sich die Aussage?	Personenbezogene Daten
Beziehung	Was passiert zwischen den Akteuren?	Muss Auskunft erteilen
Typ	Pflicht, Recht, Verbot oder Erlaubnis?	Pflicht
Bedingung	Wann oder unter welcher Voraussetzung gilt es?	Auf Antrag
Frist	Bis wann muss reagiert werden?	Innerhalb eines Monats

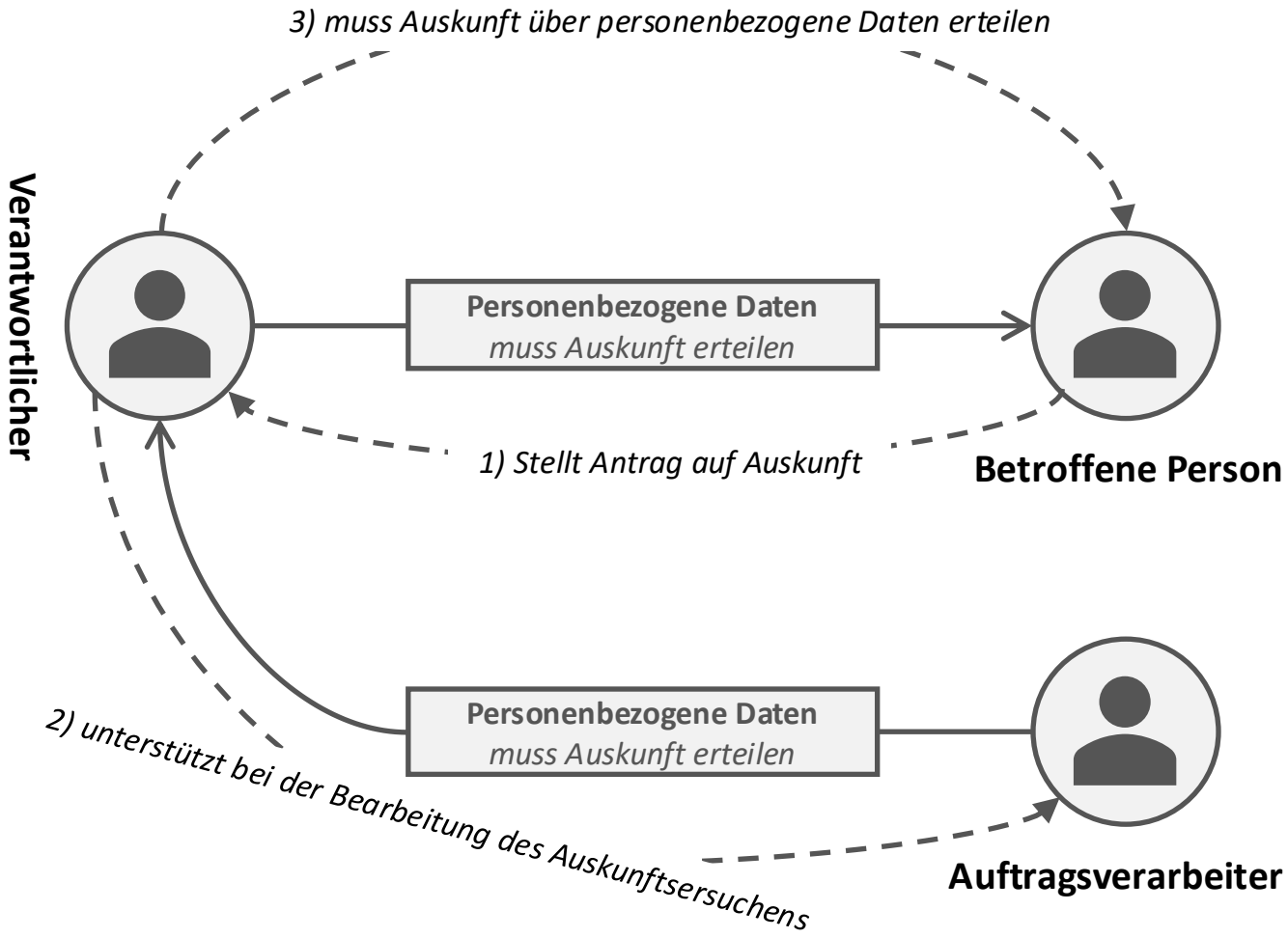
Fiktiver Normtext (in Anlehnung an GDPR)

Die betroffene Person hat das Recht, vom Verantwortlichen Auskunft über die Verarbeitung ihrer personenbezogenen Daten zu erhalten. Der Verantwortliche hat auf das Ersuchen unverzüglich, in der Regel aber innerhalb eines Monats, zu reagieren. Der Auftragsverarbeiter unterstützt den Verantwortlichen bei der Erfüllung von Betroffenenrechten.



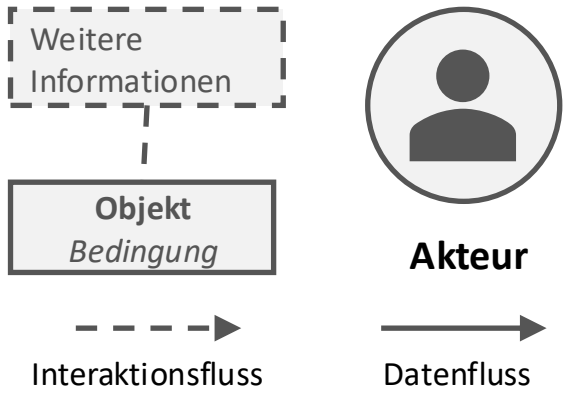
Fiktiver Normtext (in Anlehnung an GDPR)

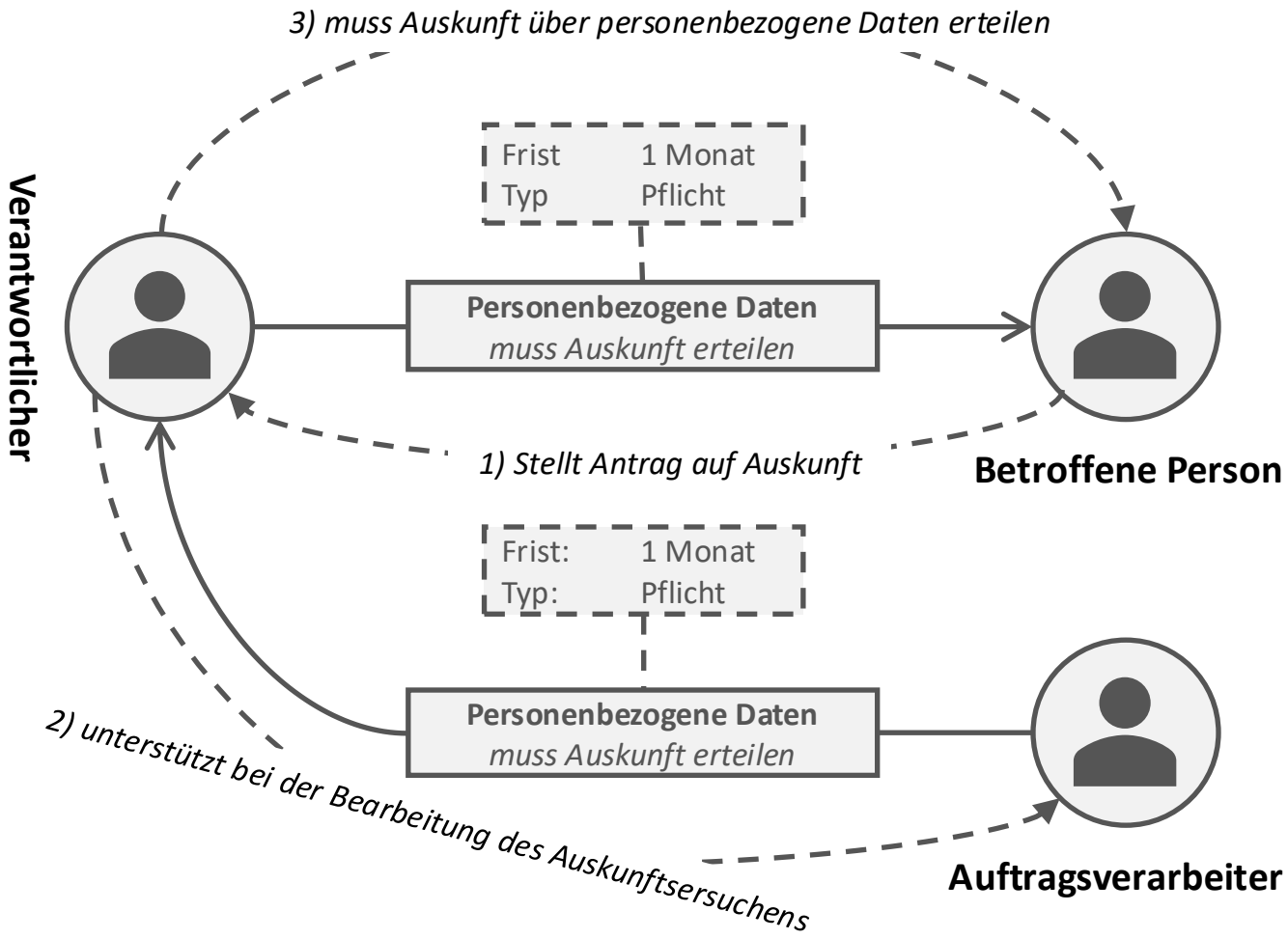
Die **betroffene Person** hat das Recht, vom **Verantwortlichen** Auskunft über die Verarbeitung ihrer personenbezogenen Daten zu erhalten. Der Verantwortliche hat auf das Ersuchen unverzüglich, in der Regel aber innerhalb eines Monats, zu reagieren. Der **Auftragsverarbeiter** unterstützt den Verantwortlichen bei der Erfüllung von Betroffenenrechten.



Fiktiver Normtext (in Anlehnung an GDPR)

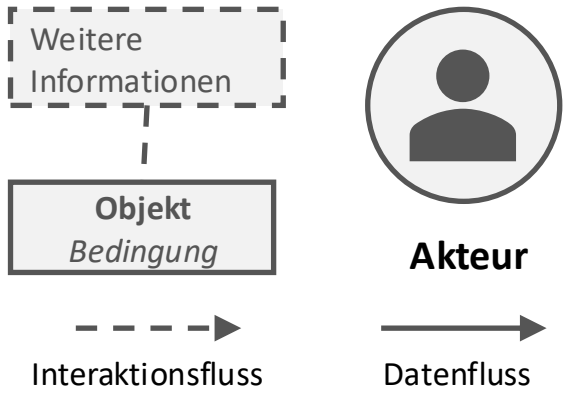
Die betroffene Person hat das Recht, vom Verantwortlichen **Auskunft über die Verarbeitung ihrer personenbezogenen Daten zu erhalten**. Der Verantwortliche hat auf das Ersuchen unverzüglich, in der Regel aber innerhalb eines Monats, zu reagieren. Der Auftragsverarbeiter **unterstützt den Verantwortlichen bei der Erfüllung von Betroffenenrechten**.

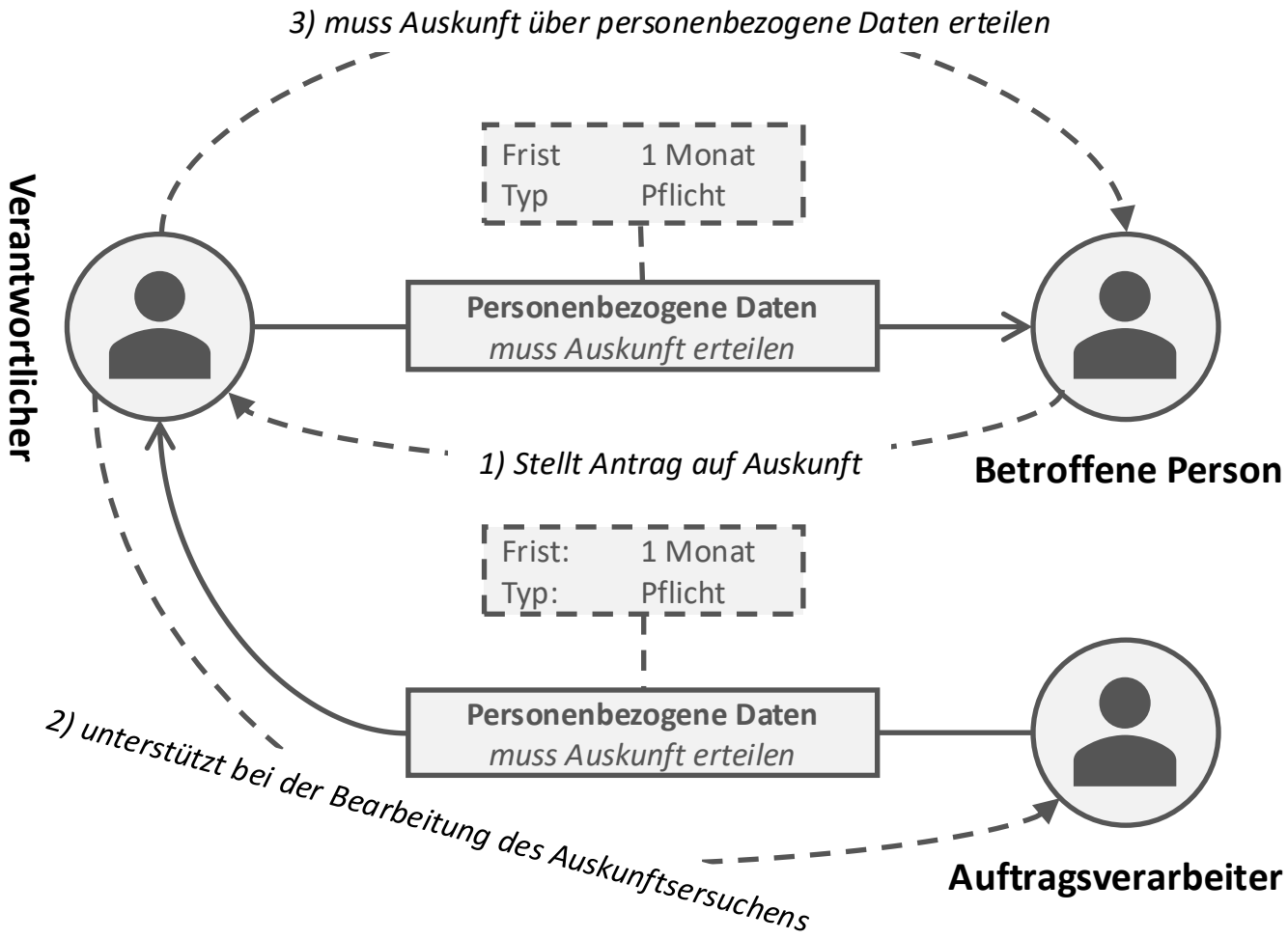




Fiktiver Normtext (in Anlehnung an GDPR)

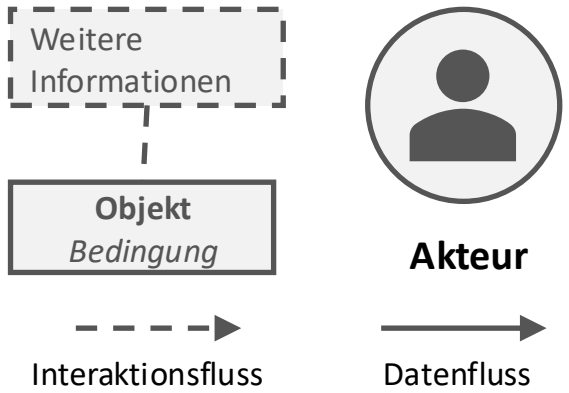
Die betroffene Person **hat das Recht**, vom Verantwortlichen Auskunft über die Verarbeitung ihrer personenbezogenen Daten zu erhalten. **Der Verantwortliche hat auf das Ersuchen unverzüglich, in der Regel aber innerhalb eines Monats, zu reagieren**. Der Auftragsverarbeiter unterstützt den Verantwortlichen bei der Erfüllung von Betroffenenrechten.





Fiktiver Normtext (in Anlehnung an GDPR)

Die betroffene Person hat das Recht, vom Verantwortlichen Auskunft über die Verarbeitung ihrer personenbezogenen Daten zu erhalten. Der Verantwortliche hat auf das Ersuchen unverzüglich, in der Regel aber innerhalb eines Monats, zu reagieren. Der Auftragsverarbeiter unterstützt den Verantwortlichen bei der Erfüllung von Betroffenenrechten.



Regulatorische Anforderungen als Treiber von Architekturstilen

Rechtliche Anforderungen beeinflussen technische Architekturentscheidungen.
Softwarearchitekturstile helfen dabei, regulatorische Vorgaben systematisch und technisch umsetzbar zu machen.

- Regularien formulieren Anforderungen an ein System, z.B. Datenschutz, Sicherheit oder Resilienz.
- Diese Anforderungen müssen nicht nur organisatorisch, sondern auch technisch umgesetzt werden.
- Die Softwarearchitektur legt fest, wie ein System aufgebaut ist und wie seine Teile zusammenarbeiten.
- Bestimmte Architekturstile können die Umsetzung regulatorischer Anforderungen teilweise erleichtern.
- Architektur ist daher nicht nur eine technische, sondern auch eine rechtlich relevante Gestaltungsentscheidung.

📝 **Regularien geben das Ziel vor – die Architektur schafft die technische Grundlage zur Umsetzung.**

📝 Softwarearchitekturen erfüllen Gesetze nicht von selbst, aber sie schaffen technische Strukturen, mit denen rechtliche Anforderungen gezielter umgesetzt, isoliert und überprüft werden können.

Systembeschreibung → Anforderungen

Beschreibung des Systems

Ein Unternehmen möchte ein webbasiertes **Bewerbungsmanagementsystem** einführen. Bewerberinnen und Bewerber sollen ihre Unterlagen online hochladen und den Status ihrer Bewerbung einsehen können. Recruiter und Fachabteilungen sollen Bewerbungen sichten, kommentieren und bewerten können. Zusätzlich soll ein KI-Modul Bewerbungen voranalysieren und z. B. Qualifikationen extrahieren oder eine erste thematische Zuordnung zu offenen Stellen vornehmen.

Anforderungsliste

1	Bewerber können ein Benutzerkonto anlegen.
2	Bewerbungsunterlagen können hochgeladen werden.
3	Recruiter können Bewerbungen sichten und bearbeiten.
4	Das KI-Modul analysiert Lebensläufe und ordnet Fähigkeiten Stellenprofilen zu.

🤔 Welche rechtlichen Anforderungen resultieren daraus?

Beispiele für Fragen

- Wer darf darauf zugreifen?
- Wie lange werden die Unterlagen gespeichert?
- Wann müssen die Unterlagen gelöscht werden?
- Müssen die Bewerber darüber informiert werden?
- …

5	Das System muss ein Rollen- und Berechtigungskonzept umsetzen.
6	Bewerbungsunterlagen dürfen nur Nutzern angezeigt werden, die für das jeweilige Bewerbungsverfahren autorisiert sind.
7	Nach Fristablauf muss eine automatische Löschung oder Anonymisierung der Daten erfolgen.

Regulatorische Anforderungen

Systembeschreibung → Anforderungen

Beschreibung des Systems

Ein Un
webba
Bewer
einfüh
Bewer
online
ihrer B
Recruiter und Fachabteilungen
sollen
komm
könne
Bewer
B. Qualifikationen extrahieren oder
eine erste thematische Zuordnung
zu offenen Stellen vornehmen.

Anforderungsliste

1	Bewerber können ein Benutzerkonto anlegen.
2	Bewerbungsunterlagen können hochgeladen werden.
6	Bewerbungsunterlagen dürfen nur Nutzern angezeigt werden, die für das jeweilige Bewerbungsverfahren autorisiert sind.
7	Nach Fristablauf muss eine automatische Löschung oder Anonymisierung der Daten erfolgen.

📝 **Aus (funktionalen) Anforderungen können weitere Anforderungen abgeleitet werde insb. im regulatorischen Kontext**

- Benutzerdaten werden gespeichert → Löschfristen bzw. Löschkonzept, Zweckbindung
- Mitarbeiter können Daten einsehen → Rollen- und Berechtigungskonzept, Protokollierung von Zugriffen
- Dokumente werden mit KI analysiert → Transparenz über KI-Einsatz, Nachvollziehbarkeit

🤔 Welche rechtlichen Anforderungen resultieren daraus?

📝 **Regulatorische Anforderungen sind, neben Stakeholder, Geschäftsziele oder technische Randbedingungen, eine weitere Quelle von Anforderungen.**

Beispiele für Fragen

- Wer darf darauf zugreifen?
- Wie lange werden die Unterlagen gespeichert?
- Wann müssen die Unterlagen gelöscht werden?
- Müssen die Bewerber darüber informiert werden?
- …

Wie UML-Diagramme regulatorische Anforderungen unterstützen

- Regularien fordern z.B. Datenschutz, Sicherheit, Nachvollziehbarkeit und Kontrolle.
- UML-Diagramme machen diese Anforderungen in der Systembeschreibung sichtbar.
- Unterschiedliche Diagrammarten unterstützen unterschiedliche Compliance-Fragen.
- UML beweist keine Compliance, hilft aber bei Dokumentation, Kommunikation und Prüfung regulatorischer Anforderungen.

📝 **UML-Diagramme zeigen, wie regulatorische Anforderungen darin konkret sichtbar und umsetzbar werden.**

📝 **Nicht jedes UML-Diagramm beantwortet dieselbe Compliance-Frage:**
Klassendiagramme helfen bei Datenstrukturen, **Aktivitätsdiagramme** bei Prozessen, **Sequenzdiagramme** bei Datenflüssen, **Komponentendiagramme** bei Verantwortlichkeiten und **Deployment-Diagramme** bei Sicherheit und Resilienz.

UML als Mittel zur Dokumentation Compliance-relevanter Aspekte

1	Wer greift auf welche Funktionen und Daten zu?	Sequenzdiagramm	A
2	Wie läuft ein Freigabe- oder Löschprozess ab?	Komponentendiagramm	B
3	Welche Systeme tauschen personenbezogene Daten aus?	Use-Case-Diagramm	C
4	Wo liegen Systemgrenzen, externe Dienste, Cloud-Komponenten?	Klassendiagramm	D
5	Welche Datenobjekte existieren, welche Felder sind sensibel?	Aktivitätsdiagramm	E

UML als Mittel zur Dokumentation Compliance-relevanter Aspekte

1	Wer greift auf welche Funktionen und Daten zu?	Sequenzdiagramm	A
2	Wie läuft ein Freigabe- oder Löschprozess ab?	Komponentendiagramm	B
3	Welche Systeme tauschen personenbezogene Daten aus?	Use-Case-Diagramm	C
4	Wo liegen Systemgrenzen, externe Dienste, Cloud-Komponenten?	Klassendiagramm	D
5	Welche Datenobjekte existieren, welche Felder sind sensibel?	Aktivitätsdiagramm	E

UML als Mittel zur Dokumentation Compliance-relevanter Aspekte

1	Wer greift auf welche Funktionen und Daten zu?	Sequenzdiagramm	A
2	Wie läuft ein Freigabe- oder Löschprozess ab?	Komponentendiagramm	B
3	Welche Systeme tauschen personenbezogene Daten aus?	Use-Case-Diagramm	C
4	Wo liegen Systemgrenzen, externe Dienste, Cloud-Komponenten?	Klassendiagramm	D
5	Welche Datenobjekte existieren, welche Felder sind sensibel?	Aktivitätsdiagramm	E

UML als Mittel zur Dokumentation Compliance-relevanter Aspekte

1	Wer greift auf welche Funktionen und Daten zu?	Sequenzdiagramm	A
2	Wie läuft ein Freigabe- oder Löschprozess ab?	Komponentendiagramm	B
3	Welche Systeme tauschen personenbezogene Daten aus?	Use-Case-Diagramm	C
4	Wo liegen Systemgrenzen, externe Dienste, Cloud-Komponenten?	Klassendiagramm	D
5	Welche Datenobjekte existieren, welche Felder sind sensibel?	Aktivitätsdiagramm	E

UML als Mittel zur Dokumentation Compliance-relevanter Aspekte

1	Wer greift auf welche Funktionen und Daten zu?	Sequenzdiagramm	A
2	Wie läuft ein Freigabe- oder Löschprozess ab?	Komponentendiagramm	B
3	Welche Systeme tauschen personenbezogene Daten aus?	Use-Case-Diagramm	C
4	Wo liegen Systemgrenzen, externe Dienste, Cloud-Komponenten?	Klassendiagramm	D
5	Welche Datenobjekte existieren, welche Felder sind sensibel?	Aktivitätsdiagramm	E

UML als Mittel zur Dokumentation Compliance-relevanter Aspekte

1	Wer greift auf welche Funktionen und Daten zu?	Sequenzdiagramm	A
2	Wie läuft ein Freigabe- oder Löschprozess ab?	Komponentendiagramm	B
3	Welche Systeme tauschen personenbezogene Daten aus?	Use-Case-Diagramm	C
4	Wo liegen Systemgrenzen, externe Dienste, Cloud-Komponenten?	Klassendiagramm	D
5	Welche Datenobjekte existieren, welche Felder sind sensibel?	Aktivitätsdiagramm	E

* by Default / * by Design

- Anforderungen an ein System sollten nicht erst nachträglich ergänzt, sondern frühzeitig im Entwurf berücksichtigt werden.
- **by Design** bedeutet, dass gewünschte Eigenschaften bereits in Architektur, Datenmodell, Prozessen und Schnittstellen eingeplant werden.
- **by Default** bedeutet, dass Systeme in ihren Standardeinstellungen bereits so konfiguriert sind, dass Anforderungen bestmöglich eingehalten werden.
- Das Prinzip lässt sich auf verschiedene Bereiche übertragen, z. B.:
  - Privacy by Design / by Default
  - Security by Design / by Default
  - Compliance by Design / by Default
  - AI Governance by Design

📝 **Wichtige Systemeigenschaften sollten nicht nachträglich aufgesetzt, sondern in Entwurf und Voreinstellungen verankert werden.**

Accountability und Nachweisbarkeit

- Es reicht nicht, compliant zu handeln — Compliance muss auch belegbar sein.
- Wichtige Nachweise in Softwareprojekten:
  - Verzeichnis von Verarbeitungstätigkeiten
  - Rollen- und Berechtigungskonzept
  - Löschkonzept
  - Architektur- und Datenflussdiagramme
  - Audit-Logs
  - Test- und Freigabedokumentation
- Gute Engineering-Artefakte unterstützen daher auch die Nachweisbarkeit regulatorischer Anforderungen.

📝 **Aus Umsetzung wird erst durch Evidenz prüfbare Compliance.**

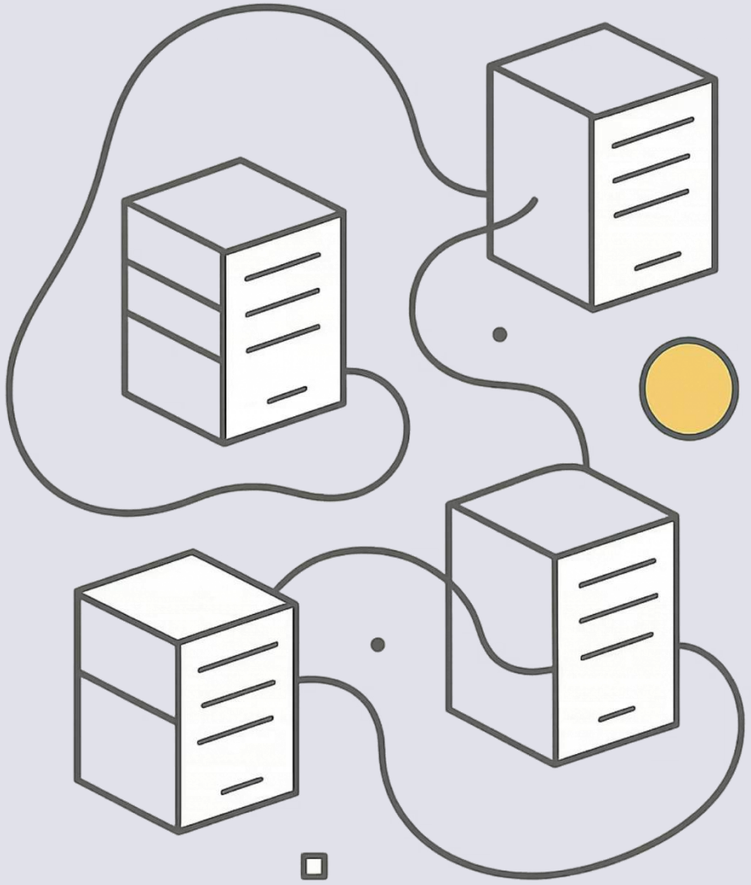
Verifikation und Testbarkeit

- Regulatorische Anforderungen müssen nicht nur beschrieben, sondern auch überprüfbar gemacht werden.
- Beispiele:
  - Zugriffsbeschränkung → Berechtigungstests
  - Löschfristen → Tests auf automatische Löschung oder Anonymisierung
  - Transparenzpflichten → Prüfung von Hinweisen und Informationen im UI
  - Protokollierung → Tests, ob sicherheitsrelevante Zugriffe erfasst werden
  - KI-Nutzung → Prüfung von Nachvollziehbarkeit und menschlicher Übersteuerbarkeit
- Regulatorische Anforderungen werden dadurch zu testbaren Qualitätsanforderungen.

📝 **Compliance-Anforderungen sind nur dann wirksam, wenn sie auch systematisch geprüft werden können.**

Patterns und Konzepte für verteilte Anwendungen

Softwaretechnik 2



Quelle: Kirstian Köhler: „Softwaredesign und Entwurfsmuster - Das umfassende Handbuch“, Rheinwerkverlag, 2025

Lernziele dieser Vorlesung

1

Konsistenzmodelle verstehen

CAP-Theorem, PACELC und Eventual Consistency als theoretische Grundlage verteilter Systeme

2

Designmuster kennen

8 zentrale Patterns für Zustandsmanagement, Konsistenz, Performance und Monitoring

3

Anwendung auf Client-Seite

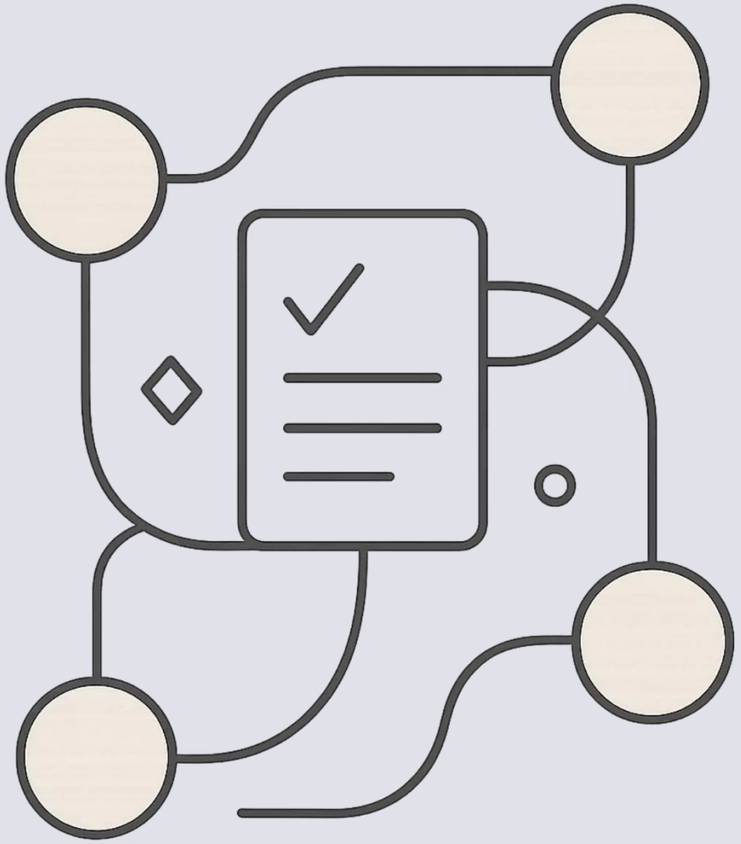
Konzepte gelten nicht nur für Server – auch Standalone-Anwendungen greifen auf verteilte Services zu

Übersicht der behandelten Patterns

Pattern	Kategorie	Fokus
Stateless Architecture	Zustandsmanagement	Skalierbarkeit, Vereinfachung
Database per Service	Datenbank	Entkopplung, Autonomie
Optimistic Locking	Konsistenz	Parallelitätskontrolle
Saga	Konsistenz	Fehler- und Rollback-Handling
Transactional Outbox	Konsistenz	Gesicherte Nachrichtenzustellung
Event Sourcing	Zustandsmanagement	Historisierung, Rückverfolgbarkeit
CQRS	Performance	Effiziente Lesetransaktionen
Distributed Tracing	Monitoring	Debugging, Performanceanalyse

Konsistenz in verteilten Systemen

Grundlagen, Theoreme und Konsistenzmodelle



Was bedeutet Konsistenz?

Definition

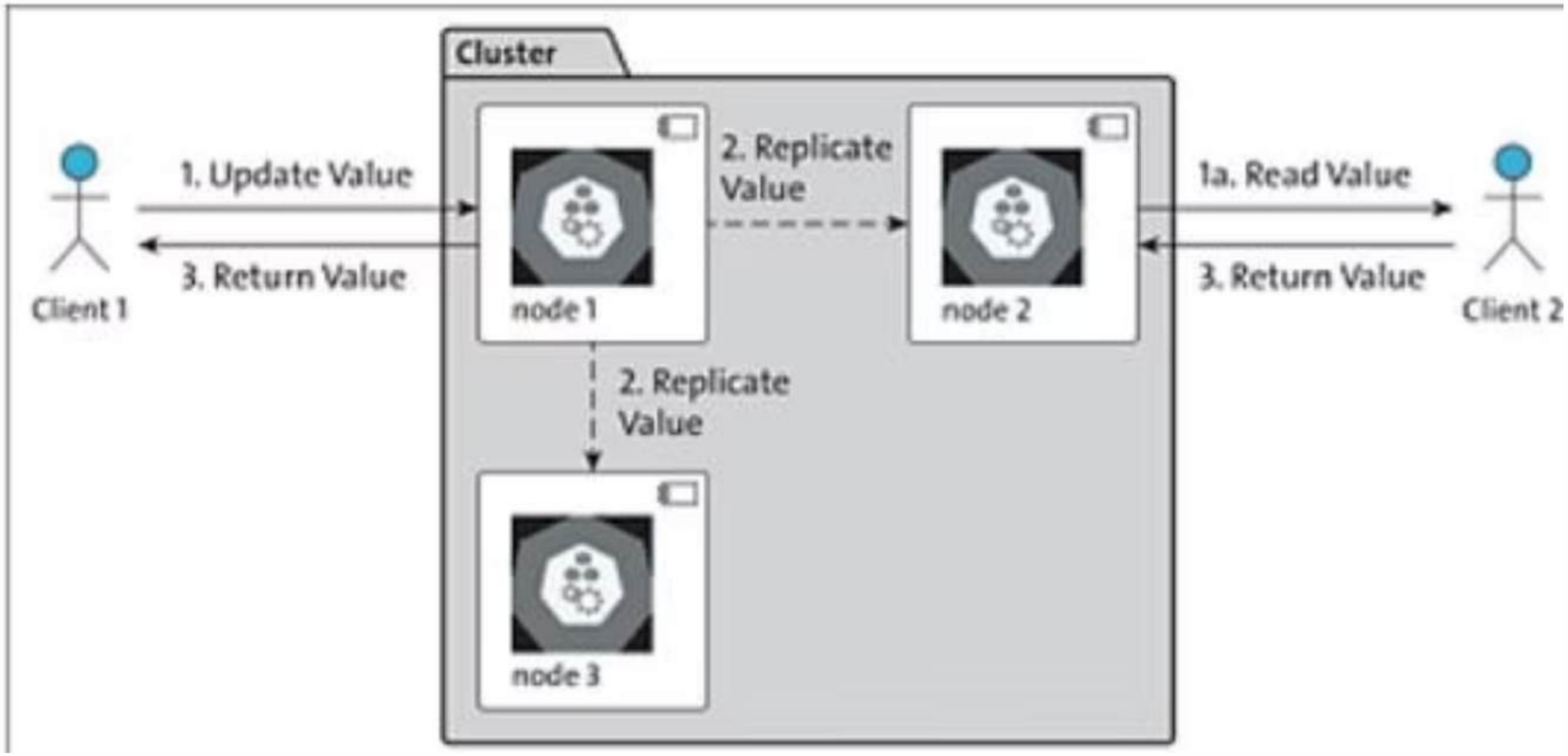
Konsistenz bezeichnet den Zustand, in dem alle Kopien bzw. Replikate eines Datensatzes identisch sind und festgelegte Regeln einhalten.
Jede Leseoperation liefert – unabhängig von der genutzten Kopie – immer den zuletzt geschriebenen Wert.

Warum ist das wichtig?

Nur mit korrekten Daten können darauf aufbauende Entscheidungen korrekt getroffen werden.

Beispiel Bankanwendung: Eine Abbuchung auf einem Konto muss zwingend eine Gegenbuchung auf einem anderen Konto auslösen. Erst wenn beide Aktionen durchgeführt sind, sind die Daten konsistent.

Konsistenz im Cluster – Ablauf



Bei starker Konsistenz muss eine Schreiboperation auf alle Knoten repliziert werden, bevor ein weiterer Client die Daten lesen kann. Das Cluster verhält sich als geschlossenes System – kein Client erhält veraltete Daten.

Das CAP-Theorem

Das von **Eric Brewer** vorgestellte CAP-Theorem besagt: In verteilten Systemen können maximal **zwei der drei** folgenden Eigenschaften gleichzeitig garantiert werden.

Consistency (C)

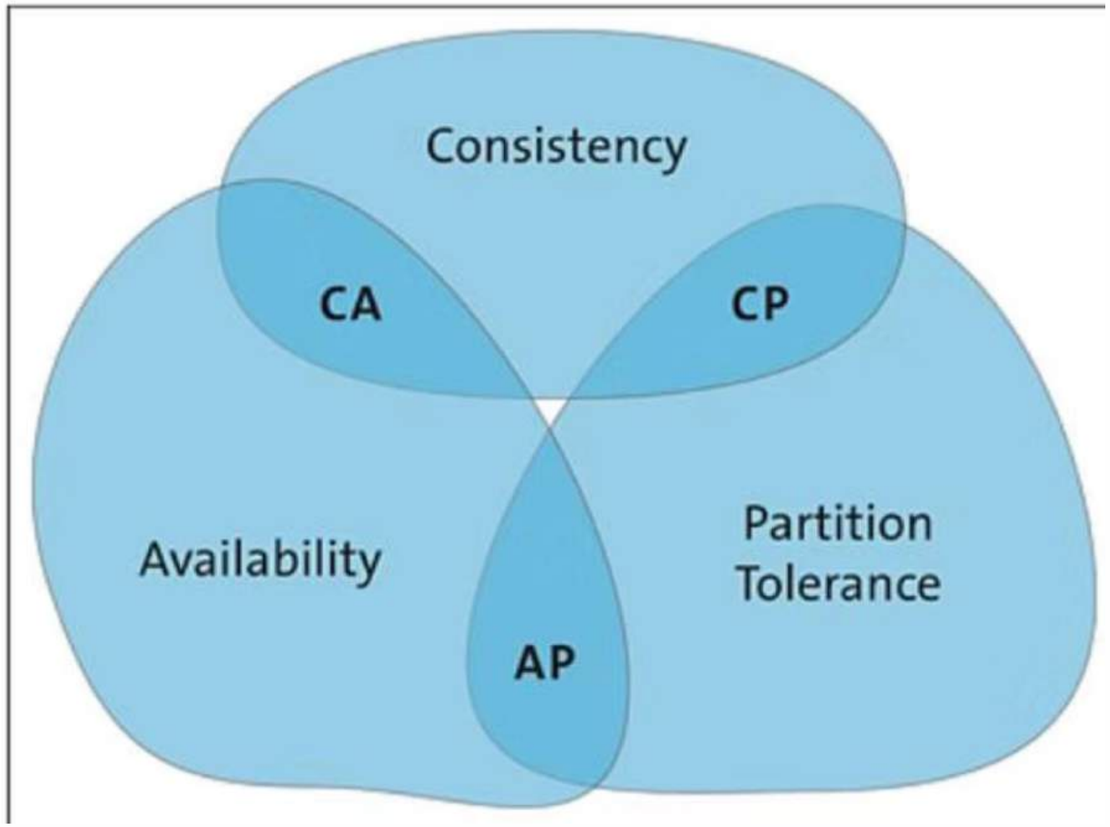
Jeder Lesezugriff erhält entweder aktuelle Daten oder einen Fehler.

Availability (A)

Jede Anfrage an ein nicht ausgefallenes System erhält eine Antwort.

Partition Tolerance (P)

Das System arbeitet auch bei Netzwerkpartitionierung weiter.



CAP-Theorem – Konsequenzen für verteilte Systeme

In verteilten Systemen muss stets mit unzuverlässigen Netzwerken gerechnet werden. **Partition Tolerance ist daher keine Option, sondern eine Notwendigkeit.** Die eigentliche Entscheidung lautet:

CP – Consistency + Partition Tolerance

Bei Netzwerkpartitionierung werden Anfragen mit einer Fehlermeldung beantwortet, da Datenkonsistenz nicht mehr garantiert werden kann.

AP – Availability + Partition Tolerance

Das System liefert auch bei Partitionierung weiterhin Daten – diese könnten jedoch veraltet sein, da Replikation fehlgeschlagen ist.

CA (Consistency + Availability ohne Partition Tolerance) ist in realen verteilten Systemen praktisch nicht umsetzbar.

Das PACELC-Theorem

PACELC erweitert das CAP-Theorem um die Dimension **Latenz**. Auch wenn das System fehlerfrei läuft (keine Partitionierung), muss zwischen Konsistenz und Latenz abgewogen werden.

PA/EL

Verfügbarkeit bei Partition, Latenz sonst

PA/EC

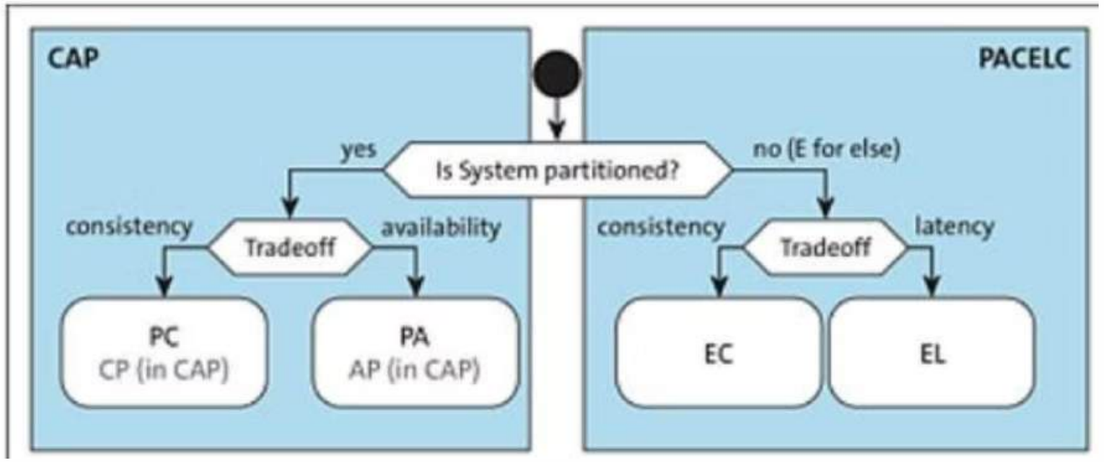
Verfügbarkeit bei Partition, Konsistenz sonst

PC/EL

Konsistenz bei Partition, Latenz sonst

PC/EC

Konsistenz in beiden Fällen

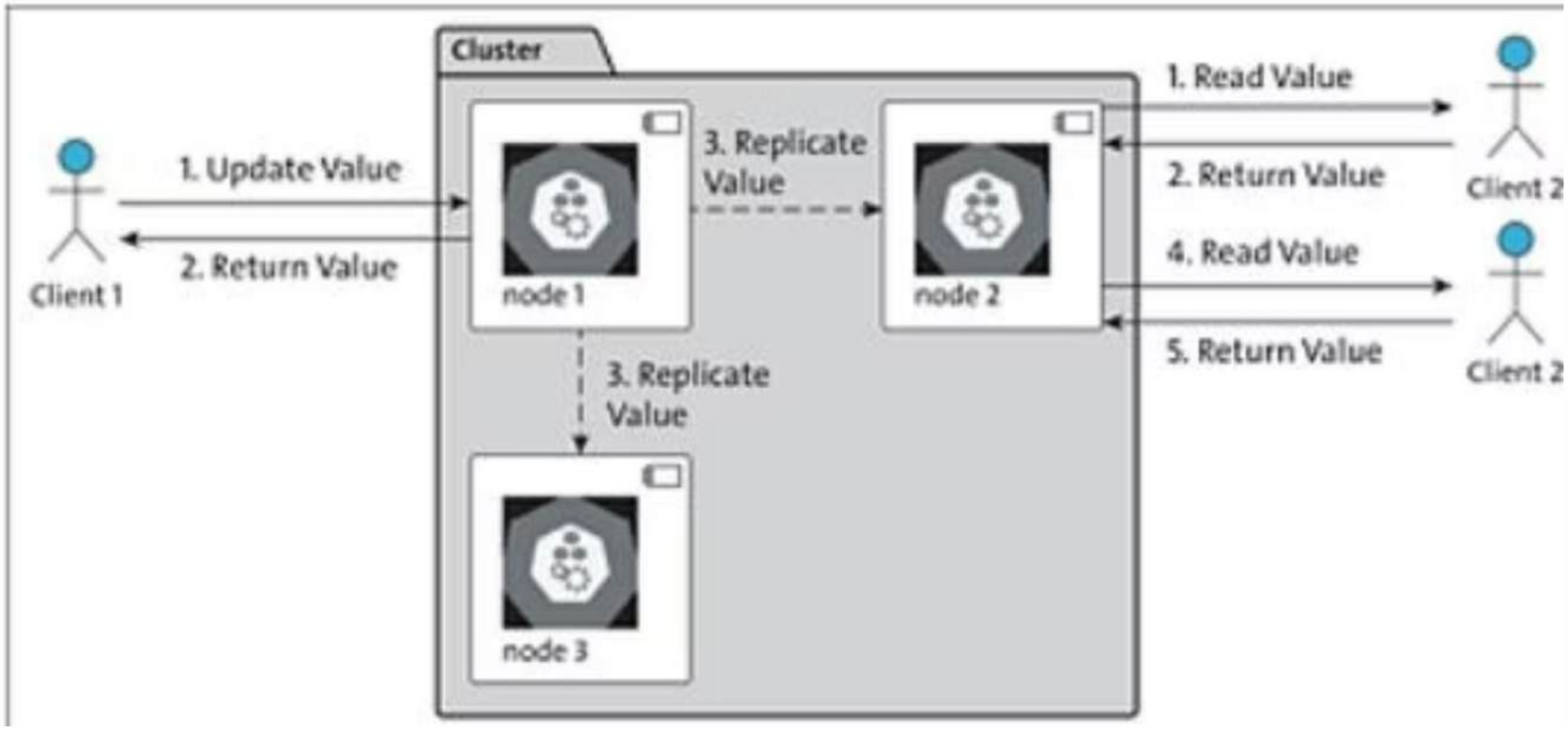


Eventual Consistency

Eventual Consistency ist ein Kompromissmodell: Das System garantiert nicht, dass Lesezugriffe unmittelbar nach einem Schreibvorgang den aktualisierten Wert liefern. Der Zeitraum zwischen Aktualisierung und vollständiger Replikation heißt **Inconsistency Window**.

Bekanntestes Beispiel: Das Domain Name System (DNS) – Aktualisierungen werden zeitgesteuert über die Welt verteilt, sind aber nicht sofort überall konsistent.

Eventual Consistency – Ablauf



Client 1 aktualisiert einen Wert; Client 2 liest zeitgleich noch den alten Wert. Nach der Replikation erhält Client 3 den aktualisierten Wert. Die Konsistenz wird *schließlich* (eventually) hergestellt.

Vorteile der Eventual Consistency

Hohe Verfügbarkeit

Anfragen werden auch bei Netzwerkausfällen oder Partitionierung weiterhin verarbeitet.

Niedrige Latenz

Schreibzugriffe müssen nicht über alle Knoten repliziert werden; Lesezugriffe erfordern keine Sperren.

Gute Skalierbarkeit

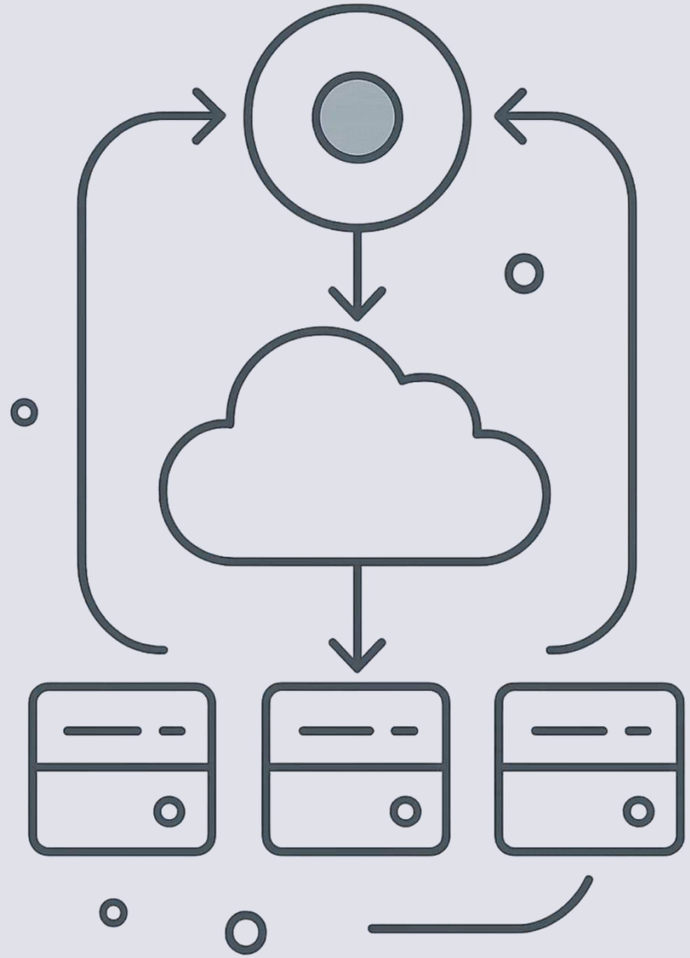
Neue Knoten müssen nicht bei jeder Schreiboperation synchron verfügbar sein.

Bessere Fehlertoleranz

Das System bleibt auch bei einzelnen nicht verfügbaren Knoten funktionsfähig.

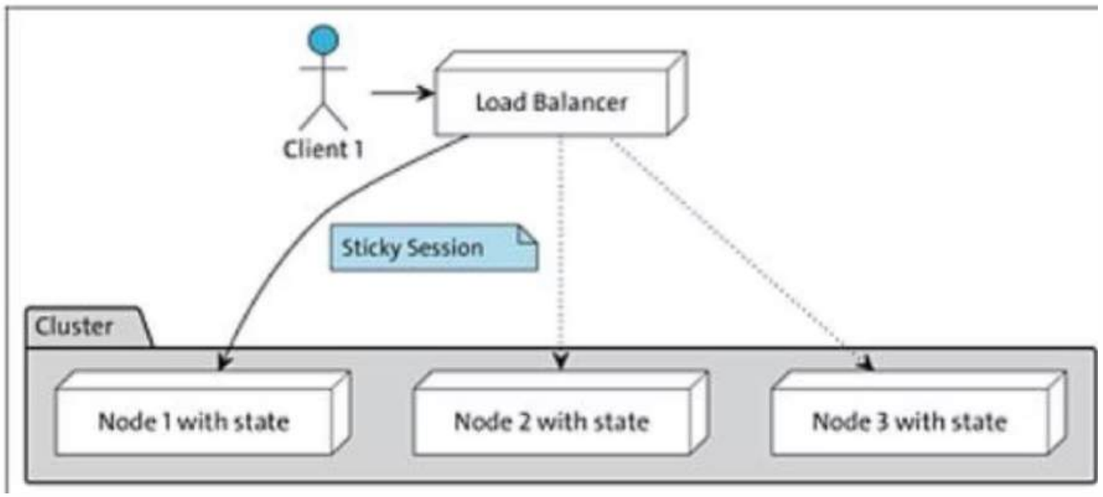
Stateless Architecture Pattern

Zustandsmanagement · Skalierbarkeit · Cloud-native Design



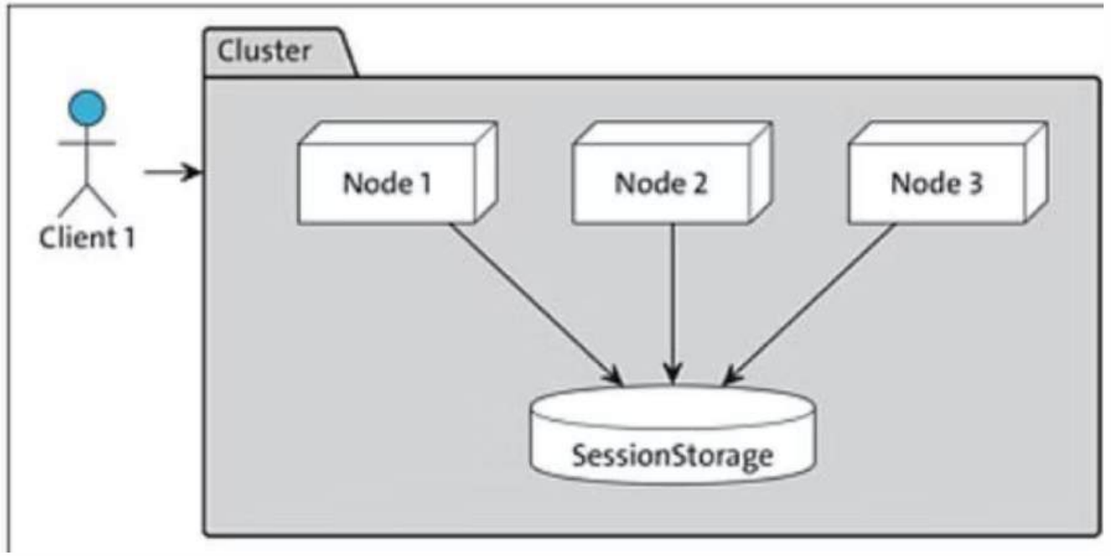
Problem: Zustandsbehaftete Anwendungen

Sticky Sessions



Clients werden per Sticky Session an dedizierte Server gebunden – horizontale Skalierung wird erschwert.

Session-Datenbank



Zentraler SessionStorage vermeidet Replikation, schafft aber eine neue Abhängigkeit und einen Single-Point-of-Failure.

Lösung: Stateless Architecture

Statuslose Anwendungen speichern **keinen anwenderabhängigen Zustand** zwischen Anfragen. Jede Anfrage ist eine vollständig unabhängige Transaktion – alle benötigten Parameter werden mitgegeben.

Application State (vermeiden)	Resource State (erlaubt)
Serverseitig für einen Client vorgehaltener Zustand, z. B. Warenkörbe. Muss repliziert oder synchronisiert werden.	Zustand einer vom Server verwalteten Ressource, unabhängig von Client-Interaktion, z. B. Artikeldaten oder Lagerbestand.

ⓘ **Authentifizierung per JWT-Token gilt als statuslos, solange der Server keine Session-Informationen speichert.**

Statuslose vs. statusbehaftete HTTP-Anfragen

Statusbehaftet

```
# Suche
GET /search?q=Golang

# Nächste Seite – Server braucht Session
GET /nextPage
Cookie: ...
```

Der Server muss den Kontext des Clients kennen.

Statuslos

```
# Alle Informationen in der URL
GET /search?q=GoLang&page=1
GET /search?q=GoLang&page=2
GET /search?q=GoLang&page=3
```

Jede Anfrage ist vollständig eigenständig – kein Serverseitiger Kontext nötig.

Vorteile und Konsequenzen: Stateless Architecture

Einfache horizontale Skalierung

Jede Instanz kann die Aufgaben einer anderen nahtlos übernehmen – kein Zustandsabgleich nötig.

Besseres Caching

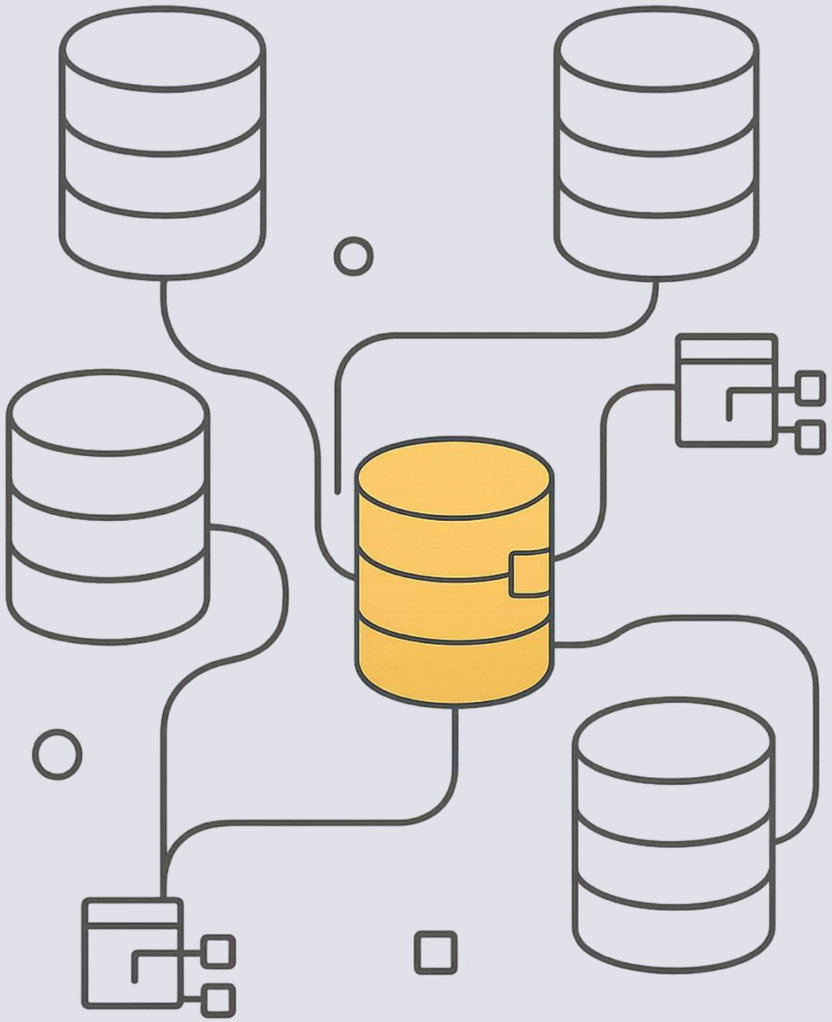
Da alle Parameter in der Anfrage enthalten sind, können Anfragen eindeutig unterschieden und gecacht werden.

Overhead pro Anfrage

Alle Parameter müssen bei jeder Anfrage erneut übertragen werden – größerer Payload pro Request.

Ideal für Cloud und Container

Einsatz in Kubernetes, serverlosen Umgebungen und Container-Plattformen ohne Einschränkungen möglich.



Database per Service Pattern

Entkopplung · Autonomie · Microservice-Datenbanken

Problem: Gemeinsame Datenbank

Nutzen mehrere Services eine gemeinsame Datenbank, entstehen strukturelle Abhängigkeiten:

→ Strukturelle Kopplung

Änderungen an Tabellen oder Schemata betreffen alle abhängigen Services gleichzeitig.

→ Skalierungsprobleme

Fremdschlüssel und Datenbanksperren über Service-Grenzen hinweg behindern die Skalierung.

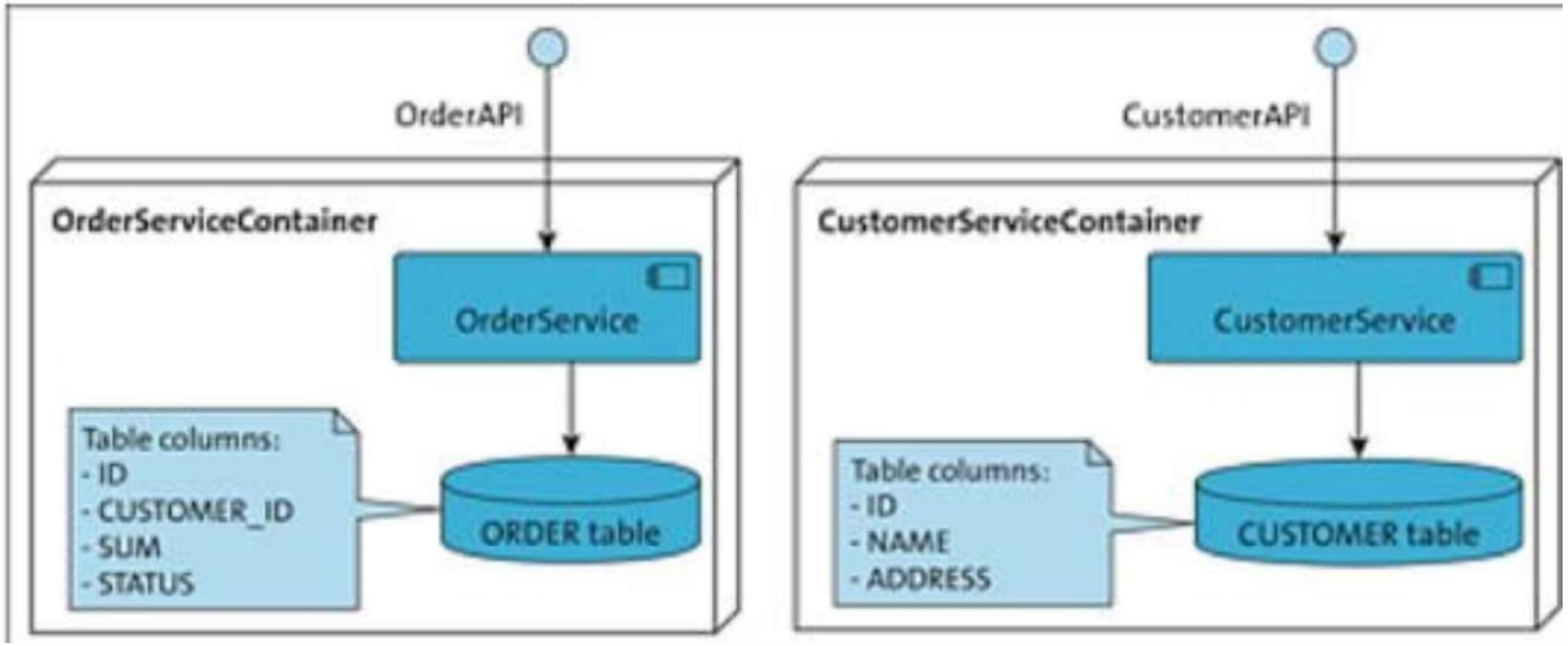
→ Single Point of Failure

Fällt die zentrale Datenbank aus, sind alle Services betroffen.

→ Technologiezwang

Alle Services müssen dieselbe Persistenz-Technologie verwenden, auch wenn eine andere besser geeignet wäre.

Lösung: Database per Service



Jeder Service erhält eine eigene Datenbank. Zugriffe auf Daten eines anderen Service erfolgen ausschließlich über dessen veröffentlichte API. Im Beispiel: **OrderService** und **CustomerService** sind vollständig getrennt.

Strategien für SQL-Datenbanken

Private Tables per Service

Gemeinsame Datenbank, aber separate Tabellen ohne Fremdschlüssel zwischen Services.

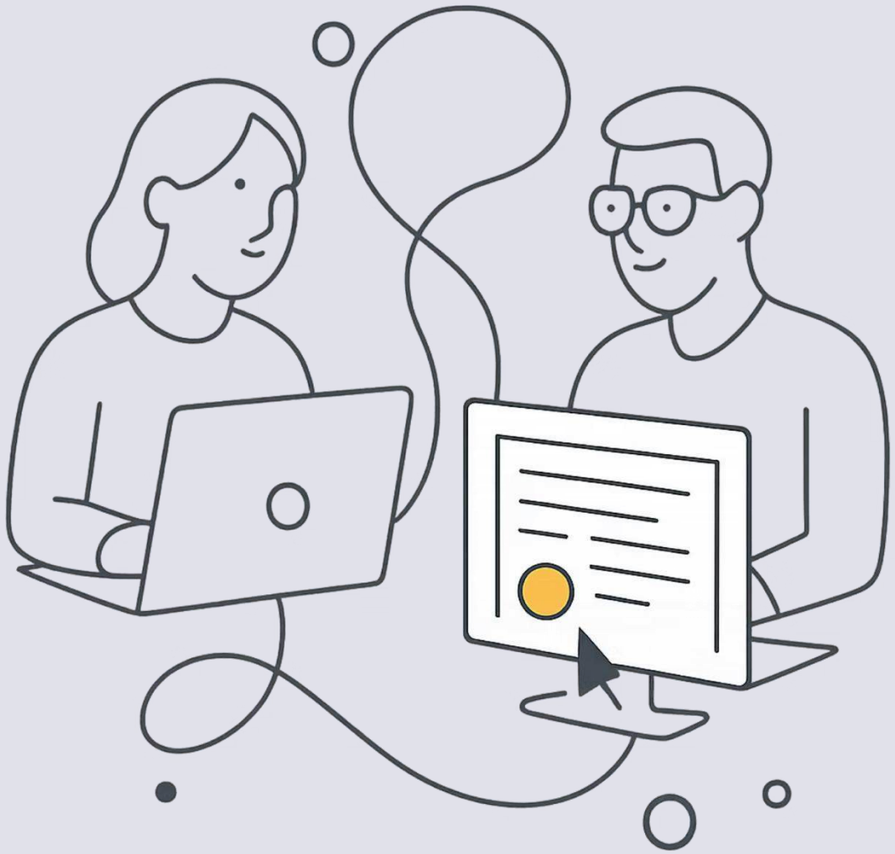
Schema per Service

Zentrale Datenbank mit getrennten Schemata – ermöglicht explizite Sicherheitseinstellungen.

Database Server per Instance

Jeder Service nutzt eine eigene Datenbankinstanz – maximale Trennung, höchster Infrastrukturaufwand.

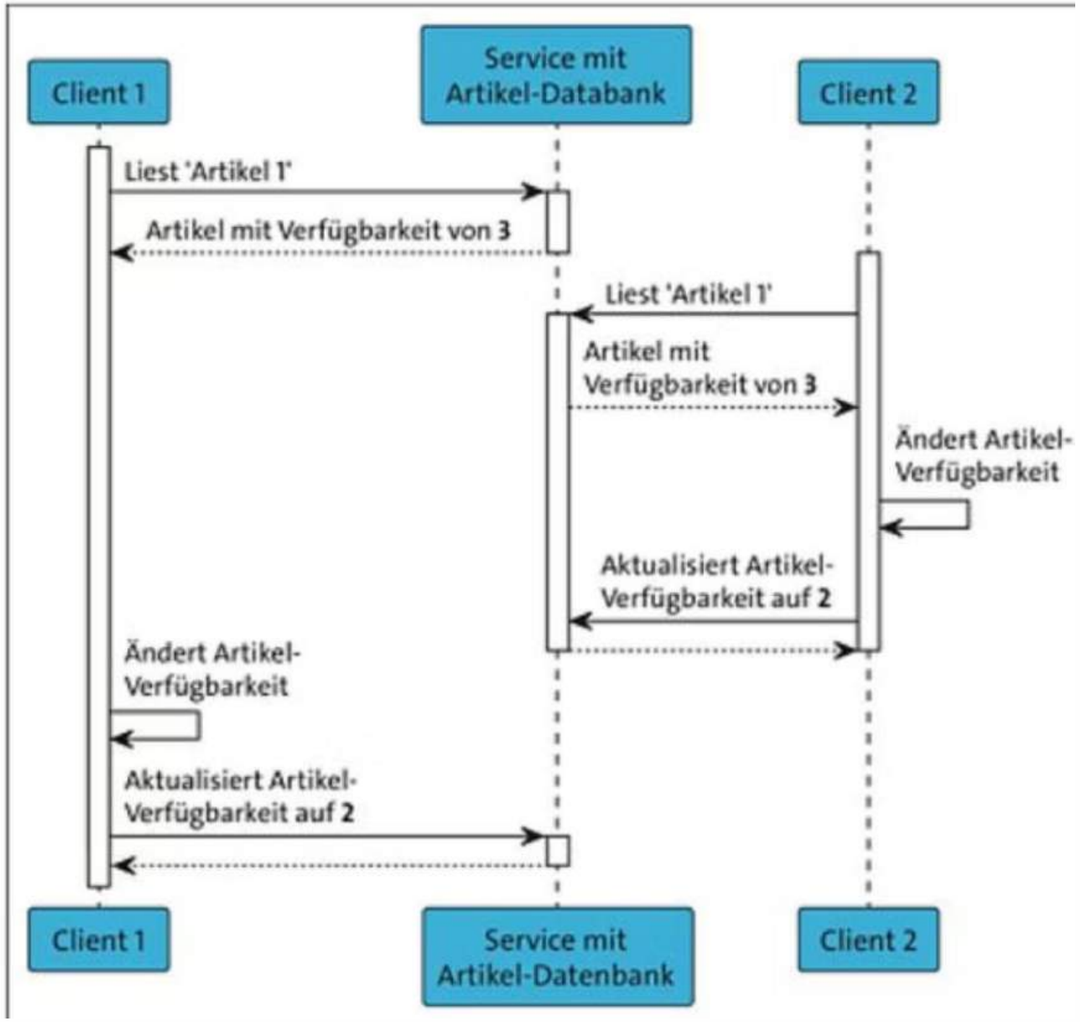
Konsequenz: Transaktionen über mehrere Services werden komplexer – das Saga Pattern adressiert dieses Problem.



Optimistic Locking Pattern

Konsistenz · Parallelitätskontrolle · Lost Update Problem

Problem: Das Lost Update Problem



Lösung: Optimistic Locking mit Versionsnummer

Jeder Datensatz erhält ein Versionsfeld. Vor dem Speichern prüft das System, ob die Version noch aktuell ist. Client 1 scheitert, weil Client 2 die Version bereits auf 2 erhöht hat – Konflikt wird erkannt, keine Daten gehen verloren.

Optimistic vs. Pessimistic Locking

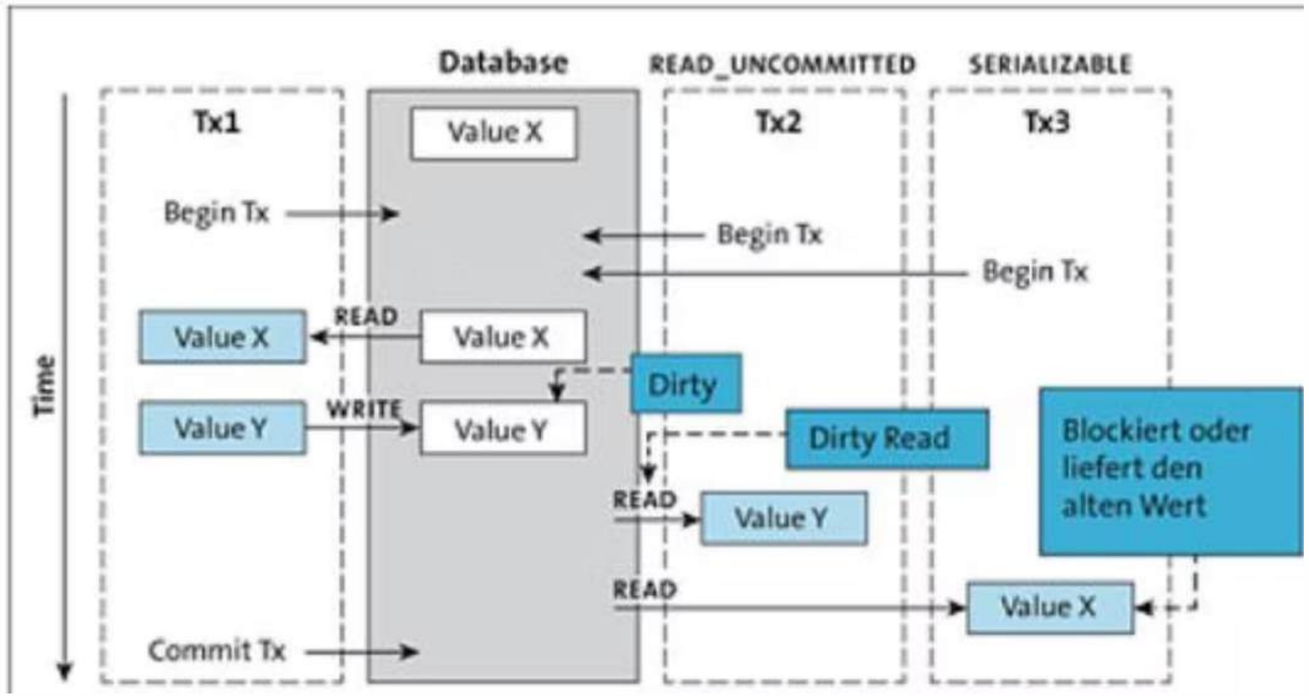
Optimistic Locking

- Kein Sperren beim Lesen
- Konflikterkennung beim Schreiben via Versionsnummer
- Gut bei wenigen Schreibkonflikten
- Skalierbar, keine Deadlocks
- Client muss Konflikt auflösen

Pessimistic Locking

- Exklusive Sperre beim Lesen (`SELECT FOR UPDATE`)
- Andere Clients warten oder erhalten Fehler
- Gut bei hoher Konfliktrate (z. B. Buchungssysteme)
- Risiko von Deadlocks und Performance-Einbußen
- Automatischer Schutz durch Transaktion

Isolation-Levels in relationalen Datenbanken



01

Read Uncommitted

Liest auch nicht commitete Änderungen → Dirty Read möglich

02

Read Committed

Liest nur commitete Änderungen

03

Repeatable Read

Wiederholte Lesevorgänge liefern konsistente Ergebnisse

04

Serializable

Höchste Isolation – Transaktionen laufen seriell

Saga Pattern

Verteilte Transaktionen · Kompensationslogik · Fehlerbehandlung

Problem: Verteilte Transaktionen

In Microservice-Architekturen mit dem Database-per-Service-Pattern ist eine klassische ACID-Transaktion über mehrere Services nahezu unmöglich.

Atomic	Consistent	Isolated	Durable
Alles oder nichts	Gültiger Zustand vor und nach	Transaktionen unabhängig	Änderungen dauerhaft

⚠ **Two-Phase-Commit (2PC) wäre eine Alternative, ist aber technologieabhängig, erfordert einen zentralen Transaktionsmanager und beeinträchtigt die Performance erheblich.**

Lösung: Das Saga Pattern

Ein Geschäftsprozess wird als **Sequenz lokaler, abgeschlossener Transaktionen** umgesetzt. Bei Fehlern werden **Kompensationsaktionen** ausgeführt, die vorherige Schritte rückgängig machen.

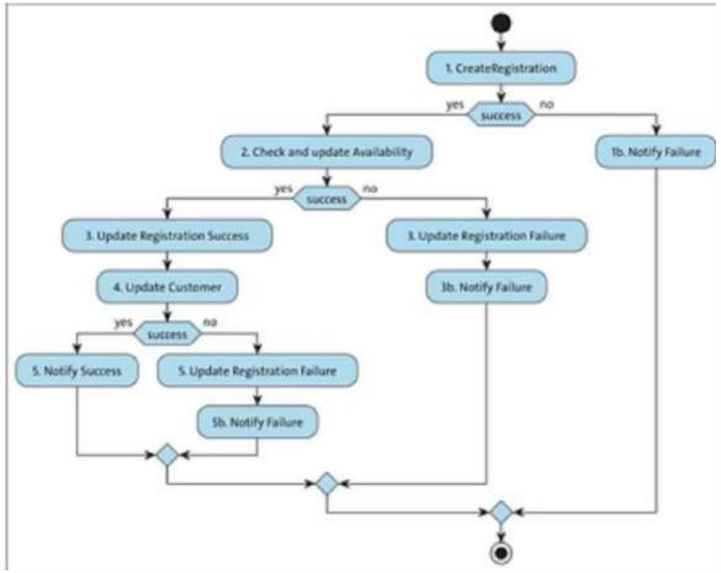
Choreography (event-basiert)

Services kommunizieren über Nachrichten/Events. Kein zentraler Koordinator – jeder Service reagiert auf Events und sendet neue.

Orchestration (orchestriert)

Eine zentrale Instanz koordiniert den Gesamtprozess und steuert die einzelnen Services durch direkte Aufrufe.

Saga – Beispiel: Schulungsregistrierung



Vier Services (Registration, Administration, Customer, Notification) arbeiten zusammen.

Bei fachlichen Fehlern (z. B. kein Platz verfügbar) werden Kompensationsaktionen ausgelöst – die Registrierung wird storniert und der Kunde benachrichtigt.

Konsequenzen des Saga Patterns

Langlaufende Transaktionen möglich

Geschäftsprozesse über mehrere Services hinweg können konsistent durchgeführt werden.

Kein automatischer Rollback

Kompensationsaktionen müssen explizit implementiert werden – höherer Entwicklungsaufwand.

ACID-Isolation verletzt

Zwischen den Schritten einer Saga sind Zwischenzustände für andere Prozesse sichtbar.

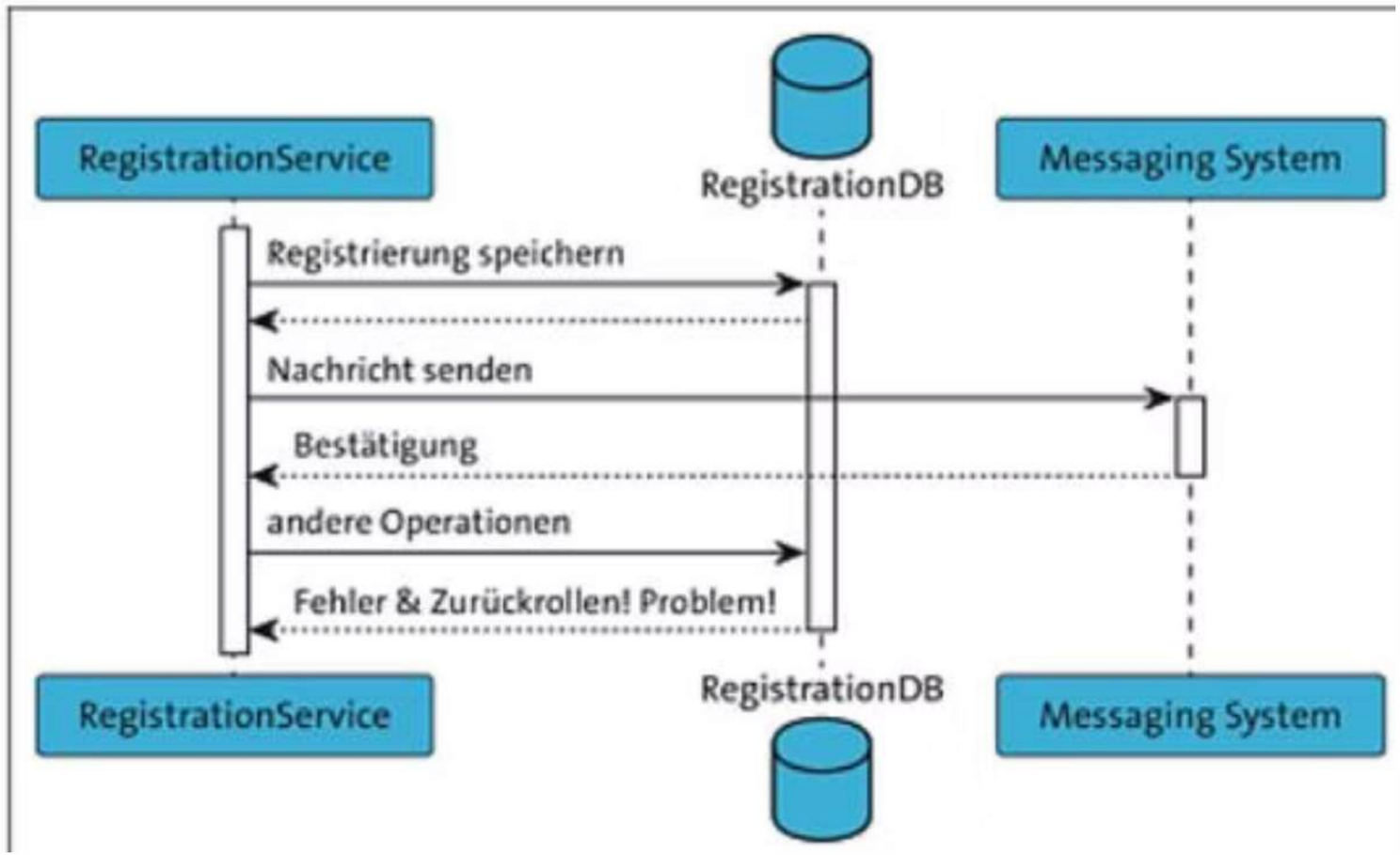
Asynchrone Kommunikation erforderlich

Nachrichtenauslieferung muss transaktional gesichert sein – das Transactional Outbox Pattern hilft dabei.

Transactional Outbox Pattern

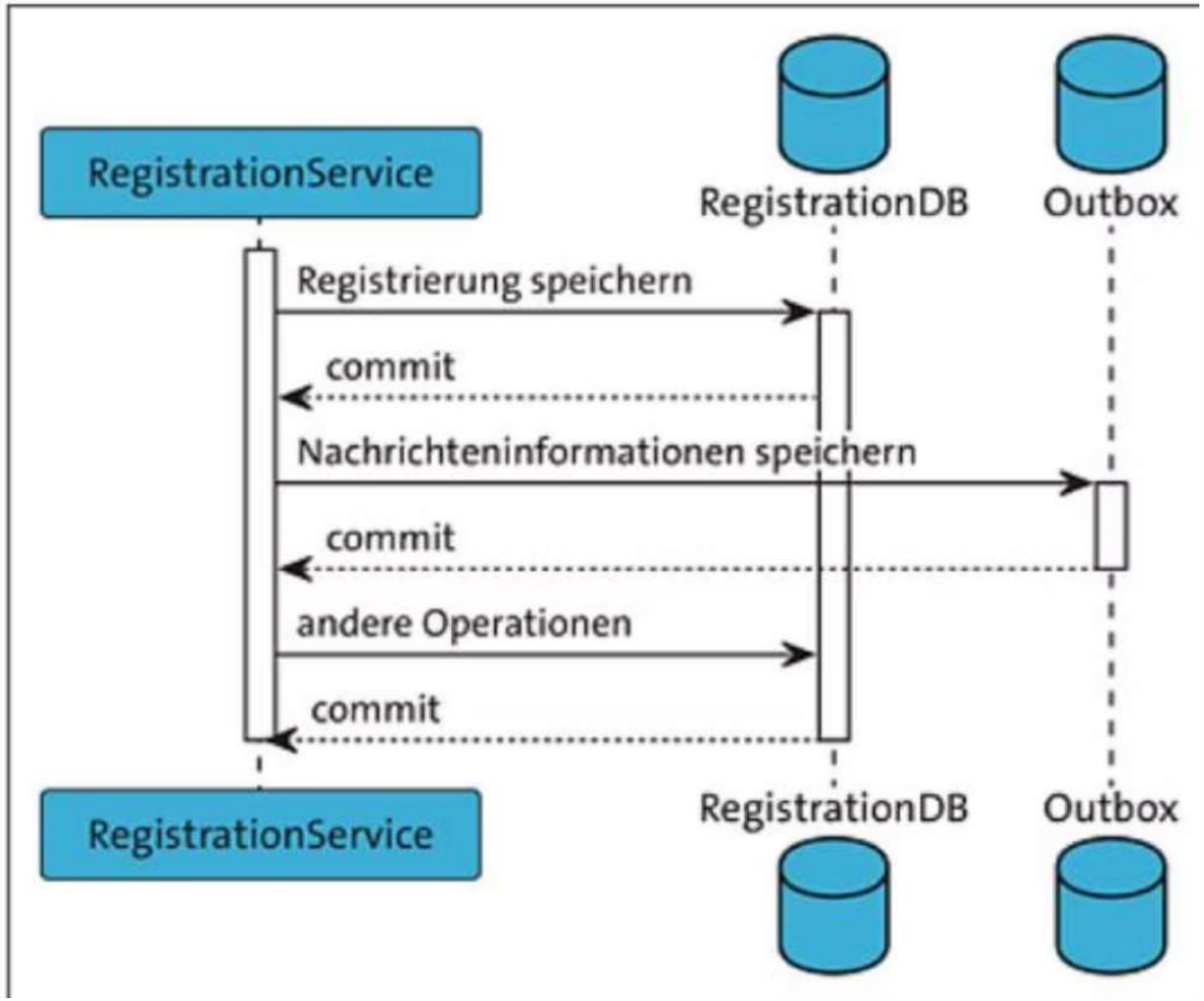
Konsistenz · Gesicherte Nachrichtenzustellung · Messaging

Problem: Inkonsistenz bei Nachrichtenversand



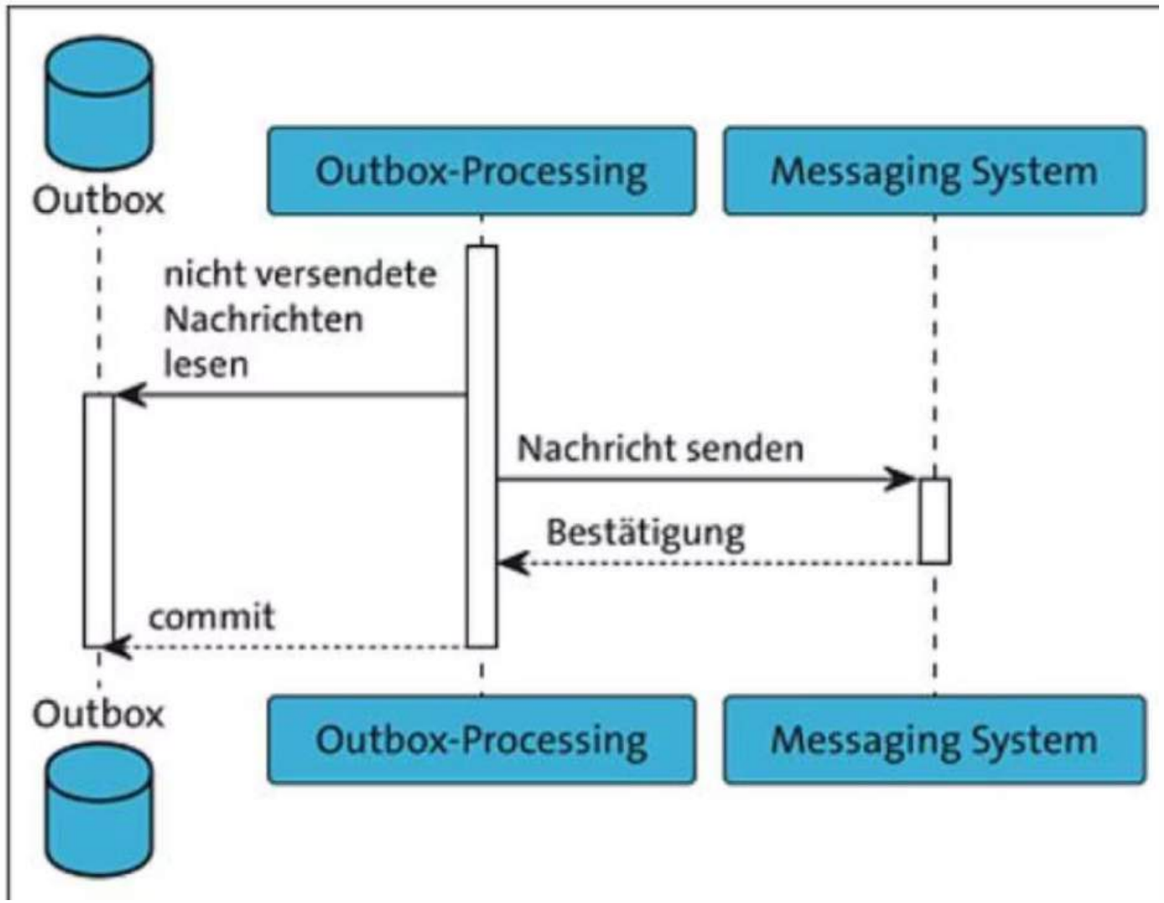
Eine Nachricht wird versendet, danach tritt ein Fehler auf und die Datenbanktransaktion wird zurückgerollt. Die Nachricht kann **nicht zurückgeholt** werden – Dateninkonsistenz droht.

Lösung: Transactional Outbox – Schritt 1



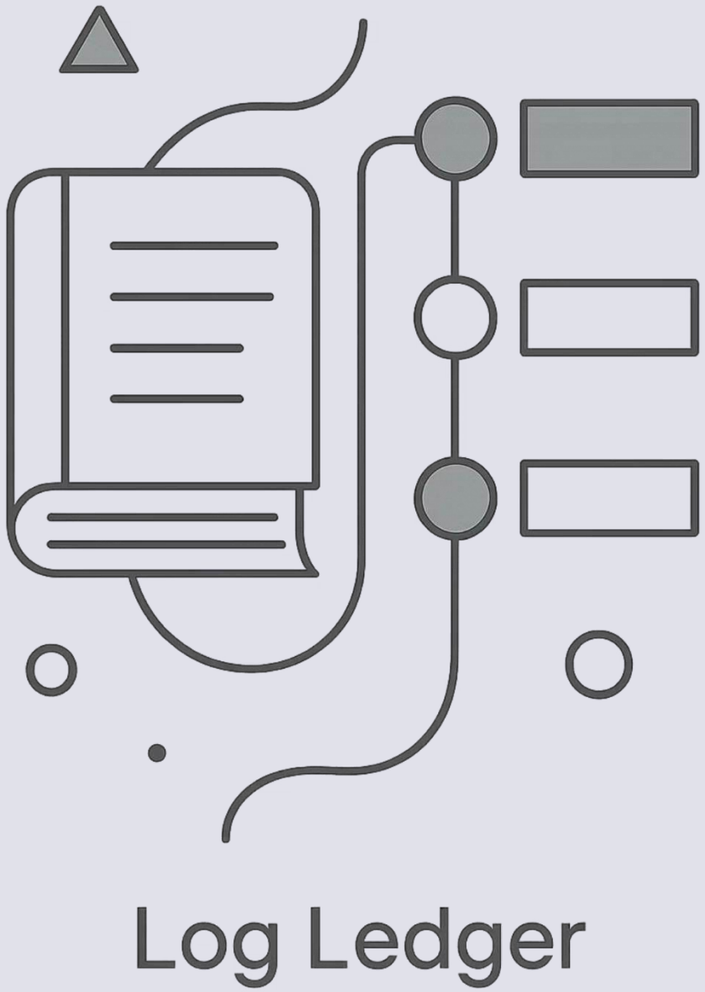
Statt die Nachricht direkt zu versenden, wird sie in einer **Outbox-Tabelle** in derselben Datenbanktransaktion gespeichert. Bei einem Rollback werden sowohl Datenänderung als auch Nachrichteninformation gemeinsam zurückgerollt.

Lösung: Transactional Outbox – Schritt 2



Eine separate **Outbox-Processing-Komponente** liest periodisch unversendete Nachrichten aus der Outbox und übergibt sie dem Messaging-System. Erst nach erfolgreicher Zustellung wird die Nachricht als versendet markiert oder gelöscht.

ⓘ Nachrichten können doppelt versendet werden – Empfänger müssen idempotent implementiert sein (eindeutige Message-ID).



Event Sourcing Pattern

Zustandsmanagement · Historisierung · Rückverfolgbarkeit

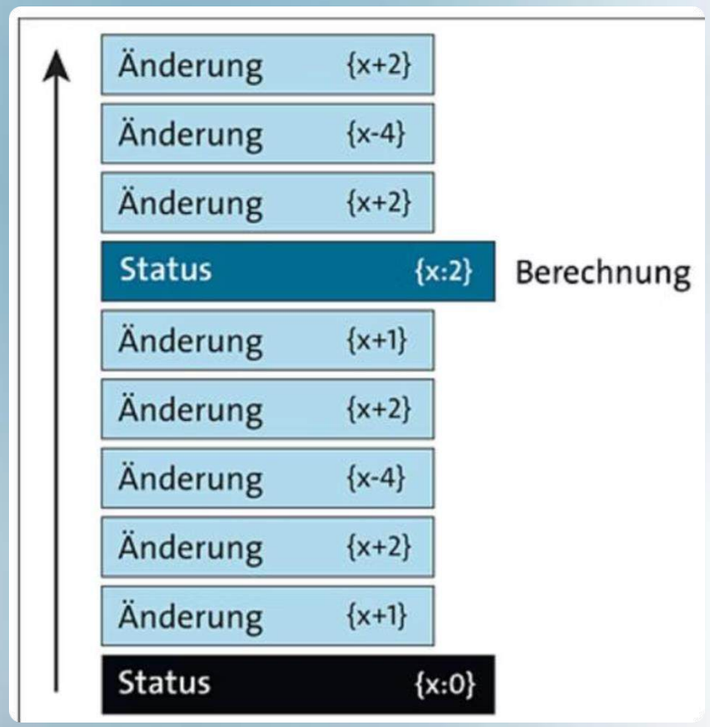
Idee: Zustand aus Ereignissen rekonstruieren

Klassischer Ansatz

Die Datenbank speichert den **aktuellen Zustand** eines Objekts. Bei jeder Aktualisierung wird der alte Zustand überschrieben – die Historie geht verloren.

Event Sourcing

Die Datenbank speichert die **vollständige Liste aller Änderungen**. Der aktuelle Zustand wird aus der Summe aller Ereignisse rekonstruiert. Alte Zustände bleiben jederzeit abrufbar.



Event Sourcing – Wann einsetzen?

Auditierbarkeit

Vollständige Nachvollziehbarkeit aller Änderungen ist gefordert (z. B. Finanz- oder Medizinsysteme).

Replay von Events

Anpassungen an der Verarbeitungslogik sollen nachträglich auf historische Daten angewendet werden können.

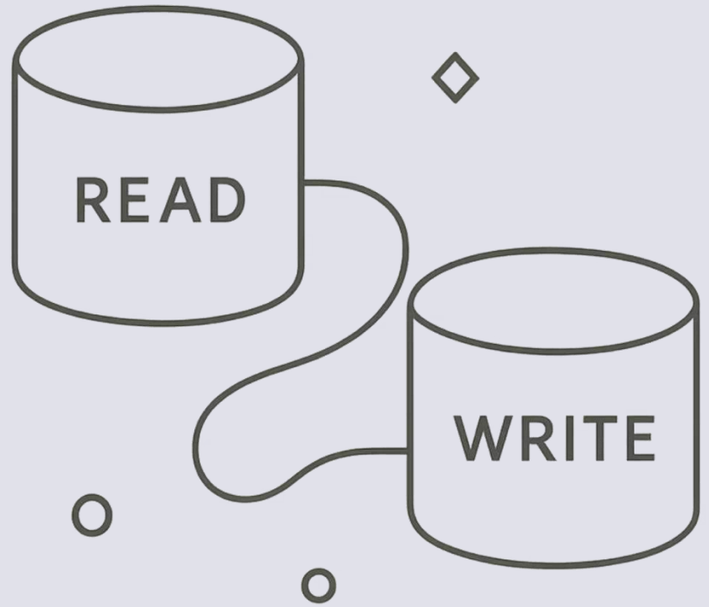
Entkopplung Lesen/Schreiben

Viele Leseoperationen, wenige Schreiboperationen – Zustand wird im Speicher gehalten, Updates asynchron verarbeitet.

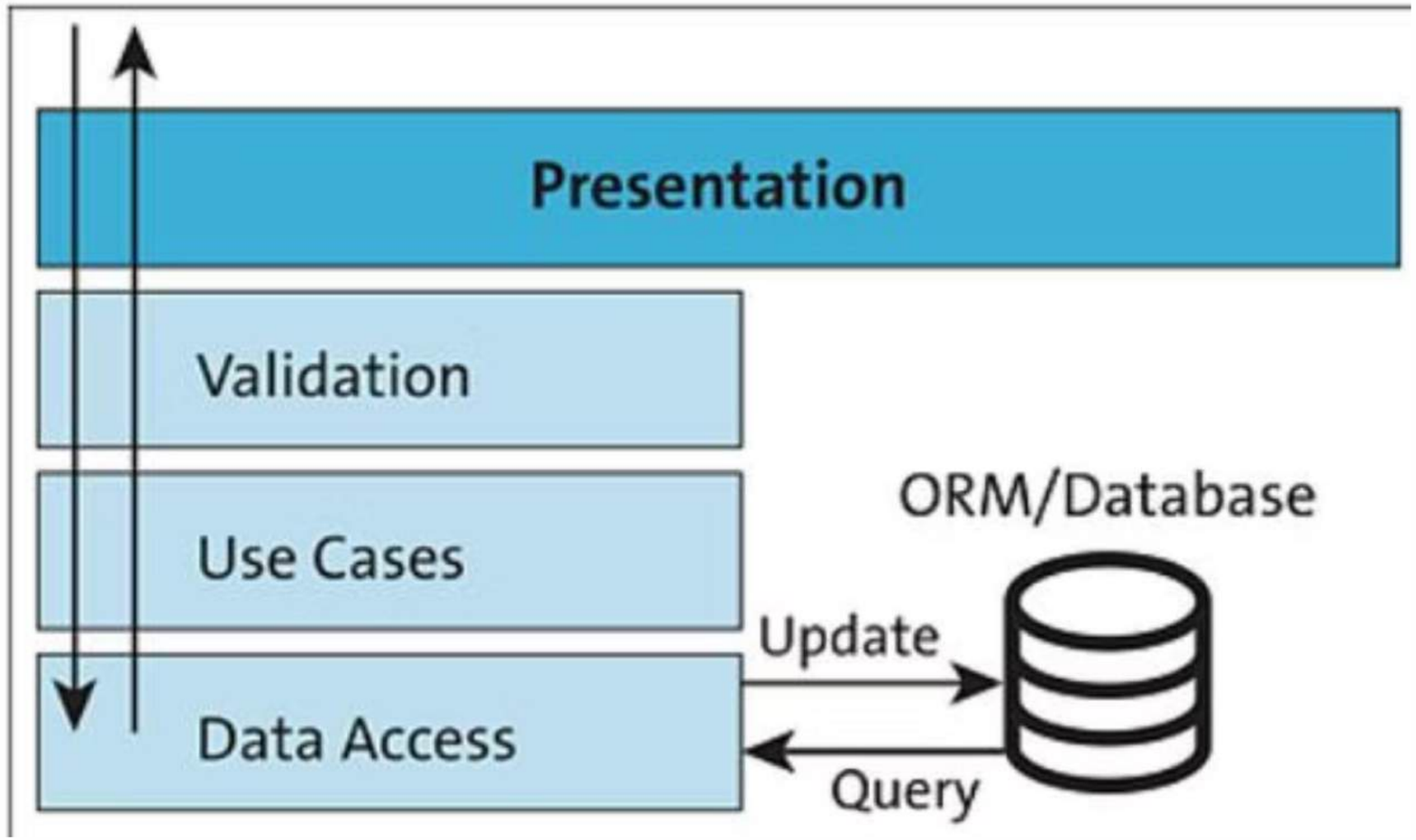
⚠ **Achtung: Speicher- und Rechenbedarf steigt kontinuierlich. Zwischenzustände (Snapshots) können die Berechnung optimieren.**

CQRS Pattern

Command Query Responsibility Segregation · Performance · Skalierbarkeit

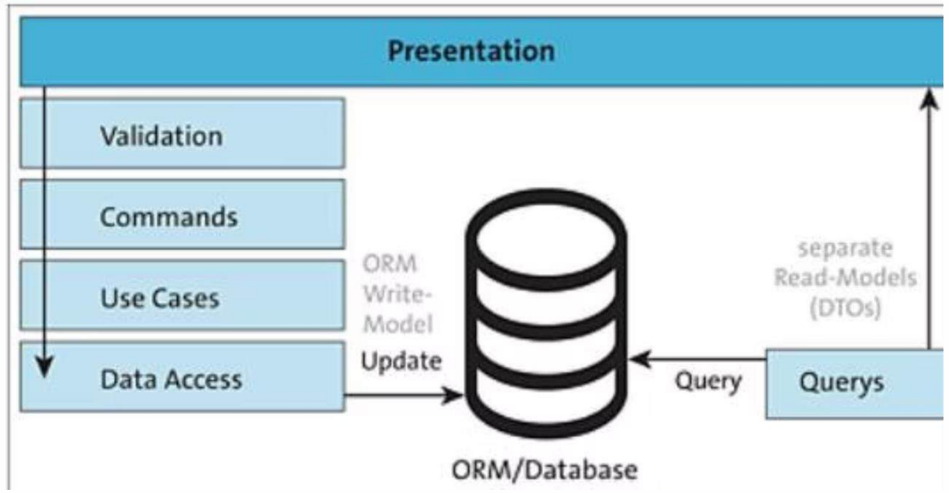


Problem: Ein Modell für alles



Wenn dasselbe Datenmodell für Lese- und Schreiboperationen verwendet wird, entstehen Probleme: unnötig geladene Daten, komplexe Mappings, Validierungen für irrelevante Felder und Serialisierungskonflikte bei gleichzeitigen Zugriffen.

Lösung: CQRS – Trennung von Commands und Queries



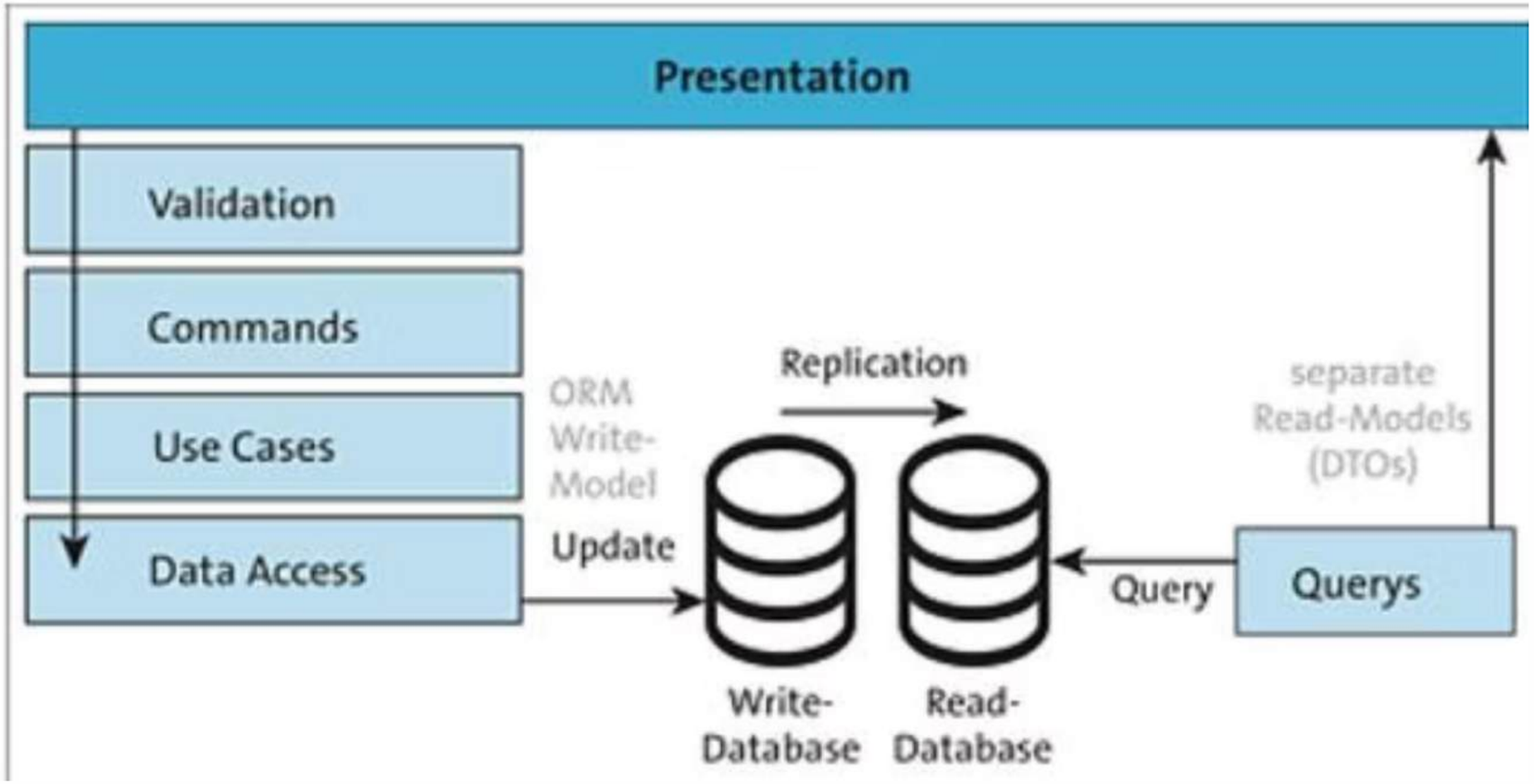
Commands (Schreiben)

Aufgabenbezogen, nicht datenzentriert. Beispiel: *„Melde User XY bei Schulung an"* **statt** *„Setze Verfügbarkeit auf X"*.

Queries (Lesen)

Für den jeweiligen Anwendungsfall optimierte Lesemodelle (DTOs). Keine Validierung nötig, da keine Daten verändert werden.

CQRS mit getrennten Datenbanken



Schreib- und Lesedatenbank werden getrennt. Die Lesedatenbank wird durch Replikation oder Messaging aktuell gehalten. **Materialized Views** speichern vorab berechnete Abfrageergebnisse – Joins bei Lesezugriffen entfallen.

CQRS – Konsequenzen

Vorteile

- Lese- und Schreibmodelle unabhängig optimierbar
- Bessere Performance bei leselastigen Anwendungen
- Klare Trennung der Verantwortlichkeiten

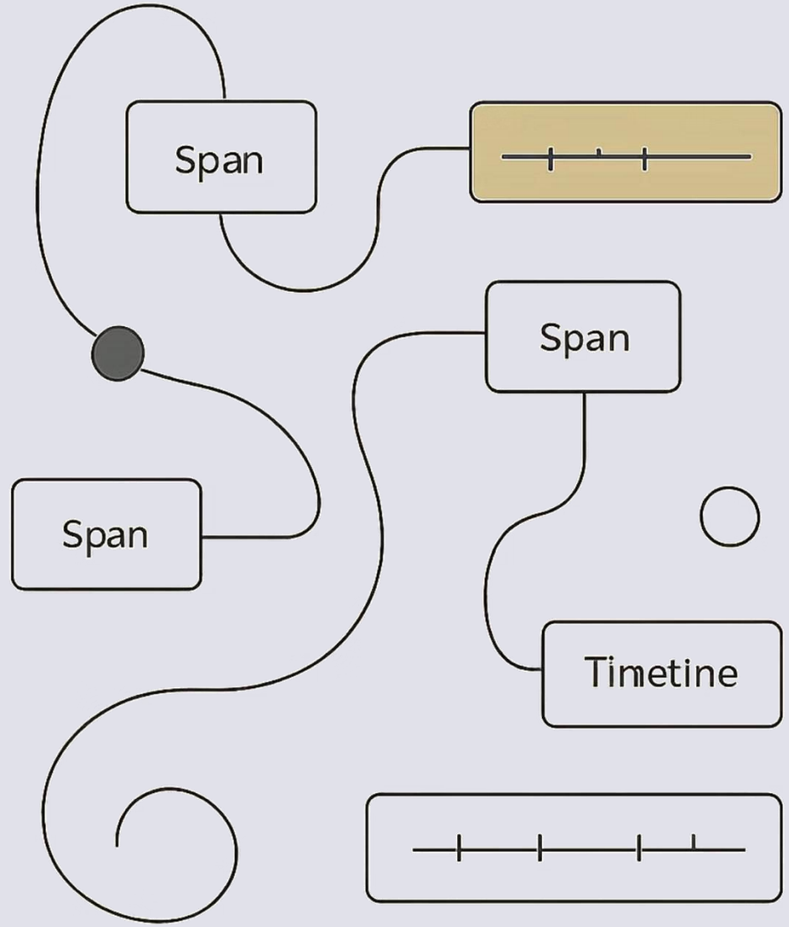
Nachteile

- Höhere Komplexität durch zwei Modelle
- Synchronisation mehrerer Datenbanken nötig
- Eventual Consistency zwischen Schreib- und Lesedatenbank möglich

CQRS wird häufig in Kombination mit Event Sourcing eingesetzt – Events dienen als Schreibquelle, Materialized Views als optimierte Lesemodelle.

Distributed Tracing Pattern

Monitoring · Debugging · Performanceanalyse in verteilten Systemen



Network trace visaalization

Problem: Nachverfolgung in verteilten Systemen

In Microservice-Architekturen durchläuft eine Anfrage ein komplexes Netz von Services. Klassische Verfahren wie lokale Log-Dateien oder Profiling-Tools sind für eine lokale Auswertung konzipiert und versagen bei verteilten Systemen.

→ Verteilte Log-Dateien

Müssen zuerst gesammelt, dann aggregiert und schließlich analysiert werden – zeitaufwendig.

→ Fehlende Korrelation

Ohne gemeinsame Kennung für Aufrufe ist eine Aggregation über Service-Grenzen hinweg kaum möglich.

→ Dynamische Umgebungen

Jeder Service wird unabhängig skaliert – die Ausführungsumgebung ändert sich ständig.

Kernkonzepte des Distributed Tracing

Trace

Gesamter Lebenszyklus einer Anfrage durch das verteilte System – identifiziert durch eine eindeutige Trace-ID.

Span

Einzelne Operation innerhalb eines Trace. Enthält Start-/Endzeit, Dauer und Metadaten. Referenziert den Parent-Span.

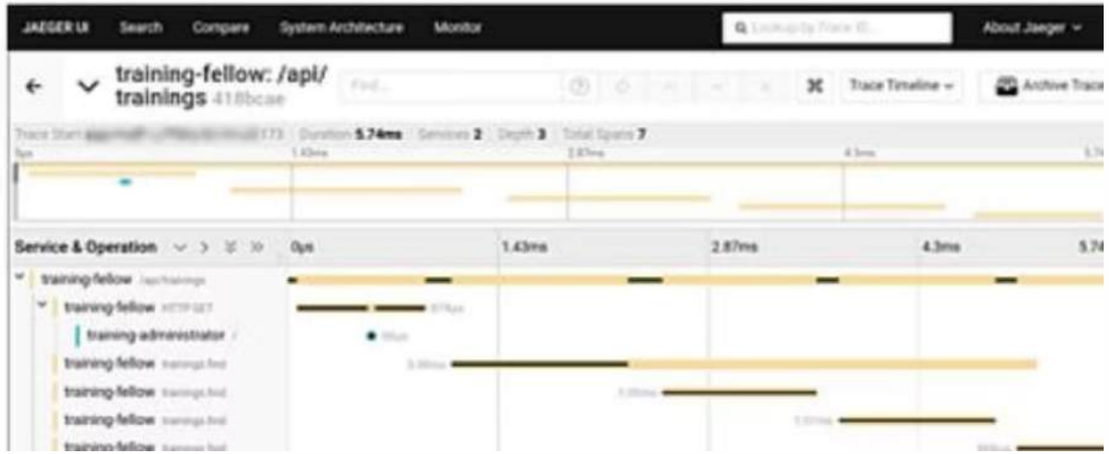
Tag / Attribut

Kontextabhängige Schlüssel-Wert-Paare pro Span, z. B. HTTP-Methode, Status-Code oder Datenbankabfrage.

Distributed Tracing – Ablauf und Jaeger UI

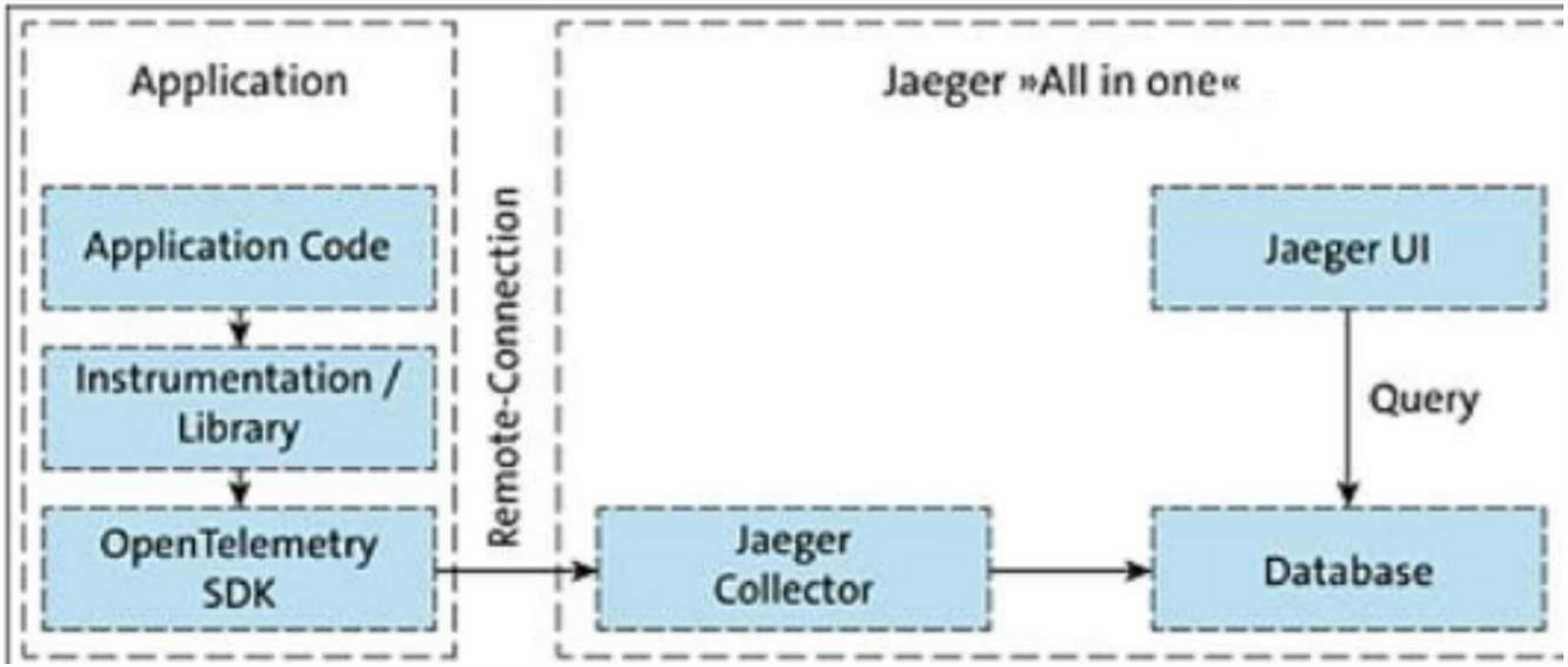
Ablauf

Service A erzeugt eine Trace-ID und einen Span. Beim Aufruf von Service B wird die Trace-ID im HTTP-Header `traceparent` (W3C-Standard) mitgegeben. Service B erzeugt einen neuen Span, der mit dem Parent-Span verknüpft ist.



Jaeger UI: Visualisierung eines Trace über 2 Services und 7 Spans mit Dauer 5,74 ms.

OpenTelemetry und Jaeger



OpenTelemetry (OTel) ist der Open-Source-Standard für Distributed Tracing, entstanden 2019 aus OpenTracing und OpenCensus. Es definiert das OTLP-Protokoll, eine API und SDKs. **Jaeger** ist eine Open-Source-Tracing-Plattform, die OTel-Daten verarbeitet und über eine Web-UI visualisiert.

Konsequenzen: Distributed Tracing

Transparenz

Prozesse in verteilten Systemen werden vollständig nachvollziehbar und überwachbar.

Fehlerdiagnose

Fehlersuche in verteilten Systemen wird vereinfacht – Engpässe und Fehler sind präzise lokalisierbar.

Zusätzliche Infrastruktur

Zentrale Tracing-Lösung wird benötigt; Ressourcenbedarf der Anwendung steigt durch Erzeugung und Übermittlung der Tracing-Daten.

Standardisierung

Durch OpenTelemetry wird Tracing interoperabel – Daten verschiedener Systeme können einheitlich erfasst und ausgewertet werden.

Zusammenfassung

Konsistenz ist ein Kompromiss

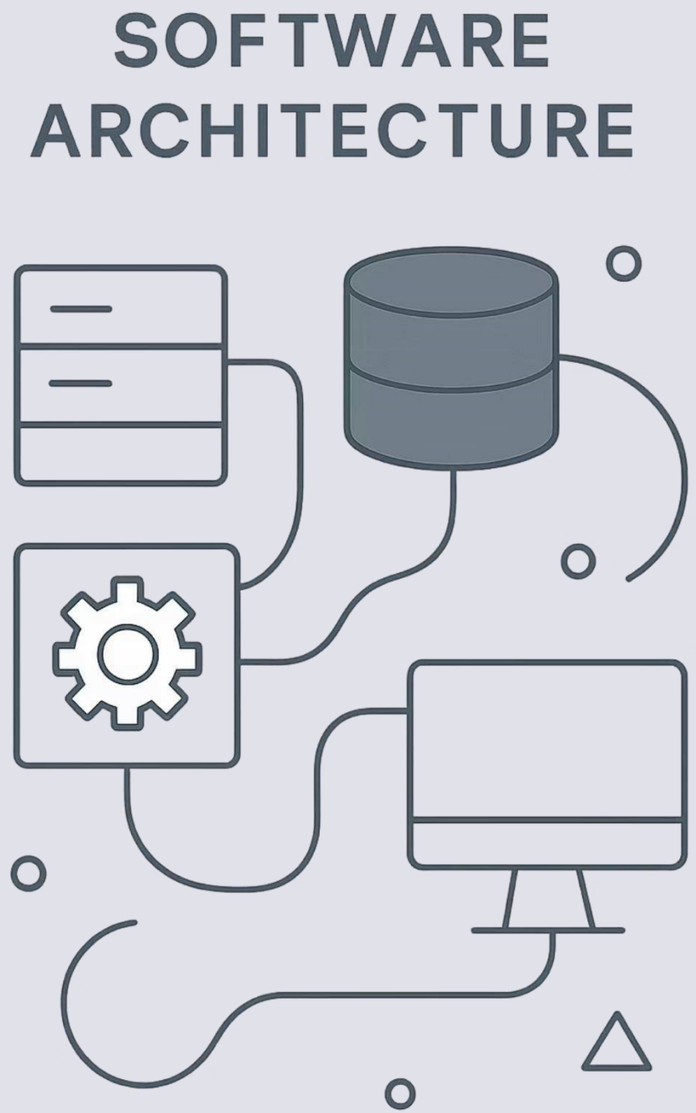
CAP und PACELC zeigen: Starke Konsistenz kostet Verfügbarkeit oder Latenz. Eventual Consistency ist oft der pragmatische Weg.

Patterns lösen konkrete Probleme

Stateless, Database per Service, Optimistic Locking, Saga, Outbox, Event Sourcing, CQRS und Distributed Tracing adressieren spezifische Herausforderungen.

Patterns kombinieren sich

CQRS + Event Sourcing, Saga + Transactional Outbox – die Patterns ergänzen sich gegenseitig zu robusten Architekturen.

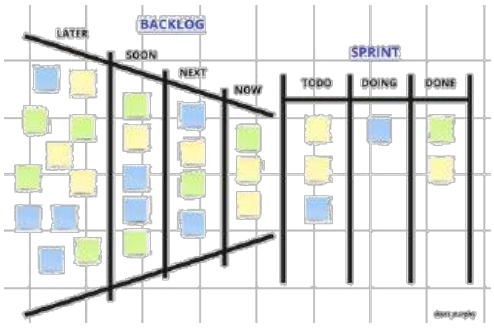


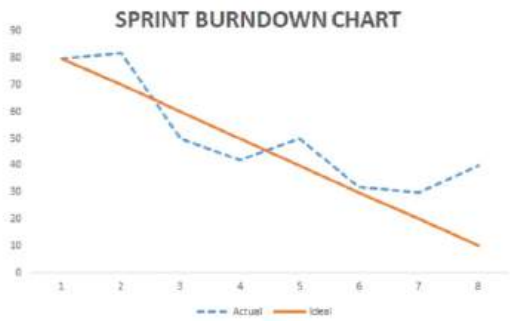
Praktikumsaufgabe für KW 18

Praktikumsaufgabe

Die Praktika werden in diesem Semester wie folgt ablaufen. Sie bearbeiten die Praktikumsaufgabe und bringen ihre Lösung mit zum Praktikumstermin. Dort stellen Sie ihre Lösung vor und erhalten direkt Hinweise zur Verbesserung.

1. **Kanban-Board und Burndown-Chart**
   Legen Sie für Ihr Team ein **Kanban-Board** zur Sprintplanung und ein **Burndown-Chart** an. Die Boards/Charts könnten beispielsweise folgende Struktur haben:





2. **Epics**
   Formulieren Sie bitte Epics (und User Stories vgl. Aufgabe 3) zu ihrem Schwerpunkt der MyStudyApp. Zur Erinnerung:
   - Ein Epic ist **ein großes Anforderungspaket**, das ein übergeordnetes Ziel beschreibt.
     Man kann sich ein Epic wie ein **Thema oder Feature-Block** vorstellen, der noch zu groß ist, um ihn direkt umzusetzen.
   - Beispiel-Epic: „Studierende können ihr Studium digital organisieren."

3. **User Stories**
   Eine **User Story** ist eine konkrete, **kleine Anforderung aus Sicht des Nutzers**. Sie folgt meistens diesem Schema: „Als [Nutzer] möchte ich [Funktion], damit [Nutzen]." Und besitzt folgende Eigenschaften:
   - klein und umsetzbar (oft in wenigen Tagen)
   - klar verständlich
   - beschreibt echten Mehrwert für Nutzer

Beispiel-User Stories:

- „Als Student möchte ich meinen Stundenplan sehen, damit ich weiß, wann meine Vorlesungen stattfinden.“
- Als Student möchte ich meinen Stundenplan importieren, damit ich ihn nicht manuell eingeben muss.“
- „Als Student möchte ich Vorlesungen nach Tagen filtern.“
- „Als Student möchte ich Push-Benachrichtigungen vor Vorlesungen erhalten.“

Viel Erfolg!

Praktikumsaufgabe für KW 18 / KW 19

Praktikumsaufgabe

Sie haben in der Vorlesung Maven als Build-Management-Tool kennengelernt. Nutzen Sie Maven, um folgende Aufgaben zu bearbeiten:

1. **Projektstruktur in Maven anlegen**
   Legen Sie Ihre Projektstruktur fest. Berücksichtigen Sie, dass Sie zwei Systeme im Backend benötigen, die über einen MQTT-Broker kommunizieren. Dazu gibt es mindestens ein (progressive) Web-Frontend und eine Datenbank.

   Hinweis:
   - Denken Sie daran, dass neue Maven-Projekte in der pom.xml als Module hinzugefügt werden müssen.

2. **Test des Build-Prozesses**
   Legen Sie in den Backend-Projekten eine main-Funktion an, die den Namen des Projektes in der Console ausgibt, um den definierten Build-Prozess zu testen.

   Hinweise:
   - Leiten Sie einen Feature-Branch vom Main-Branch in Git für die Implementierung ab.

   - Nach der Implementierung der Main-Funktionen veröffentlichen Sie Ihre Änderungen in dem jeweiligen Branch im Git Repository und erstellen einen Pull-Request für den Main-Branch.

   - Bei einer komplexeren Implementierung käme dann an dieser Stelle ein Code-Review

   - Überführen Sie nun den Pull-Request in den Main-Branch, dazu müssen zuvor ihre Actions erfolgreich durchgelaufen sein.

3. **Planen Sie Ihren ersten Sprint (Sprint-Planning)**
   Weisen Sie dazu die erfassten User Stories (letztes Praktikum) Teammitgliedern zu, die diese implementieren sollen.

   Hinweis:
   - Nutzen Sie bei GitHub die Funktion Milestones, um die Sprints abzubilden. Jeder Sprint sollte einen eigenen Milestone haben.

Viel Erfolg!

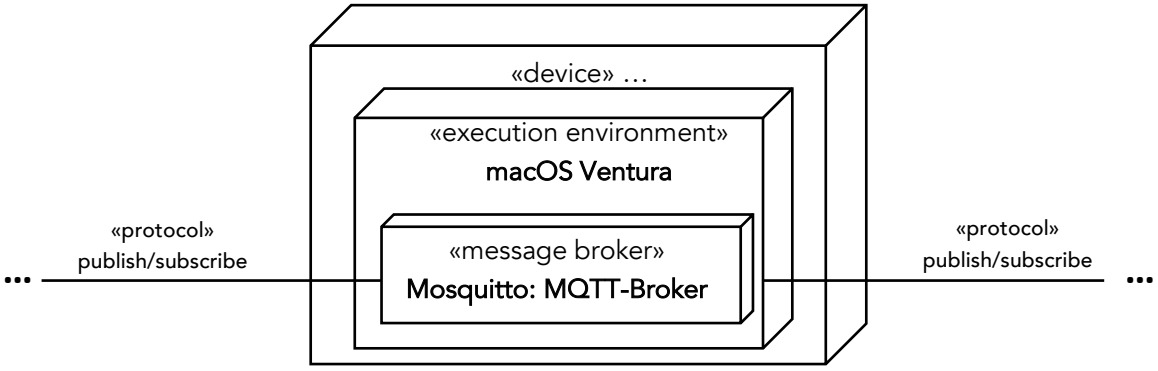
Praktikumsaufgabe für KW 20 und 23

Praktikumsaufgabe

Sie haben bisher ihr Projekt von der Anforderungsseite geplant und die Infrastruktur aufgesetzt. Jetzt erstellen Sie bitte ein Verteilungsdiagramm und legen das API für Ihre Businesslogik fest.

1. **Verteilungsdiagramm**
   Das Verteilungsdiagramm soll alle verwendeten Technologien darstellen, die Sie für die Implementierung nutzen. Planen Sie zwei Backend-Systeme, die asynchron über einen MQTT-Broker (Vorlesung dazu kommt noch) kommunizieren. Die Backends sollen mit Spring realisiert werden. Ein System ist das Backend zu ihrem Frontend und das zweite System simuliert ein weiteres System, welches Ihnen Daten liefert (z.B. Studentenwerk, ÖPNV, …).

   Hier finden Sie eine Hilfestellung zur Modellierung der asynchronen Kommunikation über MQTT:



2. **Abhängigkeiten**
   Ermitteln Sie welche Abhängigkeiten Sie in Spring zur Anbindung von Mosquitto benötigen. Und passen Sie Ihre Maven-Konfiguration entsprechend an.

3. **Implementierung der REST-Schnittstellen**
   Beginnen Sie mit der Implementierung als Modulith nach der Vertical-Slicing-Architektur. Definieren Sie für jedes Modul einen REST-Controller mit den dazugehörigen Endpunkten als Schnittstelle.

Viel Erfolg!

Fachhochschule Dortmund
University of Applied Sciences and Arts

we focus on students

Hirsch/Sachweh/Zeppenfeld Dortmund, 8.5.2026

Praktikumsaufgabe 4

Softwaretechnik 2

Sommersemester 2026

Bearbeitungszeitraum Informatik Dual ab KW 20 / Informatik ab KW 20

Aufgabenstellung

Aufgabe 1 – OOD-Klassendiagramme für Backend-Systeme

Im Rahmen der Projektdokumentation haben Sie bereits ein Verteilungsdiagramm modelliert. Um die Dokumentation zu ergänzen und zu verfeinern, erstellen Sie bitte für jedes Backend-System ein objektorientiertes Entwurfs-Klassendiagramm (OOD) gemäß der korrekten UML-Notation.

Verwenden Sie als Ausgangsbasis die in SWT1 erstellten objektorientierten Analyse-Klassendiagramme (OOA). Verfeinern Sie diese unter Berücksichtigung der in der Vorlesung vorgestellten Desginprinzipien für den Feinentwurf.

Achten Sie darauf, dass die UML-Klassendiagramme mit Ihrer Implementierung konsistent sind. Insbesondere gilt:

- Alle im Modell dargestellten Klassen müssen in der Implementierung vorhanden sein.
- Assoziationen aus dem Modell sind korrekt in Code umzusetzen.
- Vererbungsbeziehungen sind entsprechend der Modellierung korrekt zu implementieren oder bei Bedarf durch alternative Entwurfsmuster (siehe Aufgabe 2) zu ersetzen.
- Schnittstellen (*Interfaces*) und abstrakte Klassen sind gemäß der UML-Notation zu modellieren und in der Implementierung entsprechend zu realisieren.
- **Halten Sie ihr Klassendiagramm übersichtlich!** Methoden und Attribute sollen grundsätzlich nicht im Klassendiagramm mit modelliert werden.
  Ausnahme: Bei den Klasssen, die an einem Entwurfsmuster beteiligt sind (siehe Aufgabe 2), müssen Sie alle relevanten Methoden und Attribute mit modellieren. Achten Sie dabei auf die korrekten Sichtbarkeiten (public, private, protected) von Attributen und Methoden gemäß den UML-Konventionen.

Aufgabe 2 – Integration von Entwurfsmustern in das OOD-Klassendiagramm

In der Vorlesung haben Sie verschiedene Kategorien von Entwurfsmustern kennengelernt: **Erzeugungsmuster (Creational Patterns)**, **Strukturmuster (Structural Patterns)** und **Verhaltensmuster (Behavioral Patterns)**.

Bitte analysieren Sie Ihr bestehendes OOD-Klassendiagramm und identifizieren Sie geeignete Stellen, an denen Entwurfsmuster sinnvoll und zweckmäßig eingebunden werden können. Ziel ist es, die Struktur und Wartbarkeit Ihres Systems durch den Einsatz bewährter Designlösungen zu verbessern.

Integrieren Sie mindestens **drei unterschiedliche Entwurfsmuster** in Ihr OOD-Klassendiagramm, wobei aus jeder der oben genannten Kategorien mindestens ein Muster vertreten sein muss:

- Nutzen Sie die in der Vorlesung vorgestellten Entwurfsmuster als Referenz.
- Verwenden Sie **nicht** das Singleton-Muster.
- Kennzeichnen Sie die eingesetzten Entwurfsmuster im Klassendiagramm deutlich, beispielsweise durch farbliche Markierungen oder entsprechende Notationen.
- In der Abschlusspräsentation muss jedes Team-Mitmitglied in der Lage sein, für jedes eingesetzte Muster:
  - Den Namen des Entwurfsmusters,
  - Die Motivation für dessen Einsatz an der jeweiligen Stelle,
  - Eine kurze Beschreibung, wie das Muster in Ihrem Design umgesetzt wurde,

  zu erklären.